INTERVIEWHUB · INTERVIEW PREP

Linux Systems Administration Interview Guide

Platform, SRE & DevOps interview guide

Questions	**175 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/linux_interview_guide.html**
Printed	**2026-10-23**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

certs.eknathalabs.com

Q1 How do you find and kill a process consuming excessive CPU or memory?

Processes

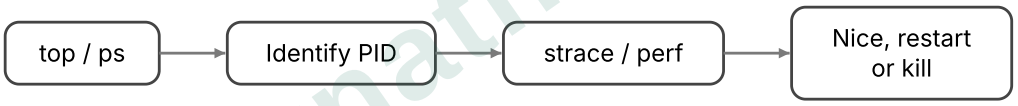
🎙 Spoken answer · ~83 s

To find a process hogging CPU or memory, I start with top, sorted by CPU, or by memory by pressing M, or htop for an easier view. For a snapshot I use ps aux sorted by CPU or memory and take the top few lines. If I know the name, pgrep minus fl shows PIDs with their command lines. Before killing, I check what it is: its parent, how long it has been running, and whether it's a legitimate busy service or a runaway. Then I terminate gracefully: kill with SIGTERM, signal fifteen, which lets the process clean up, flush buffers and close connections. I wait a few seconds and verify it exited. Only if it ignores that do I use SIGKILL, signal nine, which can't be caught and may leave temp files or corrupt state. For a service managed by systemd, I restart it through systemctl rather than killing the PID, otherwise systemd may just restart it or lose track. Afterwards I find the root cause, a leak, a loop, or a traffic surge, using logs and profiling, and set limits with cgroups or systemd, such as MemoryMax and CPUQuota, so one process can't take down the host.

Likely follow-ups

- Why try SIGTERM before SIGKILL?
- How would you limit a process's resources so this doesn't recur?

🗺 Runaway process



IDENTIFY

```
top -o %CPU          # sort by CPU (press M to sort by memory)
```

```
htop                 # interactive, easier to read
```

```
ps aux --sort=-%cpu | head -15   # snapshot sorted by CPU
```

```
ps aux --sort=-%mem | head -15   # snapshot sorted by memory
```

```
pidof nginx          # find PID by name
```

```
pgrep -fl java       # find PIDs matching pattern + show cmdline
```

certs.eknathalabs.com

KILL

```
kill -15 <pid>      # SIGTERM — graceful shutdown (try this first)
```

```
kill -9  <pid>      # SIGKILL — force kill (use when -15 fails)
```

```
killall nginx        # kill all processes named nginx
```

```
pkill -f "java.*myapp"  # kill by regex matching full command line
```

Always try SIGTERM (15) first — it allows the process to clean up (flush buffers, close connections, remove temp files). SIGKILL (9) cannot be caught or ignored — the kernel terminates the process immediately with no cleanup. Use SIGKILL only when SIGTERM fails or the process is unkillable.

Interview tip: Explain the difference between SIGTERM and SIGKILL. A process ignoring SIGTERM is either buggy or intentionally handling it (e.g., daemontools keeping a service alive). SIGKILL always works except for zombie processes — which cannot be killed at all (must reap the parent).

📚 **Official docs**

- Kubernetes: Pod lifecycle & termination
- Linux man-pages: signal(7)

certs.eknathalabs.com

Q2 What is a zombie process — why does it appear and how do you deal with it? Processes

🎙 **Spoken answer · ~81 s**

A zombie is a process that has finished executing, but still has an entry in the process table because its parent hasn't read its exit status with wait. It holds no memory or CPU, only a PID slot, so a few are harmless, but thousands can exhaust the PID space. In ps they show state Z or defunct. You can't kill a zombie, since it's already dead; the fix is to deal with the parent. I find the parent PID from the status file or ps, and check why it isn't reaping: usually a bug where the program never calls wait or doesn't handle SIGCHLD. Options: send SIGCHLD to the parent to prompt it to reap; fix or restart the parent, after which init adopts the orphaned zombies and reaps them immediately. In containers, zombies often appear because the main process isn't a real init and doesn't reap adopted orphans; the fix is running a small init such as tini, or Docker's init option. Distinguish zombies from processes in uninterruptible D state, which are stuck on I/O and are a different problem. Long term, fix the code to reap children and monitor the zombie count.

Likely follow-ups

- Why does killing the parent clear the zombies?
- How is a zombie different from a D-state process?

A zombie process has finished executing but still has an entry in the process table because its parent has not yet read its exit status via wait(). It holds no resources — just a PID slot.

```
ps aux | grep 'Z'          # Z state = zombie
```

```
ps -el | grep defunct      # another way to list zombies
```

```
cat /proc/<zombie-pid>/status | grep PPid  # find parent PID
```

```
kill -SIGCHLD <parent-pid>  # signal parent to reap children
```

You cannot kill a zombie with kill -9 — it has no running code. The fix is to signal or restart the parent process so it calls wait() to reap the zombie. If the parent is buggy and never reaps children, the parent itself must be killed — init (PID 1) will then adopt and reap the orphaned zombie. A few zombies are harmless; hundreds indicate a parent process leak.

PID table exhaustion: Each zombie holds one PID. Linux has a PID limit (default 32768, check /proc/sys/kernel/pid_max). Thousands of zombies can exhaust PIDs, preventing any new process from starting — a full system outage. Monitor zombie count in production.

📚 **Official docs**

- Linux man-pages: fork(2)/wait(2) (zombies)

Q3 What is the difference between a process and a thread — how does Linux manage them?

Processes

🎙 **Spoken answer · ~86 s**

In Linux, both processes and threads are tasks created by the clone system call; the difference is what they share. A process has its own address space, file descriptor table, and signal handlers, while threads created within a process share the memory space, open files, and the process ID, but each has its own stack, registers, and thread ID. In the kernel, each thread is a task with its own entry, which is why ps with the L flag shows a light-weight process ID per thread, and slash proc slash pid slash task lists one directory per thread. The scheduler schedules tasks, whether processes or threads, independently, so threads of one process can run on different CPUs in parallel. Sharing memory makes threads cheap to create and communicate between, but it also means a bug in one thread, such as a crash or memory corruption, can bring down the whole process, and shared data requires locking. Separate processes give isolation, so a crash is contained, at the cost of heavier creation and needing inter-process communication. For debugging, I check the thread count in the status file, and use top with the H option to see per-thread CPU. Containers use namespaces and cgroups on top of this.

Likely follow-ups

- What does clone do differently for threads and processes?
- How would you see per-thread CPU usage?

cors.eknathalabs.com

In Linux, threads and processes are both represented as "tasks" and created with clone(). The difference is what they share.

```
ps -eLf | grep <pid>     # show all threads of a process (LWP = Light Weight Process)
```

```
cat /proc/<pid>/status | grep Threads  # thread count for a process
```

```
ls /proc/<pid>/task/      # each subdirectory = one thread
```

```
pstree -p <pid>           # show process tree with PIDs
```

Process: separate address space, file descriptors, signal handlers. Thread: shares address space, heap, file descriptors with siblings — has its own stack and registers. fork() creates a new process (copy-on-write). clone() with CLONE_VM|CLONE_FILES creates a thread. In Linux, the scheduler sees threads and processes equally — both are schedulable tasks. This matters when diagnosing CPU usage: a Java app with 200 threads can show 200 entries in /proc/<pid>/task.

Practical use: When a Java or Go application is using unexpected CPU, use `top -H -p <pid>` to see per-thread CPU usage. This pinpoints which specific thread (and thus which goroutine or Java thread) is spinning.

📚 **Start here (official docs)**

- Linux man-pages project

Q4 How does process scheduling work in Linux — what is load average and when is it a problem?

Processes

🎙 **Spoken answer · ~75 s**

Load average is the average number of tasks that are runnable or waiting in uninterruptible sleep, usually disk I/O, over one, five, and fifteen minutes. On Linux that means it includes processes waiting for CPU and also processes blocked on I/O, so a high load doesn't always mean a CPU problem. The number only makes sense relative to the number of cores: a load of four on a four-core machine is fully busy, while a load of four on a sixteen-core machine is light. As a rule of thumb, sustained load above the core count suggests saturation, and I look at the trend across the three numbers. If the one-minute value is higher than the fifteen-minute, load is rising; if lower, it's recovering. When load is high, I check CPU stats: high user or system time points to CPU-bound work, while high iowait with modest CPU use points to disk or network storage problems. The tools are top, vmstat for the run queue and blocked processes, and iostat. Load average on its own is a symptom, not a diagnosis.

Likely follow-ups

- Load is high but CPU is idle. What is going on?
- How would you find which process is blocked on I/O?

certs.eknathalabs.com

```
uptime                    # load average: 1min 5min 15min
```

```
cat /proc/loadavg         # same, plus running/total tasks and last PID
```

```
nproc                     # number of CPU cores
```

```
sar -u 1 5                # CPU utilization over 5 samples
```

```
vmstat 1 5                # r column = runnable processes (key metric)
```

Load average = average number of tasks in runnable OR uninterruptible sleep (D state, waiting for I/O) state. It is NOT CPU percentage. A load of 1.0 on a 4-core system means 25% utilization — fine. A load of 8.0 on a 4-core system means the queue is 2× overloaded. Rule of thumb: load / nproc > 1.0 for sustained periods = investigate. High load with low CPU% = I/O bound (disk or network). High load with high CPU% = CPU bound. Check vmstat's 'r' (runnable) and 'b' (blocked on I/O) columns to distinguish.

D-state processes: Processes in D (uninterruptible sleep) are waiting for I/O and cannot be killed. They inflate load average significantly. `ps aux | awk '$8=="D"'` lists D-state processes. High D-state count with high load and low CPU = storage I/O bottleneck.

📚 **Official docs**

- Linux kernel docs: CFS scheduler
- Brendan Gregg: Linux performance

certs.eknathalabs.com

Q5 What are Linux signals — which ones matter most for DevOps/Platform engineers?

Processes

🎙 Spoken answer · ~85 s

Signals are the kernel's way to notify a process of an event. The ones that matter in operations: SIGTERM, fifteen, is the polite request to shut down, which can be caught, so the application can drain connections and clean up; it's what Kubernetes and systemd send first. SIGKILL, nine, can't be caught or ignored and ends the process immediately, so it's the last resort since there is no cleanup. SIGHUP, one, traditionally means the terminal hung up, but many daemons such as nginx and sshd reload their configuration on it, so it enables reloads without downtime. SIGINT, two, is Ctrl-C from the terminal. SIGSTOP pauses a process and can't be caught, while SIGCONT resumes it; Ctrl-Z sends SIGTSTP, which can be caught. SIGCHLD tells a parent that a child has terminated, which it should handle by reaping. SIGUSR1 and SIGUSR2 are application-defined, for example log rotation or dumping state. SIGPIPE appears when writing to a closed pipe. In containers the important point is that process ID one doesn't get default signal handlers, so an application running as PID one must handle SIGTERM explicitly, or the container will wait for the grace period and then be killed. I list signals with kill minus l.

Likely follow-ups

- Why does a container's PID 1 need to handle SIGTERM explicitly?
- What is SIGHUP typically used for in daemons?

```
kill -l               # list all signals
```

```
SIGHUP  (1)  — reload config (nginx, sshd respond to this)
```

```
SIGINT  (2)  — Ctrl+C (interrupt from terminal)
```

```
SIGTERM (15) — graceful shutdown request (catchable)
```

```
SIGKILL (9)  — force kill (cannot be caught or ignored)
```

```
SIGSTOP (19) — pause process (cannot be caught, Ctrl+Z sends SIGTSTP)
```

```
SIGCHLD (17) — child process changed state (parent notified)
```

```
SIGUSR1/2   — user-defined (apps use for log rotation, graceful reload)
```

SIGHUP is commonly used to reload configuration without restart — nginx, sshd, and many daemons reload their config on SIGHUP. SIGUSR1 is used by nginx for log rotation (reopens log files after they've been moved). In containers: Kubernetes sends SIGTERM when terminating a pod, then SIGKILL after terminationGracePeriodSeconds. Make sure your application handles SIGTERM to drain connections gracefully — otherwise you get 502 errors during deployments.

certs.eknathalabs.com

Kubernetes graceful shutdown: When a pod is deleted, Kubernetes sends SIGTERM to PID 1 inside the container. If your app is not PID 1 (it's a child of a shell script), it may never receive SIGTERM. Use exec form in Dockerfile (ENTRYPOINT ["app"]) not shell form (ENTRYPOINT app) so your app is PID 1.

📚 **Official docs**

- Kubernetes: Pod lifecycle & termination
- Docker: Dockerfile reference

Q6 What is /proc filesystem — what useful information can you extract from it?

Processes

🎙 **Spoken answer · ~83 s**

The slash proc filesystem is a virtual filesystem, not stored on disk, in which the kernel exposes live information about the system and each process as files. System-wide: cpuinfo for CPU details, meminfo for the memory breakdown, loadavg, uptime, mounts, and slash proc slash sys which holds tunable kernel parameters, the same ones sysctl modifies. Per process, under slash proc slash pid: cmdline gives the full command line, environ the environment variables, status gives memory use, thread count, and signal masks, fd lists the open file descriptors, which is how lsof works, maps shows memory mappings, limits shows resource limits, cwd and exe link to the working directory and the binary, and net shows its network details. In practice I use it for debugging: seeing which files a stuck process has open, finding deleted files still held open, checking whether a process hit its file descriptor limit, reading the environment of a running service, or seeing why a process is in a certain state. Because it's the source for tools like ps, top, and free, it's useful when those tools aren't installed, such as in minimal containers. Some entries, like environ, need appropriate permissions because they can contain secrets.

Likely follow-ups

- How can you use /proc to find deleted files that are still open?
- Which /proc entry shows a process's resource limits?

```
cat /proc/cpuinfo         # CPU model, cores, flags
```

```
cat /proc/meminfo         # memory breakdown: MemTotal, MemFree, Cached, Buffers
```

```
cat /proc/<pid>/cmdline  # full command line of a process (null-separated)
```

```
cat /proc/<pid>/environ  # environment variables of a running process
```

```
cat /proc/<pid>/fd/ | ls -la  # open file descriptors
```

```
cat /proc/<pid>/net/tcp  # TCP connections for a process's namespace
```

```
cat /proc/<pid>/limits   # ulimits for the process
```

certs.eknathalabs.com

```
cat /proc/sys/vm/swappiness  # kernel tunable
```

```
cat /proc/net/dev         # network interface statistics
```

/proc is a virtual filesystem exposing kernel data structures as files. Everything is readable as text. Key use cases: inspect a running process's open files and connections without any external tool, read kernel tunables (all /proc/sys/ entries are tunable via sysctl), and debug networking issues per-namespace. In containers, /proc/1/environ reveals environment variables of PID 1 — useful for debugging misconfigured containers.

Security note: /proc/<pid>/environ may expose secrets passed as env vars to processes. Ensure container runtimes and OS permissions restrict cross-process /proc access. This is why Kubernetes recommends using mounted Secrets files instead of env vars for sensitive values.

📚 **Official docs**

- Linux man-pages: proc(5)

Q7 Filesystem is full but du shows available space — how do you find and recover the space?

Filesystem

🎙 **Spoken answer · ~77 s**

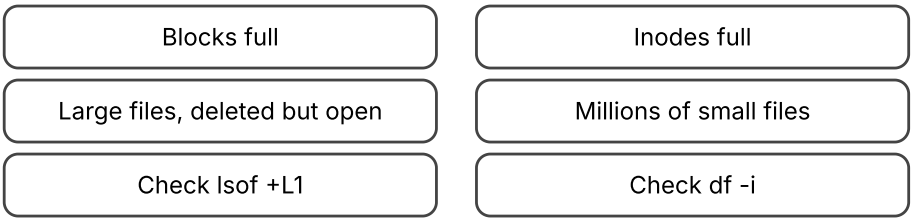
When df says the filesystem is full but du can't find the space, there are a few classic explanations. The most common is a deleted file still held open by a running process: the file is unlinked, so du doesn't see it, but the space isn't released until the process closes it. I find those with lsof, filtering for deleted files, or by looking at the process's file descriptors under /proc, and then I restart the process or truncate the file through its descriptor to free the space. Second is inode exhaustion: df with the inodes flag shows a hundred percent used, usually from millions of tiny files, which blocks new files even though blocks are free. Third is data hidden under a mount point, written to a directory before another filesystem was mounted over it; I can bind-mount the root filesystem to look underneath. Fourth, reserved blocks on ext4, or snapshots and container layers holding space. To prevent this, I use log rotation that signals processes to reopen files, alerts on both disk and inode usage, and separate partitions for volatile data like logs.

Likely follow-ups

- How do you free space from a deleted-but-open log file without restarting?
- What causes inode exhaustion in practice?

certs.eknathalabs.com

🗺 df vs du



This classic problem occurs when a file is deleted but a process still has it open — the inode is not freed until the file handle is closed.

```
df -h                          # shows filesystem usage (may show 100% full)
```

```
du -sh /* 2>/dev/null | sort -rh | head -20  # find largest directories
```

```
du -sh /var/log/* | sort -rh | head -10      # common culprit
```

```
# Find deleted-but-open files (holding space):
```

```
lsof +L1                       # list files with link count 0 (deleted but open)
```

```
lsof | grep deleted            # same, different filter
```

```
# Recover without restart: truncate the file via its fd:
```

```
truncate -s 0 /proc/<pid>/fd/<fd-number>
```

The truncate trick is invaluable in production — it empties the file in-place without closing it, immediately freeing disk space without restarting the process holding it open. Common culprits: log files being written to after logrotate moved them, application keeping a large temp file open, or a core dump file being written by a crashing process.

Inode exhaustion: df -h shows space available but writes fail with "No space left on device". Check inodes: `df -i`. Inode exhaustion happens when millions of tiny files are created (email queues, session files, package manager caches). Fix: delete many small files or reformat with larger inode ratio.

📚 **Official docs**

- Linux man-pages: inode(7)

certs.eknathalabs.com

Q8 What are hard links vs soft links — when does the distinction matter?

Filesystem

🎙 **Spoken answer · ~86 s**

A hard link is another directory entry pointing to the same inode, so both names are equal and share the data; the link count in the inode increases, and the data is freed only when the count reaches zero and no process has it open. A symbolic link is a separate small file, with its own inode, containing a path to the target. The distinction matters in several ways. Hard links can't cross filesystems, since inode numbers are per filesystem, and normally can't point to directories; symlinks can do both. If the target is deleted, a symlink dangles and breaks, whereas a hard link keeps the data alive. If you replace a file through an editor that writes a new file, a hard link will keep pointing to the old inode, while a symlink follows the path to the new file. Symlinks are visible with ls minus l, while hard links are seen by identical inode numbers with ls minus i. Practical uses: symlinks for versioned deployments, for example current pointing to a release directory, enabling atomic switches, and for config files; hard links for backup tools such as rsync link-dest, which save space for unchanged files. Also watch for symlink attacks in world-writable directories like tmp.

Likely follow-ups

- Why can't hard links cross filesystems?
- How do symlinks enable atomic deployments?

```
ln /etc/hosts /tmp/hosts_hard    # hard link (same inode)
```

```
ln -s /etc/hosts /tmp/hosts_soft # soft link (symlink, new inode)
```

```
ls -li /etc/hosts /tmp/hosts_hard  # same inode number = hard link
```

```
ls -la /tmp/hosts_soft          # shows -> /etc/hosts
```

```
stat /etc/hosts                 # see link count (nlink)
```

Hard link: another directory entry pointing to the same inode. The file is not deleted until ALL hard links are removed AND no process has it open. Hard links cannot span filesystems. Soft link: a special file containing a path to the target. If the target is deleted, the symlink dangles. Symlinks can span filesystems and can point to directories. Practical distinction: logrotate uses hard links when rotating logs to ensure no log entries are lost. Container image layers use hard links to share files between layers without duplication.

Finding dangling symlinks: `find /etc -type l ! -exec test -e {} ; -print` — finds symlinks whose targets don't exist. Dangling symlinks from removed packages are common and harmless but can confuse diagnostics.

📚 **Official docs**

- Linux man-pages: inode(7)

certs.eknathalabs.com

Q9 How do Linux file permissions work — explain chmod, chown, SUID, SGID, and sticky bit?

Filesystem

🎙 Spoken answer · ~84 s

Linux permissions give three sets, owner, group, and others, each with read, write, and execute. Octal notation encodes them: read is four, write is two, execute is one, so seven five five is rwx for the owner and r-x for the rest. chmod changes the mode, in octal or symbolic form such as u plus x, and chown changes owner and group. For directories, read lets you list names, write lets you create or delete entries, and execute lets you traverse into it. Three special bits: SUID on an executable makes it run with the file owner's privileges, which is how passwd can modify the shadow file, and is also a classic privilege-escalation risk, so I audit for SUID binaries. SGID on an executable runs with the group; on a directory, new files inherit the directory's group, which is great for shared project folders. The sticky bit on a directory, as on tmp, means only the file's owner, the directory owner, or root can delete or rename files in it, even though others can write. Beyond that: umask sets default permissions, ACLs give finer control, and capabilities split root's powers. Best practice is least privilege, and never chmod 777 as a fix.

Likely follow-ups

- Why are SUID binaries a security concern?
- What does the sticky bit do on /tmp?

```
ls -la /etc/passwd           # -rw-r--r-- root root
```

```
chmod 755 script.sh          # rwxr-xr-x (octal)
```

```
chmod u+x,g-w,o-rwx file     # symbolic notation
```

```
chown user:group file        # change owner and group
```

```
chmod u+s /usr/bin/sudo      # set SUID bit (runs as file owner)
```

```
chmod g+s /shared/dir        # set SGID on dir (new files inherit group)
```

```
chmod +t /tmp                # sticky bit (only owner can delete their files)
```

```
find / -perm -4000 2>/dev/null  # find all SUID files
```

Permission bits: 3 sets of rwx for owner/group/other. Octal: 4=read, 2=write, 1=execute. SUID on executables: runs with file owner's privileges (passwd, sudo use this). SGID on directories: new files inherit the directory's group — used for shared project directories. Sticky bit on directories: users can only delete their own files (used on /tmp to prevent users deleting each other's temp files).

SUID security risk: SUID binaries run with elevated privileges — a vulnerability in a SUID binary is a privilege escalation vector. Audit with `find / -perm /4000` and investigate unexpected SUID files. This

certs.eknathalabs.com

is a standard security hardening step and CKS exam topic.

📚 **Official docs**

- Linux man-pages: path_resolution(7)

Q10 What is the Linux Virtual Filesystem (VFS) — why does everything in Linux appear as a file?

Filesystem

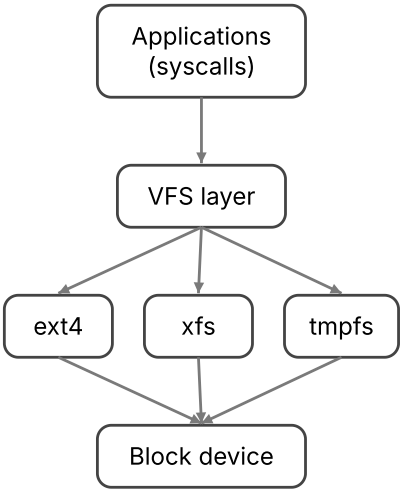
🎙 **Spoken answer · ~86 s**

The Virtual Filesystem, VFS, is a kernel abstraction layer that gives every filesystem and many other resources the same interface of open, read, write, and close. That's why everything appears as a file, which follows the Unix philosophy: a uniform interface means the same tools work on many things. Regular files on ext4 or XFS, directories, devices under slash dev, and pseudo-filesystems like slash proc and slash sys, which expose kernel and process information, all go through VFS. Programs don't need to know what is behind a path. Network sockets are file descriptors too, and bash can even open a TCP connection through its slash dev slash tcp feature. This design means you can read a device with cat, redirect output to a file or a pipe, and control kernel parameters by writing to a file under slash proc slash sys. For operations, it's useful because standard tools, grep, cat, and echo, can inspect and configure the system, and the mount command shows all mounted filesystems, including virtual ones. Under the hood each filesystem registers its operations with VFS, which handles caching of inodes, directory entries, and pages. Understanding it helps explain behaviours such as file descriptors, redirection, and why a deleted file stays alive while open.

Likely follow-ups

- How does VFS let tools work across different filesystems?
- Give an example of configuring the kernel by writing to a file.

🗺 **VFS**



certs.eknathalabs.com

```
ls /proc           # process info as files
```

```
ls /sys/class/net  # network interfaces as files
```

```
ls /dev            # hardware devices as files
```

```
cat /dev/urandom | head -c 10 | od -An -tu1  # read from random device
```

```
echo "data" > /dev/tcp/example.com/80  # write to network socket as file (bash)
```

```
mount | head -10   # see all mounted filesystems including virtual ones
```

The VFS (Virtual File System) is an abstraction layer in the kernel that provides a uniform interface (open, read, write, close) for all I/O — whether talking to disk, a device, a network socket, or kernel data structures. "Everything is a file" means you can use standard Unix tools on all of these. tmpfs is a RAM-backed filesystem (used by /run, /tmp). cgroups and namespaces are exposed via /sys/fs/cgroup and /proc/<pid>/ns — this is how container runtimes manage container isolation.

Container relevance: Kubernetes uses /proc and /sys to inspect container processes. The kubelet reads cgroup memory limits from /sys/fs/cgroup. Understanding VFS helps debug "why does my container think it has 128GB RAM" (it's reading from /proc/meminfo which shows host memory, not container limits).

📚 **Official docs**

- Linux man-pages: proc(5)
- Linux man-pages: cgroups(7)

certs.eknathalabs.com

Q11 How do you find large files, find files modified recently, and find files by type across a filesystem?

Filesystem

🎙 Spoken answer · ~85 s

The find command is the tool for this. For large files, find on the filesystem with type f and size plus one hundred megabytes, redirecting errors to dev null; I add minus xdev to stay within one filesystem, and often pipe to ls minus lh or use du, sorted, to see sizes. For recently modified files, minus mmin minus sixty finds changes in the last hour, minus mtime for days, and minus newer compares against a reference file, which is great for after an incident or a deploy. By type, minus type f for files, d for directories, l for symlinks, and minus name with a pattern, or minus iname to ignore case. I can combine conditions, and act on results with minus exec or minus delete, for example deleting log files older than thirty days, though I run it without the delete first to check what matches. For large directories, du minus ah sorted by size and head is quicker for finding what fills a disk, and ncdu gives an interactive view. For performance with many files, using exec with a plus sign batches arguments, and xargs with null separation handles odd filenames safely. Watch out for quoting names and the proc filesystem.

Likely follow-ups

- Why run find without -delete first?
- How would you find what is filling a disk quickly?

```
# Large files:
```

```
find / -type f -size +100M 2>/dev/null | sort -k5 -rn
```

```
find /var -type f -size +500M -exec ls -lh {} ;
```

```
# Recently modified (last 60 minutes):
```

```
find /etc -type f -mmin -60 2>/dev/null
```

```
find / -newer /tmp/reference_file 2>/dev/null  # modified after reference file
```

```
# By type:
```

```
find / -type f -name "*.log" -mtime +30 -delete  # delete logs older than 30 days
```

```
find /proc -type l -name "exe" -exec ls -la {} ; 2>/dev/null  # running binaries
```

```
# By permission:
```

```
find /etc -perm /o+w 2>/dev/null  # world-writable files (security audit)
```

certs.eknathalabs.com

find is the Swiss Army knife of filesystem queries. Key options: -type (f=file, d=dir, l=symlink), -size (+/-N[ckMG]), -mtime/-mmin (modified N days/minutes ago), -perm, -user, -group. Combine with -exec to act on results. Use 2>/dev/null to suppress permission errors. For disk cleanup automation, combine find with -delete or pipe to xargs rm.

locate vs find: `locate filename` searches a pre-built database (fast but stale). `find` walks the filesystem in real time (slower but current). Use locate for quick lookups, find for scripts and when recency matters. Update locate db with `updatedb`.

📚 **Official docs**

- Git LFS

Q12 **What is LVM (Logical Volume Manager) — how do you extend a filesystem online?** Filesystem

🎙 **Spoken answer · ~80 s**

LVM, the Logical Volume Manager, adds a layer between disks and filesystems: physical volumes are disks or partitions, they're pooled into volume groups, and logical volumes are carved from the pool and hold filesystems. The benefit is flexibility: you can resize volumes, span multiple disks, and take snapshots without repartitioning. To extend a filesystem online: first check that the volume group has free space with vgdisplay, or add a new disk with pvcreate and vgextend. Then extend the logical volume with lvextend, for example plus ten gigabytes, and resize the filesystem: resize2fs for ext4, which works online, or xfs_growfs for XFS, which takes the mount point. The lvextend command has a flag, minus r, that resizes the filesystem in the same step. Note that XFS can only grow, not shrink, while ext4 can shrink offline, and shrinking is risky, so take a backup first. Snapshots are useful for consistent backups. In the cloud, the typical flow is to expand the virtual disk, rescan, grow the partition, then the physical volume with pvresize, then the logical volume and filesystem. I verify with df afterwards. LVM adds some complexity, so know the layout with lsblk.

Likely follow-ups

- What is the difference between growing ext4 and XFS?
- How do you add a new disk to an existing volume group?

```
pvdisplay                       # physical volumes
```

```
vgdisplay                       # volume groups
```

```
lvdisplay                       # logical volumes
```

```
# Extend an LV and resize filesystem online:
```

```
lvextend -L +10G /dev/vg0/lv_data          # extend LV by 10GB
```

certs.eknathalabs.com

```
resize2fs /dev/vg0/lv_data                  # resize ext4 filesystem (online)
```

```
xfs_growfs /mount/point                     # resize XFS (must use mount point)
```

```
# Add a new disk to an existing VG:
```

```
pvcreate /dev/sdb                            # initialize new disk as PV
```

```
vgextend vg0 /dev/sdb                       # add PV to VG
```

```
lvextend -l +100%FREE /dev/vg0/lv_data      # use all new free space
```

LVM adds a virtualization layer between physical disks and filesystems: Physical Volumes (disks) → Volume Groups (pools) → Logical Volumes (virtual disks). Benefits: online resize, snapshots (for consistent backups), striping across disks. Ext4 supports online grow (resize2fs on mounted filesystem). XFS supports online grow but NOT shrink. Neither supports online shrink — unmount first.

Cloud disk resize workflow: In GCP/AWS, resize the persistent disk in the console/CLI → resize the partition (if needed, using growpart or parted) → resize the LV (if LVM) → resize the filesystem. All steps can be done on a running system for ext4 and XFS.

📚 **Official docs**

- Linux kernel docs: LVM (device-mapper)

certs.eknathalabs.com

Q13 How do you diagnose network connectivity issues on Linux — what is your systematic approach?

Networking

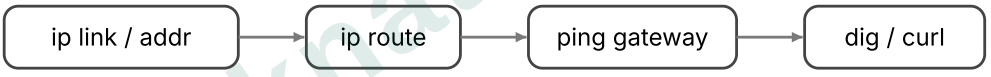
🎙 Spoken answer · ~81 s

For network problems on Linux, I go layer by layer from the bottom. Layer two: is the interface up and the carrier detected, using ip link. Layer three: does it have the right IP address, ip addr, and a default route, ip route. Then connectivity: ping the gateway to test the local segment, then an external IP like 8.8.8.8 to test routing beyond, which separates a local problem from an upstream one. If the IP works but names don't, it's DNS: check resolv.conf and test with dig or getent hosts. Then reach the specific service: use nc or curl to test the port, which tells me whether it's refused, meaning reachable but not listening, or timing out, meaning something drops the packets. traceroute, or the TCP variant, shows where the path breaks, and mtr gives continuous per-hop statistics. Then check local firewall rules with iptables or nft, and the other side's listening sockets with ss. If still unclear, capture with tcpdump to see whether packets leave, arrive, and get answered. I also check for MTU problems, ARP issues, and conntrack. This ordered approach avoids guessing and tells me which team or layer owns the problem.

Likely follow-ups

- How do you distinguish a firewall drop from a closed port?
- What would you check if IP connectivity works but hostnames fail?

🗺 Network debug




```
# Layer by layer approach:

ip link show                    # L2: interfaces up? carrier detected?

ip addr show                    # L3: IP addresses assigned?

ip route show                   # L3: default route exists?

ping -c 4 8.8.8.8               # L3: can reach internet?

ping -c 4 <gateway-ip>          # L3: can reach gateway?

traceroute -T -p 443 <host>     # L3: where does routing break?

nc -zv <host> <port>            # L4: is the port open?

curl -v https://<host>          # L7: does the app respond?

ss -tlnp                        # what is listening on which port?
```

certs.eknathalabs.com

```
iptables -L -n -v                 # any firewall rules blocking?
```

Work bottom-up: physical link → IP config → routing → port reachability → application. Each layer narrows the problem. If ping works but nc fails: the network path is fine but the service isn't listening or is firewalled. If ping fails but the gateway pings: routing issue. If the gateway doesn't ping: either IP misconfigured or L2/L1 issue.

ss vs netstat: `ss` is the modern replacement for `netstat` — faster (reads from kernel directly), more features. Learn ss: `ss -tlnp` (TCP listen), `ss -tnp state established` (active connections), `ss -s` (summary stats).

📚 **Official docs**

- RFC 792: ICMP

Q14 How do iptables work — how do you debug and write firewall rules?

Networking

🎙 **Spoken answer · ~83 s**

iptables is the classic Linux packet filter built on netfilter hooks. Rules are organised into tables, filter for allow and drop decisions, nat for address translation, and mangle for packet alteration, and each table has chains such as INPUT, OUTPUT, FORWARD, PREROUTING, and POSTROUTING. Rules in a chain are evaluated in order, and the first match decides; the chain's default policy applies if nothing matches. To debug: list rules with minus L minus n minus v and line numbers, where the packet and byte counters show which rules are actually matching; add a LOG rule temporarily to see dropped packets; and use the order carefully, since an early broad DROP can shadow later ACCEPTs. Writing rules: allow established and related connections first, accept SSH and the needed service ports, and then drop everything else by default. Insert with minus I to place a rule at a specific position, append with minus A. Rules don't persist across reboot unless saved with iptables-save and restored, or managed through a tool. Modern systems use nftables, and Kubernetes and Docker add their own chains, so be careful not to flush them. Test remote changes with a safety timer, to avoid locking myself out.

Likely follow-ups

- Why does rule order matter in iptables?
- How do you avoid locking yourself out when changing firewall rules remotely?

```
iptables -L -n -v --line-numbers    # list all rules with packet counts
```

```
iptables -t nat -L -n -v            # NAT table rules (DNAT, SNAT)
```

```
iptables -t mangle -L -n            # mangle table (QoS, TTL)
```

```
# Allow incoming SSH:
```

certs.eknathalabs.com

```
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

```
# Block specific IP:
```

```
iptables -I INPUT 1 -s 1.2.3.4 -j DROP
```

```
# Log dropped packets (for debugging):
```

```
iptables -A INPUT -j LOG --log-prefix "DROPPED: " --log-level 4
```

```
iptables -A INPUT -j DROP
```

```
# Save rules (Debian/Ubuntu):
```

```
iptables-save > /etc/iptables/rules.v4
```

iptables processes packets through chains (INPUT, OUTPUT, FORWARD) in tables (filter, nat, mangle). Rules are processed in order — first match wins. Default chains have a policy (ACCEPT or DROP). Kubernetes uses iptables extensively: kube-proxy writes DNAT rules for every Service (ClusterIP → pod IPs), and NetworkPolicies generate iptables rules per pod. Understanding iptables is essential for debugging why a Kubernetes service isn't reachable.

iptables vs nftables: Modern Linux (RHEL 8+, Debian 11+) uses nftables as the backend. iptables commands still work via iptables-nft compatibility layer. Kubernetes is migrating kube-proxy to nftables mode. Know both exist; iptables syntax still dominates CKA/CKS exams.

📚 **Official docs**

- Netfilter documentation

certs.eknathalabs.com

Q15 How do you capture and analyze network traffic on Linux?

Networking

🎙 Spoken answer · ~85 s

To capture and analyse traffic on Linux I use tcpdump on the command line. I choose the interface, or any, and filter with BPF expressions to keep capture focused: by host, port, protocol, or TCP flags, for example port eighty, host a certain IP, or packets with the RST flag set. I use minus n to avoid DNS lookups, which would slow down and pollute the capture. For deeper analysis I write the capture to a pcap file and open it in Wireshark, or use tshark on the server. Typical uses: confirm whether packets arrive at a host, check the TCP handshake, look at retransmissions and resets, inspect DNS queries and responses, and capture on both client and server to find where packets are lost. For encrypted traffic, I can still see handshake timing, and sizes. Practical tips: limit capture size and duration with minus c or rotation, because captures grow quickly; use a snap length for headers only if payloads aren't needed; run as root; be aware that capturing may expose sensitive data, so handle files carefully and follow policy; and on busy hosts the capture itself has overhead. In Kubernetes I might use an ephemeral debug container sharing the pod's network namespace.

Likely follow-ups

- Why use -n with tcpdump?
- How do you capture traffic from inside a Kubernetes pod?

```
tcpdump -i eth0 -n port 80              # capture HTTP on eth0
```

```
tcpdump -i any host 10.0.0.5            # all traffic to/from IP
```

```
tcpdump -i eth0 -w /tmp/cap.pcap        # write to file for Wireshark
```

```
tcpdump -i eth0 'tcp[tcpflags] & tcp-rst != 0'  # capture RST packets
```

```
tcpdump -i any port 53 -n               # DNS queries
```

```
# tshark (Wireshark CLI):
```

```
tshark -i eth0 -f "port 443" -T fields -e http.request.uri  # extract URIs
```

```
# ss with detailed TCP info:
```

```
ss -tin dst 10.0.0.5                    # TCP internals: rtt, cwnd, retransmits
```

tcpdump is the primary packet capture tool. Filters use Berkeley Packet Filter (BPF) syntax: host, port, proto, src/dst, and/or/not. Key patterns: capture DNS (port 53) to debug resolver issues, capture RST packets to find rejected connections, capture SYN packets to see connection attempts. Always write to file (-w) for complex analysis in Wireshark. In Kubernetes, use kubectl-sniff or a debug container in the pod's network namespace to capture from a running pod.

certs.eknathalabs.com

Bandwidth measurement: `iperf3 -s` on server, `iperf3 -c <server>` on client. Use `nload` or `iftop` for real-time bandwidth monitoring per interface. `nethogs` shows bandwidth per process — invaluable for finding which process is saturating the network.

📚 **Official docs**

- pcap-filter(7)
- tcpdump manual

Q16 How do you configure static IP, routes, and DNS on a Linux server?

Networking

🎙 **Spoken answer · ~80 s**

There are two layers: runtime changes and persistent configuration. For runtime, ip addr add assigns an address, ip route add sets routes, including the default gateway, but these vanish on reboot, so they're only for testing or emergency. For persistence it depends on the distribution. On modern Ubuntu, netplan uses a YAML file under etc slash netplan, defining addresses, routes, and nameservers, applied with netplan apply, and best tested first with netplan try, which rolls back if I lose connectivity. On RHEL-family systems, NetworkManager manages connections, via nmcli or nmtui, with older ifcfg files as the legacy approach. Systems using systemd-networkd have network unit files. DNS settings come from the network configuration, which updates resolv.conf, often managed by systemd-resolved, so editing resolv.conf directly may be overwritten. I verify with ip addr, ip route, and resolvectl status, and test connectivity and name resolution. Careful points: a mistake in remote network config can lock me out, so use netplan try or a console, keep a rollback, and check there's no conflicting config from DHCP or cloud-init. In cloud environments, cloud-init or the provider's tooling often controls networking, so manual changes may be reset.

Likely follow-ups

- Why use netplan try instead of netplan apply on a remote host?
- Why might your resolv.conf edit get overwritten?

certs.eknathalabs.com

```
# Modern (systemd-networkd / netplan on Ubuntu 18+):
```

```
cat /etc/netplan/01-netcfg.yaml
```

```
netplan apply               # apply config changes
```

```
# Legacy (ifcfg files on RHEL/CentOS):
```

```
cat /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
# Runtime changes (not persistent):
```

```
ip addr add 10.0.0.100/24 dev eth0
```

```
ip route add default via 10.0.0.1
```

```
ip route add 192.168.1.0/24 via 10.0.0.254
```

```
# DNS:
```

```
cat /etc/resolv.conf         # current nameservers
```

```
resolvectl status            # systemd-resolved status
```

```
resolvectl dns eth0 8.8.8.8  # set DNS for interface
```

Network configuration has fragmented across tools: legacy ifupdown (/etc/network/interfaces), RHEL's NetworkManager (/etc/sysconfig), and modern systemd-networkd with Netplan (Ubuntu). Cloud instances typically use DHCP with cloud-init setting initial config. Changes to /etc/resolv.conf may be overwritten by systemd-resolved or NetworkManager — edit the source config, not resolv.conf directly.

Cloud metadata routes: Cloud VMs have a link-local route to 169.254.169.254 (metadata server). Never remove this route — it's used by the guest agent, credential fetching, and Workload Identity. Check with `ip route show | grep 169.254`.

📚 **Official docs**

- Linux man-pages: ip(7)

certs.eknathalabs.com

Q17 What is /etc/hosts and /etc/nsswitch.conf — how does DNS resolution order work?

Networking

🎙 Spoken answer · ~83 s

Name resolution order is controlled by nsswitch.conf, specifically the hosts line, which typically reads files then dns. That means the system first checks etc slash hosts, the static mapping file, and only then queries DNS servers listed in resolv.conf. Applications that use the standard library, through getaddrinfo, follow this order; some tools such as dig query DNS directly and bypass it, which is why dig and an application can disagree. To test what applications really see, I use getent hosts, which respects nsswitch. If an application can't resolve a name, check for a stale or wrong entry in etc hosts overriding DNS, a missing dns entry in nsswitch, or search domains and ndots in resolv.conf expanding the name unexpectedly. With systemd-resolved, there's a local stub resolver and caching, and mdns may appear in the order too. To see exactly what happens I can use strace on getent, tracing network calls. Practical uses: etc hosts for quick overrides in testing, like pointing a hostname at a new server before DNS changes, or blocking a domain; but overusing it creates hidden configuration drift, so I keep it minimal and documented. Also cache layers like nscd, sssd, or resolved can serve stale answers.

Likely follow-ups

- Why can dig succeed while an application fails to resolve?
- What does getent hosts test that dig doesn't?

```
cat /etc/nsswitch.conf | grep hosts  # resolution order
```

```
# Typical: hosts: files dns  (check /etc/hosts first, then DNS)
```

```
cat /etc/hosts               # static hostname → IP mappings
```

```
getent hosts example.com    # test resolution using NSS (respects nsswitch)
```

```
strace -e trace=network getent hosts example.com  # trace exactly what happens
```

```
# Override DNS for testing (add to /etc/hosts):
```

```
echo "1.2.3.4 staging.example.com" >> /etc/hosts
```

nsswitch.conf controls resolution order for hosts, users, groups, and more. "files dns" means: check /etc/hosts first, then query nameservers in /etc/resolv.conf. "dns files" reverses this. In containers: /etc/hosts is writable by the container runtime (Kubernetes adds pod IPs to /etc/hosts). The ndots option in /etc/resolv.conf controls how many dots a hostname needs before it's treated as fully qualified — critical for Kubernetes DNS performance.

getent vs nslookup: `nslookup` and `dig` bypass nsswitch and query DNS directly. `getent hosts` uses the full NSS stack including /etc/hosts. Use getent when you want to test exactly what the application sees.

certs.eknathalabs.com

Use dig when you want to test DNS specifically.

📚 **Official docs**

- Linux man-pages: nsswitch.conf(5)
- RFC 1035: DNS implementation and specification
- Cloudflare Learning: What is DNS?

Q18 How do you diagnose high CPU usage on a Linux server?

Performance

🎙 **Spoken answer · ~83 s**

To diagnose high CPU, I first look at where the CPU time goes. In top I check the per-CPU breakdown: user, system, iowait, softirq, and steal. High user time points to application code; high system time points to kernel work such as syscalls or networking; iowait suggests the CPU is idle waiting for disk; softirq can indicate network interrupt handling concentrated on one core; and steal means a noisy neighbour on a virtual machine. mpstat per CPU shows whether the load is spread or pinned to one core, and sar gives history to see when it began. Then identify the process with top or ps sorted by CPU, and drill in: for a user-space process, use perf top or perf record with flame graphs to find hot functions, strace to see syscalls, and application profilers. For per-thread detail use top with H. Common causes: a runaway loop, a regex problem, garbage collection thrash, too many threads or context switches, a traffic increase, or something unexpected such as a cron job or malware. Check load average in context of core count. Mitigation: restart or scale out, limit with cgroups, fix the code, and alert on sustained saturation rather than brief spikes.

Likely follow-ups

- What does high steal time mean?
- How would you find which function is burning CPU?

```
top                          # real-time CPU per process
```

```
top -1                       # show per-CPU breakdown (press 1 in top)
```

```
mpstat -P ALL 1 5            # per-CPU utilization: usr, sys, iowait, si (softirq)
```

```
sar -u 1 5                   # CPU utilization history
```

```
ps -eo pid,ppid,cmd,%cpu,%mem --sort=-%cpu | head -15
```

```
# Find which CPU is hot (for IRQ affinity issues):
```

```
cat /proc/interrupts | sort -k2 -rn | head -10
```

```
# Profile what the CPU is actually doing:
```

certs.eknathalabs.com

```
perf top                    # live kernel/userspace profiling (needs perf pkg)
```

```
perf record -g -p <pid> sleep 10 && perf report  # flame graph data
```

CPU breakdown: us (user), sy (system/kernel), ni (niced), id (idle), wa (I/O wait), hi (hardware IRQ), si (software IRQ), st (stolen by hypervisor). High sy = kernel-intensive work (syscall-heavy, iptables rules, context switching). High si = network interrupt processing (high-traffic network interface). High wa = I/O bound. High us = application CPU. Use perf for deep profiling — it shows exactly which function the CPU spends time in.

Stolen CPU (st): In VMs, high %st means the hypervisor is not scheduling your VM's vCPUs promptly — the physical host is overcommitted. No application-level fix; escalate to the cloud provider or move to a dedicated host. On GCP/AWS, st > 5% consistently signals a noisy-neighbor problem.

📚 **Official docs**

- Brendan Gregg: Linux performance

Q19 **How do you diagnose memory issues — OOM killer, swap usage, and memory leaks?** Performance

🎙 **Spoken answer · ~79 s**

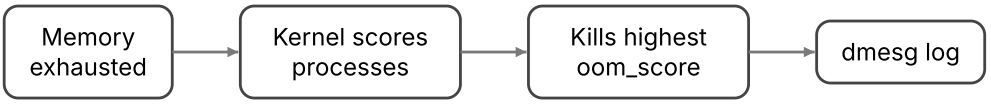
For memory issues I start with free minus h, reading the available column rather than free, since cache is reclaimable. Then proc meminfo and vmstat for details, and swap activity with vmstat columns si and so, since heavy active swapping is what hurts. For OOM events, I search dmesg or the kernel journal for out of memory and killed process, which shows the victim and the memory state at the time. Per-process, I check RSS and virtual size from the status file, ps sorted by memory, or smem for proportional shares. To distinguish a leak from a legitimate need, I watch memory over time: steady growth that never plateaus, even at constant load, indicates a leak, and I use heap profilers, valgrind, or language-specific tools to find it. Consider other consumers: page cache, tmpfs, kernel slab memory, shared memory, and for containers, the cgroup limit rather than host memory. Mitigations: fix the leak, right-size limits and caches, adjust swappiness, add RAM, protect critical processes with oom_score_adj, and make services restart cleanly. Monitor available memory, swap activity, and memory pressure information from PSI to get warning before the OOM killer acts.

Likely follow-ups

- Why is 'available' more useful than 'free'?
- How do you tell a memory leak from a growing cache?

certs.eknathalabs.com

🗺 OOM killer




```
free -h                          # overview: total, used, free, buff/cache, available
```

```
cat /proc/meminfo                # detailed breakdown
```

```
vmstat -s                        # memory stats summary
```

```
# OOM killer events:
```

```
dmesg | grep -i "oom|killed process|out of memory"
```

```
journalctl -k | grep -i oom   # systemd journal kernel messages
```

```
# Per-process memory:
```

```
cat /proc/<pid>/status | grep -E "VmRSS|VmSwap|VmPeak"
```

```
smem -r -k | head -15        # accurate memory accounting per process
```

```
# Swap:
```

```
swapon --show                    # swap devices and usage
```

```
cat /proc/sys/vm/swappiness   # 60=default, 1=avoid swap, 100=aggressive swap
```

Available memory (not Free) is what matters — Linux uses RAM for cache aggressively. Available = free + reclaimable cache. A system with 500MB free but 20GB available is healthy. OOM killer: when available memory is exhausted, the kernel scores processes by OOM score (larger RSS and lower priority = higher score) and kills the highest scorer. Check dmesg for "Killed process" to identify OOM victims.

Kubernetes OOM: A container that hits its memory limit gets OOM-killed (Exit Code 137). The kernel's OOM killer operates inside the container's cgroup. Set memory limits accurately — too low causes constant OOMKills, too high allows memory leaks to grow unchecked. Always set both requests and limits for memory in production pods.

📚 Official docs

- Linux kernel docs: /proc/sys/vm

certs.eknathalabs.com

Q20 How do you diagnose I/O performance issues — high disk latency or throughput problems?

Performance

🎙 Spoken answer · ~84 s

For disk I/O problems I use iostat with extended statistics, watching await, which is average I/O latency in milliseconds, utilisation percent, queue size, and read and write IOPS and throughput. As a rule of thumb, await over about twenty milliseconds on SSD or a hundred on spinning disks is a problem, and utilisation near one hundred percent means saturation, though for NVMe and arrays utilisation is a less reliable signal. To find who is causing it, iotop or pidstat minus d shows per-process I/O. I also check vmstat's wa column for CPU waiting on I/O, and processes in D state. Then dig: is it reads or writes, random or sequential, small or large blocks? Causes include a noisy neighbour process, a backup or log flood, too little memory so the page cache thrashes, swapping, a database doing full scans, a filesystem nearly full or fragmented, a failing disk, which smartctl and dmesg errors reveal, or cloud volume limits like exhausted IOPS credits or throughput caps. Fixes: reduce or reschedule the heavy I/O, tune the application and queries, add cache or memory, change the I/O scheduler, move to faster or provisioned-IOPS storage, or separate workloads onto different disks. Benchmark with fio to validate.

Likely follow-ups

- What does await tell you in iostat?
- Why is utilisation misleading on NVMe devices?

```
iostat -xz 1 5                  # per-disk: r/w IOPS, throughput, await, util%
```

```
iotop -o                        # per-process I/O (like top but for disk)
```

```
iotop -bod 1                    # batch mode for logging
```

```
pidstat -d 1 5                  # per-process disk I/O stats
```

```
# Key iostat columns:
```

```
await   = average I/O wait time (ms). > 20ms SSD / > 100ms HDD = problem
```

```
util%   = disk utilization. > 80% = saturated
```

```
r_await = read latency, w_await = write latency
```

```
# Check for I/O scheduler:
```

```
cat /sys/block/sda/queue/scheduler   # current I/O scheduler
```

```
# Benchmark disk:
```

```
fio --name=randread --rw=randread --bs=4k --size=1G --runtime=30 --iodepth=32
```

certs.eknathalabs.com

iostat await is the most important metric — it measures how long I/O requests wait in the device queue plus actual I/O time. High await with low util% = high queue depth or device latency. High util% = device saturated. Use iotop to identify which process is causing I/O. For cloud VMs: disk IOPS and throughput are limited by the disk type and VM type — check GCP/AWS console for throttling metrics.

Page cache tuning: Linux caches disk reads/writes in RAM. `vm.dirty_ratio` (default 20%) = max RAM used for dirty (unwritten) pages before sync. High dirty_ratio delays writes but improves throughput. Lower it for databases that need durable writes. `vm.dirty_background_ratio` = when background writeback starts.

📚 **Official docs**

- Brendan Gregg: Linux performance

Q21 What are the key sysctl parameters every Platform Engineer should know? Performance

🎙 **Spoken answer · ~70 s**

Sysctl parameters tune kernel behaviour at runtime, viewed with sysctl minus a and set temporarily with minus w, and persisted in etc sysctl.conf or a file in sysctl.d. The important groups. Networking: ip_forward enables routing, which Kubernetes nodes need; bridge-nf-call-iptables lets iptables see bridged traffic; somaxconn and tcp_max_syn_backlog control the listen queue for busy servers; ip_local_port_range sets the ephemeral port range; tcp_tw_reuse helps with many outbound connections; and conntrack max sizes the connection tracking table. Memory: vm.swappiness controls swap eagerness, vm.overcommit_memory and overcommit_ratio affect allocation behaviour, vm.max_map_count matters for Elasticsearch and similar, and dirty ratios control writeback of dirty pages. File system: fs.file-max and inotify limits, since many watchers can exhaust inotify. Kernel: pid_max, and kernel.panic to reboot on panic. Security: rp_filter, disabling source routing, and ASLR. I treat tuning carefully: change one thing at a time, measure before and after, understand the trade-offs, and record changes in configuration management instead of making ad hoc edits. Defaults are usually fine; tune on evidence from a real bottleneck.

Likely follow-ups

- Which sysctls would you check for a busy web server?
- Why persist sysctl changes in a drop-in file?

```
sysctl -a | grep <param>       # view current value
```

```
sysctl -w net.ipv4.ip_forward=1 # set temporarily
```

```
echo "net.ipv4.ip_forward = 1" >> /etc/sysctl.conf  # persist
```

```
sysctl -p                       # reload /etc/sysctl.conf
```

certs.eknathalabs.com

```
# Networking:
net.ipv4.ip_forward = 1                 # enable IP routing (required for K8s)
net.bridge.bridge-nf-call-iptables = 1  # iptables sees bridged traffic (K8s CNI)
net.ipv4.tcp_tw_reuse = 1               # reuse TIME_WAIT sockets
net.core.somaxconn = 65535              # max listen backlog (for high-conn services)
net.ipv4.tcp_max_syn_backlog = 65535    # SYN queue size
net.ipv4.ip_local_port_range = 1024 65535  # ephemeral port range
# Memory:
vm.swappiness = 1                       # minimize swap (for DB servers)
vm.overcommit_memory = 1                # allow overcommit (Redis requirement)
vm.max_map_count = 262144               # max mmap regions (Elasticsearch)
# File descriptors:
fs.file-max = 2097152                   # system-wide fd limit
fs.inotify.max_user_watches = 524288    # for IDE/file watchers (K8s needs this)
```

These parameters are the standard tuning set for production Kubernetes nodes. net.ipv4.ip_forward and bridge-nf-call-iptables are required for pod networking. inotify limits are hit by development tools and Kubernetes controllers watching many files. Redis requires vm.overcommit_memory=1 to avoid background save failures. Elasticsearch requires vm.max_map_count=262144 for memory-mapped index files.

> **Kubernetes node requirements:** kubeadm preflight checks validate several of these sysctl values. Failing preflight on a new node is almost always a missing sysctl setting. Keep a node preparation playbook with all required sysctl values for your Kubernetes version.

> 📚 **Official docs**
>
> - Linux kernel docs: sysctl

certs.eknathalabs.com

Q22 How do you use strace and lsof to debug a misbehaving application?

Performance

🎙 Spoken answer · ~90 s

strace and lsof are my main tools when an application misbehaves and the logs don't explain it. strace shows system calls a process makes, so I can see exactly what it does with the kernel: which files it opens and fails to open, which network connections it makes, what it waits on. I attach to a running process with minus p, filter to relevant categories such as network or file, use minus T to show time spent per call to find slow syscalls, and minus c for a summary count of calls. For example, a hang may show the process blocked in a read or connect call; a startup failure may show an open that returns ENOENT for a missing config file; a permission error shows EACCES. lsof lists open files, including sockets: which files a process has open, who is listening on a port, who holds a deleted file consuming disk space, and how many file descriptors are in use against the limit. Combined, they answer what the process is doing and what it's holding. Cautions: strace slows the traced process significantly, so use it carefully on production, with filters, and for short periods; for lower overhead, consider perf trace or eBPF tools. It also needs ptrace permissions, which may be restricted in containers.

Likely follow-ups

- How would strace reveal a missing config file?
- Why can strace be risky on a production process?

```
# strace: trace system calls made by a process
```

```
strace -p <pid>                    # attach to running process
```

```
strace -p <pid> -e trace=network   # only network syscalls
```

```
strace -p <pid> -e trace=file      # only file syscalls
```

```
strace -c -p <pid> sleep 10       # count syscalls over 10s (summary)
```

```
strace -T -p <pid> 2>&1 | grep -v "0.0000"  # show slow syscalls only
```

```
# lsof: list open files and connections
```

```
lsof -p <pid>                      # all open files for process
```

```
lsof -i :8080                      # what process is using port 8080
```

```
lsof -u www-data                   # all files opened by user
```

```
lsof -i TCP -s TCP:ESTABLISHED     # all established TCP connections
```

```
lsof +D /var/log                   # all processes with files open in /var/log
```

certs.eknathalabs.com

strace reveals exactly what a process is doing at the kernel level — which files it's opening, which sockets it's connecting to, what errors it's getting. Use -T to show time spent per syscall (find slow operations). Use -e trace=network to focus on network activity. lsof answers "what has this port/file/directory open?" — essential for finding port conflicts and leaked file descriptors.

strace overhead: strace adds significant overhead (2-10x slowdown) — it intercepts every syscall. Use it for debugging, not production monitoring. For production-safe tracing, use bpftrace or perf with eBPF probes.

📚 **Official docs**

- Linux man-pages: ptrace(2) (strace)

Q23 How do you audit and harden SSH configuration on a Linux server?

Security

🎙 **Spoken answer · ~80 s**

To harden SSH, I review sshd configuration, using sshd minus T to see the effective settings. Key settings: disable root login with PermitRootLogin no; turn off password authentication so only keys are accepted, which eliminates brute-force password attacks; allow only specific users or groups with AllowUsers or AllowGroups; limit MaxAuthTries and set LoginGraceTime; use strong modern algorithms, disabling old ciphers and protocol versions; disable unneeded features like X11 forwarding and agent forwarding where not needed; set ClientAliveInterval to drop idle sessions; and consider two-factor authentication through PAM or hardware keys. Changing the port is only obscurity, so don't rely on it. Additional layers: restrict access by network with firewall rules or security groups, use a bastion host or a session service instead of exposing SSH publicly, and use short-lived certificates from an SSH certificate authority instead of long-lived keys. Run fail2ban or equivalent to block repeated failures, and keep OpenSSH patched. For auditing, log authentication events, centralise logs, review authorized_keys files for stale keys, and use tools like ssh-audit or CIS benchmarks. Test configuration changes with a second session open and validate with sshd minus t before restarting, to avoid lockout.

Likely follow-ups

- Why open a second session before restarting sshd?
- What advantages do SSH certificates have over static keys?

```
cat /etc/ssh/sshd_config       # current config
```

```
# Key hardening settings:
```

```
PermitRootLogin no            # never allow direct root SSH
```

```
PasswordAuthentication no     # keys only (no password brute force)
```

```
PubkeyAuthentication yes
```

certs.eknathalabs.com

```
AllowUsers deployer admin     # whitelist specific users
```

```
Port 22                       # consider non-standard port (obscurity only)
```

```
MaxAuthTries 3
```

```
ClientAliveInterval 300       # disconnect idle sessions after 5 min
```

```
X11Forwarding no
```

```
UseDNS no                     # skip DNS lookup (faster login)
```

```
systemctl reload sshd          # apply config changes
```

```
# Audit login attempts:
```

```
journalctl -u sshd | grep "Failed|Accepted" | tail -20
```

```
lastb | head -20               # failed login attempts
```

```
last | head -20                # successful logins
```

Disable password authentication immediately on any internet-facing server — this eliminates 99% of brute-force attacks. Use key-based authentication exclusively. AllowUsers creates an explicit whitelist — accounts not listed cannot SSH even with valid keys. Consider fail2ban for automatic IP banning after repeated failures. For cloud environments, prefer IAP (Identity-Aware Proxy) or Systems Manager Session Manager over direct SSH — no need to expose port 22 at all.

GCP OS Login: OS Login ties SSH access to GCP IAM identity — add roles/compute.osLogin to a user's IAM policy instead of managing authorized_keys files. Login events appear in Cloud Audit Logs. No SSH key management required. Enable org-wide with Organization Policy.

📚 **Official docs**

- OpenSSH manual pages
- Linux audit documentation (auditd)

certs.eknathalabs.com

Q24 What are Linux capabilities — how do they reduce attack surface compared to running as root?

Security

🎙 Spoken answer · ~80 s

Traditionally root can do everything, and a process that is either root or not is a coarse model. Linux capabilities split root's powers into distinct privileges, such as binding to low ports, which is CAP_NET_BIND_SERVICE, changing file ownership, CAP_CHOWN, loading kernel modules, CAP_SYS_MODULE, using raw sockets, CAP_NET_RAW, and a catch-all CAP_SYS_ADMIN, which is almost root and should be avoided. A process can be given only the capabilities it needs, so a compromise yields much less power. For example, instead of running a web server as root to bind port eighty, I give the binary only CAP_NET_BIND_SERVICE with setcap, or run it as a normal user with that capability. I inspect with capsh print, the Cap lines in a process's status file, and getcap on binaries. In containers, the default set is already reduced, and best practice is to drop all capabilities and add back only what is required, in the Kubernetes securityContext, plus run as non-root, with no privilege escalation and a read-only root filesystem. Privileged containers give all capabilities and should be avoided. Capabilities reduce the attack surface through least privilege, and they complement seccomp, AppArmor or SELinux, and user namespaces.

Likely follow-ups

- Why is CAP_SYS_ADMIN considered dangerous?
- How do you give a web server port 80 without running as root?

```
capsh --print                # current process capabilities
```

```
cat /proc/<pid>/status | grep Cap  # hex-encoded capability sets
```

```
capsh --decode=0000000000003000   # decode capability hex
```

```
getcap /usr/bin/ping           # view capabilities on a binary
```

```
setcap cap_net_bind_service+ep /usr/bin/node  # bind ports <1024 without root
```

```
# In containers (K8s securityContext):
```

```
securityContext:
```

```
  capabilities:
```

```
    drop: ["ALL"]
```

```
    add: ["NET_BIND_SERVICE"]
```

Linux capabilities split root privileges into ~40 distinct units. CAP_NET_BIND_SERVICE: bind ports below 1024. CAP_SYS_PTRACE: trace/debug processes. CAP_NET_ADMIN: configure network interfaces, iptables. CAP_SYS_ADMIN: almost everything (avoid). Running as root grants all capabilities. Running as non-root with

certs.eknathalabs.com

specific capabilities is far safer — compromise of the process gives only those specific privileges. In Kubernetes, drop ALL capabilities and add back only what's needed — this is the "restricted" Pod Security Standard requirement.

Interview depth: Explaining capabilities vs root shows security maturity. A common follow-up is "how does a container escape work?" — many escapes rely on containers having CAP_SYS_ADMIN or running as root with --privileged. Dropping capabilities and running as non-root eliminates most container escape vectors.

📚 **Official docs**

- Linux man-pages: capabilities(7)

Q25 **How do you use auditd to track file access and command execution on a Linux system?** Security

🎙 **Spoken answer · ~85 s**

auditd is the Linux audit framework, which records security-relevant events at the kernel level, so it's useful for compliance and forensics. I check that the service is running, then add rules with auditctl, or persistently in rules.d. File watches use minus w with a path and permissions, for example watching etc passwd and etc sudoers for writes and attribute changes, with a key to tag the events. System call rules record things like every execve, which logs all commands executed, filtered by architecture. I list active rules with minus l. To query the log, ausearch finds events by key, time range, user, or message type, and aureport produces summaries such as authentication failures, file access, and executables run. Practical considerations: audit generates a lot of data, so rules must be targeted, as auditing every syscall hurts performance and fills disks; set sensible log rotation and size limits; and ship audit logs to a central, tamper-resistant store, since an attacker with root can alter local logs. Make the rules immutable with minus e two so they can't be changed without a reboot, if policy allows. It complements other tools like osquery and eBPF-based runtime security for containers, and it's central to frameworks like CIS and PCI.

Likely follow-ups

- Why ship audit logs off the host?
- What does making audit rules immutable do?

```
systemctl status auditd        # auditd running?
```

```
# Add audit rules:
```

```
auditctl -w /etc/passwd -p wa -k passwd_changes  # watch file for writes/attribs
```

```
auditctl -w /etc/sudoers -p wa -k sudoers_changes
```

certs.eknathalabs.com

```
auditctl -a always,exit -F arch=b64 -S execve -k command_exec  # log all commands
```

```
auditctl -l                    # list current rules
```

```
# Query audit log:
```

```
ausearch -k passwd_changes     # search by key
```

```
ausearch -ua 1000              # all events for UID 1000
```

```
aureport --summary             # summary report
```

```
aureport -x --summary          # executable summary
```

```
cat /var/log/audit/audit.log | grep execve  # raw log
```

auditd provides kernel-level audit logging — it captures syscall events with full context (who, what, when, from which process). Rules watch files (-w) or syscalls (-a). The -k flag tags rules with a searchable key. Use for compliance requirements (PCI-DSS, CIS benchmarks), incident forensics, and detecting privilege escalation attempts. Export logs to SIEM (Splunk, Chronicle) for long-term retention and alerting.

Performance impact: Auditing every execve syscall generates enormous log volume on busy servers. Be selective: watch sensitive files (/etc/passwd, /etc/sudoers, /etc/ssh/) and specific directories rather than logging every command system-wide. Use rate limiting with -F to reduce noise.

📚 **Official docs**

- Linux audit documentation (auditd)

certs.eknathalabs.com

Q26 What is SELinux / AppArmor — how do mandatory access controls work?

Security

🎙 Spoken answer · ~80 s

SELinux and AppArmor are mandatory access control systems: unlike normal permissions, where the owner decides, a central policy defines what each process may do, and even root-owned processes are confined. SELinux, common on RHEL and Fedora, labels every process and file with a security context and the policy states which types may access which; modes are enforcing, permissive, which logs but allows, and disabled. AppArmor, used on Ubuntu and SUSE, is path-based, with profiles per program listing allowed files, capabilities, and network access. The value is containment: if a web server is compromised, the policy stops it from reading other users' files or spawning shells. Operationally, when something fails mysteriously with correct permissions, check for denials: for SELinux, ausearch for AVC messages, or sealert for plain explanations; fix by correcting labels with restorecon, setting a boolean, or adding a policy module, not by turning SELinux off. Setting permissive temporarily helps diagnosis, but should not stay in production. Common problems arise after moving files, which keep the wrong context, or using non-standard ports or directories. In containers, SELinux and AppArmor profiles are another isolation layer. Best practice: keep it enforcing and learn the tooling.

Likely follow-ups

- Why is disabling SELinux a poor fix?
- How do you diagnose an SELinux denial?

```
# SELinux (RHEL/CentOS):
getenforce                    # Enforcing / Permissive / Disabled
sestatus                      # detailed SELinux status
setenforce 0                  # set Permissive (for testing, NOT production)
ausearch -m avc -ts recent    # SELinux denial messages
sealert -a /var/log/audit/audit.log  # human-readable denial explanations
restorecon -Rv /var/www/html    # fix file context after copy
# AppArmor (Ubuntu/Debian):
aa-status                     # profile status
aa-complain /usr/sbin/nginx   # put nginx profile in complain mode
aa-enforce /usr/sbin/nginx    # enforce nginx profile
journalctl | grep apparmor    # AppArmor denials
```

certs.eknathalabs.com

DAC (Discretionary Access Control) = traditional Unix permissions — file owner controls access. MAC (Mandatory Access Control) = kernel enforces policies that override file permissions. SELinux labels every file, process, and port with a security context — access is only allowed when the policy explicitly permits it. A web server process confined by SELinux cannot read /etc/shadow even if the file permissions allow it. In containers: seccomp and AppArmor profiles restrict syscalls and filesystem access — GKE's default runtime/default seccomp profile blocks ~100 risky syscalls.

📚 **Official docs**

- Kubernetes: AppArmor tutorial
- RHEL 9 documentation

Never permanently disable SELinux. "setenforce 0" is a debug step, not a fix. If SELinux is blocking a legitimate operation, generate the correct policy addition with audit2allow. Disabling SELinux on a RHEL production server is a security failure and a compliance violation.

Q27 How do you manage disks and partitions on Linux — MBR vs GPT?

Storage & disks

🎙 **Spoken answer · ~83 s**

Disk management starts with seeing what's there: lsblk for the block device tree, fdisk minus l for partitions, and blkid for UUIDs and filesystem types. For new disks, choose the partition table: MBR, the legacy format, supports disks only up to two terabytes, a maximum of four primary partitions, and stores its table in one place; GPT is modern, supports huge disks, many partitions, has redundant headers with checksums, and is required for UEFI boot. So I use GPT, with gdisk or parted. The workflow: create the partition table, create partitions, format with mkfs, ext4 for general use or XFS for large files and scalability, and mount it. For persistence, add an entry in etc fstab using the UUID, not the device name, which can change, and add nofail for non-critical disks; test with mount minus a before rebooting. Consider LVM for flexibility, and alignment for performance. In the cloud, a new volume appears as a device; I format it once, and be careful not to format a disk with existing data, so I check with blkid or file minus s first. Also monitor usage, and use labels or UUIDs in automation. Always double-check device names before destructive commands.

Likely follow-ups

- Why use UUIDs in fstab rather than /dev/sdX names?
- What limits does MBR have compared with GPT?

```
lsblk                          # block devices tree view
```

```
fdisk -l                       # list all disks and partitions
```

```
blkid                          # show UUID and filesystem type per partition
```

certs.eknathalabs.com

```
# Create partition (GPT, modern):
```

```
gdisk /dev/sdb                # GPT partition tool
```

```
parted /dev/sdb mklabel gpt
```

```
parted /dev/sdb mkpart primary ext4 0% 100%
```

```
# Format:
```

```
mkfs.ext4 /dev/sdb1           # create ext4 filesystem
```

```
mkfs.xfs /dev/sdb1            # create XFS filesystem
```

```
# Mount persistently:
```

```
echo "UUID=xxxx /data ext4 defaults,nofail 0 2" >> /etc/fstab
```

```
mount -a                      # test fstab without reboot
```

MBR (Master Boot Record): max 4 primary partitions, max 2TB disk. GPT (GUID Partition Table): 128 partitions, up to 9.4 ZB. Use GPT for all new disks. Always use UUID in /etc/fstab instead of device names (/dev/sdb) — device names can change on reboot, UUIDs are stable. The nofail mount option prevents boot failure if the disk is missing (important for data disks that may not always be attached).

Cloud disk attach workflow: Attach disk in GCP/AWS console → `lsblk` to see new device → `mkfs` if new disk → `blkid` to get UUID → add to /etc/fstab with nofail → `mount -a` to verify. This is a daily operation for platform engineers managing data volumes.

📚 **Official docs**

- Linux kernel docs: ext4

certs.eknathalabs.com

Q28 What is RAID — what levels matter and when do you use software RAID in production?

Storage & disks

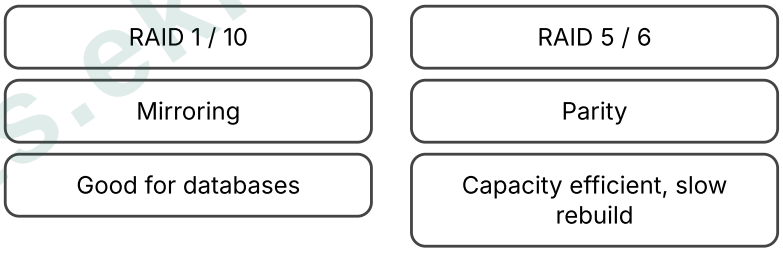
🎙 Spoken answer · ~84 s

RAID combines multiple disks for redundancy or performance. The common levels: RAID 0 stripes data across disks for speed and capacity but has no redundancy, so one failure loses everything. RAID 1 mirrors two disks, giving redundancy with half the capacity. RAID 5 stripes with single parity across at least three disks, tolerating one failure, with write penalty and long, risky rebuilds on large disks. RAID 6 uses double parity, tolerating two failures. RAID 10 combines mirroring and striping, offering good performance and redundancy at fifty percent capacity, and is a common choice for databases. Software RAID with mdadm is used when there's no hardware controller or in cloud and bare-metal setups: for example mirrored boot disks. I create arrays with mdadm, monitor them via proc mdstat and mdadm detail, and replace failed disks by removing the failed one and adding a new one, then watching the rebuild. Important caveats: RAID is not a backup, since it doesn't protect against deletion, corruption, or ransomware; rebuild time stresses the remaining disks; and I should monitor and alert on degraded arrays. In the cloud, redundancy usually comes from the storage layer or replication across zones, so RAID is less common than on bare metal.

Likely follow-ups

- Why is RAID not a substitute for backups?
- Why is RAID 10 often preferred for databases?

🗺 RAID levels




```
# Software RAID with mdadm:
```

```
mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc  # RAID 1
```

```
cat /proc/mdstat                # RAID status
```

```
mdadm --detail /dev/md0         # detailed array status
```

```
mdadm --manage /dev/md0 --add /dev/sdd   # add spare/replace failed disk
```

```
# Common RAID levels:
```

```
RAID 0: striping — max performance/capacity, zero redundancy (any disk failure = data loss)
```

certs.eknathalabs.com

```
RAID 1: mirroring — 2 disks, full redundancy, 50% capacity
```

```
RAID 5: striping + parity — 3+ disks, 1 disk fault tolerance, (n-1) capacity
```

```
RAID 6: striping + 2 parity — 2 disk fault tolerance
```

```
RAID 10: RAID 1+0 — mirrored stripes, best performance + redundancy, 50% capacity
```

In cloud environments, RAID is largely replaced by cloud storage redundancy (GCP Persistent Disks, AWS EBS have built-in replication). Use software RAID on bare metal or when you need: local NVMe RAID 0 for extreme IOPS (databases, scratch storage), or RAID 10 for bare-metal database servers. RAID is not a backup — it protects against disk failure, not data corruption or accidental deletion.

RAID 5 rebuild risk: During rebuild after a disk failure, the remaining disks are under stress. A second failure during rebuild (common for old drives from the same batch) causes total data loss. RAID 6 or RAID 10 is safer for production. Many DBAs prefer RAID 10 over RAID 5 for write-heavy databases.

📚 **Official docs**

- Linux kernel docs: md (software RAID)

Q29 How do you diagnose and fix filesystem corruption on Linux?

Storage & disks

🎙 **Spoken answer · ~82 s**

For suspected filesystem corruption, I first look for evidence: kernel messages from dmesg or the journal showing ext4 or XFS errors, I/O errors, or remount to read-only, which the kernel does to protect data. I also check the underlying hardware with smartctl and logs, since corruption often has a failing disk, controller, or cable behind it, and in virtual environments the storage layer. Before repairing, protect data: if the data is valuable, take an image or snapshot of the device first, because repair tools can make things worse. Then unmount the filesystem; fsck must not run on a mounted filesystem. For ext4, run e2fsck, first without changes to see what it finds, then with repair. For XFS, use xfs_repair, again unmounted, with xfs_repair minus n for a dry run. For the root filesystem, boot into rescue mode or from live media. After repair, check lost and found for recovered fragments and verify the application data, restore from backup if needed. Then fix the root cause: replace failing hardware, address unclean shutdowns and power issues, and review the storage layer. Prevention: reliable backups, monitoring SMART and filesystem errors, journalling filesystems, UPS or cloud durability, and tested restores.

Likely follow-ups

- Why must you unmount before running fsck?
- Why take an image of the device before repairing?

```
dmesg | grep -i "error|corrupt|EXT4-fs|XFS"   # kernel filesystem errors
```

certs.eknathalabs.com

```
journalctl -k | grep -i "filesystem|ext4|xfs"  # journal filesystem messages
```

```
# Check and repair ext4 (must be unmounted):
```

```
umount /dev/sdb1              # unmount first
```

```
fsck -y /dev/sdb1             # check and auto-fix ext4
```

```
e2fsck -f -y /dev/sdb1        # more thorough ext4 check
```

```
# XFS repair (must be unmounted):
```

```
xfs_check /dev/sdb1           # check (read-only)
```

```
xfs_repair /dev/sdb1          # repair XFS
```

```
# Remount read-only if filesystem errors detected:
```

```
mount -o remount,ro /data     # prevent further corruption
```

Filesystem corruption causes: improper shutdown (power loss during writes), hardware failure (failing disk, bad RAM causing bit flips), kernel bugs, or full filesystem (writes failing partially). When the kernel detects errors, it remounts the filesystem read-only to prevent further damage. Fix: fsck/xfs_repair on unmounted filesystem. For production systems: always have journaling enabled (ext4 journal, XFS journal) to minimize corruption on unclean shutdown.

> **Root filesystem fsck:** You can't unmount the root filesystem while the system is running. Boot into rescue/single-user mode, or use a live CD, to run fsck on the root partition. Modern Linux runs fsck automatically on boot if the filesystem was not cleanly unmounted.

> 📚 **Official docs**
>
> - Linux kernel docs: ext4
> - Linux kernel docs: XFS

certs.eknathalabs.com

Q30 Explain the Linux boot process — from power-on to login prompt.

Boot & systemd

🎙 Spoken answer · ~85 s

The Linux boot process runs in stages. First, firmware, BIOS or UEFI, initialises hardware, runs a self-test, and finds a bootable device. Second, the bootloader, usually GRUB2, is loaded from disk, presents a menu, and loads the kernel and the initramfs from the boot partition. Third, the kernel decompresses, initialises the CPU, memory, and drivers, and mounts the initramfs as a temporary root filesystem. Fourth, the initramfs contains the drivers and scripts needed to find and mount the real root filesystem, for example for LVM, RAID, encryption, or special storage controllers. Fifth, the kernel switches to the real root filesystem, found from the command line or fstab. Sixth, it starts the first process, PID one, which on modern systems is systemd. Systemd then brings up the system by activating targets and units in dependency order, mounting filesystems, starting networking and services, until it reaches the default target, such as multi-user or graphical. Finally, a login prompt or display manager appears. Knowing this helps troubleshooting: a failure before the kernel is firmware or bootloader, a failure dropping to initramfs is usually missing storage drivers or a wrong root device, and a failure after is a service or mount issue, which the journal and systemd-analyze reveal.

Likely follow-ups

- What is the purpose of the initramfs?
- Where in the boot process would a wrong root= parameter cause a failure?

```
1. BIOS/UEFI — hardware init, finds bootable disk
```

```
2. Bootloader (GRUB2) — loads kernel + initramfs from /boot
```

```
3. Kernel init — decompresses, initializes hardware, mounts initramfs
```

```
4. initramfs — temporary root FS with drivers to mount real root FS
```

```
5. Kernel mounts real root filesystem (from /etc/fstab or kernel cmdline)
```

```
6. systemd (PID 1) starts — reads /etc/systemd/system/default.target
```

```
7. systemd activates services in dependency order
```

```
8. getty (login prompt) or display manager starts
```

```
# Debug boot:
```

```
journalctl -b              # all logs from current boot
```

```
journalctl -b -1           # logs from previous boot (useful after crash)
```

```
systemd-analyze            # total boot time
```

certs.eknathalabs.com

```
systemd-analyze blame       # time per service (find slow boot services)
```

```
systemd-analyze critical-chain  # critical path of boot
```

GRUB2 is the bootloader on most modern Linux. It reads /boot/grub/grub.cfg, loads the kernel image and initrd. The kernel cmdline (visible in /proc/cmdline) can pass parameters: root= (root device), quiet (suppress messages), ro (mount root read-only initially). systemd replaces sysvinit — it parallelizes service startup and tracks dependencies. Containers skip all of this — the kernel is provided by the host, the container runtime starts PID 1 directly.

Rescue mode: If systemd fails to start (corrupted unit file, bad fstab), boot with `systemd.unit=rescue.target` on the kernel cmdline in GRUB. Or append `init=/bin/bash` for a root shell before systemd starts. These rescue techniques are tested in CKA.

📚 **Official docs**

- systemd: boot process

Q31 **How do you manage services with systemd — what are the key commands and unit file structure?** Boot & systemd

🎙 **Spoken answer · ~76 s**

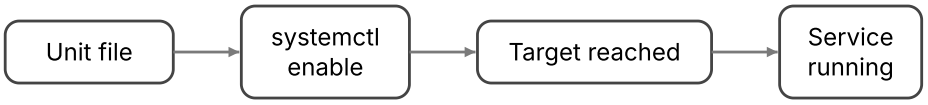
Systemd is the init system and service manager on most modern Linux distributions. Key commands: systemctl status shows the state and the recent log lines; start, stop, and restart control the service now; enable and disable control whether it starts at boot by creating or removing symlinks; daemon-reload makes systemd re-read unit files after editing; list-units with the failed state filter shows what's broken; and journalctl minus u with follow reads the service's logs. A unit file has sections. The Unit section has the description and dependencies and ordering, using After, Requires, and Wants. The Service section defines Type, such as simple, forking, or oneshot, the ExecStart command, User, environment, Restart policy such as on-failure with RestartSec, and resource and security settings, like MemoryMax, LimitNOFILE, and sandboxing options. The Install section says which target wants it, usually multi-user.target. Custom overrides go in drop-in directories, created by systemctl edit, instead of modifying vendor units. I verify units with systemd-analyze verify, and analyse boot time with systemd-analyze blame. Good practice: run services as non-root users, set restart policies, add limits, and log to the journal.

Likely follow-ups

- Why use drop-in overrides instead of editing vendor unit files?
- What is the difference between Requires and Wants?

certs.eknathalabs.com

🗺 systemd




```
systemctl status nginx         # service status + recent logs
```

```
systemctl start/stop/restart nginx
```

```
systemctl enable nginx         # start on boot (creates symlink in wants/)
```

```
systemctl disable nginx        # remove from boot
```

```
systemctl daemon-reload        # reload unit files after editing
```

```
systemctl list-units --type=service --state=failed
```

```
journalctl -u nginx -f         # follow nginx service logs
```

```
journalctl -u nginx --since "1 hour ago"
```

```
# Create a custom service unit:
```

```
cat /etc/systemd/system/myapp.service
```

```
[Unit]
```

```
Description=My Application
```

```
After=network.target
```

```
[Service]
```

```
Type=simple
```

```
User=myapp
```

```
ExecStart=/usr/bin/myapp --config /etc/myapp/config.yaml
```

```
Restart=on-failure
```

```
RestartSec=5
```

```
[Install]
```

```
WantedBy=multi-user.target
```

certs.eknathalabs.com

Unit files define services, timers, sockets, and mounts. The [Service] section key settings: Type (simple, forking, notify, oneshot), ExecStart/ExecStop/ExecReload, Restart (always/on-failure/on-abnormal), User/Group for privilege dropping, Environment/EnvironmentFile for config. After=network.target ensures networking is up before the service starts. systemd replaces both /etc/init.d scripts and cron (use .timer units instead of cron for systemd-managed services).

Restart policies: Restart=on-failure restarts only on non-zero exit code. Restart=always restarts even on clean exit (good for long-running services). RestartSec=5 adds a delay to prevent rapid restart loops from hammering the system. Combine with StartLimitIntervalSec and StartLimitBurst to limit restart frequency.

📚 **Official docs**

- systemd.service(5)

Q32 A server is not booting — how do you diagnose and recover from boot failures?

Boot & systemd

🎙 **Spoken answer · ~81 s**

When a server won't boot, I work out which stage fails. From the console, if the kernel or the bootloader doesn't load, I look at GRUB messages. At the GRUB menu, I can press e to edit the kernel command line: remove quiet and splash to see all messages, append systemd.unit=rescue.target to boot to a rescue shell, or emergency.target for a minimal environment, or init=/bin/bash to bypass systemd entirely as a last resort, remounting root read-write afterwards. In the rescue shell I read the journal for the current boot filtered to errors, check failed units, and look at common culprits: a bad fstab entry, a full or corrupted filesystem, a broken initramfs after a kernel update, a missing or renamed disk, or a service misconfiguration. Fixes: correct fstab, run fsck on unmounted filesystems, regenerate the initramfs and reinstall GRUB from a chroot using a live or rescue image, or select an older kernel from the GRUB menu. In the cloud, use the serial console, or detach the root volume and attach it to a healthy instance to repair it. Afterwards, add safeguards: nofail on non-critical mounts, keep previous kernels installed, and test changes before rebooting.

Likely follow-ups

- How do you boot an older kernel when the new one fails?
- What would you do in a cloud VM with no console access?

```
# From GRUB: edit kernel cmdline (press 'e' on GRUB entry)
```

```
Append: systemd.unit=rescue.target   # boot to rescue shell
```

```
Append: init=/bin/bash               # bypass systemd entirely
```

```
Remove: quiet splash                  # see all boot messages
```

certs.eknathalabs.com

```
# In rescue shell:
mount -o remount,rw /               # remount root read-write
journalctl -b -p err                # errors from this boot
systemctl list-units --failed       # which units failed
fsck /dev/sda1                      # check root filesystem
cat /etc/fstab                      # check for bad entries
# For cloud VMs — rescue disk approach:
# Snapshot the boot disk → detach → attach to rescue VM → mount → fix → reattach
```

Common boot failures: bad /etc/fstab entry (missing nofail on non-critical disk causes boot hang), corrupted filesystem (fsck needed), failed systemd service in required boot chain (disable or fix the unit), kernel panic (bad kernel module — boot old kernel from GRUB), or full root filesystem (can't write to disk). For cloud VMs: use serial console access (GCP Serial Console, AWS EC2 Serial Console) to see boot messages without needing SSH.

fstab mistakes: A typo in /etc/fstab is one of the most common ways to break a server. Always test with `mount -a` before rebooting. Use `nofail` for non-root filesystems so a missing disk doesn't halt boot. Never reboot after editing fstab without testing.

📚 **Official docs**

- GNU GRUB manual

certs.eknathalabs.com

Q33 How do you manage users, groups, and sudo on a Linux system?

Users & permissions

🎙 Spoken answer · ~82 s

User management starts with creating accounts: useradd with a home directory, a shell, and supplementary groups; passwd to set a password, though for servers I prefer key-only SSH; usermod with append to add groups, where forgetting the append flag replaces the groups; and userdel with remove to delete the user and home. id shows UID, GID, and groups. Account data lives in etc passwd, shadow for hashed passwords, and group. For privileges, sudo lets users run commands as root with an audit trail, configured in etc sudoers and sudoers.d using visudo, which validates syntax so a typo doesn't break sudo. Best practice is to grant least privilege: specific commands to specific groups rather than unlimited access, avoid NOPASSWD except where needed for automation, log sudo usage, and use groups such as sudo or wheel. Beyond local accounts, in organisations I centralise identity through LDAP, SSSD, or an identity provider, with SSH keys or certificates, and offboarding processes that remove access promptly. Service accounts should be non-login system users with no shell. I audit regularly: users with UID zero, accounts with empty passwords, stale users, authorized_keys files, and sudo rules, and manage this through configuration management for consistency.

Likely follow-ups

- Why use visudo to edit sudoers?
- What happens if you run usermod -G without -a?

```
useradd -m -s /bin/bash -G sudo deployer  # create user with home dir and sudo
```

```
usermod -aG docker,sudo existing_user     # add to groups (-a = append)
```

```
passwd deployer                           # set password
```

```
userdel -r olduser                        # delete user + home dir
```

```
id username                               # show UID, GID, groups
```

```
groups username                           # list groups
```

```
cat /etc/passwd | cut -d: -f1             # list all users
```

```
# sudo configuration:
```

```
visudo                                    # edit /etc/sudoers safely
```

```
deployer ALL=(ALL) NOPASSWD: /usr/bin/systemctl  # passwordless specific command
```

```
%ops ALL=(ALL) ALL                        # all users in ops group get full sudo
```

```
cat /etc/sudoers.d/deployer               # per-user sudo config in drop-in dir
```

certs.eknathalabs.com

Always use visudo to edit sudoers — it validates syntax before saving, preventing lockout from a syntax error. Use /etc/sudoers.d/ for per-user/group configs instead of editing the main sudoers file. NOPASSWD should be limited to specific commands, never ALL, to minimize risk. For cloud environments: prefer IAM-based access (OS Login for GCP) over managing local users — it scales and audits better.

Group membership: Users must log out and back in after group changes for new groups to take effect (the login shell reads group membership at session start). Or use `newgrp docker` to start a new shell with the updated group without logging out.

📚 **Official docs**

- sudoers(5) manual

Q34 What is PAM (Pluggable Authentication Modules) — how does it control login?

Users & permissions

🎙 **Spoken answer · ~83 s**

PAM, Pluggable Authentication Modules, is the framework that many Linux programs use for authentication, so the login policy is configured centrally instead of inside each program. Each service, such as sshd, sudo, or login, has a file in etc pam.d listing modules in a stack. There are four module types: auth verifies identity with a password, LDAP, or a second factor; account checks restrictions such as expiry, time-of-day access, or lockout; password handles password changes and complexity rules; and session sets up and tears down the session, for example creating home directories or applying limits. Each line has a control flag: required means it must succeed but processing continues, requisite fails immediately, sufficient succeeds the stack if it passes, and optional is ignored for the result. The order matters. Common uses: enforcing password policy with pwquality, locking accounts after failed attempts with faillock, adding two-factor authentication with Google Authenticator or hardware keys, integrating with SSSD for LDAP or Active Directory, and restricting access through pam_access or pam_listfile. Be careful: a mistake in PAM configuration can lock everyone out, including root, so keep a root session open while testing, and change one thing at a time with a backup.

Likely follow-ups

- What is the difference between required and requisite?
- How would you add two-factor authentication using PAM?

```
cat /etc/pam.d/sshd            # PAM config for SSH login
```

```
cat /etc/pam.d/sudo            # PAM config for sudo
```

```
cat /etc/pam.d/common-auth     # included by most PAM configs
```

```
# PAM module types:
```

certs.eknathalabs.com

```
auth    — verify identity (password, LDAP, 2FA, SSH key)
account — check account restrictions (expiry, time of day, lockout)
session — setup/teardown session (mount home, set env, log)
password — manage password changes
# Control flags:
required   — must pass, failure continues chain (reveals all errors at end)
requisite  — must pass, failure stops chain immediately
sufficient — if passes, stop chain (success without later checks)
optional   — result ignored
```

PAM is the Linux authentication framework — it makes authentication pluggable without recompiling applications. SSH, sudo, login, su all use PAM. Common modules: pam_unix (local passwords), pam_ldap (LDAP auth), pam_google_authenticator (TOTP 2FA), pam_limits (enforce ulimits on login), pam_faillock (account lockout after failed attempts). When an SSH login is slow, PAM modules (especially LDAP or DNS lookups) are often the cause.

PAM misconfiguration: A syntax error or wrong module path in a PAM config file can make it impossible to log in via that method. Always test on a second open session before closing the current one. pam_tally2/pam_faillock can lock out all users including root — know how to unlock: `faillock --user root --reset`.

📚 **Official docs**

- Linux-PAM System Administrators Guide

certs.eknathalabs.com

Q35 How do you implement and audit access controls using ACLs (Access Control Lists)?

Users & permissions

🎙 Spoken answer · ~84 s

ACLs extend the basic owner, group, and other permissions with finer-grained rules for specific users and groups. I view them with getfacl and set them with setfacl: for example, give a particular user read, write, and execute on a project directory, and give a team group read and execute. Default ACLs on a directory are inherited by new files and subdirectories, which solves the common problem of shared directories where new files get the wrong permissions. I remove individual entries with minus x or everything with minus b. An ACL mask limits the maximum effective permissions for named users and groups, which sometimes explains why permissions seem not to apply. A plus sign at the end of the ls mode string shows that a file has an ACL. Auditing means reviewing ACLs recursively with getfacl, and comparing against the intended policy, ideally defined in configuration management. Considerations: the filesystem must support ACLs, which is the default in modern ones; backup and copy tools must preserve ACLs, for example with rsync minus A or tar with acls; and too many exceptions make access hard to reason about, so prefer groups where possible. ACLs add flexibility without requiring new groups for every case.

Likely follow-ups

- What does the ACL mask do?
- What are default ACLs used for?

```
getfacl /shared/project         # view current ACLs
```

```
setfacl -m u:alice:rwx /shared/project   # give alice rwx
```

```
setfacl -m g:devteam:rx /shared/project  # give devteam r-x
```

```
setfacl -m d:g:devteam:rx /shared/project  # default ACL for new files
```

```
setfacl -x u:alice /shared/project   # remove alice's ACL
```

```
setfacl -b /shared/project           # remove all ACLs
```

```
# Find files with ACLs:
```

```
find / -acl 2>/dev/null              # files with extended ACLs
```

```
ls -la /shared/project               # + sign after permissions = has ACL
```

POSIX ACLs extend standard rwx permissions — you can grant specific users or groups access to files without changing the owning user/group or using SUID/SGID. Default ACLs on directories propagate to newly created files inside — critical for shared project directories where you want consistent permissions on all

certs.eknathalabs.com

files. The + in ls -la output indicates ACLs are present. Filesystem must be mounted with acl option (ext4/XFS support ACLs natively, enabled by default in modern Linux).

Practical use case: A shared /data/project directory owned by root:engineering. Engineers need write access, CI service account needs read-only. Without ACLs: you need complex group arrangements. With ACLs: setfacl -m u:ci-user:rx and u:alice:rwx solves it cleanly without touching group membership.

📚 **Official docs**

- Linux audit documentation (auditd)

Q36 What are essential bash scripting practices for production automation scripts?

Scripting & automation

🎙 **Spoken answer · ~84 s**

Production bash scripts need defensive practices. Start with a proper shebang and set minus e, minus u, and minus o pipefail: exit on errors, treat undefined variables as errors, and fail a pipeline if any stage fails; this prevents scripts from silently continuing after a failure. Use trap to clean up temporary files on exit and report errors with the line number. Quote all variable expansions to handle spaces and special characters, use default values with parameter expansion, and check that required variables and arguments exist with clear usage messages. Prefer double brackets for conditions, local variables in functions, and arrays for lists. Avoid parsing ls output, use find with null separators or globbing for filenames. Make scripts idempotent, so rerunning is safe, and use locking, for example flock, to prevent concurrent runs. Log with timestamps, return meaningful exit codes, and avoid hardcoding secrets. For debugging, set minus x for tracing. Validate inputs, use mktemp for temp files, and check dependencies. Use shellcheck in CI to catch common bugs, and test with bats. Know when to stop: if the script has complex logic, data structures, or error handling needs, switching to Python is more maintainable. Keep scripts in version control with review.

Likely follow-ups

- What does set -euo pipefail each do?
- Why quote variable expansions?

```
#!/bin/bash
```

```
set -euo pipefail              # exit on error, undefined var, pipe failure
```

```
set -x                         # trace (print each command before execution)
```

```
# Trap for cleanup on exit:
```

```
trap 'cleanup; echo "Script failed on line $LINENO"' ERR
```

certs.eknathalabs.com

```
trap cleanup EXIT

# Safe variable use:

var="${1:-default_value}"       # parameter with default

[ -z "${VAR:-}" ] && echo "VAR is empty"

# Avoid word splitting with quotes:

file="my file.txt"; rm "$file"  # quotes prevent word splitting

# Lockfile to prevent concurrent runs:

LOCKFILE=/tmp/myscript.lock

exec 200>"$LOCKFILE"; flock -n 200 || { echo "Already running"; exit 1; }

# Logging:

log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*" | tee -a /var/log/myscript.log; }
```

set -euo pipefail is the most important line in any production bash script. -e: exit on command failure. -u: treat unset variables as errors (catches typos). -o pipefail: a pipe fails if any command in it fails (without this, `false | echo "ok"` succeeds). trap ERR catches failures and cleans up temp files, releases locks. Never write production scripts without these — a missing set -e means errors are silently ignored and the script continues in a broken state.

set -e caveats: `set -e` has surprising behavior: it doesn't trigger inside if conditions (`if grep foo file; then`), function calls assigned to variables, or expressions after || and &&. Test scripts thoroughly and consider using explicit error checking for critical commands: `cmd || { echo "cmd failed"; exit 1; }`.

📚 **Official docs**

- GNU Bash reference manual

certs.eknathalabs.com

Q37 How do you write efficient text processing pipelines with grep, awk, sed, and sort?

Scripting & automation

🎙 Spoken answer · ~85 s

Text-processing pipelines let me analyse logs quickly, and each tool has a role. grep searches and filters: recursive search, a count of matches, extended regex for alternatives, inverted matches to exclude noise, and context lines before and after a match for exceptions. awk is for columnar data: print selected fields, filter by conditions on fields, and aggregate, for example summing bytes per client or counting status codes using an associative array. sed handles stream edits, mostly substitution and deleting or printing line ranges. sort and uniq with minus c produce frequency counts, and sort minus rn then head gives a top-N, such as the top ten client IPs or slowest endpoints. cut, tr, and paste help with simpler transformations, and xargs turns output into arguments. A typical pipeline: extract the field with awk, sort, count unique values with uniq minus c, and sort numerically in reverse. Efficiency tips: filter early to reduce data, use grep before heavier processing, avoid useless use of cat, use LC_ALL equals C for faster sorting, and use ripgrep on large trees. Be careful with quoting and with delimiters in the data. For structured logs, jq is better for JSON. For repeated needs, I turn the pipeline into a script.

Likely follow-ups

- How would you find the top 10 client IPs in an access log?
- When would you use jq instead of awk?

```
# grep — search and filter:

grep -r "ERROR" /var/log/ --include="*.log" -l   # recursive, filenames only

grep -E "ERROR|WARN" app.log | grep -v "expected" # exclude pattern

grep -c "404" access.log                          # count matches

grep -A3 -B1 "Exception" app.log                  # context lines

# awk — columnar processing:

awk '{print $1, $NF}' access.log                  # first and last field

awk -F: '$3 >= 1000 {print $1}' /etc/passwd       # users with UID >= 1000

awk '{sum+=$NF} END {print "Total:", sum}' file    # sum last column

awk '/START/,/END/' logfile                        # print between patterns

# sed — stream editing:

sed -i 's/old_value/new_value/g' config.yaml      # in-place replacement
```

certs.eknathalabs.com

```
sed -n '10,20p' largefile                          # print lines 10-20
```

```
sed '/^#/d; /^$/d' config.yaml                     # remove comments + blank lines
```

```
# sort + uniq — aggregation:
```

```
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10  # top IPs
```

The Unix pipeline philosophy: each tool does one thing, compose them. grep filters lines, awk processes columns, sed transforms text, sort orders, uniq counts. Mastering these four tools means you can process any log file, config file, or data stream without writing a Python script. The top-IPs pipeline is a classic — learn it and adapt it for any log analysis task.

Performance tip: For large files, grep is faster than awk for pure pattern matching. Place the most selective filter first in a pipeline — filter early to reduce data flowing to later stages. For very large log files (>10GB), consider ripgrep (rg) which is significantly faster than GNU grep.

📚 **Official docs**

- GNU awk (gawk) manual
- GNU sed manual
- GNU grep manual

certs.eknathalabs.com

Q38 How do you schedule tasks with cron and systemd timers — what are the differences?

Scripting & automation

🎙 Spoken answer · ~89 s

Both cron and systemd timers schedule recurring tasks. Cron is the traditional approach: edit a user's crontab, or put files in etc cron.d, and the format is minute, hour, day of month, month, day of week, then the command. It's simple and universal. Its limitations: minimal environment, so PATH and variables differ from your shell, a common source of it works manually but not in cron; output goes to mail or gets lost unless redirected; no built-in logging beyond syslog; no dependency handling; and if the machine is off at the scheduled time, the job is skipped, unless anacron is used. Systemd timers pair a timer unit with a service unit. Advantages: logs in the journal, dependency and ordering control, resource limits and sandboxing through the service unit, Persistent equals true to catch up on missed runs after downtime, randomised delay to avoid thundering herds, calendar expressions, and the ability to see the next and last run with list-timers. They can also run on events relative to boot. I choose timers for services on modern systems where observability and control matter, and cron for simple portable tasks. For both: make jobs idempotent, use locking to avoid overlaps, set timeouts, send output somewhere visible, and alert on failure, for example with a heartbeat monitor.

Likely follow-ups

- Why do cron jobs behave differently from your interactive shell?
- What does Persistent=true do on a timer?

```
# cron:
```

```
crontab -e                          # edit current user's crontab
```

```
crontab -l                          # list current user's crontab
```

```
cat /etc/crontab                    # system crontab
```

```
ls /etc/cron.d/ /etc/cron.daily/    # drop-in cron directories
```

```
# Format: min hour dom month dow command
```

```
*/5 * * * * /usr/bin/check_disk.sh      # every 5 minutes
```

```
0 2 * * 0   /usr/bin/weekly_backup.sh   # Sunday at 2am
```

```
@reboot     /usr/bin/startup.sh         # on reboot
```

```
# systemd timer:
```

```
cat /etc/systemd/system/backup.timer
```

certs.eknathalabs.com

```
[Timer]
```

```
OnCalendar=Sun *-*-* 02:00:00
```

```
Persistent=true                         # run if missed while system was off
```

```
systemctl enable --now backup.timer
```

```
systemctl list-timers --all             # list all timers and next run time
```

cron advantages: simple, universal, familiar. Disadvantages: no dependency management, no logging (output goes to email or /dev/null), no retry on failure, skips if system was off. systemd timer advantages: full journal logging, dependencies (OnActiveSec, After=network.target), Persistent=true to catch missed runs, easy to disable/enable without crontab editing, integrated with systemctl status. For new automation on systemd systems, prefer timers. Keep cron for simple one-liners and scripts deployed to mixed environments.

Cron PATH: cron runs with a minimal environment — PATH is typically just /usr/bin:/bin. Scripts that work interactively fail in cron because commands aren't found. Always use absolute paths in cron scripts or set PATH explicitly at the top of the script.

📚 **Official docs**

- systemd.service(5)
- systemd.timer(5)

Q39 Server is responding slowly — walk through your systematic diagnosis approach.

Troubleshooting

🎙 **Spoken answer · ~74 s**

I use a systematic approach rather than guessing; Brendan Gregg's USE method is a good frame: check utilization, saturation, and errors for each resource. I start with uptime and load average for a quick picture, then top or htop to see whether the bottleneck is CPU, memory, or waiting. For CPU, I look at user versus system versus iowait and the top processes. For memory, free and vmstat: is it swapping, and did the OOM killer fire, which dmesg will show. For disk, iostat shows utilization and await latency, iotop shows who is doing the I/O, and df, including inodes, shows space. For network, ss and sar show connections, retransmits, and errors. If it's one process, strace, lsof, or perf can show what it's doing. I also check what changed, such as a deploy, a cron job, or a backup, and read the logs and journal around the time it started. The key is to find the saturated resource first, then the responsible process, then the cause, and to keep baselines so I know what normal looks like.

Likely follow-ups

- Everything looks idle but the server is still slow. What next?
- How do you tell memory pressure from a leak?

certs.eknathalabs.com

```
# Step 1: System-wide snapshot (60 seconds of data)

uptime                    # load average trend

vmstat 1 10               # CPU, memory, swap, I/O, processes

iostat -xz 1 5            # disk I/O breakdown

free -h                   # memory: available is the key metric

ss -s                     # socket summary (connection counts)

# Step 2: Find the bottleneck

top -b -n 1 -o %CPU       # CPU hogs

top -b -n 1 -o %MEM       # memory hogs

iotop -b -n 1 -o          # I/O hogs

iftop                     # network bandwidth hogs

# Step 3: Check for specific conditions

dmesg | tail -20          # kernel errors (OOM, disk errors, etc.)

journalctl -p err -n 50   # systemd error messages

ps aux | awk '$8=="D"' | wc -l  # D-state process count
```

The USE method (Utilization, Saturation, Errors) is the framework: for each resource (CPU, memory, disk, network), check utilization (how busy is it?), saturation (is there a queue?), and errors. A slow server is always one of these. vmstat's columns tell the story: r (runnable queue), b (blocked), wa (I/O wait), si/so (swap in/out). Swapping to disk is catastrophic for performance — si/so > 0 means memory is exhausted.

USE method in 60 seconds: This is the Brendan Gregg "60-second Linux performance analysis" checklist. In an interview, recite this flow — uptime → dmesg → vmstat → mpstat → pidstat → iostat → free → sar → top. It demonstrates structured thinking, not random command firing.

📚 **Official docs**

- Brendan Gregg: the USE Method

certs.eknathalabs.com

Q40 How do you investigate a server that was compromised — what do you look for?

Troubleshooting

🎙 Spoken answer · ~89 s

For a suspected compromise, I first decide on containment versus evidence: isolate the host from the network if possible, but avoid powering off or wiping, since volatile evidence like memory and network connections is lost on reboot. Ideally I capture a disk snapshot and memory image first. Then I reconstruct a timeline. Logins: last and lastb show successful and failed logins, the auth log and journal show SSH activity and sudo use. Audit logs show commands executed if auditd was configured. Then look for persistence and changes: files modified recently, new or changed users, especially UID zero or accounts with shells, new SSH authorized keys, cron jobs and systemd units or timers, modified startup files and shell profiles, suspicious SUID binaries, and changes to ld.so.preload. Check running processes and network connections for unfamiliar listeners or outbound connections to odd addresses, with ps, ss, and lsof; hidden processes and rootkits can fool local tools, so compare with an external view. Verify package integrity with rpm minus V or debsums. Then determine the entry point: an exposed service, weak credentials, or an application vulnerability. Recovery: rebuild from a known-good image rather than cleaning, rotate all credentials and keys the host had access to, patch the entry point, and write up the incident. Involve the security team.

Likely follow-ups

- Why avoid rebooting a suspected-compromised host?
- Why rebuild rather than clean the system?

```
# Timeline reconstruction:
```

```
last -F                       # login history with timestamps
```

```
lastb -F                      # failed logins
```

```
journalctl --since="2024-01-15" -p warning  # journal from incident time
```

```
ausearch -ts 01/15/2024 -te 01/16/2024 -m execve  # audit log commands
```

```
# Find recently modified files:
```

```
find / -mtime -7 -type f 2>/dev/null | grep -v /proc | sort
```

```
find /tmp /var/tmp /dev/shm -type f  # common malware drop locations
```

```
# Check for persistence mechanisms:
```

```
crontab -l; cat /etc/cron*
```

```
systemctl list-units --type=service | grep -v "loaded active"
```

certs.eknathalabs.com

```
cat /etc/rc.local; ls /etc/init.d/
```

```
# Running processes from unusual locations:
```

```
ls -la /proc/*/exe 2>/dev/null | grep -v '/usr|/bin|/lib|/sbin'
```

```
# Network connections:
```

```
ss -tnp                         # all TCP connections with PIDs
```

```
ss -tnp | grep -v "127.0.0.1|::1"  # external connections
```

Incident response order: (1) Isolate (remove from network if actively compromised), (2) Preserve evidence (snapshot disk image, copy logs), (3) Investigate (timeline, modified files, persistence mechanisms, network connections), (4) Remediate, (5) Harden. Look for: SUID binaries in unexpected locations, cron entries you don't recognize, services/processes from /tmp or /dev/shm, outbound connections to unusual IPs, and unauthorized SSH keys in ~/.ssh/authorized_keys.

Don't trust the running system: An attacker may have replaced system binaries (ls, ps, netstat) with rootkitted versions that hide their presence. Boot from a live CD or trusted external media for forensic investigation. Use immutable audit logs shipped to an external SIEM — logs on the compromised server may have been tampered with.

📚 **Official docs**

- NIST: Computer Security Incident Handling Guide (SP 800-61)

certs.eknathalabs.com

Q41 "No space left on device" error — but df shows space available. What are the possibilities?

Troubleshooting

🎙 Spoken answer · ~85 s

When writes fail with no space left on device but df shows free space, the first suspect is inode exhaustion: df minus i shows inode usage, and each file consumes one inode regardless of size, so millions of tiny files can exhaust inodes while bytes remain. To locate the culprit directory, I count files per directory. The second possibility: deleted files still held open by a process. The space is not freed until the process closes the file, and du won't see it, but lsof with plus L one lists them. Fix by restarting the process or truncating through its file descriptor. Third: a separate small filesystem such as tmpfs, with its own size limit, or a different mount than I'm looking at, like boot or var being full. Fourth: quotas, either user or project quotas, are exhausted. Fifth: reserved blocks, as ext4 reserves a percentage for root, so non-root users may hit the limit earlier. Sixth: other filesystem-level conditions such as XFS allocation-group fragmentation preventing large allocations, a Btrfs metadata exhaustion, or a read-only remount after errors. Also a container's overlay filesystem or ephemeral storage limit. I check each systematically, then fix the cause and add monitoring for both bytes and inodes.

Likely follow-ups

- How do you find files deleted but still held open?
- Why can non-root users run out of space earlier than root?

```
df -h                          # check disk space
```

```
df -i                          # check inode usage (THIS is often the culprit)
```

```
lsof +L1                       # deleted files still held open
```

```
ls -la /proc/*/fd 2>/dev/null | grep "(deleted)"  # same via /proc
```

```
# Inode exhaustion: find directory with most files
```

```
find / -xdev -printf '%h
' 2>/dev/null | sort | uniq -c | sort -rn | head -10
```

```
# tmpfs size limit:
```

```
df -h | grep tmpfs              # /run, /dev/shm may be full
```

```
# Kernel reserved space (5% by default on ext4):
```

```
tune2fs -l /dev/sda1 | grep "Reserved block"  # blocks reserved for root
```

```
tune2fs -m 1 /dev/sda1         # reduce reserved to 1% (on data disks)
```

certs.eknathalabs.com

Four causes: (1) Inode exhaustion — millions of tiny files consumed all inodes; df -i shows 100% inode usage. (2) Deleted-but-open files — process holds a file open, space not freed; lsof +L1 reveals it. (3) Reserved blocks — ext4 reserves 5% for root by default; a non-root process can't use this space even if df shows it. (4) tmpfs or /run full — these are RAM-backed filesystems with their own limits. This is a common interview question because it tests whether you understand the difference between space and inodes.

Fast inode cleanup: Finding and deleting millions of small files is slow with standard rm. Use `find . -delete` (faster than piping to rm) or `rsync --delete empty_dir/ target_dir/` to bulk-delete efficiently. For mail/session queues: check application documentation for purge commands that use internal indexes rather than filesystem walks.

📚 **Official docs**

- Linux kernel docs: ext4
- Linux man-pages: inode(7)

Q42 SSH connection hangs or is extremely slow to authenticate — how do you diagnose?

Troubleshooting

🎙 **Spoken answer · ~87 s**

For slow or hanging SSH, I begin with ssh minus vvv, whose verbose output shows each step and where the time goes. If it stalls before the banner, suspect network or DNS problems. If it stalls after the connection but before the password or key prompt, a common culprit on the server is reverse DNS lookup: with UseDNS enabled, sshd tries to resolve the client's address, and if DNS is slow it waits for a timeout. Another is GSSAPI authentication trying to contact a Kerberos server that isn't reachable; disabling GSSAPIAuthentication on the client or server fixes it. In the verbose output I see which keys are tried, since too many keys offered can hit MaxAuthTries before the right one, and which authentication methods the server accepts. On the server I read the sshd logs in the journal and check effective settings with sshd minus T. Other causes: PAM modules that do slow lookups, such as LDAP or sssd timeouts, a slow or full filesystem for home directory access, high load on the server, MOTD scripts that run slow commands, and network issues like MTU or packet loss, which show up after the key exchange. Fix the specific cause, and monitor SSH login time as a health check.

Likely follow-ups

- Why does UseDNS cause login delays?
- What does ssh -vvv tell you about authentication?

```
ssh -vvv user@host              # verbose mode (shows every auth step)
```

```
# Common culprits from verbose output:
```

```
debug1: Trying private key: ~/.ssh/id_rsa  # key being tried
```

certs.eknathalabs.com

```
debug1: Authentications that can continue  # what server accepts
```

```
# On the server side:
```

```
journalctl -u sshd -f           # sshd log in real time
```

```
sshd -T | grep -E "usedns|gssapi|challengeresponse"  # effective sshd config
```

```
# DNS reverse lookup (common culprit):
```

```
UseDNS no                       # in /etc/ssh/sshd_config — disables PTR lookup
```

```
# GSSAPI timeout:
```

```
GSSAPIAuthentication no         # disable if not using Kerberos
```

```
# Check /etc/hosts.allow / hosts.deny (TCP wrappers):
```

```
cat /etc/hosts.deny
```

```
# PAM timeout (LDAP/AD unreachable):
```

```
time ssh -o PasswordAuthentication=no user@host  # skip password auth
```

SSH slowness causes: UseDNS=yes causing PTR (reverse DNS) lookup for the client IP (slow or failing DNS = slow login), GSSAPIAuthentication=yes trying Kerberos negotiation, PAM module timing out (LDAP server unreachable), /etc/hosts.allow timeout for TCP wrappers, or many SSH keys in agent forcing the server to try each key (use IdentitiesOnly=yes in ~/.ssh/config). The -vvv flag shows exactly which step is slow — use it first.

> **Quick wins:** Adding `UseDNS no` and `GSSAPIAuthentication no` to sshd_config (plus systemctl reload sshd) instantly fixes the majority of slow SSH login issues. These are safe to disable on most systems not using Kerberos or relying on DNS-based access control.

> 📚 **Start here (official docs)**
>
> - Linux man-pages project

certs.eknathalabs.com

Q43 How do you handle a runaway process that is consuming all disk space with log writes?

Troubleshooting

🎙 Spoken answer · ~89 s

When a runaway process is filling the disk with logs, I act in a sequence that stops the damage without losing the process. First identify the file being written: lsof on the mount point sorted by size, or inotifywait on the log directory, and find which process owns it. For immediate relief without killing the process, truncate the file in place with truncate minus s zero or a redirect; this frees space right away and the process keeps its file handle. Don't just rm the file, because the process holds it open and the space isn't freed until it closes, as you'd see with lsof plus L one; if that already happened, truncate via slash proc slash pid slash fd. Then stop the source of the flood: find why it's logging so much, perhaps a crash loop, a debug level left on, or an error repeated millions of times, and fix it, or lower the log level, or restart the service after the fix. Then prevent a recurrence: configure logrotate with size limits and compression, or journald limits, use log rate limiting, ship logs off-host, put logs on a separate filesystem so they can't fill root, and alert on disk usage trends with a predicted fill time as well as on log volume.

Likely follow-ups

- Why is truncating better than rm for a file a process holds open?
- How do you stop it recurring?

```
# Identify the file being written:
```

```
lsof | grep -i "REG.*<mountpoint>" | sort -k7 -rn | head -10  # largest open files
```

```
inotifywait -m /var/log/        # watch directory for writes in real time
```

```
# Immediate relief without killing the process:
```

```
truncate -s 0 /var/log/runaway.log      # empty the file immediately
```

```
> /var/log/runaway.log               # bash redirect to empty
```

```
# Rate-limit log output (if using systemd):
```

```
# In unit file [Service] section:
```

```
StandardOutput=journal
```

```
LogRateLimitIntervalSec=30
```

```
LogRateLimitBurst=1000
```

```
# Redirect stdout to /dev/null temporarily:
```

certs.eknathalabs.com

```
gdb -p <pid> -batch -ex 'call close(1)' -ex 'call open("/dev/null",1)'
```

A runaway logger can fill disk in minutes. The truncate trick empties the file in-place without restarting the process — the process continues writing from offset 0. This gives immediate disk relief while you investigate the root cause. Common root causes: debug logging left on in production (check log level config), a loop generating errors that each trigger a log line, or a core dump being written repeatedly. Long-term: configure log rotation (logrotate), set max log size in the application config, and set disk usage alerts at 80%.

Journal filling disk: systemd-journald can fill disk if not limited. Set SystemMaxUse=2G in /etc/systemd/journald.conf. Check current usage with `journalctl --disk-usage`. Vacuum old entries with `journalctl --vacuum-size=1G`.

📚 **Official docs**

- systemd.service(5)
- journalctl(1)

Q44 How do you tune Linux kernel parameters for a Kubernetes node?

Performance

🎙 **Spoken answer · ~79 s**

Tuning a Kubernetes node starts with the required settings, then adjusts for scale based on evidence. Required: ip_forward enabled so the node routes pod traffic, and bridge-nf-call-iptables and its IPv6 counterpart set to one so iptables rules, used by kube-proxy and network policies, see bridged traffic; some CNI plugins also want rp_filter relaxed. Swap is traditionally disabled, although newer Kubernetes supports it with care. For high connection counts: raise somaxconn and tcp_max_syn_backlog for busy listeners, widen the ephemeral port range, and enlarge the conntrack table, because kube-proxy and NAT depend on connection tracking, and a full table drops packets. For memory and files: raise fs.inotify limits, since many pods and watchers can exhaust inotify, raise fs.file-max and the process limits, and set vm.max_map_count for workloads like Elasticsearch. Also keep kernel.pid_max high enough for many containers, and consider vm.swappiness low. Apply them in a sysctl drop-in file through node bootstrap or configuration management, or for namespaced sysctls through the pod's securityContext. The key discipline is to change defaults only for a measured reason, test on one node pool, document the change, and monitor conntrack usage, file handles, and network drops.

Likely follow-ups

- Why does Kubernetes need bridge-nf-call-iptables?
- Which sysctls matter most for a node with many pods?

certs.eknathalabs.com

```
# Required for Kubernetes networking:
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.conf.all.rp_filter = 0    # for some CNI plugins
# High connection count (for services with many pods):
net.core.somaxconn = 32768
net.ipv4.tcp_max_syn_backlog = 32768
net.ipv4.ip_local_port_range = 1024 65535
net.ipv4.tcp_tw_reuse = 1
# inotify for Kubernetes controllers:
fs.inotify.max_user_watches = 524288
fs.inotify.max_user_instances = 512
# File descriptor limits:
fs.file-max = 2097152
# Disable swap (K8s requires this or kubelet --fail-swap-on=false):
swapoff -a; sed -i '/swap/d' /etc/fstab
vm.swappiness = 0
```

Kubernetes nodes require specific kernel settings. The bridge-nf-call-iptables settings ensure iptables sees bridged traffic — required for pod-to-pod networking with most CNI plugins. Without ip_forward, routing between pods is impossible. Kubernetes strongly recommends disabling swap — memory limits on containers become unreliable when swap is available (a container hitting its limit swaps instead of being OOM-killed). Store these in /etc/sysctl.d/99-kubernetes.conf for persistence across reboots.

> **Node preparation playbook:** Always version-control your node preparation steps (Ansible, Terraform user_data). Any K8s node joining the cluster should have identical kernel parameters. Drift between nodes causes subtle, hard-to-diagnose issues where pods behave differently depending on which node they land on.

certs.eknathalabs.com

📚 **Official docs**

- Linux kernel docs: sysctl

Q45 What is a Linux namespace — how do containers use them for isolation?

Security

🎙 **Spoken answer · ~83 s**

A Linux namespace isolates one kind of global system resource, so processes inside see their own private view of it. Containers are essentially processes in a set of namespaces plus cgroups. The main types: mount gives a separate filesystem tree, so the container sees its own root; PID gives separate process IDs, so the container's first process is PID one inside, though it has a different PID on the host; network gives separate interfaces, routing tables, firewall rules, and ports; IPC isolates shared memory and message queues; UTS isolates hostname; user maps user IDs, so root inside can be an unprivileged user outside; and cgroup namespaces virtualise the cgroup view. I can inspect them with ls on slash proc slash self slash ns and lsns, and enter them with nsenter, which is handy for debugging a container from the host. Important point: namespaces provide isolation of visibility, not resource limits, which cgroups handle, and not complete security, since containers share the host kernel, unlike VMs. So hardening still needs seccomp, capabilities, and read-only filesystems. In Kubernetes, pods share a network namespace among their containers, and options like hostNetwork or hostPID deliberately break isolation and should be avoided unless required.

Likely follow-ups

- How is a namespace different from a cgroup?
- Why do containers share less isolation than VMs?

```
ls -la /proc/self/ns/          # namespaces for current process
```

```
lsns                          # list all namespaces on the system
```

```
lsns -t net                   # only network namespaces
```

```
# Namespace types:
```

```
mnt  — filesystem mounts (each container has own mount tree)
```

```
pid  — process IDs (container PID 1 maps to host PID N)
```

```
net  — network interfaces, routes, iptables rules
```

```
ipc  — inter-process communication (shared memory, semaphores)
```

```
uts  — hostname and domain name
```

certs.eknathalabs.com

```
user — UID/GID mappings (rootless containers)
```

```
cgroup — cgroup root view
```

```
# Enter a container's namespace:
```

```
nsenter -t <container-pid> -n  # enter network namespace
```

```
nsenter -t <container-pid> --all  # enter all namespaces (like exec)
```

Linux namespaces are what make containers possible — each container gets isolated views of processes, networking, filesystem, hostname, and IPC. cgroups (control groups) complement namespaces: namespaces provide isolation (what you can see), cgroups provide resource limits (how much you can use). Together they form the foundation of container technology. nsenter is invaluable for debugging — it lets you enter a container's network namespace to run network diagnostic tools that aren't in the container image.

Debug without exec: If a container doesn't have debugging tools (distroless images), use `nsenter -t $(docker inspect -f '{{.State.Pid}}' <container>) -n -- ss -tlnp` to run ss inside the container's network namespace from the host. No exec into container needed — you use the host's tools in the container's namespace.

📚 **Official docs**

- Linux man-pages: namespaces(7)

certs.eknathalabs.com

Q46 What are cgroups — how do they enforce resource limits for containers and Kubernetes pods?

Security

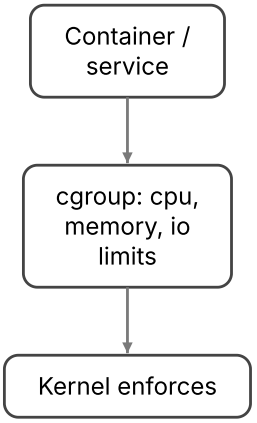
🎙 Spoken answer · ~84 s

Cgroups, control groups, are the kernel mechanism that limits and accounts for resources used by groups of processes: CPU, memory, I/O, and the number of processes. Namespaces control what a process can see; cgroups control how much it can use. Containers and Kubernetes pods use them to enforce limits. For memory, a limit means that when the group exceeds it and the kernel can't reclaim, the OOM killer kills a process in the group, which appears as OOMKilled. For CPU there are two controls: shares or weight, which decide proportional division when there's contention and correspond to Kubernetes requests, and the quota with period, which is a hard cap and corresponds to limits; exceeding the quota causes throttling, which hurts latency even when the node has idle CPU. I/O controllers limit disk bandwidth, and the pids controller caps process count to stop fork bombs. I inspect them under slash sys slash fs slash cgroup, where cgroup v2 has a unified hierarchy, with files such as memory.max, cpu.max, and cpu.stat that shows throttling counts. Kubernetes sets pod and container cgroups and uses QoS classes, Guaranteed, Burstable, and BestEffort, to decide eviction and OOM priority. Good practice: set requests and limits from measured usage.

Likely follow-ups

- What is the difference between CPU requests and limits at the cgroup level?
- How do you see whether a container is being throttled?

🗺 cgroups




```
cat /proc/self/cgroup             # cgroup membership of current process
```

```
ls /sys/fs/cgroup/                # cgroup v1 hierarchy
```

```
ls /sys/fs/cgroup/memory/         # memory controller
```

```
# For a container:
```

certs.eknathalabs.com

```
cat /sys/fs/cgroup/memory/docker/<id>/memory.limit_in_bytes
```

```
cat /sys/fs/cgroup/cpu/docker/<id>/cpu.cfs_quota_us  # CPU quota
```

```
# Kubernetes pod cgroup:
```

```
cat /sys/fs/cgroup/kubepods/pod<uid>/memory.limit_in_bytes
```

```
# Check cgroup version:
```

```
stat -fc %T /sys/fs/cgroup/        # tmpfs=v1, cgroup2fs=v2
```

```
mount | grep cgroup                # cgroup mounts
```

cgroups (control groups) are the kernel mechanism for resource accounting and limiting. For each container, the runtime creates cgroup entries setting: memory.limit_in_bytes (OOM kill threshold), cpu.cfs_quota_us / cpu.cfs_period_us (CPU limit as a fraction — 100ms/100ms = 1 full CPU), blkio.throttle.read/write_bps_device (disk I/O limits). Kubernetes translates pod resource limits directly into cgroup settings. Understanding this explains why a container behaves differently from a VM: cgroup limits are enforced in kernel, not in the application.

cgroup v1 vs v2: cgroup v2 (unified hierarchy) is default on modern kernels (5.4+, used by GKE COS nodes). Some older tools only support v1. Kubernetes fully supports v2. The memory accounting differs between v1 and v2 — cached memory counts differently, affecting OOM behavior. Verify your monitoring tools support v2 if your nodes run modern kernels.

📚 **Official docs**

- Linux man-pages: cgroups(7)

certs.eknathalabs.com

Q47 How do you use journalctl effectively — what are the most useful query patterns?

Boot & systemd

🎙 Spoken answer · ~85 s

journalctl reads the systemd journal, and a few query patterns cover most needs. For a service, minus u with the unit name, and add minus f to follow live. For time ranges, since and until accept absolute times or relative ones like one hour ago. Minus b shows the current boot, and minus b minus one the previous boot, which is invaluable after a crash or unexpected reboot. Minus p filters by priority, such as err and above, and minus k shows kernel messages, the equivalent of dmesg, which is where I find OOM kills and disk errors. I can filter by fields, for example by process ID, executable, or user, and use output formats: json for scripts, short-iso for timestamps, and cat for just the message. Combine with grep for text patterns, or use minus g for built-in pattern matching. Useful housekeeping: journalctl disk-usage and vacuum options to control size, and configuring persistent storage in journald so logs survive reboots, since the default on some systems is volatile. For fleets I ship the journal to a central logging system, because local logs are lost if the host dies. And when troubleshooting a failing unit, systemctl status plus journalctl minus xe gives the context quickly.

Likely follow-ups

- How do you look at logs from before the last reboot?
- How do you make journal storage persistent?

```
journalctl -f                          # follow live (like tail -f syslog)
```

```
journalctl -u nginx                    # specific service
```

```
journalctl -u nginx --since "1 hour ago"
```

```
journalctl -u nginx --since "2024-01-15 10:00" --until "2024-01-15 11:00"
```

```
journalctl -b                          # current boot
```

```
journalctl -b -1                       # previous boot (crash investigation)
```

```
journalctl -p err                      # error priority and above
```

```
journalctl -p warning -n 50            # last 50 warnings+
```

```
journalctl -k                          # kernel messages only (dmesg equivalent)
```

```
journalctl _PID=1234                   # logs from specific PID
```

```
journalctl _UID=1000                   # logs from specific UID
```

```
journalctl -o json-pretty -u sshd -n 5  # JSON output for parsing
```

certs.eknathalabs.com

```
journalctl --disk-usage              # journal size on disk
```

```
journalctl --vacuum-size=500M        # trim journal to 500MB
```

journald is the central logging daemon on systemd systems — it collects stdout/stderr from all services, kernel messages, and systemd unit events. Unlike syslog, entries have structured fields (unit, PID, UID, priority) enabling precise filtering. The -b -1 flag is invaluable for crash investigations — logs from the last boot before a crash. Priority levels: 0=emerg, 1=alert, 2=crit, 3=err, 4=warning, 5=notice, 6=info, 7=debug.

Persistent journal: By default, journal is stored in /run/log/journal (lost on reboot). To persist across reboots: `mkdir /var/log/journal && systemctl restart systemd-journald`. Or set Storage=persistent in /etc/systemd/journald.conf. Essential for post-reboot debugging.

📚 **Official docs**

- journalctl(1)

Q48 How do you write Ansible-style idempotent automation versus imperative scripts?

Scripting & automation

🎙 **Spoken answer · ~87 s**

Idempotent automation means running it once or ten times leaves the system in the same desired state, with no errors or duplicate changes. Imperative scripts describe steps, so a second run might fail, for example useradd failing because the user exists, or mkdir because the directory exists, or worse, append a line to a config twice. Declarative tools like Ansible describe the desired state and check the current state before acting, reporting changed or ok. In shell, I get idempotency by guarding actions: create the user only if id fails, use mkdir minus p, use install or ln minus sfn, replace a config using a template with a comparison instead of appending, and use grep to check before adding a line. Idempotency matters because it makes reruns safe after partial failures, allows configuration to be applied continuously to fix drift, makes automation testable, and enables convergence at scale. Also consider check mode and dry runs to preview changes, and handlers so services restart only when config actually changed. Not everything is naturally idempotent, such as running a database migration or sending an email, so those need tracking of state, like a migrations table. For complex logic with many conditions, use a proper configuration management tool, not a hand-rolled script.

Likely follow-ups

- How do you make a shell script idempotent?
- Why do handlers matter for service restarts?

```
# Imperative (not idempotent — fails if run twice):
```

```
useradd deployer        # fails with "user already exists" on second run
```

certs.eknathalabs.com

```
mkdir /app/data         # fails with "file exists" on second run
```

```
# Idempotent (safe to run multiple times):
```

```
id deployer || useradd deployer          # only create if not exists
```

```
mkdir -p /app/data                       # -p = no error if exists
```

```
install -m 644 -o root config.yaml /etc/app/  # sets perms atomically
```

```
grep -q "setting=value" /etc/app.conf || echo "setting=value" >> /etc/app.conf
```

```
# Better: use Ansible tasks which are inherently idempotent:
```

```
- name: Create deployer user
```

```
  ansible.builtin.user:
```

```
    name: deployer
```

```
    state: present            # present = ensure exists (not create)
```

```
    groups: docker,sudo
```

```
    append: yes
```

Idempotency means running an operation multiple times produces the same result as running it once. This is critical for automation: deployment scripts run in CI/CD, Ansible playbooks run for compliance enforcement, and provisioning scripts may be re-run after failures. All Ansible modules are designed to be idempotent — they check current state before making changes and report changed/ok/skipped. In bash, implement idempotency with conditional checks before every modifying operation.

Platform Engineer standard: All infrastructure automation should be idempotent. Playbooks should be runnable against production at any time to enforce desired state without fear of side effects. This is the foundation of GitOps thinking applied to OS-level configuration.

📚 Official docs

- RFC 9110: HTTP Semantics (status codes, methods)
- Ansible documentation

certs.eknathalabs.com

Q49 What is the difference between CPU throttling and CPU stealing — how do you detect each?

Performance

🎙 Spoken answer · ~85 s

CPU throttling and CPU steal both make a workload slow with CPU that looks available, but they have different causes and different fixes. Throttling happens inside the container platform: a cgroup CPU quota limits how much CPU time the group can use per period, typically a hundred milliseconds, and when it's exhausted the processes are paused until the next period. It shows up as latency spikes even when average usage is below the limit. I detect it from the cgroup cpu.stat counters, the number of throttled periods and throttled time, or the Prometheus metric for CFS throttled seconds. The fix is to raise or remove the CPU limit, tune thread counts, or adjust requests. CPU steal happens on virtual machines: the hypervisor didn't schedule my virtual CPU because the physical host is oversubscribed or a noisy neighbour is using it. I see it as the st column in top or vmstat, or in mpstat. The fix is outside the guest: move to a different host or instance type, use dedicated or larger instances, or avoid burstable instance types that exhaust their credits. A key difference: throttling is something I configured, so I can fix it in my own manifests, while steal is the provider's capacity.

Likely follow-ups

- How do you detect CFS throttling in Kubernetes?
- What can you do about high steal time?

```
# CPU throttling (cgroup CPU limits exceeded):
```

```
cat /sys/fs/cgroup/cpu/<container>/cpu.stat | grep throttled
```

```
# Kubernetes: check via metrics
```

```
container_cpu_cfs_throttled_seconds_total  # Prometheus metric
```

```
kubectl top pod <pod> -n <ns>  # current CPU usage vs limit
```

```
# CPU stealing (hypervisor not scheduling our vCPU):
```

```
top                            # %st column = steal percentage
```

```
vmstat 1 5                     # st column
```

```
sar -u 1 5 | grep -i steal     # sar history of steal
```

CPU throttling: the container's cgroup CPU quota is exhausted — the kernel pauses the container until the next quota period (default 100ms). Even if the physical host has idle CPUs, a throttled container waits. Fix: increase the container's CPU limit or optimize the application. CPU stealing: the hypervisor is not scheduling

certs.eknathalabs.com

our VM's vCPU because physical CPUs are overcommitted. The VM is ready to run but waits for a physical CPU. Fix: scale down the physical host, move to a less loaded host, or buy dedicated/reserved instances. Throttling is under your control; stealing is the cloud provider's problem.

Prometheus alert for throttling: `rate(container_cpu_cfs_throttled_seconds_total[5m]) / rate(container_cpu_cfs_periods_total[5m]) > 0.25` — alert when more than 25% of CPU periods are throttled. This is a better SLI than CPU utilization% for container workloads.

📚 **Official docs**

- Kubernetes: Resource management for Pods & containers
- Linux kernel docs: CFS scheduler

Q50 Walk through a complete methodology for diagnosing a production Linux incident from alert to resolution.

Troubleshooting

🎙 **Spoken answer · ~87 s**

My methodology for a production Linux incident is structured, fast, and communicative. Phase one, triage in the first two minutes: acknowledge the alert, state that I'm on it, and run a handful of commands for system-wide health, uptime for load, free for memory, df for disk, ss for sockets, dmesg for kernel errors, and list failed systemd units. Assess impact: who is affected and is it getting worse? Phase two, mitigate before diagnosing deeply if there's a known safe action such as rolling back a recent change, restarting a crashed service, failing over, or shedding load; stopping the bleeding comes first. Phase three, diagnose with the USE method: for each resource, CPU, memory, disk, network, check utilisation, saturation, and errors, then go to the offending process and use tools like perf, strace, and lsof, and correlate with recent changes and logs. Form a hypothesis, test it, and avoid changing multiple things at once. Phase four, resolve and verify with metrics, not just that the command ran. Throughout, communicate: update a channel at regular intervals with what is known, what's being tried, and next steps, and keep a timeline. Phase five, follow up: a blameless postmortem with root cause, contributing factors, and tracked action items for detection and prevention.

Likely follow-ups

- What is the USE method?
- Why mitigate before finding the root cause?

This is the Principal-level systems thinking question. Structure, speed, and communication matter as much as technical depth.

INCIDENT RESPONSE METHODOLOGY

```
Phase 1 — Triage (first 2 minutes):
```

```
  uptime; free -h; df -h; ss -s  # four commands, system-wide health
```

certs.eknathalabs.com

```
  dmesg | tail -20              # any kernel errors?
```

```
  systemctl list-units --failed  # any failed services?
```

```
Phase 2 — Identify bottleneck (next 5 minutes):
```

```
  vmstat 1 5           # r/b columns: CPU queue or I/O blocked?
```

```
  top -b -n 1          # which process is the culprit?
```

```
  iostat -xz 1 3       # which disk is saturated?
```

```
  iftop                # which connection is saturating network?
```

```
Phase 3 — Confirm hypothesis:
```

```
  strace -c -p <pid> sleep 5  # what syscalls is the process making?
```

```
  lsof -p <pid>              # what files/sockets are open?
```

```
  journalctl -u <service> --since "10 min ago"  # service logs
```

```
Phase 4 — Immediate relief + root cause fix:
```

```
  # Immediate: kill/restart/scale the problematic component
```

```
  # Root cause: fix config, add resource, fix code
```

```
Phase 5 — Validate + document:
```

```
  # Confirm metrics return to baseline
```

```
  # Write postmortem: timeline, root cause, fix, prevention
```

certs.eknathalabs.com

Common patterns by symptom: high load + high I/O wait + D-state processes = storage bottleneck (check iostat await). High load + low I/O wait + high %sy = iptables/networking overhead (many rules, high connection rate). High load + low I/O wait + high %us = CPU-bound application (profile with perf). Memory exhausted + swap in use = OOM risk imminent (free up memory or add swap). Filesystem full + lsof shows deleted files = truncate open files. SSH not working but ping works = service issue, firewall, or PAM problem.

Principal-level deliverable: After every incident: write a postmortem (5 whys root cause analysis), add the missing alert that would have caught this earlier, and improve the runbook. Convert reactive incidents into proactive prevention. The best Platform Engineers make their own on-call rotation quieter over time — not because they silence alerts, but because they fix the underlying issues permanently.

📚 **Official docs**

- Brendan Gregg: Linux performance

Q51 Explain process states (R, S, D, Z, T). Why can't you kill a process in D state?

Processes

🎙 **Spoken answer · ~78 s**

Linux process states appear in the STAT column of ps. R means running or runnable, either on a CPU or in the run queue. S is interruptible sleep, waiting for an event like network input, and can be woken by a signal. D is uninterruptible sleep, where the process is inside a kernel call, usually waiting on disk or network storage I/O, and it can't be interrupted by signals. Z is zombie, finished but not yet reaped by its parent. T is stopped, by SIGSTOP, job control, or being traced by a debugger. You can't kill a process in D state because signals are only acted on when the process returns to user mode or reaches an interruptible point; while it waits in the kernel on I/O, the signal stays pending, even SIGKILL. Forcing it could corrupt kernel state or data. The way out is to resolve what it waits for: restore the NFS server or network, fix the failing disk, or in the worst case reboot. Processes in D state also count in the load average, which is why high load with idle CPU suggests I/O stalls.

Likely follow-ups

- Why do D-state processes inflate the load average?
- How do you find what a D-state process is waiting on?

THE STATES (COLUMN STAT IN PS)

```
R  Running / runnable (on CPU or in run queue)
S  Interruptible sleep (waiting for an event, e.g. network, can be woken)
D  Uninterruptible sleep (in a kernel syscall, usually disk/NFS I/O)
Z  Zombie (finished, waiting for parent to reap exit status)
T  Stopped (SIGSTOP / job control, or being traced)
# suffixes: < high-pri, N low-pri (nice), s session leader, + foreground, l multithreaded
```

WHY D CAN'T BE KILLED

```
# A process in D is blocked inside the kernel and is NOT accepting signals
#   (not even SIGKILL) until the syscall returns. kill -9 is queued but
#   won't act until the I/O completes — or never, if the device/NFS hangs.
ps -eo pid,stat,wchan,cmd | awk '$2 ~ /D/'   # wchan shows what it's waiting on
```

Real cause: lots of D-state processes = stuck I/O — a dead NFS mount, a failing disk, or saturated storage. The fix is at the I/O layer (recover the mount/device), not `kill -9`. Persistent D-state often needs the underlying device fixed or a reboot.

Interview tip: The headline: D = uninterruptible kernel sleep, immune to signals including SIGKILL, almost always disk/NFS I/O. Zombies, by contrast, are already dead — you can't kill them; you kill or fix the *parent* so it reaps them. Reading `wchan` to see what a D process waits on shows real depth.

certs.eknathalabs.com

📚 **Official docs**

- Linux man-pages: fork(2)/wait(2) (zombies)

Q52 **What is a zombie process and what is an orphan? How do you deal with each?** Processes

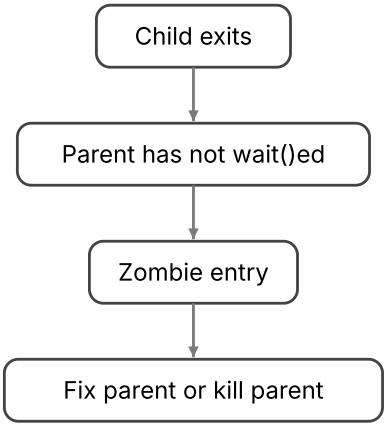
🎙 **Spoken answer · ~88 s**

A zombie is a child that has exited, but whose parent hasn't called wait to collect the exit status. The entry stays in the process table as defunct, holding just a PID and the status. It uses no CPU or memory. You can't kill it, since it's already dead; you deal with the parent: send SIGCHLD to prompt it to reap, or fix or restart the parent, at which point init adopts the zombies and reaps them. The root cause is a parent bug where it doesn't wait on children. An orphan is the opposite situation: a child that is still running, but whose parent has exited. The kernel re-parents orphans to init, PID one, or to a subreaper, which reaps them when they eventually finish. So orphans are normal and usually harmless; they become relevant if they keep running unintentionally, such as leftover worker processes after a crashed supervisor, which I find by looking for processes whose parent is one and which should not exist, then terminate them. In containers, the main process often isn't a proper init, so adopted orphans may later become zombies that nothing reaps; the fix is to run a small init like tini. Monitor the zombie count, since too many exhaust the PID space.

Likely follow-ups

- What happens to a zombie when its parent exits?
- What is a subreaper?

🗺 **Zombie process**



ZOMBIE (DEFUNCT)

certs.eknathalabs.com

```
# Child has exited but the parent hasn't called wait() to read its exit code.
# The entry stays in the process table (just a PID + exit status) -> <defunct>.
ps aux | grep -w Z          # or STAT shows Z
# You CAN'T kill a zombie (already dead). Fix the PARENT:
#   - signal the parent to reap (SIGCHLD), or
#   - kill/restart the parent -> children get reparented to init/PID 1, which reaps them.
```

ORPHAN

```
# Parent died before the child. The child is "orphaned" and immediately
#   reparented to PID 1 (init/systemd), which becomes responsible for reaping it.
# Orphans are normal and harmless — init handles them.
```

When zombies pile up: a buggy parent that forks but never `wait()`s. A few zombies are harmless (tiny), but thousands can exhaust the PID space. In containers, PID 1 is often your app, not init — if it doesn't reap, zombies accumulate. Use an init shim (`tini`, `--init`) so PID 1 reaps.

Interview tip: Crisp distinction: zombie = child dead, parent didn't reap (fix the parent); orphan = parent dead, child reparented to init (normal). The container angle — PID 1 must reap, use `tini`/`docker --init` — is a strong modern signal that separates you from a textbook answer.

📚 **Official docs**

- Linux man-pages: fork(2)/wait(2) (zombies)

certs.eknathalabs.com

Q53 Explain nice, renice, and ionice. How do you deprioritize a backup job without killing it?

Processes

🎙 Spoken answer · ~83 s

nice, renice, and ionice let me deprioritise a job without killing it. nice sets CPU scheduling priority: values from minus twenty to nineteen, where higher means nicer, meaning lower priority. I start a backup with nice minus n nineteen so it only uses CPU that others don't need, and I can change a running process with renice by PID, or for all processes of a user. Only root can lower the niceness value, i.e. raise priority. But CPU priority doesn't help when the contention is on disk, which is typical for backups, so I use ionice: class three, idle, gives the process disk time only when nobody else wants it, while class two, best effort, has priority levels zero to seven. Combining them, nice plus ionice, makes a backup unobtrusive. Caveats: ionice works with certain I/O schedulers, and not all, such as with no scheduler on fast NVMe devices; priority doesn't limit total consumption, so for hard limits use cgroups through systemd, with CPUQuota, CPUWeight, and IOWeight. The best approach is to run the job via a systemd service or timer with these settings, schedule it for off-peak hours, and monitor to confirm it isn't affecting latency for production traffic.

Likely follow-ups

- Why doesn't nice help when a backup is I/O bound?
- When would you use cgroup limits instead of nice?

CPU PRIORITY: NICE / RENICE

```
nice -n 19 ./backup.sh        # start with lowest CPU priority (range -20..19)
renice -n 10 -p 12345         # change priority of a running PID (raise = lower pri)
renice -n 5 -u alice          # all of a user's processes
# Only root can LOWER niceness (raise priority, negative values).
```

I/O PRIORITY: IONICE

```
ionice -c 3 -p 12345         # class 3 = idle (only gets disk when nothing else wants it)
ionice -c 2 -n 7 ./backup.sh # best-effort, lowest priority within it
# Classes: 1=realtime, 2=best-effort(default), 3=idle
```

DEPRIORITIZE A BACKUP SAFELY

```
# Backups are usually I/O-bound, so ionice matters more than nice:
ionice -c 3 nice -n 19 ./backup.sh
# CPU-idle AND disk-idle -> backup runs only with spare capacity, never killed.
# Or via systemd: CPUWeight=, IOWeight=, or Nice= in the unit.
```

Interview tip: The key insight: `nice` is CPU only — for an I/O-heavy job like a backup, `ionice -c3` (idle I/O class) is what actually keeps it out of the way. Combining both, or using systemd's `IOWeight`/`CPUWeight` with cgroups, is the production-grade answer (resource control without killing the job).

certs.eknathalabs.com

📚 **Start here (official docs)**

- Linux man-pages project

Q54 What are signals? Explain SIGTERM vs SIGKILL vs SIGHUP and graceful shutdown.

Processes

🎙 **Spoken answer · ~83 s**

Signals are asynchronous notifications to a process. SIGTERM, fifteen, is the polite request to terminate; it can be caught, so the application can finish in-flight work, close connections, and flush data, and it's the default for kill. SIGKILL, nine, is forceful: it can't be caught or ignored and gives no chance to clean up, so it risks lost data and leftover files; it's the last resort. SIGHUP, one, originally meant the terminal hung up, but many daemons use it to reload their configuration without restarting. Graceful shutdown means handling SIGTERM correctly: stop accepting new work, finish or hand off in-flight requests, close resources, and exit within a deadline. Orchestrators implement this pattern: Kubernetes sends SIGTERM, waits for the termination grace period, then sends SIGKILL. systemd does the same with a stop timeout. For it to work, the application must actually handle SIGTERM; in containers the entrypoint shouldn't be a shell wrapper that swallows the signal, so use exec, and PID one has special signal handling that requires explicit handlers. I design services to drain connections, using readiness to stop traffic first, perhaps with a preStop delay, and I test shutdown behaviour under load to confirm there are no dropped requests.

Likely follow-ups

- Why doesn't a shell-script entrypoint always forward SIGTERM?
- What does a graceful shutdown sequence look like?

COMMON SIGNALS

```
SIGTERM (15) : polite "please shut down" — catchable, app can clean up. DEFAULT of kill.
SIGKILL (9)  : forced kill — NOT catchable, NOT ignorable, no cleanup. Last resort.
SIGINT  (2)  : Ctrl-C (catchable)
SIGHUP  (1)  : terminal hangup — often repurposed to "reload config" (nginx, etc.)
SIGSTOP/SIGCONT : pause / resume (SIGSTOP also uncatchable)
SIGCHLD : sent to parent when a child exits (used for reaping)
```

GRACEFUL SHUTDOWN PATTERN

```
# Send SIGTERM first, give it time, THEN SIGKILL if still alive:
kill -TERM 12345; sleep 10; kill -KILL 12345 2>/dev/null
# This is exactly what systemd (TimeoutStopSec) and Kubernetes
#   (terminationGracePeriodSeconds) do: SIGTERM -> grace period -> SIGKILL.
```

Why -9 is harmful as a default: SIGKILL gives the app zero chance to flush buffers, finish in-flight requests, release locks, or remove pid/lock files — risking corruption and orphaned resources. Always try SIGTERM first.

certs.eknathalabs.com

Interview tip: Stress that SIGTERM is catchable (clean shutdown) and SIGKILL/SIGSTOP are not (no cleanup), so the correct pattern is TERM → wait → KILL. Tying it to systemd `TimeoutStopSec` and K8s `terminationGracePeriodSeconds` shows you understand graceful shutdown the same way orchestrators implement it. SIGHUP-as-reload is a nice bonus fact.

📚 **Official docs**

- Kubernetes: Pod lifecycle & termination
- Linux man-pages: signal(7)

Q55 How do processes communicate (IPC)? Briefly cover pipes, sockets, shared memory, signals.

Processes

🎙 **Spoken answer · ~83 s**

Linux offers several inter-process communication mechanisms with different trade-offs. Pipes are unidirectional byte streams between related processes, as in shell pipelines. Named pipes, FIFOs, created with mkfifo, give a pipe a filesystem name so unrelated processes can use it. Unix domain sockets are bidirectional, local-only, and fast because they bypass the TCP stack; Docker, databases, and php-fpm commonly use them, and they allow permission control via file permissions and can pass file descriptors and credentials. TCP and UDP sockets are network IPC, usable locally or across hosts. Shared memory is the fastest, since processes map the same memory region, but requires synchronisation with semaphores or mutexes. Message queues provide discrete messages with priorities. Signals are small asynchronous notifications, good for events like reload or terminate, not for data. Files and memory-mapped files are also used. How I choose: for simple streaming between related processes, pipes; for a local service API, Unix sockets, which are faster than loopback TCP; for high-throughput data sharing, shared memory with care; for cross-host, TCP. Operationally, I inspect with ss for sockets, ipcs for System V IPC, and lsof. Security matters: socket file permissions and namespace boundaries, since containers can share IPC namespaces only deliberately.

Likely follow-ups

- Why use a Unix socket rather than TCP on localhost?
- Why does shared memory need synchronisation?

certs.eknathalabs.com

THE MECHANISMS

```
Pipes (|)        : unidirectional byte stream between related processes (shell pipelines)
Named pipes/FIFO : mkfifo — pipe with a filesystem name, unrelated processes
Unix sockets     : bidirectional, local-only, fast (no TCP stack) — /var/run/*.sock
                   (Docker, databases, php-fpm use these)
TCP/UDP sockets  : network IPC (also works locally via loopback)
Shared memory    : fastest — processes map the same memory (SysV shm, POSIX, /dev/shm)
Signals          : tiny async notifications (no data payload beyond the signal number)
Message queues / semaphores : structured messaging + synchronization
```

CHOOSING

```
# Speed vs convenience:
shared memory  -> fastest, but you manage synchronization yourself
unix socket    -> fast local, bidirectional, easy (most local services)
pipe           -> simple stream, related processes
network socket -> when processes are on different hosts
```

Interview tip: Know that Unix domain sockets are the workhorse for local service IPC (Docker daemon, DBs, fpm) — faster than TCP loopback because they skip the network stack. Shared memory is fastest but needs your own locking. Signals carry no data (just a number). Matching mechanism to need (local vs remote, stream vs structured, speed vs simplicity) is what's being tested.

📚 **Start here (official docs)**

- Linux man-pages project

Q56 How do you debug a process: strace, ltrace, /proc/, lsof? Give a workflow.

Processes

🎙 **Spoken answer · ~85 s**

My workflow for debugging a misbehaving process goes from least to most intrusive. First, inspect without attaching, via the proc filesystem: the open file descriptors, the status file for state, threads and memory, the limits file for the effective ulimits, the working directory and executable links, and the environment. Those answer many questions at no risk: is it near its file descriptor limit, is it stuck in D state, what config did it start with? Second, lsof, to see files, sockets, and deleted-but-open files it holds. Third, strace to trace system calls: attach with minus p, filter by category, use timing to find slow calls, and a summary count to see where it spends time; this shows what it's blocked on, which files fail to open, or which connections fail. ltrace traces library calls, useful for dynamically linked programs. Fourth, for CPU hotspots, perf; for memory, heap profilers; for stuck threads, a thread dump or gdb backtrace. Cautions: strace can slow a process substantially, so keep traces short and filtered on production, and lower-overhead tools like perf or eBPF are better for busy services. Also check permission restrictions, such as ptrace limits in containers. After finding the cause, confirm by reproducing and verifying the fix.

Likely follow-ups

- Why start with /proc before attaching strace?
- What are the risks of running strace on production?

INSPECT WITHOUT ATTACHING

```
ls -l /proc/12345/fd        # open file descriptors (files, sockets, pipes)
cat /proc/12345/status      # state, threads, memory, uid
cat /proc/12345/limits      # ulimits actually in effect
readlink /proc/12345/cwd /proc/12345/exe   # working dir + binary
cat /proc/12345/environ | tr '\0' '\n'      # its environment
```

TRACE ACTIVITY

cours.eknathalabs.com

```
strace -p 12345 -f          # trace syscalls (what files/network it touches)
strace -e trace=open,connect -p 12345   # filter to specific calls
ltrace -p 12345             # trace library calls (higher level)
lsof -p 12345               # everything the process has open
lsof -i :8080               # who's using a port
```

WORKFLOW FOR "PROCESS IS STUCK"

```
1. ps stat -> R/D/S/Z? (D = stuck I/O)
2. strace -p PID -> is it blocked on a syscall (read/futex/connect)?
3. /proc/PID/fd + lsof -> what file/socket is it blocked on?
4. cat /proc/PID/stack (if permitted) -> kernel stack for D-state
```

Caution: `strace` significantly slows the target (every syscall is trapped) — use briefly on production, and prefer `-e trace=` filters. For low-overhead production tracing, reach for eBPF tools (bpftrace, `perf trace`).

Interview tip: Show a layered method: cheap first (`/proc`, `lsof`, `ps`), then `strace` to see the blocking syscall. The standout point is that strace adds heavy overhead, so on production you'd use bpftrace/eBPF instead — that distinguishes someone who's actually traced a live prod process from someone reciting commands.

📚 **Official docs**

- Linux man-pages: proc(5)
- Linux man-pages: ptrace(2) (strace)

certs.eknathalabs.com

Q57 Explain fork, exec, and copy-on-write. Why is fork() cheap on Linux?

Processes

🎙 Spoken answer · ~83 s

fork creates a child process that is a copy of the parent, with a new PID, and exec replaces the current process image with a new program. Together they are how every command starts: the shell forks, the child execs the command, and the parent waits. Fork is cheap on Linux because of copy-on-write. The kernel doesn't actually copy the parent's memory at fork; it duplicates the page tables and marks the pages as shared and read-only. Only when either process writes to a page does the kernel make a private copy of that page. So a fork followed immediately by exec copies almost nothing, since the child replaces its address space anyway. This also explains behaviours such as Redis background saves using fork: the child sees a consistent snapshot of memory, and memory use grows only as the parent modifies pages, which under heavy writes can nearly double memory and trigger the OOM killer. Related: vfork and posix_spawn are optimisations for the fork-exec pattern, and clone with flags is the generalisation used for threads and containers. In operations, large processes can see fork latency due to page-table copying, and overcommit settings affect whether fork succeeds for big processes.

Likely follow-ups

- Why can Redis memory spike during a background save?
- What does exec do to the process?

FORK + EXEC

```
fork()  : creates a child that is a COPY of the parent (same code, new PID)
exec()  : REPLACES the current process image with a new program
# The classic pattern: shell forks, child execs the command, parent waits().
# That's how every command you run from a shell starts.
```

COPY-ON-WRITE (COW)

```
# fork() doesn't actually copy the parent's memory up front.
# Parent + child SHARE the same physical pages, marked read-only.
# A page is only physically copied when one of them WRITES to it.
# -> fork is cheap even for large processes; you pay only for what changes.
```

WHY IT MATTERS

COW makes forking a multi-GB process fast and memory-light — until it writes. This is why a big app (e.g. Redis) can `fork()` to snapshot: the child sees a frozen copy, and only changed pages get duplicated. But heavy writes after fork cause a memory spike as pages get copied.

Interview tip: The key fact: fork is cheap because of copy-on-write — memory is shared read-only and only duplicated on write. Name the fork+exec pattern (how shells launch programs) and the Redis fork-to-snapshot example. The gotcha worth mentioning: a fork followed by heavy writes triggers mass page-copying and a RAM spike (relevant to Redis BGSAVE OOMs).

certs.eknathalabs.com

📚 **Official docs**

- Docker: Storage drivers (overlayfs)

Q58 What is the OOM killer? How does it decide what to kill and how do you influence it?

Processes

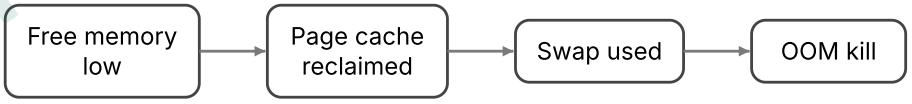
🎙 **Spoken answer · ~86 s**

The OOM killer is the kernel's last resort when memory runs out and it can't reclaim enough: it selects a process to kill so the system stays alive. I see its work in dmesg or the kernel log, with lines saying killed process, plus a dump of memory state and a process table. It picks victims by oom_score, which is mostly proportional to the process's memory use, adjusted by oom_score_adj, a value from minus a thousand to a thousand that I can set per process. Setting minus a thousand exempts a process, though exempting too much can leave the kernel nothing to kill; raising the value makes a process a preferred victim. In containers, a cgroup memory limit triggers a local OOM kill within that group, without affecting the host, which shows as OOMKilled in Kubernetes with exit code 137. To influence it: set oom_score_adj through systemd's OOMScoreAdjust for critical services, set sensible memory limits so a single workload is killed rather than the host, and in Kubernetes use QoS classes so BestEffort pods die first. Better still is avoiding OOM: right-size memory, find leaks, limit caches, monitor available memory and pressure stall information, and consider userspace killers like systemd-oomd or earlyoom for earlier, more graceful action.

Likely follow-ups

- What does exit code 137 indicate?
- How do QoS classes affect OOM kill order in Kubernetes?

🗺 **OOM decision**



certs.eknathalabs.com

WHAT IT DOES

```
# When the kernel runs out of memory (and can't reclaim), the OOM killer
#   picks a process to kill to free RAM and keep the system alive.
dmesg | grep -i "killed process"        # see what got OOM-killed and why
grep -i oom /var/log/syslog             # OOM events
```

HOW IT CHOOSES

```
# Scores each process by oom_score (mostly proportional to memory used),
#   adjusted by oom_score_adj (-1000 .. +1000). Highest score dies.
cat /proc/12345/oom_score
echo -1000 > /proc/12345/oom_score_adj   # -1000 = never kill (protect it)
echo  500  > /proc/12345/oom_score_adj   # more likely to be the victim
```

Gotcha: the OOM killer often kills the *biggest* process, which may not be the culprit — it can take down your database instead of the leaking worker. In cgroup v2 / containers, memory limits cause a *cgroup-local* OOM kill (only that container's processes), which is usually what you want.

Interview tip: Explain the score is memory-driven and tunable via `oom_score_adj` (protect critical services with -1000). The senior points: prefer cgroup memory limits so an OOM is contained to the offending container instead of the host picking a random big victim, and disabling overcommit or adding swap changes when/whether OOM triggers. Reading `dmesg` for the kill is the first debugging step.

📚 **Official docs**

- Linux kernel docs: /proc/sys/vm

Q59 Explain inodes. What happens when you run out of inodes but still have free space?

Filesystem

🎙 **Spoken answer · ~85 s**

An inode stores a file's metadata: permissions, owner, timestamps, size, and pointers to the data blocks, but not the filename. The directory entry maps a name to an inode number, which is why hard links work and why renaming is cheap. Commands: ls minus i shows the inode number, stat shows the metadata, and df minus i shows inode usage per filesystem. On ext4 the number of inodes is fixed when the filesystem is created, so if you create millions of tiny files, you can run out of inodes while plenty of space remains. Symptoms: writes fail with no space left on device, even though df shows free space. You confirm with df minus i showing one hundred percent inode use. The remedy is to find the directories with huge file counts, such as caches, session files, mail queues, or temp files, and delete or archive them, noting that deleting millions of files needs care, using find with delete or rsync from an empty directory. Longer term: fix the application producing those files, add cleanup jobs, restructure storage, or recreate the filesystem with more inodes, using mkfs options for bytes-per-inode; or use XFS or Btrfs, which allocate inodes dynamically. And monitor inode usage alongside space.

Likely follow-ups

- Why does a directory with millions of tiny files cause problems?
- How do you choose inode density at filesystem creation?

WHAT AN INODE IS

```
# An inode stores a file's METADATA (permissions, owner, timestamps, size,
#   and pointers to data blocks) — but NOT the filename. The directory entry
#   maps name -> inode number.
ls -i file        # show inode number
stat file         # full inode metadata
df -i             # inode usage per filesystem (vs df -h for space)
```

OUT OF INODES, SPACE FREE

certs.eknathalabs.com

```
# inodes are allocated at mkfs time (fixed count on ext4). Millions of TINY
#   files exhaust inodes while bytes remain free -> "No space left on device"
#   even though df -h shows free space. The tell: df -i at 100%.
# Find the culprit directory (lots of small files):
for d in /var/*; do echo "$(find "$d" 2>/dev/null | wc -l) $d"; done | sort -rn | head
```

Classic cause: a runaway mail/session/cache dir, or millions of tiny log/temp files. The error message ("No space left") is misleading — always check `df -i`, not just `df -h`. ext4's inode count is fixed at format time; XFS allocates inodes dynamically and rarely hits this.

Interview tip: The signature interview trap: "disk has free space but writes fail with ENOSPC." Answer: inode exhaustion — `df -i` confirms it. Inodes hold metadata + block pointers, not the name. Mention that ext4 fixes inode count at `mkfs` while XFS is dynamic — explaining why this bug is an ext4 phenomenon shows filesystem depth.

📚 **Official docs**

- Linux man-pages: inode(7)

Q60 Hard links vs symbolic links: how do they differ and when do you use each? Filesystem

🎙 **Spoken answer · ~89 s**

A hard link is a second name for the same inode, and a symbolic link is a small file containing a path. With a hard link, both names are equal and point at the same data; the data survives until the link count reaches zero and no process has it open. Hard links can't cross filesystems because inode numbers are local to a filesystem, and normally can't link directories. A symlink points by path, so it can cross filesystems and point to directories, but it breaks if the target is moved or deleted, and it resolves relative to its own location unless absolute. When I use each: symlinks are the common choice for everyday needs, such as current pointing to a versioned release directory, so a deployment can switch atomically by replacing the symlink, for libraries versions, and for configuration enabling, like sites-enabled in nginx. Hard links are used for deduplicating backups, for example rsync with link-dest creating snapshot directories where unchanged files share data, and for keeping a file alive under another name. Gotchas: editors that save by writing a new file break hard-link relationships; symlink targets resolve at access time; and be careful with symlinks in world-writable directories and following links during recursive operations, which can cause security and deletion surprises.

Likely follow-ups

- How do symlinks enable atomic deploy switches?
- Why can editors break hard links?

THE DIFFERENCE

coris.eknathalabs.com

```
ln  target link        # HARD link: another name pointing to the SAME inode
ln -s target link      # SYMlink: a small file containing a PATH to the target
# Hard link  -> same inode, same data. Deleting one name leaves the data
#               until the link count hits 0. Can't cross filesystems or link dirs.
# Symlink    -> points to a path. If the target is moved/deleted -> dangling link.
#               Can cross filesystems and point to directories.
```

OBSERVING

```
ls -li file            # 2nd column = link count; same inode # = hard linked
stat file              # "Links:" shows hard-link count
ls -l symlink          # shows  link -> /path/to/target
find / -inum 1234567   # find all hard links sharing an inode
```

WHEN TO USE

Symlinks for almost everything (cross-fs, point to dirs, human-readable, common in `/etc/alternatives`, current-release pointers). Hard links for space-efficient deduplication on one filesystem (backup snapshots like rsync `--link-dest`, where unchanged files share inodes).

Interview tip: Core facts: hard link = same inode (data persists until link count 0, same-fs only, not dirs); symlink = a path pointer (can dangle, crosses filesystems, points to dirs). The practical kicker: rsync/backup tools use hard links (`--link-dest`) for dedup, and deleting a file with multiple hard links doesn't free space until the last one goes.

📚 **Official docs**

- Linux man-pages: inode(7)

certs.eknathalabs.com

Q61 A file was deleted but disk space wasn't freed. Why, and how do you recover the space? Filesystem

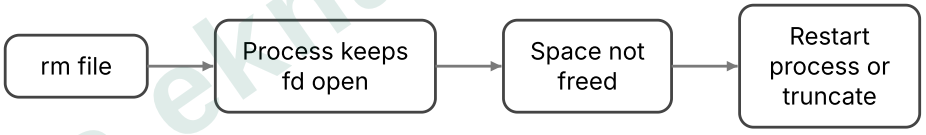
🎙 Spoken answer · ~87 s

On Linux, a file's space is freed only when two conditions are met: all directory entries pointing to it are removed, and no process still has it open. If a process holds a deleted file open, the inode and blocks persist, which shows as deleted entries in lsof or in the process's file descriptor directory. The classic case is a log file removed while the application keeps writing to it; du doesn't count it, but df does, creating the mismatch. To find it, lsof with plus L one lists open files with zero links. To recover the space without downtime, truncate the file through the process's file descriptor under proc, for example by redirecting nothing into slash proc slash pid slash fd slash number, which empties it while the process keeps writing. Or signal the process to reopen its logs, with SIGHUP for many daemons, or restart it if that's acceptable. Prevention: don't rm active log files; truncate instead, or use logrotate with the right post-rotate action, such as copytruncate, or signalling the service to reopen files. Also remember that the same mechanism explains why a deleted file can sometimes be recovered from proc while the process is alive, which can be a useful trick after an accidental deletion.

Likely follow-ups

- How can you recover an accidentally deleted file that a process still has open?
- Why does du disagree with df in this situation?

🗺 Deleted file still holds space



WHY SPACE ISN'T FREED

```
# On Linux, space is freed only when BOTH are true:
#   1. all directory entries (names) are removed, AND
#   2. no process still has the file OPEN.
# If a process holds the deleted file open, the inode + blocks persist
#   (you'll see "(deleted)" entries). Classic: a log file rm'd while the
#   app keeps writing to its open fd -> disk fills, df shows full, du doesn't.
```

FIND AND FIX

```
lsof +L1                        # files with link count 0 still held open
lsof | grep deleted             # show "(deleted)" files + the holding PID
# Recover space WITHOUT killing the app — truncate via the proc fd:
: > /proc/<pid>/fd/<fd-number>   # empties the still-open deleted file
# Or, if acceptable, restart/HUP the process to release the fd.
```

The tell: `df` shows the filesystem full but `du` of its contents is much smaller — the difference is space held by deleted-but-open files. This is extremely common with rotated logs when logrotate doesn't

certs.eknathalabs.com

signal the app to reopen.

Interview tip: This is a top-tier troubleshooting question. The mechanism: an open fd to a deleted file keeps its blocks allocated. Diagnose with `lsof +L1` / `lsof | grep deleted`; the `df`-vs-`du` mismatch is the fingerprint. Recover by truncating the proc fd or restarting the holder — and prevent it with proper logrotate `copytruncate` or a post-rotate reload signal.

📚 **Official docs**

- Linux man-pages: ptrace(2) (strace)

Q62 Explain mount, /etc/fstab, and bind mounts. How do you safely add a new disk mount?

Filesystem

🎙 **Spoken answer · ~85 s**

Mounting attaches a filesystem to a directory in the tree. mount with a device and mountpoint attaches it, umount detaches it and fails if busy, in which case lsof or fuser shows who is using it, and findmnt displays the tree clearly. A bind mount makes a directory or file visible at another path, used heavily by containers to expose host directories. Persistent mounts go in etc fstab with device, mountpoint, filesystem type, options, dump, and pass fields. To safely add a new disk: identify it with lsblk, double-check that it's the right, empty device so I don't overwrite data, partition it, create a filesystem, create the mountpoint, and get the UUID with blkid. Add an fstab entry using the UUID rather than a device name, which can change between boots, with suitable options, and nofail for non-critical data disks so a missing device doesn't block boot. Then test without rebooting using mount minus a, or findmnt verify, which validates the syntax and the entries, check with df, and set ownership and permissions. A bad fstab can leave a server unbootable, so I keep console access available and test carefully. Also consider noexec and nosuid for data volumes, and regular monitoring of space.

Likely follow-ups

- Why use UUIDs in fstab?
- How do you find what is keeping a mount busy?

MOUNTING

```
mount /dev/sdb1 /data           # attach a filesystem at a mountpoint
umount /data                    # detach (fails if busy; lsof /data to find users)
findmnt                         # tree of all mounts (clearer than mount)
mount --bind /src /dst          # bind mount: make a dir appear at another path
```

PERSISTENT MOUNTS: /ETC/FSTAB

```
# <device>  <mountpoint>  <fstype>  <options>  <dump> <pass>
UUID=abc-123  /data  ext4  defaults,noatime  0  2
# Use UUID (or LABEL), NOT /dev/sdb1 — device names reorder across reboots!
blkid /dev/sdb1                 # get the UUID
```

certs.eknathalabs.com

SAFELY ADD A DISK

```
1. mkfs.ext4 /dev/sdb1; blkid to get UUID
2. Add the UUID line to /etc/fstab
3. mount -a            # mount everything in fstab NOW — catches typos
4. Verify: findmnt /data, df -h
# CRITICAL: test mount -a before rebooting. A bad fstab line can drop the
#   box to emergency mode on boot. (Use 'nofail' for non-critical mounts.)
```

Interview tip: Two safety points score here: (1) reference disks by UUID/LABEL, not `/dev/sdX` (which can reorder), and (2) always run `mount -a` to validate fstab *before* rebooting, since a bad entry can hang boot in emergency mode — add `nofail` for optional mounts. Knowing bind mounts (and that they're how containers share host dirs) is a nice extra.

📚 **Official docs**

- Linux man-pages: mount_namespaces(7)
- Docker: Volumes

Q63 **What's the difference between ext4, XFS, and Btrfs/ZFS? When would you pick each?** Filesystem

🎙 **Spoken answer · ~80 s**

Choosing a filesystem depends on workload. ext4 is the mature, reliable default: journaling, well understood, a fixed number of inodes, and it can be shrunk as well as grown. It's the safe, boring choice for general purpose. XFS excels at large files and parallel I/O, scales to very large filesystems, allocates inodes dynamically, and is the default on RHEL; it can grow but not shrink, and repair needs the filesystem unmounted. Btrfs is copy-on-write with built-in snapshots, checksums, compression, and subvolumes, which is attractive for snapshots and data integrity, though some features such as RAID5 or 6 are considered less mature. ZFS offers pooled storage, strong end-to-end checksums, snapshots, replication, and compression, but is out of tree on Linux, with licensing and memory considerations, and is excellent for storage servers and backups. How I choose: ext4 for boot and simple needs; XFS for databases, media, and big data volumes where throughput and scale matter; Btrfs or ZFS when snapshots, checksumming, and send and receive replication are valuable. Operational factors also matter: the team's familiarity, tooling support, recovery procedures, and the cloud provider's recommendations. Whatever the choice, test with realistic workloads using fio.

Likely follow-ups

- Why can't XFS be shrunk?
- When would ZFS be worth the extra complexity?

THE MAIN FILESYSTEMS

certs.eknathalabs.com

```
ext4  : mature, reliable default. Fixed inode count, journaling. Shrinkable.
        Great general-purpose; the safe boring choice.
XFS   : high-performance, great for large files + parallel I/O, dynamic inodes.
        Default on RHEL. Can GROW but NOT shrink. Scales to huge filesystems.
Btrfs : CoW, snapshots, checksums, built-in RAID, subvolumes. More features,
        historically less trusted for some RAID modes.
ZFS   : CoW, snapshots, checksums, compression, RAID-Z, self-healing. Excellent
        data integrity; heavy RAM use; licensing keeps it out of mainline kernel.
```

CHOOSING

```
General server / root fs   -> ext4 (or XFS on RHEL)
Big data, large files, DBs -> XFS (parallel I/O, scales)
Need snapshots/checksums/  -> ZFS (integrity) or Btrfs (in-kernel)
  data integrity, NAS
Need to shrink a fs        -> ext4 (XFS can't shrink)
```

Interview tip: Have a one-line opinion on each: ext4 = safe default, XFS = performance/scale (RHEL default, grow-only), ZFS/Btrfs = snapshots + checksums + integrity (CoW). The most-tested specifics: XFS can grow but not shrink, ext4 has fixed inodes, and CoW filesystems give you snapshots and end-to-end checksums (self-healing) at the cost of complexity/RAM.

📚 **Official docs**

- Linux kernel docs: ext4
- Linux kernel docs: XFS

certs.eknathalabs.com

Q64 Explain file permissions deeply: SUID, SGID, sticky bit, and umask.

Filesystem

🎙 Spoken answer · ~86 s

Beyond basic read, write, and execute, there are special bits and the umask. SUID, shown as an s in the owner execute position, makes an executable run with the owner's privileges, like passwd running as root to update the shadow file; it is powerful and risky, so I audit for unexpected SUID binaries, especially root-owned ones. SGID on an executable runs with the file's group; on a directory it makes new files inherit the directory's group, perfect for shared team folders. The sticky bit on a directory restricts deletion: users can delete only their own files even if the directory is world-writable, as in tmp. Octal prefix digits are four, two, and one for these bits. The umask defines which permission bits are removed from newly created files and directories: with a umask of zero two two, new files get six four four and directories seven five five; with zero zero seven, files are private to owner and group. It's set per shell, in profile files, in systemd units with UMask, or in PAM. Together these control default security. Best practices: least privilege, avoid world-writable files, use groups and ACLs for sharing, set the umask appropriately for services, and scan for risky bits as part of hardening.

Likely follow-ups

- What does a umask of 022 produce for new files?
- How do you audit a system for SUID binaries?

BEYOND RWX: SPECIAL BITS

```
SUID (4xxx)  : run the file AS THE OWNER (e.g. /usr/bin/passwd runs as root)
               shows as 's' in owner-exec: -rwsr-xr-x
SGID (2xxx)  : on a file -> run as the group; on a DIR -> new files inherit
               the dir's group (great for shared team folders)
Sticky (1xxx): on a dir -> only the file's owner can delete it (e.g. /tmp)
               shows as 't': drwxrwxrwt
chmod u+s / g+s / +t file_or_dir
```

UMASK

```
# umask SUBTRACTS permissions from defaults (666 files, 777 dirs):
umask 022    -> new files 644, new dirs 755   (typical)
umask 027    -> new files 640, new dirs 750   (more private; group read only)
umask        # show current
```

Security: SUID-root binaries are a major attack surface — a bug in one = root. Audit them: `find / -perm -4000 -type f 2>/dev/null`. Modern systems replace many SUID binaries with capabilities (finer-grained). Never set SUID on shell scripts.

Interview tip: Be precise: SUID = run as owner (the `passwd` example), SGID on a directory = group inheritance (shared folders), sticky bit = owner-only delete (why anyone can write to `/tmp` but not delete others' files). umask *subtracts* from defaults. The security-aware close — audit SUID binaries, prefer capabilities — is what senior interviewers want to hear.

certs.eknathalabs.com

📚 **Official docs**

- Linux man-pages: path_resolution(7)

Q65 How do you find what's filling a disk, and handle 'No space left on device'? Filesystem

🎙 **Spoken answer · ~85 s**

When a disk is full, I find the big consumers methodically. First df shows which filesystem is full, and df minus i checks whether it's actually inodes. Then du with the one-filesystem flag and limited depth, sorted by size, drills down directory by directory; ncdu gives an interactive explorer that's much faster to navigate; and find with a size filter locates individual large files. The deleted-but-open trap: if df shows more usage than du explains, a process is holding deleted files; lsof with plus L one finds them, and I free the space by truncating through proc or restarting the process. Typical culprits are logs, core dumps, old container images and layers, package caches, temp files, database data or WAL files, and backups. Quick relief: rotate or compress logs, clear package and container caches such as docker system prune with care, remove old kernels and core dumps, and journalctl vacuum. Then the fix: log rotation with limits, retention policies, separate filesystems for var and data so one can't fill the root, monitoring with alerts on predicted fill time, and capacity planning. If it's an emergency on production, free a small amount safely first so the service can recover, then investigate. Always double-check before deleting.

Likely follow-ups

- What is the 'deleted but open' trap?
- Why put logs on a separate filesystem?

FIND THE BIG CONSUMERS

```
df -h                          # which filesystem is full?
df -i                          # OR is it inodes? (millions of tiny files)
du -xh /var --max-depth=1 | sort -rh | head   # -x stays on one filesystem
ncdu /var                      # interactive disk usage explorer (fast)
find / -xdev -size +500M -exec ls -lh {} \;   # individual large files
```

THE DELETED-BUT-OPEN TRAP

```
# If du is much smaller than df-used, space is held by open deleted files:
lsof +L1                       # or: lsof | grep deleted
: > /proc/<pid>/fd/<n>         # truncate to reclaim without killing the app
```

COMMON CULPRITS + QUICK WINS

```
journalctl --disk-usage; journalctl --vacuum-size=200M   # systemd journal bloat
docker system df; docker system prune                    # images/volumes/build cache
/var/log (rotate), /tmp, old kernels, package caches (apt clean / dnf clean)
```

Interview tip: Show a decision tree: `df -h` (space) vs `df -i` (inodes) vs `df`-beats-`du` (deleted-open files) — three different "disk full" root causes. Then locate with `du -x`/`ncdu`. Naming the usual offenders

certs.eknathalabs.com

(journal, Docker, logs, old kernels) and reclaiming them shows you've actually firefought a full root filesystem.

📚 **Start here (official docs)**

- Linux man-pages project

Q66 Explain the Linux page cache, dirty pages, and how writes are flushed to disk.

Filesystem

🎙 **Spoken answer · ~88 s**

Linux caches file data in free RAM in the page cache, so reads are served from memory instead of disk. That's why free shows little free memory but a lot of buff and cache: it is reclaimable, not wasted, and the number that matters is available. Writes go first to the page cache and mark the pages dirty; a background writeback mechanism flushes them to disk later, controlled by thresholds such as dirty_background_ratio, where background flushing begins, and dirty_ratio, where processes generating writes are forced to block and write synchronously. There are also time limits like dirty_expire and the writeback interval. This makes writes fast, but means data in the cache can be lost on a crash or power failure unless it has been flushed; applications needing durability call fsync or fdatasync, or open with sync flags, and databases do this for their logs. Operational implications: large dirty buffers can cause latency stalls when the threshold is hit and a flush storm happens, which can be mitigated by tuning the ratios lower on servers with lots of RAM and slow disks. Drop-caches is for testing only, not a fix. Monitoring dirty and writeback counters in meminfo helps diagnose stalls. Also memory-mapped files and O_DIRECT bypass or use the cache differently.

Likely follow-ups

- Why is free memory a misleading metric on Linux?
- What happens when dirty_ratio is reached?

certs.eknathalabs.com

PAGE CACHE

```
# Linux caches file data in free RAM (page cache). Reads hit RAM, not disk.
# This is why 'free -h' shows little "free" but lots of "buff/cache" —
#   that memory is reclaimable, NOT wasted. available is the number that matters.
free -h          # look at "available", not "free"
```

DIRTY PAGES + WRITEBACK

```
# A WRITE first goes to the page cache (marked "dirty"), then is flushed
#   to disk asynchronously by kernel writeback threads -> writes feel fast.
cat /proc/meminfo | grep -i dirty           # Dirty: pending bytes
sync                                         # force flush all dirty pages now
# Tunables: vm.dirty_ratio / vm.dirty_background_ratio control when flush starts.
```

Durability gap: because writes are buffered, data in the page cache is lost on power failure unless `fsync()` has committed it. Databases call `fsync`/use `O_DIRECT` for durability. A sudden `dirty_ratio`-

triggered flush can also cause an I/O stall ("write storm").

Interview tip: Two things land well: (1) `buff/cache` in `free` is reclaimable, so "low free memory" is usually fine — watch `available` instead; and (2) writes are buffered as dirty pages and flushed asynchronously, which is why `fsync` exists and why a power loss can lose un-synced data. Mentioning `vm.dirty_ratio` tuning to avoid write-storm stalls is a strong performance signal.

📚 **Official docs**

- Linux kernel docs: /proc/sys/vm

Q67 The modern net-tools vs iproute2: what replaced ifconfig, netstat, route, arp?

Networking

🎙 **Spoken answer · ~83 s**

The older net-tools suite has been replaced by iproute2, which is more capable and actively maintained. ifconfig is replaced by ip addr and ip link; route by ip route; arp by ip neigh; and netstat by ss, which is faster because it reads socket information directly from the kernel instead of parsing proc, and handles large numbers of sockets better. netstat statistics map to ip minus s link and nstat. Everyday commands: ip a shows all interfaces and addresses; ip route get with a destination shows which route, interface, and source IP would be used, very helpful for debugging asymmetric routing; ss minus tlnp lists listening TCP sockets with the owning process; ss with state filters shows connection states and counts, for example finding TIME_WAIT buildup; and ip neigh shows the ARP table. Newer additions include ip rule for policy routing, ip netns for namespaces, and bridge commands. The old tools may not be installed on minimal images and don't support newer features like multiple addresses per interface properly. In an interview, the point is showing current practice: using ss and ip, understanding their output, and knowing that these tools are essential for container and Kubernetes network debugging.

Likely follow-ups

- Why is ss preferred over netstat?
- What does ip route get tell you?

OLD -> NEW

```
ifconfig        -> ip addr / ip link
route -n        -> ip route
netstat -tlnp   -> ss -tlnp          (ss is faster, reads /proc directly)
netstat -i      -> ip -s link
arp -a          -> ip neigh
netstat -rn     -> ip route show
```

THE EVERYDAY COMMANDS

certs.eknathalabs.com

```
ip a                        # all interfaces + addresses
ip route get 8.8.8.8        # which route/interface/source IP is used for a dest
ip -s link show eth0        # interface stats (errors, drops)
ss -tlnp                    # listening TCP sockets + owning process
ss -tn state established    # established connections
ss -s                       # socket summary stats
```

Why it matters: `net-tools` (ifconfig/netstat) is deprecated and often not installed on minimal/container images. `ss` is dramatically faster than `netstat` on busy hosts (netstat parses `/proc` slowly). Knowing iproute2 is now table stakes.

Interview tip: Just demonstrate fluency in the modern tools — `ip a`, `ip route get`, `ss -tlnp`, `ip neigh`. The single best one-liner to know is `ip route get <dest>`, which shows exactly which route, interface, and source IP the kernel will use — invaluable for routing/asymmetric-path debugging. Note that ifconfig/netstat are deprecated and may be absent on containers.

📚 **Official docs**

- Linux man-pages: ip(7)
- RFC 826: ARP

Q68 How do you debug 'can't connect to a service'? Give a layered checklist.

Networking

🎙 **Spoken answer · ~86 s**

For can't connect to a service, I work up the stack in order, each step isolating one layer. First DNS: does the name resolve, and to the right IP, using getent or dig. Second routing: is there a path, with ip route get, and the right interface and source address. Third reachability with ping, remembering that ICMP may be blocked, so a failed ping isn't conclusive. Fourth the port: test a TCP connection with nc or curl, which separates refused, meaning the host is reachable but nothing listens or a firewall rejects, from timeout, meaning packets are being dropped. Fifth, on the server, is anything listening and on which address, using ss; a service bound to localhost only is a common culprit. Sixth the firewalls at every level: host rules with iptables or nftables, cloud security groups and network ACLs, and any intermediate firewall. Seventh TLS, if applicable, with openssl s_client for certificate and handshake problems. Eighth the application itself: its logs, health, and whether it is overloaded. If the cause remains unclear, tcpdump on both sides shows where packets stop. I also compare with a working client to find what differs. This ordering is quick, avoids guesswork, and tells me which team owns the fix.

Likely follow-ups

- What is the difference between connection refused and a timeout?
- What would a service bound to 127.0.0.1 look like in ss output?

WORK UP THE STACK

certs.eknathalabs.com

```
1. DNS:     getent hosts host   /  dig host        # name resolves? right IP?
2. Route:   ip route get <ip>                       # is there a path? right iface?
3. Reach:   ping <ip>                               # L3 up? (ICMP may be blocked)
4. Port:    nc -vz host 443  /  curl -v telnet://host:443   # is the port open?
5. Listen:  (on server) ss -tlnp | grep :443        # is anything bound? on 0.0.0.0?
6. Firewall: iptables/nftables, cloud SG/NACL, host ufw/firewalld
7. App:     curl -v https://host/health             # does the app actually answer?
```

READ THE FAILURE MODE

```
Connection REFUSED  -> reached host, nothing listening on that port (or RST)
Connection TIMEOUT  -> firewall dropping / wrong route / host down (no reply)
No route to host    -> routing/ARP problem, or firewall REJECT with icmp
Name resolution err -> DNS, before any connection is attempted
```

Interview tip: Lead with the layered method (DNS → route → reach → port → listener → firewall → app) and especially the **refused vs timeout** distinction: refused = got there, nothing listening (fast); timeout = packets dropped silently (firewall/down). A frequent gotcha to mention: the service is listening on `127.0.0.1` instead of `0.0.0.0`, so it works locally but refuses remotely.

📚 **Start here (official docs)**

- Linux man-pages project

Q69 Explain iptables vs nftables and the netfilter chains. How does a packet traverse them?

Networking

🎙 **Spoken answer · ~83 s**

Packets traverse netfilter hooks in a defined order. For incoming packets, they hit PREROUTING first, where destination NAT happens, then the routing decision: if the packet is for this host it goes to INPUT; if it's to be forwarded it goes through FORWARD and then POSTROUTING, where source NAT or masquerade is applied. Locally generated packets go through OUTPUT, then POSTROUTING. Tables attach rules to these hooks by purpose: filter for allow and deny, nat for address translation, mangle for packet modification, and raw for exemptions from connection tracking. iptables is the classic interface, with rules matched in order and a default policy per chain. nftables is its successor: a single framework replacing iptables, ip6tables, ebtables, and arptables, with a cleaner syntax, sets and maps for efficient matching of many addresses, atomic rule-set updates, and better performance. Many distributions now run iptables as a compatibility layer on top of nftables. For troubleshooting, I list rules with counters to see which match, check the chain order, and use logging or trace to follow a packet. Kubernetes and Docker add their own chains, so I avoid flushing blindly. The key concept is understanding where in the path a rule acts.

Likely follow-ups

- Where does DNAT happen relative to the routing decision?
- What advantages does nftables have over iptables?

certs.eknathalabs.com

NETFILTER CHAINS (THE HOOKS)

```
PREROUTING  -> (routing decision) -> INPUT    (packets FOR this host)
                                  -> FORWARD  (packets THROUGH this host) -> POSTROUTING
locally generated           -> OUTPUT -> POSTROUTING
# Tables apply at these hooks: filter (allow/deny), nat (SNAT/DNAT), mangle, raw.
```

IPTABLES EXAMPLE

```
iptables -L -n -v --line-numbers       # list filter rules + counters
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
iptables -A INPUT -j DROP              # default-deny the rest
# Rules are evaluated top-to-bottom; FIRST match wins. Order matters!
```

NFTABLES (THE REPLACEMENT)

```
# nft unifies iptables/ip6tables/arptables/ebtables into one framework,
#   with sets, maps, and better performance. Modern distros use nft underneath.
nft list ruleset
nft add rule inet filter input tcp dport 22 accept
```

Interview tip: Draw the path: PREROUTING → routing → INPUT (to us) or FORWARD (through us) → POSTROUTING. Stress first-match-wins and that a misplaced DROP above your ACCEPT silently blocks traffic. Note nftables is the modern replacement (sets/maps, one tool for v4+v6) and that even `iptables` commands now often drive the nft backend. Knowing where NAT happens (PRE/POSTROUTING) ties into the SNAT/DNAT questions.

📚 **Official docs**

- nftables wiki
- Netfilter documentation

certs.eknathalabs.com

Q70 How do you capture and analyze traffic with tcpdump? Give a practical workflow.

Networking

🎙 Spoken answer · ~87 s

My tcpdump workflow is: define the question, capture narrowly, and analyse. First decide what I'm testing: does traffic arrive, is the handshake completing, are there resets or retransmissions, are DNS queries answered? Then capture with a BPF filter on the right interface, using minus nn to avoid name lookups: filter by host and port, or by flags, such as SYNs and RSTs only. For intermittent problems, write to a file with rotation, a size limit and a file count, so I don't fill the disk, and use a full snap length when payload matters. Then read the capture back with tcpdump or open it in Wireshark or tshark for deeper analysis, like follow stream, retransmission analysis, TCP window sizes, and timing between request and response. Capturing on both client and server tells me where packets get lost or delayed. Reading patterns: SYN with no SYN-ACK means blocked or dropped; SYN then RST means nothing listening; many retransmissions mean packet loss; zero window means the receiver is slow; long gaps between request and response mean the server is slow. Security and cost: captures may contain sensitive data, so follow policy, delete them after use, and be aware of capture overhead on busy hosts. In Kubernetes, use an ephemeral debug container.

Likely follow-ups

- What does SYN followed by RST indicate?
- How do you capture an intermittent problem without filling the disk?

CAPTURE

```
tcpdump -i eth0 -nn host 10.0.0.5 and port 443    # filter by host+port (-nn no DNS)
tcpdump -i any -nn 'tcp[tcpflags] & (tcp-syn|tcp-rst) != 0'  # SYNs and RSTs
tcpdump -i eth0 -w cap.pcap -C 100 -W 5           # write to file, rotate 100MB x5
tcpdump -r cap.pcap -nn                            # read back a capture
# -s0 captures full packets; add -v for more header detail.
```

WHAT THE FLAGS TELL YOU

```
SYN sent, no SYN-ACK back   -> firewall drop / nothing listening / wrong route
SYN -> RST                  -> port closed (refused)
Retransmissions             -> packet loss / congestion
FIN/RST mid-stream          -> who closed first (server vs client)?
```

WORKFLOW

Capture on **both ends** when possible: if the client sends SYN but the server never sees it, the drop is in between (firewall/route); if the server replies but the client doesn't get it, the return path is broken. Move the capture to Wireshark for TCP stream analysis.

Caution on prod: capture with tight filters and rotation (`-C/-W`) — an unfiltered capture on a busy host can fill the disk and add load. Be mindful that payloads may contain sensitive data.

certs.eknathalabs.com

Interview tip: The pro move is capturing on both ends to localize where packets vanish (sender-side vs path vs return path). Know the flag-reading shorthand: SYN with no reply = dropped/no listener; SYN->RST = refused; retransmits = loss. Mention tight filters + file rotation so you don't take down a prod box with the capture itself.

📚 **Official docs**

- pcap-filter(7)
- tcpdump manual

Q71 Explain network namespaces and veth pairs. How does container networking actually work?

Networking

🎙 **Spoken answer · ~85 s**

A network namespace is an isolated network stack with its own interfaces, routing table, ARP table, firewall rules, and ports; it is the primitive that containers use for network isolation. A veth pair is a virtual cable with two ends: whatever enters one end comes out the other. To connect a namespace to the outside, one end of a veth pair is placed in the container's namespace, appearing as its eth0, and the other end stays in the host namespace, attached to a bridge such as docker0 or a CNI bridge. The bridge acts as a virtual switch connecting all containers on the host. For outbound traffic, the host performs NAT with masquerade through iptables, so containers can reach the internet; for inbound, port publishing uses DNAT rules. In Kubernetes, all containers in a pod share one network namespace, so they communicate over localhost, and CNI plugins then connect the pod's namespace across nodes using routing, overlays like VXLAN, or cloud-native networking. I can explore this with ip netns, nsenter into a container's network namespace to run ip and tcpdump, and look at the host-side veth interfaces. Understanding it helps with debugging DNS, MTU issues from overlays, conntrack and iptables problems, and connectivity between pods.

Likely follow-ups

- How does a container reach the internet through the host?
- How can you run tcpdump inside a container's network namespace from the host?

certs.eknathalabs.com

NETWORK NAMESPACES

```
# A netns is an isolated network stack: its own interfaces, routes,
#   ARP table, iptables, ports. This is the primitive containers use.
ip netns add blue
ip netns exec blue ip a        # run a command inside that namespace
ls /var/run/netns              # named namespaces
```

VETH PAIRS (THE CABLE)

```
# A veth pair is a virtual cable: two ends; what goes in one comes out the other.
# One end lives in the container's netns, the other on the host (often on a bridge).
ip link add veth0 type veth peer name veth1
ip link set veth1 netns blue   # push one end into the namespace
# Host end -> docker0/cni bridge -> NAT/routing to the outside.
```

THE FULL CONTAINER PICTURE

```
container netns --(veth)--> host bridge (docker0/cni0) --(iptables NAT)--> eth0 -> world
# "publish a port" = a DNAT rule mapping host:port -> container IP:port.
# Kubernetes CNI plugins automate exactly this (plus pod-to-pod routing).
```

Interview tip: The insight that impresses: containers aren't magic — they're network namespaces connected by veth pairs to a host bridge, with iptables doing NAT/port-publishing. Being able to say "`docker run -p` is just a DNAT rule" and "a pod is a netns; CNI wires the veth" shows you understand the plumbing under Docker/Kubernetes, not just the CLI.

📚 **Official docs**

- Linux man-pages: namespaces(7)
- Docker: Networking

certs.eknathalabs.com

Q72 What is the difference between a bridge, bond, VLAN, and tunnel interface? Networking

🎙 **Spoken answer · ~83 s**

These are different virtual and physical interface types that solve different problems. A bridge is a virtual layer-two switch that connects multiple interfaces, such as veth ends or VM interfaces, into one broadcast domain; docker0 and VM host bridges are examples. A bond aggregates multiple physical NICs into one logical interface for redundancy, so it survives a NIC, cable, or switch failure, and sometimes for throughput; modes include active-backup, which needs no switch support, and 802.3ad LACP, which needs switch configuration. A VLAN interface uses 802.1Q tags to split one physical link into multiple isolated layer-two networks, so different traffic types share the same cable but stay separated. A tunnel interface encapsulates packets inside other packets to carry them across a network: VXLAN, GRE, WireGuard, and IPsec tunnels, used for overlay networks, VPNs, and connecting sites; tunnels add overhead, so MTU must be reduced to avoid fragmentation or black holes. These combine: a bond of two NICs, with VLAN interfaces on top, attached to bridges for VMs. I inspect with ip link and ip minus d link for details, bridge commands, and the proc bonding status. Knowing them helps with cloud and Kubernetes networking, where overlays and bridges are everywhere.

Likely follow-ups

- When would you choose active-backup over LACP bonding?
- Why do tunnels require MTU adjustments?

THE INTERFACE TYPES

```
Bridge : a virtual L2 switch — connects multiple interfaces/veths into one
         segment (docker0, br0 for VMs/containers).
Bond   : aggregates multiple NICs into one for redundancy and/or throughput
         (a.k.a. link aggregation / LACP). Survives a NIC/cable failure.
VLAN   : tags frames (802.1Q) to split one physical link into many logical L2
         segments (eth0.100 = VLAN 100).
Tunnel : encapsulates packets to traverse another network — VXLAN, GRE, IPsec,
         WireGuard. Basis of overlay networks.
```

BONDING MODES (MOST ASKED)

```
active-backup : one NIC active, others standby (pure redundancy, no extra bw)
802.3ad (LACP): switch-negotiated aggregation (redundancy + bandwidth)
balance-alb/tlb : load-balance without switch config
```

WHERE EACH SHOWS UP

Bridges back container/VM networking; bonds give NIC redundancy on servers; VLANs segment a datacenter L2; tunnels (VXLAN) build the overlay fabrics that Kubernetes CNIs and SDNs use to make pods on different nodes look like one L2.

Interview tip: One sentence each: bridge = virtual switch (containers/VMs), bond = NIC aggregation for HA/throughput (know active-backup vs LACP), VLAN = logical L2 segmentation via tagging, tunnel/VXLAN = overlay encapsulation. Connecting VXLAN tunnels to Kubernetes overlay networking, and bonds to server NIC redundancy, shows you map these primitives to real infrastructure.

📚 **Official docs**

- RFC 7348: VXLAN
- Kustomize reference

certs.eknathalabs.com

Q73 How do you diagnose high latency or packet loss between two hosts?

Networking

🎙 Spoken answer · ~86 s

To diagnose latency or packet loss between hosts, I first quantify and locate. ping with a count gives baseline round-trip time, loss percentage, and jitter, though ICMP may be deprioritised. mtr is the key tool: it combines ping and traceroute, showing per-hop loss and latency over many cycles. Reading it correctly matters: loss that starts at a hop and continues to the final destination is real loss at that hop, while loss at one middle hop that doesn't continue to later hops is usually just ICMP rate limiting by that router, not real loss. High latency that starts at a hop and persists suggests the congested or distant link there. Then look at the end hosts: interface errors and drops with ip minus s link, retransmissions with ss minus ti or nstat, CPU saturation, or buffer problems; and check for bufferbloat or microbursts. Use TCP-based probes if ICMP is filtered, and test with iperf for throughput. Capture packets on both ends to confirm where loss occurs. Typical causes: congested link or oversubscription, faulty cable or NIC, duplex mismatch, wireless issues, firewall or middlebox overload, or a routing change. If it's within the provider's network, I give them the mtr evidence. Keep a baseline and monitor continuously.

Likely follow-ups

- How do you tell real packet loss from ICMP rate limiting in mtr?
- What other tests complement mtr?

FIND WHERE IT HURTS

```
ping -c 100 host          # baseline RTT + loss % (watch for variance/jitter)
mtr -rwzbc 100 host       # per-hop loss + latency over 100 cycles (THE tool)
traceroute host           # path; -T for TCP if ICMP is filtered
```

READING MTR

```
# Loss that STARTS at a hop and CONTINUES to the end -> real loss at that hop.
# Loss at ONE middle hop but 0% after it -> that router just deprioritizes ICMP
#   to its own control plane (NOT a real problem) — a classic misread.
# So only trust loss that persists to the destination.
```

LOCAL-SIDE CHECKS

```
ip -s link show eth0      # interface errors/drops/overruns (bad NIC/cable/driver)
ethtool -S eth0           # detailed NIC counters
ss -ti                    # per-socket RTT, retransmits, cwnd (TCP-level loss)
# Also: MTU/PMTUD black hole (big packets lost), conntrack full, NIC ring buffers.
```

Interview tip: `mtr` is the headline tool, but the senior point is reading it correctly: mid-path ICMP deprioritization looks like loss but isn't — only loss that persists to the final hop is real. Separate latency (distance/queuing) from loss (errors/congestion) from jitter (variance). Checking local NIC error counters and per-socket TCP retransmits (`ss -ti`) rounds out a complete answer.

certs.eknathalabs.com

📚 **Official docs**

- RFC 792: ICMP

Q74 Explain ephemeral ports, TIME_WAIT, and 'cannot assign requested address' under load.

Networking

🎙 **Spoken answer · ~84 s**

Each outbound connection uses a local source port from the ephemeral range, about twenty-eight thousand ports by default, and a connection is identified by the four-tuple of source and destination IP and port. To a single destination IP and port, the client is limited by the number of available source ports. When a connection closes, the side that closes first enters TIME_WAIT for around sixty seconds to catch stray packets, which holds the port. A client making thousands of short connections per second to one backend can therefore accumulate tens of thousands of TIME_WAIT sockets and run out of ports, producing cannot assign requested address. I check with ss counts by state and the port range. Fixes: reuse connections through keep-alive and connection pooling, which addresses the cause; widen the ephemeral range; enable tcp_tw_reuse so outbound connections can reuse TIME_WAIT sockets safely; spread the load across multiple source IPs or destination IPs and ports; and fix connection leaks. Don't use the old recycle option, which was removed because it breaks clients behind NAT. In Kubernetes and cloud NAT, the NAT layer has its own port limits per destination, so add NAT addresses. Monitor socket states so I see it before users do.

Likely follow-ups

- Why does the client rather than the server run out of ports?
- Why was tcp_tw_recycle removed?

EPHEMERAL PORTS

```
# Each OUTBOUND connection uses a local source port from the ephemeral range:
cat /proc/sys/net/ipv4/ip_local_port_range    # e.g. 32768  60999  (~28k ports)
# A connection is the 4-tuple: src_ip:src_port -> dst_ip:dst_port.
# To ONE destination ip:port you're limited by available source ports.
```

TIME_WAIT

```
# The side that closes first holds the socket in TIME_WAIT (~60s) to absorb
#   stray packets. Under high churn, thousands of TIME_WAIT sockets tie up
#   source ports -> new outbound connects fail:
#   "cannot assign requested address" (EADDRNOTAVAIL).
ss -tan state time-wait | wc -l
```

FIXES

```
- Use connection pooling / keep-alive (fewer, reused connections) — the real fix
- Widen ip_local_port_range
- net.ipv4.tcp_tw_reuse=1  (safe: reuse TIME_WAIT for new OUTBOUND conns)
- Do NOT use tcp_tw_recycle (removed; broke NAT)
```

corts.eknathalabs.com

Interview tip: The chain to articulate: high connection churn → pile of TIME_WAIT on the active closer → source-port exhaustion → EADDRNOTAVAIL. The correct primary fix is connection reuse/pooling, not tuning. If asked about sysctls, `tcp_tw_reuse` is safe but `tcp_tw_recycle` is a trap (NAT-breaking, removed) — knowing that difference is a strong signal.

📚 **Official docs**

- RFC 9293: TCP
- IANA: Service name and port number registry
- Terraform: Sensitive data in state

Q75 **What is SSH multiplexing, agent forwarding, and a bastion/jump host? Show a secure setup.** Networking

🎙 **Spoken answer · ~83 s**

A secure SSH setup uses a bastion host as the controlled entry point, and ProxyJump to reach private hosts through it without placing private keys on the bastion: the client connects to the bastion, which just forwards the TCP connection, and the end-to-end SSH session is encrypted between my machine and the target. In the client config I define the private host group with ProxyJump pointing to the bastion. Multiplexing reuses one TCP connection for multiple SSH sessions, with ControlMaster, ControlPath, and ControlPersist: the first connection authenticates and later ones start instantly, which speeds up repeated commands and tools like Ansible. Agent forwarding lets the remote host use my local agent's keys, but it's risky: anyone with root on the intermediate host can use my agent while connected, so avoid it and prefer ProxyJump, or use it only with trusted hosts and confirm-on-use keys. For security of the bastion itself: key-only login, MFA, restricted source IPs, minimal software, full logging and session recording, short-lived SSH certificates, and no shared accounts. Even better, replace long-lived bastions with session managers such as AWS Systems Manager, which need no inbound ports. Add timeouts, audit access, and remove access promptly at offboarding.

Likely follow-ups

- Why is ProxyJump safer than agent forwarding?
- What does ControlPersist do?

JUMP HOST (PROXYJUMP)

```
# Reach a private host THROUGH a bastion, without landing keys on the bastion:
ssh -J bastion.example.com private-host
# ~/.ssh/config:
Host private-*
    ProxyJump bastion.example.com
    User deploy
```

MULTIPLEXING (REUSE ONE CONNECTION)

certs.eknathalabs.com

```
Host *
    ControlMaster auto
    ControlPath ~/.ssh/cm-%r@%h:%p
    ControlPersist 10m
# Subsequent ssh/scp to the same host reuse the open channel -> instant, no re-auth.
```

AGENT FORWARDING — AND WHY TO AVOID IT

```
# ssh -A forwards your local agent so the remote can use your keys onward.
# RISK: root on that remote can hijack your agent socket to impersonate you.
# Prefer ProxyJump (keys never touch the intermediate host) over -A.
```

Interview tip: Recommend **ProxyJump** over agent forwarding for bastions — ProxyJump tunnels through without exposing your keys on the jump box, whereas `-A` lets a compromised intermediate hijack your agent. Multiplexing (`ControlMaster`) is the performance win for repeated connections (Ansible uses it). Mentioning the agent-forwarding risk is a security-maturity marker. Modern shops increasingly replace bastions with SSM/IAP (no open SSH port at all).

📚 **Official docs**

- RFC 9113: HTTP/2
- AWS Systems Manager Session Manager

certs.eknathalabs.com

Q76 **How do you harden SSH on a production server? Give concrete sshd_config settings.** Security

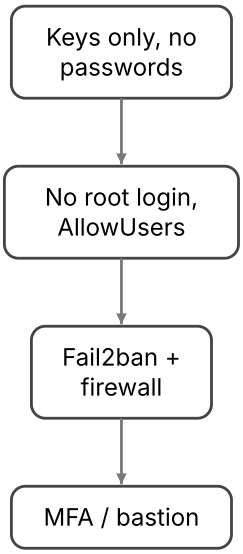
🎙 **Spoken answer · ~79 s**

Hardening SSH starts with sshd_config. Set PermitRootLogin no, so administrators log in as named users and use sudo, giving accountability. Set PasswordAuthentication no and disable keyboard-interactive, so only keys are accepted, eliminating password brute force. Keep PubkeyAuthentication yes and restrict who can log in with AllowUsers or AllowGroups. Limit MaxAuthTries to three and shorten LoginGraceTime. Disable unneeded features: X11 forwarding, agent forwarding, and TCP forwarding where not required. Set ClientAliveInterval and ClientAliveCountMax to drop idle or dead sessions. Use modern algorithms only, disabling legacy ciphers, MACs, and key exchange methods; ssh-audit checks this. Changing the port is only obscurity and not real security. Beyond the config: restrict network access with firewall rules, security groups, or a VPN, and avoid exposing SSH publicly; use a bastion or session manager; add MFA; use short-lived certificates from an SSH CA instead of scattered authorized_keys; run fail2ban or similar to throttle attacks; keep OpenSSH patched; log to a central system; and audit authorized keys regularly. Before restarting, validate the config with sshd minus t and keep an existing session open to avoid locking myself out. Document and enforce through configuration management and compliance scanning.

Likely follow-ups

- Why is changing the SSH port not real security?
- How do you apply sshd changes without risking lockout?

🗺 **SSH hardening**



CORE SSHD_CONFIG HARDENING

certs.eknathalabs.com

```
PermitRootLogin no                # no direct root; sudo from a named user
PasswordAuthentication no         # keys only — kills brute force
PubkeyAuthentication yes
KbdInteractiveAuthentication no
AllowUsers deploy ops             # explicit allowlist (or AllowGroups)
Port 22                           # (obscurity only; not real security)
MaxAuthTries 3
LoginGraceTime 20
X11Forwarding no
AllowAgentForwarding no           # unless explicitly needed
```

LAYERED CONTROLS

```
- firewall: restrict 22 to known CIDRs / VPN / bastion only
- fail2ban / sshguard: ban IPs after repeated failures
- MFA for SSH (e.g. TOTP PAM) for human access
- short-lived SSH certificates (CA-signed) instead of static authorized_keys
- centralize + monitor auth logs (failed logins, new keys)
```

Always keep an active session open and test the new config from a second terminal (`sshd -t` to validate syntax) before closing — a bad `sshd_config` can lock you out.

Interview tip: The two highest-impact settings are `PasswordAuthentication no` (keys only — eliminates brute-force) and `PermitRootLogin no`. Then layer: firewall to bastion/VPN, fail2ban, MFA. The modern senior answer mentions SSH certificates (CA-signed, short-lived) over static `authorized_keys` for scalable, revocable access — and always validate with `sshd -t` before disconnecting.

📚 **Official docs**

- OpenSSH manual pages

certs.eknathalabs.com

Q77 Explain Linux capabilities. Why are they better than SUID-root?

Security

🎙 Spoken answer · ~82 s

With traditional SUID-root, a binary runs with all of root's powers even if it needs only one, so any bug in it can lead to full system compromise: privilege is all or nothing. Linux capabilities split root's power into distinct units, such as CAP_NET_BIND_SERVICE for binding low ports, CAP_CHOWN for changing ownership, CAP_NET_RAW for raw sockets, or CAP_SYS_TIME for setting the clock. A web server that only needs to bind port eighty can be given just that one capability, so if it's compromised, the attacker doesn't get root. That's least privilege applied. I assign them to binaries with setcap, view with getcap, and inspect running processes via capsh or the status file. In containers, the runtime already drops many by default, and best practice is to drop all and add back only what's needed, run as non-root, and block privilege escalation. Caveats: some capabilities are nearly equivalent to root, particularly CAP_SYS_ADMIN, which is a catch-all, and CAP_SYS_PTRACE and CAP_DAC_OVERRIDE, so granting those defeats the purpose. Ambient and inheritable sets control how capabilities pass through exec. I audit for SUID binaries and replace them with capabilities where possible, and combine them with seccomp and mandatory access control.

Likely follow-ups

- Which capabilities are close to full root?
- How would you let a non-root service bind to port 443?

THE PROBLEM WITH SUID-ROOT

```
# SUID-root = the binary runs with ALL root powers, even if it only needs one.
# A bug in it = full root compromise. All-or-nothing privilege.
```

CAPABILITIES = ROOT SPLIT INTO PIECES

```
CAP_NET_BIND_SERVICE  # bind ports <1024 without being root
CAP_NET_RAW           # raw sockets (ping, packet capture)
CAP_SYS_ADMIN         # huge catch-all ("the new root" — avoid granting)
CAP_CHOWN, CAP_KILL, CAP_SYS_TIME, ...
# Grant just the capability a program needs, not full root:
setcap cap_net_bind_service+ep /usr/bin/myserver
getcap /usr/bin/myserver       # inspect
```

WHY BETTER

Least privilege: a web server that only needs to bind port 80 gets `CAP_NET_BIND_SERVICE` instead of running as root. A bug then leaks one capability, not the whole system. Containers use this heavily — drop all caps, add back only what's needed.

Interview tip: Frame capabilities as "root chopped into ~40 independent privileges" enabling least privilege. The killer example: bind to port 80 with `CAP_NET_BIND_SERVICE` instead of full root. The container tie-in is gold: `--cap-drop=ALL --cap-add=NET_BIND_SERVICE`, and that `CAP_SYS_ADMIN` is effectively root so granting it defeats the purpose.

certs.eknathalabs.com

📚 **Official docs**

- Linux man-pages: capabilities(7)
- Linux man-pages: path_resolution(7)

Q78 What are SELinux and AppArmor? How do you debug an SELinux denial?

Security

🎙 **Spoken answer · ~86 s**

SELinux and AppArmor are mandatory access control systems: the kernel enforces a central policy that even root-owned processes can't override, confining what a process may access, as a defence-in-depth layer. SELinux, on RHEL and Fedora, is label-based: every process and file has a context with a type, and policy rules say which process types can access which file types. AppArmor, on Ubuntu and SUSE, is path-based, with a profile per program, and is simpler to write. To debug an SELinux denial: first confirm it's SELinux, because permissions and ownership look right but access fails, and check the mode with getenforce. Search the audit log for AVC denials with ausearch, and use sealert or audit2why to explain them in plain language. Then choose the right fix: restore the default file context with restorecon, which is the usual fix after files were moved or created in the wrong place; set a boolean for a supported behaviour, such as allowing the web server to connect to the network; label a non-standard directory with semanage fcontext, or allow a non-standard port with semanage port; and only as a last resort generate a custom policy module with audit2allow. Setting permissive mode temporarily helps diagnosis, but disabling SELinux isn't an acceptable fix.

Likely follow-ups

- What is the first thing to do after files are moved and SELinux denies access?
- When is a custom policy module justified?

WHAT THEY ARE (MAC)

```
# Mandatory Access Control: the kernel enforces policy even root can't override,
#   confining what a process may access — a strong defense-in-depth layer.
# SELinux (RHEL/Fedora) : label-based (every file/process has a context/type)
# AppArmor (Ubuntu/SUSE): path-based profiles per program (simpler model)
```

DEBUGGING SELINUX DENIALS

```
getenforce                          # Enforcing / Permissive / Disabled
ausearch -m AVC -ts recent          # the actual denials (AVC messages)
sealert -a /var/log/audit/audit.log  # human-readable with suggested fixes
# Common real fix: relabel a file to the right context
restorecon -Rv /web/html
chcon -t httpd_sys_content_t /web/file
setsebool -P httpd_can_network_connect on   # toggle a policy boolean
```

Anti-pattern: `setenforce 0` / disabling SELinux "to make it work" throws away a real security layer. The symptom "works with SELinux off" means a labeling/policy issue — fix the context (`restorecon`) or the boolean, don't disable it. Use Permissive mode to log-without-blocking while you diagnose.

ocrts.eknathalabs.com

Interview tip: Define MAC as kernel-enforced confinement that even root obeys (vs discretionary owner-set perms). For debugging, the workflow is `ausearch/sealert` → usually `restorecon` (wrong label) or a `setsebool` toggle. The maturity signal is explicitly NOT disabling SELinux — using Permissive mode to diagnose, then fixing the label/policy properly.

📚 **Official docs**

- Kubernetes: AppArmor tutorial
- RHEL 9 documentation

Q79 **How do you manage sudo safely? Explain sudoers, NOPASSWD risks, and auditing.** Security

🎙 **Spoken answer · ~87 s**

Managing sudo safely means least privilege, auditability, and care when editing. Always edit with visudo, which syntax-checks before saving, since a broken sudoers file can lock out every administrator; prefer drop-in files in sudoers.d, edited with visudo minus f, so changes are modular and manageable by configuration management. Grant specific commands to specific users or groups rather than blanket ALL; for example, allow the deploy user to restart one service and read its logs. Be careful with commands that can escape to a shell, such as editors, pagers, find, or anything with a shell escape, because allowing them effectively grants full root; use sudoedit for editing files. NOPASSWD removes the password prompt, which is convenient for automation but means a compromised account gets root silently, so limit it to narrow, specific commands, never ALL, and prefer service identities with tight scopes. For auditing: sudo logs each command to the auth log or journal, so ship those logs centrally; enable sudo I/O logging where required; and review sudoers and group membership regularly. Additional hardening: require MFA for privileged access, use short-lived elevation or just-in-time access, set a secure path, and remove sudo from accounts that don't need it. Understand that sudo with a wildcard in arguments can be abused.

Likely follow-ups

- Why is NOPASSWD ALL dangerous?
- Why use sudoedit rather than allowing sudo vim?

EDITING SUDOERS

```
visudo                          # ALWAYS — it syntax-checks before saving
                                # (a broken sudoers can lock out all admins)
# Prefer drop-in files over editing the main file:
visudo -f /etc/sudoers.d/deploy
```

LEAST-PRIVILEGE RULES

coris.eknathalabs.com

```
# Grant specific commands, not blanket ALL:
deploy  ALL=(ALL) /usr/bin/systemctl restart myapp, /usr/bin/journalctl -u myapp
%ops    ALL=(ALL) ALL          # group-based (manage membership, not rules)
# NOPASSWD removes the password prompt — convenient but risky:
deploy  ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart myapp   # scope it tightly!
```

Risks: `NOPASSWD: ALL` means a stolen session = instant root with no second factor. Also beware commands that can spawn shells (editors, `less`, `find -exec`, `vi`) — granting those effectively grants root. Restrict to exact, non-shell-escaping commands.

AUDITING

```
journalctl _COMM=sudo           # who ran what via sudo
# sudo logs each invocation; ship these to central logging/SIEM.
```

Interview tip: Hit three things: always use `visudo` (syntax check prevents lockout), grant *specific commands* not `ALL`, and be wary of `NOPASSWD` plus shell-escaping commands (an editor or `less` in a sudo rule = root). Mention auditing sudo usage to central logs. This shows you treat privilege escalation as an attack surface, not a convenience.

📚 **Official docs**

- sudoers(5) manual

certs.eknathalabs.com

Q80 How do you respond to a suspected compromised Linux server? Walk through the steps.

Security

🎙 Spoken answer · ~79 s

For a suspected compromised Linux server, I follow contain, preserve, investigate, eradicate, recover. Contain without tipping off the attacker: isolate the host at the network layer with a security group or firewall change, but don't power it off, because volatile evidence like memory, live connections, and running processes would be lost. Preserve evidence by snapshotting the disk or VM and capturing memory if possible, and keep a written timeline. Investigate: who is or was logged in, with w, last, and lastb; active connections and listeners with ss; unusual processes and their executables; recently modified files; new users, SSH keys, cron jobs, systemd units, and shell profile changes that provide persistence; and SUID binaries or library preload tampering. Compare package files with the package manager's verification. A rootkit can hide things from local tools, so analyse the disk offline when in doubt. Determine the initial access vector and the blast radius. Eradicate and recover by rebuilding from a known-good image rather than cleaning, patching the entry point, rotating all credentials the host could reach, and restoring data from clean backups. Involve the security team, follow legal obligations, and write a postmortem.

Likely follow-ups

- Why snapshot the disk before touching a compromised host?
- Why rotate credentials even if you rebuild the host?

1. CONTAIN (DON'T TIP OFF THE ATTACKER)

```
# Isolate at the network layer (SG/firewall), DON'T power off
#   (loses volatile evidence: memory, live connections, running procs).
# Preserve: snapshot the disk/VM; capture RAM if you can.
```

2. TRIAGE / INVESTIGATE

```
w; last; lastb              # who's/was logged in, failed logins
ss -tnp                     # active connections + processes (C2 beacon?)
ps auxf                     # process tree — odd parents, miners
ls -la /tmp /dev/shm /var/tmp   # common dropper locations
crontab -l; ls /etc/cron.*; systemctl list-timers   # persistence
find / -mtime -2 -type f 2>/dev/null   # recently modified files
grep -r authorized_keys ~ /home /root  # added backdoor keys
journalctl/auth.log         # but assume local logs may be tampered
```

3. ERADICATE + RECOVER

```
# Don't "clean" a rooted box — rebuild from known-good images/IaC,
#   rotate ALL credentials/keys/secrets it could have touched, patch the
#   entry vector, restore data from pre-compromise backups.
```

Key principle: a compromised host can't be trusted to report on itself (rootkits hide processes/files). Investigate from outside where possible (network logs, disk image offline), and **rebuild rather than**

certs.eknathalabs.com

clean — you can never be sure you found everything.

Interview tip: Structure it: contain (isolate, preserve volatile evidence — don't just power off), investigate (connections, process tree, persistence: cron/timers/authorized_keys, recent file changes), then rebuild-not-clean + rotate all secrets + fix the entry point. The two senior insights: preserve memory/connections before acting, and never trust a rooted box's own logs/tools.

📚 **Official docs**

- NIST: Computer Security Incident Handling Guide (SP 800-61)

Q81 Explain disk encryption (LUKS) and file integrity monitoring. When do you need each?

Security

🎙 **Spoken answer · ~81 s**

LUKS provides encryption at rest for block devices: cryptsetup luksFormat encrypts a partition, luksOpen unlocks it to a mapped device on which I create a filesystem and mount it. It protects data if the disk is stolen, lost, or decommissioned without wiping, which matters for laptops, removable media, and compliance requirements. It does not protect a running system: once unlocked, the OS can read everything, so malware or a logged-in attacker sees plaintext. Key management is the hard part: passphrases, key files, or keys from a TPM or a network key server, and unattended boots need a secure approach. In the cloud, provider-managed volume encryption with a KMS key serves a similar role. File integrity monitoring answers a different question: has something changed that shouldn't have? Tools like AIDE, Tripwire, or auditd-based and eBPF-based monitors take a baseline of file hashes and permissions for critical files such as binaries, configs, and authorized_keys, and alert on unexpected changes, helping detect tampering, rootkits, and persistence. I need both in different situations: encryption for confidentiality of data at rest, integrity monitoring for detecting compromise and meeting compliance such as PCI. Neither replaces patching, access control, backups, and logging.

Likely follow-ups

- What threat does LUKS not protect against?
- How does AIDE detect tampering?

LUKS — ENCRYPTION AT REST

```
cryptsetup luksFormat /dev/sdb1        # encrypt a block device
cryptsetup luksOpen /dev/sdb1 secure   # unlock -> /dev/mapper/secure
mkfs.ext4 /dev/mapper/secure; mount ...
# Protects data if the DISK is stolen/decommissioned. Does NOT protect a
#   running system (once unlocked, data is readable to the OS).
# Key management: passphrase, keyfile, or TPM/network-bound (clevis/tang) for
#   unattended boot.
```

FILE INTEGRITY MONITORING (FIM)

certs.eknathalabs.com

```
# Detect unauthorized changes to critical files (binaries, configs):
# AIDE / Tripwire build a baseline of hashes, then alert on changes.
aide --init        # baseline
aide --check       # compare current vs baseline -> what changed
# auditd can watch specific paths in real time (watch /etc/passwd, /etc/sudoers).
```

WHEN

LUKS for laptops, removable media, and compliance (encryption-at-rest mandates) — its real value is at decommission/theft. FIM for detecting intrusion/tampering and meeting compliance (PCI requires it). They cover different threats: confidentiality vs integrity.

Interview tip: Be clear about LUKS's threat model: it protects data at rest (stolen/retired disk), NOT a live unlocked system — a common misconception. FIM (AIDE/Tripwire/auditd) detects tampering and is often a compliance requirement (PCI). Mentioning TPM/tang for unattended LUKS unlock, and auditd watches on `/etc/passwd/sudoers`, shows operational depth.

📚 **Start here (official docs)**

- Linux man-pages project

Q82 **How do you keep a fleet patched and find vulnerabilities? Explain CVEs and patch strategy.** Security

🎙 **Spoken answer · ~83 s**

Keeping a fleet patched starts with knowing what is vulnerable. Package managers list pending security updates, and scanners map installed packages to known CVEs: OpenSCAP, Trivy for hosts and containers, Grype, and vendor tools. A CVE is a public identifier for a vulnerability, and CVSS gives a severity score, but I prioritise by real risk: is the vulnerable component exposed to the network, is there a known exploit, is it internet-facing, and is the asset critical? A critical CVE in an unused package matters less than a medium one in an exposed service. Patch strategy: automate. Use unattended security updates for low-risk hosts, staged rollouts for everything else, starting with dev and a canary group, then waves with health checks and automatic rollback. For immutable infrastructure, rebuild images regularly with updated base layers and replace instances, instead of patching in place; for containers, rebuild images and redeploy. Kernel updates need reboots or live patching, so plan rolling reboots with capacity in mind. Set SLAs by severity, such as critical within days, track compliance with dashboards, handle exceptions with compensating controls, and keep an inventory including software bills of materials. Test patches, because they can break things, and communicate maintenance windows.

Likely follow-ups

- How do you prioritise between CVEs with equal CVSS scores?
- Why prefer rebuilding images to patching in place?

certs.eknathalabs.com

FIND WHAT'S VULNERABLE

```
apt list --upgradable;  dnf updateinfo list security   # pending security updates
# Scanners map installed packages -> known CVEs:
#   OpenSCAP, Trivy (also containers), Grype, vendor tools (Insights, etc.)
# Track severity (CVSS) + whether it's actually exploitable/exposed in your env.
```

PATCH STRATEGY

```
- Unattended security updates for low-risk patches (unattended-upgrades / dnf-automatic)
- Staged rollout: canary hosts -> wider fleet, watch for regressions
- Maintenance windows + reboot orchestration (kernel updates need reboot;
  or livepatch / kpatch / ksplice for no-reboot kernel patching)
- Immutable infra: bake patched golden images, redeploy (don't patch in place)
```

PRIORITIZE, DON'T PATCH-EVERYTHING-NOW

Rank by CVSS severity × exploitability × exposure (is the vulnerable service even reachable?). A critical CVE in an internet-facing service beats a high CVE in an isolated internal tool. Known-exploited (CISA KEV) jumps the queue.

Interview tip: Show a system, not "run apt upgrade": scan (Trivy/OpenSCAP) → prioritize by severity + exposure + known-exploited → stage (canary then fleet) → automate low-risk + maintenance-window the rest. The modern angle is immutable infrastructure (rebuild patched golden images rather than mutate live hosts) and livepatch for no-reboot kernel CVEs. Prioritization beats "patch everything" — that's the senior judgment.

📚 **Official docs**

- Argo CD: ApplicationSet
- Trivy

certs.eknathalabs.com

Q83 Explain PAM (Pluggable Authentication Modules) and a practical use (MFA, limits, lockout).

Security

🎙 Spoken answer · ~83 s

PAM, Pluggable Authentication Modules, is the framework that programs such as login, sshd, and sudo use to authenticate users, so each program doesn't implement authentication itself, and policy lives in etc pam.d. There are four management groups: auth verifies identity, such as a password, a key, or MFA; account checks whether the account is allowed, valid, or expired; password governs password changes and complexity; and session sets up and tears down sessions, like applying limits or creating home directories. Each line names a module and a control flag, such as required, requisite, sufficient, or optional, evaluated in order. Practical uses: enforce password complexity with pwquality; lock accounts after repeated failures with faillock to slow brute force; add multi-factor authentication to SSH or sudo with a TOTP module or hardware key; apply resource limits per user with pam_limits; restrict logins by time, group, or source using pam_access or pam_time; and integrate centralised identity through SSSD with LDAP or Active Directory. Safe practice: PAM errors can lock everyone out, including root, so keep a root session open while testing, back up the files, change one thing at a time, and use configuration management. Understand ordering and the interplay of sufficient and required.

Likely follow-ups

- How would you add TOTP to SSH using PAM?
- Why keep a root session open when editing PAM?

WHAT PAM IS

```
# PAM is the framework that programs (login, sshd, sudo) use to AUTHENTICATE,
#   without each implementing auth itself. Config lives in /etc/pam.d/<service>.
# Four management groups:
auth     -> verify identity (password, key, MFA)
account  -> is the account allowed/valid/expired?
password -> password change rules (complexity)
session  -> set up the session (limits, mounts, logging)
```

PRACTICAL EXAMPLES

```
# Add TOTP MFA to SSH (pam_google_authenticator) in /etc/pam.d/sshd:
auth required pam_google_authenticator.so
# Account lockout after failed attempts (pam_faillock):
auth required pam_faillock.so deny=5 unlock_time=900
# Resource limits at login (pam_limits -> /etc/security/limits.conf):
*  hard  nofile  65535
```

Caution: a broken PAM stack can lock everyone out (including root via console). Test changes in a separate session, and know your recovery path (single-user mode / console). Order and `required` vs `sufficient` control flow matters.

certs.eknathalabs.com

Interview tip: Explain PAM as the pluggable auth layer shared by login/sshd/sudo, with stackable modules (auth/account/password/session). Give a concrete win: `pam_google_authenticator` for SSH MFA, `pam_faillock` for lockout, `pam_limits` for ulimits. The safety note — a bad PAM edit can lock everyone out, so test in a second session — signals you've actually touched it in production.

📚 **Official docs**

- Linux-PAM System Administrators Guide

Q84 What's the principle of least privilege in practice on Linux? Give concrete mechanisms.

Security

🎙 **Spoken answer · ~78 s**

Least privilege means every user, process, and service gets only the access it needs, nothing more, which shrinks the blast radius when something is compromised. Concrete Linux mechanisms: run daemons as dedicated non-root service accounts with no login shell; use capabilities, such as CAP_NET_BIND_SERVICE, instead of full root; configure sudo with specific commands for specific groups instead of blanket access; use file permissions, groups, and ACLs so each account touches only its own data; and mount data volumes with noexec and nosuid where appropriate. For services, systemd hardening options add a lot: User, ProtectSystem, ProtectHome, PrivateTmp, NoNewPrivileges, restricted address families, and system call filters. Mandatory access control with SELinux or AppArmor confines processes even if they run as root. Seccomp limits system calls, and in containers I drop all capabilities, run as non-root, use read-only root filesystems, and disable privilege escalation. For remote access, use key-only SSH, allow-lists, and short-lived credentials. Then maintain it: audit SUID binaries, review sudoers and group membership regularly, remove stale accounts, and automate through configuration management. The aim is that a compromise of one component doesn't grant the attacker the whole system.

Likely follow-ups

- Which systemd options give the most hardening for little effort?
- How do you audit for excess privilege?

certs.eknathalabs.com

THE PRINCIPLE

Every user, process, and service should have the **minimum** access needed — nothing more. It shrinks the blast radius when (not if) something is compromised.

CONCRETE LINUX MECHANISMS

```
Service accounts : run daemons as dedicated non-root users (never root)
Capabilities     : grant CAP_NET_BIND_SERVICE, not full root
systemd sandbox  : User=, ProtectSystem=strict, PrivateTmp=, NoNewPrivileges=,
                   ReadOnlyPaths=, RestrictAddressFamilies=, SystemCallFilter=
sudo             : specific commands per user/group, not ALL
MAC              : SELinux/AppArmor confinement per service
Filesystem       : tight ownership + perms; mount data noexec,nosuid,nodev
Containers       : --cap-drop=ALL, read-only rootfs, non-root USER, seccomp
```

SYSTEMD AS A SANDBOX

```
[Service]
User=myapp
NoNewPrivileges=true
ProtectSystem=strict
PrivateTmp=true
SystemCallFilter=@system-service
# A modern, easy way to confine a service without root or containers.
```

Interview tip: Don't just define it — list the mechanisms: dedicated service users, capabilities over root, `sudo` scoped to commands, SELinux/AppArmor, and especially **systemd sandboxing** directives (`NoNewPrivileges`, `ProtectSystem=strict`, `PrivateTmp`, `SystemCallFilter`), which let you confine a plain service without containers. Framing it as "shrink the blast radius" ties everything together.

📚 **Start here (official docs)**

- Linux man-pages project

certs.eknathalabs.com

Q85 Walk through the USE method for diagnosing a performance problem on Linux.

Performance

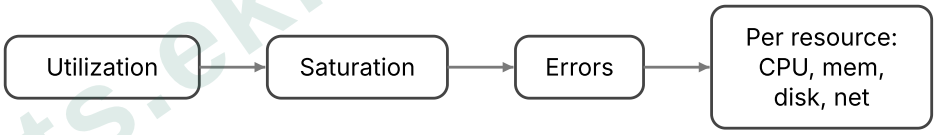
🎙 Spoken answer · ~83 s

The USE method, from Brendan Gregg, is a quick, systematic way to find performance bottlenecks: for every resource, check Utilisation, Saturation, and Errors. Utilisation is how busy the resource is, as a percentage of time in use. Saturation is whether there is extra work queued that it can't service yet. Errors are counts of error events, like drops, retransmits, or I/O errors. The resources are CPU, memory, disk, and network, plus things like file descriptors and connection pools. A quick map: for CPU, utilisation is user plus system time from top or mpstat, saturation is the run-queue length versus core count, and errors are rare. For memory, utilisation is used versus available, saturation is swapping activity and page scanning, with errors being OOM kills. For disk, utilisation is percent busy, saturation is queue depth and await in iostat, and errors appear in dmesg and SMART. For network, utilisation is bandwidth versus capacity, saturation is drops and overruns, errors are interface errors and retransmissions. The value is completeness: by walking the list I avoid anchoring on a favourite theory and quickly find the resource that's saturated or erroring. Then I drill into the process or workload responsible and form specific hypotheses.

Likely follow-ups

- What does saturation mean for CPU?
- Why is the USE method useful in an interview setting?

🗺 USE method



USE = UTILIZATION, SATURATION, ERRORS

```
# For every resource (CPU, memory, disk, network), check:
Utilization : how busy is it? (% time in use)
Saturation  : is there a queue / waiting work it can't service yet?
Errors      : any error counts? (drops, retransmits, EIO)
```

PER-RESOURCE QUICK MAP

```
CPU   : util=%us+%sy (top/mpstat); sat=run-queue/load, %st (steal); err=rare
Memory: util=used vs available; sat=swap activity (si/so), page scan; err=OOM
Disk  : util=%util (iostat -x); sat=avgqu-sz/await; err=ioerrors, SMART
Network: util=bandwidth vs link; sat=tx/rx queue drops; err=ip -s link errors
```

certs.eknathalabs.com

WHY A METHOD BEATS POKING AROUND

USE forces you to check every resource systematically instead of fixating on CPU. It quickly rules resources in or out — e.g. high disk `await` + low CPU points at storage, not the app. Brendan Gregg's USE method is the canonical systematic approach.

Interview tip: Naming the **USE method** (Utilization, Saturation, Errors per resource) signals you debug systematically, not by guessing. The high-value specifics: saturation is the early-warning sign (a queue forming before 100% util), `%st` steal time reveals a noisy-neighbor/overcommitted hypervisor, and disk `await`/`avgqu-sz` catch storage saturation. Pair it with the RED method (Rate/Errors/Duration) for services.

📚 **Official docs**

- Brendan Gregg: the USE Method
- Google SRE Book: Monitoring Distributed Systems

Q86 How do you read top/htop correctly? Explain load average, %wa, %st, and the columns.

Performance

🎙 **Spoken answer · ~84 s**

Reading top correctly begins with load average: the one, five, and fifteen minute averages of runnable plus uninterruptible-sleep processes. Compare it with the core count: a load of eight on eight cores is fully busy; on four cores it's overloaded. Importantly, load includes D-state processes, so high load can mean stuck I/O, not CPU. In the CPU line, us is user time, sy is kernel time, ni is niced processes, id is idle, wa is time waiting for I/O, hi and si are hardware and software interrupts, and st is steal time on virtual machines. High wa points to disk or network storage; high sy to heavy syscalls or networking; high si to network interrupt load; high st to a noisy neighbour. In the process list: PID, user, priority and nice values, VIRT for virtual size, RES for resident physical memory, SHR for shared memory, S for state, then CPU and memory percentages. RES is what matters for memory pressure, not VIRT. I press 1 for per-core view, H for threads, and sort by memory with M. Single-core saturation can hide behind a low overall average on many-core machines. htop adds a clearer, colour-coded view and easier filtering and tree views.

Likely follow-ups

- Why can load average be high while CPU is idle?
- What does a high %st value indicate?

LOAD AVERAGE

```
# 1/5/15-min averages of runnable + UNINTERRUPTIBLE(D) processes.
# Compare to core count: load 8 on 8 cores = fully busy; on 4 cores = overloaded.
# KEY: load counts D-state too, so high load can mean stuck I/O, not CPU!
nproc                          # core count for context
```

certs.eknathalabs.com

THE CPU LINE

```
%us user   %sy system(kernel)   %ni niced
%id idle
%wa iowait   -> CPU idle WAITING on disk I/O -> storage bottleneck
%st steal    -> hypervisor took your CPU (noisy neighbor / overcommit) — VM tell
%si/%hi      -> soft/hard interrupts (high = network/IRQ pressure)
```

READING IT

```
High %us              -> app/CPU bound
High %sy              -> syscall/kernel heavy (context switches? check vmstat)
High %wa + low %us    -> I/O bound (go to iostat)
High %st              -> you're being starved by the host (resize / move VM)
High load, low CPU    -> D-state processes -> disk/NFS stuck
```

Interview tip: The two most-missed columns: `%wa` (CPU idle waiting on disk = it's a storage problem, not CPU) and `%st` (steal = the hypervisor is taking your cycles, a cloud/VM noisy-neighbor signal). And the load-average subtlety — it includes uninterruptible (D) processes, so high load with low CPU usually means stuck I/O. Always compare load to `nproc`.

📚 **Official docs**

- Brendan Gregg: Linux performance

Q87 Diagnose a disk I/O bottleneck. What do iostat, await, and %util tell you?

Performance

🎙 **Spoken answer · ~84 s**

iostat with extended statistics, run at one-second intervals, shows per-device I/O behaviour. The key columns: percent util shows how busy the device is, with near one hundred meaning saturated for a single spinning disk, but it's misleading on SSDs, NVMe, and RAID arrays that handle parallel requests, where high util isn't necessarily bad. await is the average time in milliseconds per I/O including queue time, and it's the latency number that hurts applications; separate read and write await values help. Queue size shows how many requests are waiting, and IOPS and throughput columns show the workload. As a rule of thumb, await of several milliseconds on SSD is fine, tens of milliseconds is a problem, and on spinning disks around a hundred is bad. To find who's responsible, I use iotop or pidstat for per-process I/O. Then I characterise the pattern: random versus sequential, read versus write, small versus large blocks. Causes include a noisy process, backups, log floods, a database scanning, low memory causing cache thrash or swap, a failing disk, or cloud volume limits like exhausted burst credits. Fixes: reduce the I/O, tune the application, add caching, move to faster or provisioned storage, or isolate workloads. Validate with fio.

Likely follow-ups

- Why is %util misleading on SSDs?
- What does await measure?

certs.eknathalabs.com

THE TOOL

```
iostat -xz 1              # extended, skip idle devices, every 1s
# Key columns per device:
%util    : how busy the device is (~100% = saturated, but misleading on SSD/RAID
           which handle parallel I/O — high %util there isn't always bad)
await    : avg time (ms) per I/O incl. queue — THE latency number that hurts apps
r_await / w_await : split read vs write latency
aqu-sz   : average queue depth — requests waiting (saturation)
r/s w/s rkB/s wkB/s : IOPS and throughput
```

WHAT POINTS WHERE

```
High await + high aqu-sz   -> device saturated, requests queueing (the real pain)
High %util but low await   -> busy but keeping up (esp. SSD/RAID) — often fine
Which process?  ->  iotop -o    (top I/O consumers)
                    pidstat -d 1 (per-process disk I/O)
```

BEYOND THE DEVICE

Correlate with `%wa` in top (CPU waiting on I/O). Check if it's random vs sequential (random IOPS-bound), the filesystem/scheduler, or a noisy neighbor on shared cloud storage (EBS burst credits exhausted is a classic cloud trap).

Interview tip: `await` is the number that matters (per-I/O latency including queue time) — not `%util`, which is misleading on SSDs/RAID that service I/O in parallel. High `await` + growing `aqu-sz` = saturation. Then attribute it with `iotop`/`pidstat -d`. The cloud-savvy bonus: exhausted EBS/burst IOPS credits cause sudden latency cliffs.

📚 **Official docs**

- Brendan Gregg: Linux performance

certs.eknathalabs.com

Q88 How do you diagnose memory pressure? Explain free, swap, available, and the page cache.

Performance

🎙 Spoken answer · ~78 s

Diagnosing memory pressure means reading the numbers correctly. In free, ignore the free column and watch available, which is the memory applications can actually obtain, counting reclaimable page cache. High buff and cache is good: it's cached file data and is released under pressure. Low free with high available is healthy. Real pressure signals: active swapping, seen in vmstat's swap-in and swap-out columns, because swap used but idle isn't a problem; major page faults; direct reclaim and kswapd activity; OOM killer messages in the kernel log; and the pressure stall information files, which report the share of time tasks stall waiting for memory. Then identify consumers: ps or top sorted by resident size, smem for proportional memory, and slab usage for kernel memory. Look at the trend to separate a leak, steady growth, from a workload peak. In containers check cgroup memory usage versus limit, including page cache charged to the cgroup. Fixes: reduce consumption, fix leaks, tune caches and heap sizes, set memory limits, add RAM, and adjust swappiness; protect critical processes from the OOM killer. Alert on available memory and PSI, not on free.

Likely follow-ups

- Why is swap used not the same as memory pressure?
- What does PSI measure?

READ FREE CORRECTLY

```
free -h
# IGNORE the "free" column. Watch "available" — RAM apps can actually get,
#   counting reclaimable page cache. "buff/cache" is cached file data (good!),
#   reclaimed automatically under pressure. Low "free" + high "available" = healthy.
```

REAL PRESSURE SIGNALS

```
vmstat 1     -> si/so columns: swap IN/OUT activity = real memory pressure
              -> high 'r' run queue, low 'free', cache shrinking
sar -B 1     -> pgscan/majflt: page scanning + MAJOR faults (hitting disk)
cat /proc/pressure/memory   -> PSI: % time stalled on memory (modern, precise)
dmesg | grep -i oom         -> OOM kills (you've gone past pressure to failure)
```

PER-PROCESS

```
ps aux --sort=-%mem | head            # top memory users (RSS)
# RSS vs VSZ: RSS = real physical RAM used; VSZ = virtual (often huge, misleading).
smem -tk                              # PSS (shared memory counted fairly)
```

Interview tip: The #1 correction interviewers want: low `free` is normal — Linux uses spare RAM for page cache; watch `available` and swap activity (`si/so` in `vmstat`), not `free`. Real pressure shows as swapping, page scanning, and major faults. Modern bonus: PSI (`/proc/pressure/memory`) quantifies stall time. RSS (real) vs VSZ (virtual) trips people up — cite RSS for actual usage.

certs.eknathalabs.com

📚 **Official docs**

- Linux kernel docs: networking scaling

Q89 What is CPU steal time, context switching, and run queue? When is each a problem?

Performance

🎙 **Spoken answer · ~88 s**

These three explain different kinds of slowness. Steal time, shown as st, is time my virtual CPU was ready to run but the hypervisor ran something else; it only appears on virtual machines, and sustained high steal means an oversubscribed host or noisy neighbour. I can't fix it from inside the guest: resize, change instance type or family, use dedicated hosts, or move. Context switches, the cs column in vmstat, count how often the CPU switches between tasks; very high rates mean too many runnable threads, lock contention, or excessive interrupts, wasting CPU on switching rather than work. The run queue, the r column in vmstat or the load average, is the number of tasks ready to run; if it consistently exceeds the number of cores, CPU is saturated and tasks wait, which adds latency even if utilisation looks only moderately high. When each is a problem: steal when performance is poor on a VM while the guest looks fine; context switching when a service has thousands of threads or heavy lock contention; run queue when sustained above core count. I diagnose with vmstat, mpstat per core, pidstat minus w for per-process context switches, and perf. The fix for the last two is reducing threads, better concurrency design, or adding cores.

Likely follow-ups

- How can you tell steal time from a throttled container?
- What does a run queue longer than core count mean?

STEAL TIME (%ST)

```
# %st = time your vCPU was READY to run but the hypervisor ran someone else.
# Seen only in VMs/cloud. Sustained high %st = overcommitted host / noisy neighbor.
# You can't fix it from inside — resize, change instance type, or move hosts.
mpstat -P ALL 1     # per-core %steal
```

CONTEXT SWITCHES

```
vmstat 1   -> 'cs' column (context switches/sec), 'in' (interrupts)
# Very high cs/sec relative to work done = thrashing: too many threads,
#   lock contention, or tiny time slices. Voluntary (waiting) vs involuntary
#   (preempted) switches: pidstat -w shows per-process.
```

RUN QUEUE

```
vmstat 1   -> 'r' column = processes runnable/running.
# r consistently > number of cores -> CPU saturation (tasks waiting for CPU).
# This is "saturation" in the USE method for CPU.
```

certs.eknathalabs.com

Interview tip: Steal time is the cloud signature — high `%st` means the hypervisor is starving you, fixable only by resizing/moving, not by tuning your app. A run queue (`r` in vmstat) persistently above core count = CPU saturation. Excessive context switches (high `cs`) hint at over-threading or lock contention. These three are exactly the CPU-side USE-method metrics.

📚 **Start here (official docs)**

- Linux man-pages project

Q90 How do you profile what a process is spending CPU on? (perf, flame graphs, eBPF)

Performance

🎙 **Spoken answer · ~83 s**

To profile where a process spends CPU, I use sampling profilers because they have low overhead. perf top gives a live system-wide view of the hottest functions. perf record samples a specific process, at a frequency such as ninety-nine hertz, avoiding lock-step with periodic timers, with call graphs enabled, for some seconds; perf report then breaks down where CPU went. Flame graphs turn those samples into a visual: width is time on CPU, and the stack grows upward by caller, so the widest boxes at the top are hot code paths and I can spot them instantly. Need symbols: the application must have frame pointers or debug info, and managed languages need their own support, such as perf maps for the JVM or Node. For off-CPU time, where a process is blocked on locks, I/O, or network, a CPU profile won't show it; use eBPF tools like offcputime, or wall-clock profilers. eBPF-based tools such as bpftrace and the BCC suite give detailed, low-overhead tracing of syscalls, latency histograms, and more in production. Cautions: sampling is safe but check overhead, work on representative load, and compare against a baseline. Always follow up by validating the fix with another profile.

Likely follow-ups

- What does the width of a flame graph box represent?
- When does a CPU profile fail to explain slowness?

certs.eknathalabs.com

SAMPLE WITH PERF

```
perf top                      # live, system-wide hottest functions
perf record -F 99 -p <pid> -g -- sleep 30   # sample PID at 99Hz with stacks
perf report                   # interactive breakdown of where CPU went
# 99Hz (not 100) avoids lock-stepping with periodic timers.
```

FLAME GRAPHS

```
# Turn samples into a visual: width = time on CPU, stacked by call hierarchy.
perf script | stackcollapse-perf.pl | flamegraph.pl > out.svg
# Instantly shows the hot path — the widest tower is where the CPU goes.
```

EBPF (LOW OVERHEAD, PRODUCTION)

```
# bcc/bpftrace tools profile with minimal overhead, safe on prod:
profile -p <pid> 30           # on-CPU profiler (bcc)
offcputime -p <pid>           # where it's BLOCKED (off-CPU: locks, I/O, sleep)
bpftrace -e 'profile:hz:99 /pid==1234/ { @[ustack] = count(); }'
```

On-CPU vs off-CPU: `perf`/flame graphs show where CPU is *burned*. If the process is slow but NOT using CPU, it's blocked — use off-CPU analysis (`offcputime`) to find waits on locks, I/O, or sleep. Many "it's slow" cases are off-CPU.

Interview tip: `perf record -g` + flame graphs is the classic CPU-profiling answer; the visual makes the hot path obvious. The senior depth: distinguish on-CPU (perf) from off-CPU (blocked on locks/I/O — use eBPF `offcputime`), and prefer eBPF/bpftrace on production for its low overhead vs strace. Knowing 99Hz sampling avoids timer aliasing is a nice detail.

📚 **Official docs**

- Linux kernel docs: BPF
- ebpf.io: What is eBPF?
- Brendan Gregg: Linux performance

Q91 Explain cgroups and how systemd uses them to limit CPU, memory, and I/O.

Performance

🎙 **Spoken answer · ~80 s**

Cgroups limit and account for resources of groups of processes, and cgroup v2, with its unified hierarchy, is the modern default. They are the kernel primitive behind both container limits and systemd service limits. Systemd creates a cgroup per service, and I can set limits in the unit file or at runtime: MemoryMax for a hard memory limit, MemoryHigh for a soft throttle point, CPUQuota for a percentage cap, CPUWeight for proportional sharing under contention, IOWeight and IO device limits for disk, and TasksMax to cap processes and threads, which prevents fork bombs. systemd-cgtop shows live per-cgroup resource use, like top by service, and systemd-cgls shows the tree. Slices group services, so I can give a whole class, such as batch jobs, a share. The behaviour on exceeding: memory above the max triggers the OOM killer within the group; CPU quota causes throttling; I/O limits cause queueing. This is how I contain noisy neighbours on a host without containers, and how Kubernetes enforces requests and limits underneath. Good practice: set limits from measured usage, leave headroom, monitor throttling and OOM events, and use soft limits where graceful degradation is preferable to a kill.

Likely follow-ups

- What is the difference between MemoryHigh and MemoryMax?
- How does TasksMax protect a host?

WHAT CGROUPS DO

certs.eknathalabs.com

```
# Control groups limit + account resources for groups of processes.
# cgroup v2 (unified hierarchy) is the modern default. It's the kernel
#   primitive behind container resource limits AND systemd service limits.
systemd-cgtop          # live per-cgroup CPU/mem/IO (like top, by service)
systemd-cgls           # the cgroup tree
```

LIMIT A SERVICE VIA SYSTEMD

```
[Service]
CPUQuota=50%           # max half a core
MemoryMax=512M         # hard cap -> cgroup OOM kills inside this service only
MemoryHigh=400M        # soft throttle before the hard cap
IOWeight=50            # relative disk I/O share
TasksMax=100           # limit thread/process count
# Apply live: systemctl set-property myapp.service MemoryMax=1G
```

WHY IT MATTERS

Without limits, one runaway service can starve the whole host. cgroups contain the blast radius: a memory leak hits a per-service OOM (only that service dies) instead of the host OOM killer picking a random victim. CPU limits also explain CFS throttling in containers.

Interview tip: Tie it together: cgroups are the kernel mechanism; containers AND systemd both use them. Show you can cap a plain service with `MemoryMax`/`CPUQuota`/`IOWeight` — no container needed. The key benefit is blast-radius containment: a per-cgroup memory limit causes a local OOM kill, protecting the rest of the host. `systemd-cgtop` is the per-service `top`.

📚 **Official docs**

- Linux man-pages: cgroups(7)
- systemd.service(5)

certs.eknathalabs.com

Q92 How do you tune key kernel sysctl parameters for a high-traffic server?

Performance

🎙 Spoken answer · ~81 s

Tuning a high-traffic server is about removing specific limits that hit under load, with evidence. Network: raise somaxconn so the accept queue handles bursts, tcp_max_syn_backlog for half-open connections, the ephemeral port range for many outbound connections, enable tcp_tw_reuse for outbound reuse of TIME_WAIT sockets, raise netdev_max_backlog for NIC to kernel queues, and enlarge socket buffers for high-bandwidth, high-latency paths. Increase conntrack limits where connection tracking is used. File and process limits: raise fs.file-max and the per-process nofile limit, since each connection is a file descriptor. Memory: tune swappiness and dirty page ratios for writeback behaviour, and consider overcommit settings. The application also needs to match: the listen backlog in its config must be raised too, because the effective queue is the smaller of the two. Apply changes via sysctl drop-ins and configuration management. Discipline: measure first to find the real limit, for example listen queue overflows, SYN drops, or too many open files errors, change one thing at a time, load test before and after, and document why each value was set. Avoid cargo-cult tuning from old blog posts, because defaults on modern kernels are often already good and some old tips are harmful.

Likely follow-ups

- Why must the application's backlog match somaxconn?
- How do you know a listen queue is overflowing?

COMMON NETWORK TUNABLES

```
net.core.somaxconn = 4096                # max accept-queue backlog (raise for high conn
rate)
net.ipv4.tcp_max_syn_backlog = 8192      # half-open queue
net.ipv4.ip_local_port_range = 1024 65535  # more ephemeral ports
net.ipv4.tcp_tw_reuse = 1                # reuse TIME_WAIT for outbound
net.core.netdev_max_backlog = 16384      # NIC -> kernel queue
net.ipv4.tcp_fin_timeout = 15
fs.file-max = 2000000                    # system-wide fd limit
```

FILE DESCRIPTORS (PER-PROCESS)

```
# sysctl fs.file-max is system-wide; per-process is ulimit / systemd LimitNOFILE:
ulimit -n                                # current
# /etc/security/limits.conf or systemd [Service] LimitNOFILE=65535
# "Too many open files" = hit this, not fs.file-max, usually.
```

APPLY + PERSIST

```
sysctl -w net.core.somaxconn=4096        # now (non-persistent)
echo 'net.core.somaxconn=4096' >> /etc/sysctl.d/99-tuning.conf
sysctl --system                          # reload all
# vm.swappiness=10 (favor RAM over swap), vm.max_map_count (Elasticsearch needs high)
```

Don't cargo-cult: measure first, change one thing, re-measure. Blindly copying "tuning guides" can hurt. The most common real fix isn't exotic — it's raising `somaxconn` and `LimitNOFILE` (the "too many

certs.eknathalabs.com

open files" / connection-drop pair).

Interview tip: Know the high-value few rather than reciting dozens: `somaxconn` + `tcp_max_syn_backlog` (connection bursts), `LimitNOFILE`/`ulimit -n` (the "too many open files" killer), ephemeral port range, and `vm.swappiness`. Emphasize measuring before/after and persisting via `/etc/sysctl.d`. The per-process ulimit vs system-wide `fs.file-max` distinction is commonly confused.

📚 **Official docs**

- Linux kernel docs: sysctl

Q93 **An app is slow but CPU, memory, and disk look fine. How do you find the real bottleneck?** Performance

🎙 **Spoken answer · ~83 s**

If an app is slow while CPU, memory, and disk look fine, it is probably waiting, not working: blocked on something else. I find what it's blocked on. strace with timing on a process, including its threads, filtering network, read, and futex calls, shows which syscall stalls: a long connect or read means a slow dependency, a long futex wait means lock contention. eBPF off-CPU profiling shows where threads sleep, whether on locks, I/O, or the network. ss with TCP info shows round-trip times and retransmits to dependencies. The usual hidden culprits: a slow downstream service or database; DNS resolution delays; connection pool or thread pool exhaustion, where requests queue for a connection; lock contention or serialised sections; garbage collection pauses; network latency or packet loss; TLS handshakes without connection reuse; throttling by cgroup CPU limits, which doesn't show as high CPU; rate limits; and external API timeouts. Distributed tracing is the best tool if available, because it shows time spent per hop. I also compare latency percentiles per dependency and check pool metrics. Once found, fix with timeouts, pooling, caching, concurrency changes, or fixing the slow dependency, and add instrumentation so this wait is visible next time.

Likely follow-ups

- How does a futex wait in strace point to lock contention?
- Why can CPU throttling hide from CPU usage graphs?

IT'S PROBABLY WAITING, NOT WORKING

```
# Low resource usage + slow = the process is BLOCKED, not busy. Find on what:
strace -p <pid> -f -T -e trace=network,read,futex   # which syscall stalls (-T = time)
offcputime -p <pid>          # eBPF: where it's blocked off-CPU (locks/IO/net)
ss -ti                       # TCP rtt/retransmits to dependencies
```

USUAL HIDDEN CULPRITS

certs.eknathalabs.com

```
- Downstream latency: a slow DB/API/DNS — your app waits on the network
- Lock contention: threads blocked on a mutex (futex in strace; off-CPU stacks)
- DNS: 5s resolver timeouts on every request (check /etc/resolv.conf, ndots)
- Connection pool exhausted: requests queue waiting for a free connection
- GC pauses / single-threaded hot section / external rate limiting (429s)
```

GET END-TO-END VISIBILITY

Distributed tracing (or just timing each external call) localizes the slow hop. "Resources fine" almost always means the time is spent waiting on something external or on a lock — instrument the boundaries.

Interview tip: The reframe is the answer: "resources fine but slow" = it's **waiting**, so do off-CPU analysis, not resource hunting. Name the suspects — downstream dependency latency, lock contention (futex), DNS 5s timeouts, connection-pool exhaustion. The tools that find waits are `offcputime` (eBPF), `strace -T`, and tracing. This off-CPU mindset is exactly what separates senior debugging from "check top."

📚 **Start here (official docs)**

- Linux man-pages project

Q94 Explain LVM: physical volumes, volume groups, logical volumes. Why use it?

Storage & disks

🎙 **Spoken answer · ~84 s**

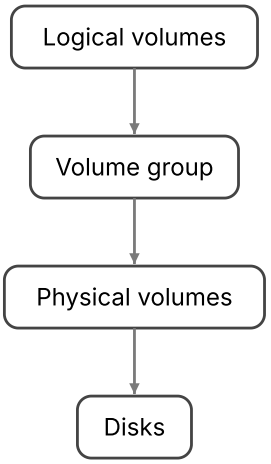
LVM has three layers. A physical volume, PV, is a disk or partition initialised for LVM. A volume group, VG, pools one or more PVs into a single storage pool. A logical volume, LV, is carved out of the VG, formatted with a filesystem, and mounted; it behaves like a partition but is not tied to physical disk boundaries. The killer feature is online resizing: extend an LV when it fills up, and grow the filesystem without unmounting, adding a new disk to the VG when the pool is full, with no downtime. Other benefits: spanning multiple disks as one volume; moving data between disks while live with pvmove, for replacing hardware; snapshots for consistent backups or quick rollbacks; striping and mirroring options; and thin provisioning, which allows overcommitting space with monitoring. Costs: extra complexity and another layer to troubleshoot, snapshot performance overhead, and the risk that a mistake in resizing, especially shrinking, can destroy data. Shrinking requires the filesystem to be shrunk first, and XFS can't shrink. In the cloud, LVM is less necessary because volumes can be resized by the provider, but it's still useful for combining volumes and for flexible layouts on bare metal. Always keep backups before changes.

Likely follow-ups

- What does pvmove let you do?
- Why is shrinking a logical volume risky?

oEits.eknathalabs.com

🗺 LVM



THE LAYERS

```
PV (Physical Volume) : a disk/partition initialized for LVM (pvcreate /dev/sdb)
VG (Volume Group)    : a pool of PVs (vgcreate data /dev/sdb /dev/sdc)
LV (Logical Volume)  : carved from a VG, formatted + mounted (lvcreate -L 50G ...)
# disks -> pooled into a VG -> sliced into resizable LVs
```

THE KILLER FEATURE: ONLINE RESIZE

```
lvextend -L +20G /dev/data/web        # grow the LV
resize2fs /dev/data/web               # grow ext4 filesystem (xfs_growfs for XFS)
# Or one step:  lvextend -r -L +20G /dev/data/web
vgextend data /dev/sdd                # add a new disk to the pool first
# All ONLINE — no unmount, no downtime.
```

WHY USE IT

Flexible capacity (grow filesystems on the fly, span multiple disks), snapshots (point-in-time, e.g. for consistent backups), and the ability to add disks to a pool without repartitioning. The cost is a layer of complexity vs raw partitions.

Interview tip: Explain the PV → VG → LV stack and lead with the practical win: **online filesystem growth** (`lvextend -r`) without downtime, plus pooling disks and snapshots. Mention XFS grows but can't shrink (so plan LV sizing), and that LVM snapshots enable consistent backups. This is bread-and-butter for anyone who's expanded a full data volume in production.

📚 **Official docs**

- Linux kernel docs: LVM (device-mapper)

ceris.eknathalabs.com

Q95 How do you safely grow a filesystem that's running out of space (cloud + LVM)?

Storage & disks

🎙 Spoken answer · ~89 s

To grow a filesystem safely in the cloud, I work from the outside in, and take a snapshot first. Without LVM: first expand the volume at the cloud level, such as modifying the EBS volume size, and wait for it to be in use or optimising state. Second, make the kernel see the new size, with a rescan if necessary. Third, grow the partition to fill the disk with growpart. Fourth, grow the filesystem: resize2fs for ext4, or xfs_growfs on the mount point for XFS, both working online. Then verify with lsblk and df. With LVM, the chain is longer: expand the cloud volume, rescan, grow the partition if one exists, run pvresize so LVM sees the extra space on the physical volume, extend the logical volume with lvextend, ideally with the option to resize the filesystem in the same step, then confirm. If I add a new disk instead, I pvcreate it, vgextend the volume group, and then lvextend. Safety: snapshot or back up first; check that the device names are right before running commands; never shrink as part of an emergency; and note that many cloud providers limit how often a volume can be modified. After growing, fix the cause of the growth, add monitoring, and consider automation through infrastructure as code.

Likely follow-ups

- Why grow the partition before the filesystem?
- What extra step does LVM add to the process?

CLOUD DISK (E.G. ATTACHED VOLUME, NO LVM)

```
1. Expand the volume in the cloud console/API (e.g. EBS modify).
2. Tell the kernel about the new size:  (rescan if needed)
   echo 1 > /sys/class/block/sdb/device/rescan   # or growpart for partitions
3. growpart /dev/nvme1n1 1          # grow the PARTITION to fill the disk
4. resize2fs /dev/nvme1n1p1         # ext4   (xfs_growfs /mount for XFS)
# All online.
```

LVM-BACKED

```
1. (If new disk) pvcreate /dev/sdd ; vgextend vg /dev/sdd
2. lvextend -r -L +50G /dev/vg/data    # -r resizes the fs in the same step
# Or grow the underlying disk, then pvresize /dev/sdb, then lvextend -r.
```

ORDER MATTERS

```
disk/volume  ->  partition (growpart)  ->  PV (pvresize)  ->  LV (lvextend)  ->  filesystem
(resize2fs/xfs_growfs)
# You must grow each layer from the bottom up. Skipping a layer = "no space" persists.
```

Gotcha: XFS can only grow (never shrink), and `xfs_growfs` takes the MOUNTPOINT, not the device. ext4 can shrink but only offline. Always have a backup/snapshot before resizing, and verify with `df -h` after.

certs.eknathalabs.com

Interview tip: The structure interviewers want is the bottom-up layer chain: cloud volume → partition (`growpart`) → PV (`pvresize`) → LV (`lvextend -r`) → filesystem (`resize2fs`/`xfs_growfs`). The common failure is growing the disk but forgetting the partition or PV layer, so space never appears. Note it's all online and XFS is grow-only. This is a very common real on-call task.

📚 **Official docs**

- Linux kernel docs: LVM (device-mapper)

Q96 Explain RAID levels (0,1,5,6,10). What are the trade-offs and the RAID-5 rebuild risk?

Storage & disks

🎙 **Spoken answer · ~86 s**

RAID levels trade capacity, performance, and redundancy. RAID 0 stripes across disks: full capacity and speed, but no redundancy, so one failure loses everything. RAID 1 mirrors: full redundancy, fifty percent capacity, survives one disk failure. RAID 5 stripes with single parity: capacity of N minus one disks, survives one failure, but has a write penalty because parity must be updated. RAID 6 uses double parity: capacity of N minus two, survives two failures. RAID 10 mirrors and then stripes: good performance and redundancy at fifty percent capacity, the usual choice for databases. The RAID 5 rebuild risk: after a disk fails, rebuilding reads every sector of all the remaining disks, which takes hours or days on large drives and stresses them. If a second disk fails, or a unrecoverable read error occurs during the rebuild, the array is lost. With large modern disks, the probability is significant, which is why RAID 6 or RAID 10 is preferred for big arrays. Other points: RAID is not a backup, since it doesn't protect against deletion, corruption, or ransomware; monitor for degraded arrays and replace failed disks promptly; keep hot spares; and use disks from different batches. In the cloud, durability comes from the storage service and replication instead.

Likely follow-ups

- Why is RAID 5 risky with large disks?
- Why is RAID not a backup?

certs.eknathalabs.com

THE LEVELS

```
RAID 0  : striping — speed + full capacity, ZERO redundancy (1 disk dies = all lost)
RAID 1  : mirroring — full redundancy, 50% capacity, survives 1 disk
RAID 5  : striping + 1 parity disk — survives 1 failure, capacity = (N-1) disks
RAID 6  : striping + 2 parity — survives 2 failures, capacity = (N-2)
RAID 10 : mirror then stripe — fast + redundant, 50% capacity (the DB favorite)
```

THE RAID-5 REBUILD RISK

```
# When a disk fails, rebuild reads EVERY block on ALL remaining disks to
#   reconstruct parity. On large modern drives this is hours of full-stress reads,
#   during which a SECOND disk failure (or an unreadable sector) loses everything.
# -> RAID 5 is discouraged for large disks; prefer RAID 6 or RAID 10.
```

OPERATIONAL

```
cat /proc/mdstat            # md (software RAID) status + rebuild progress
mdadm --detail /dev/md0
# RAID is NOT a backup — it protects against disk failure, not deletion/corruption.
```

Interview tip: Know the trade-off per level (0=speed/no safety, 1=mirror, 5=one-parity, 6=two-parity, 10=mirror+stripe for DBs). The senior point is the **RAID-5 rebuild hole**: rebuilds stress all disks for hours and a second failure/URE during rebuild is catastrophic, so RAID 6 or 10 is preferred on large drives. And always say "RAID is not a backup" — it doesn't protect against `rm` or corruption.

📚 **Official docs**

- Linux kernel docs: md (software RAID)

Q97 What is swap, swappiness, and should a database/Kubernetes node have swap?

Storage & disks

🎙 **Spoken answer · ~83 s**

Swap is disk space used as overflow when RAM is full, letting the system survive memory spikes instead of immediately killing processes. The cost is that swapped pages are very slow, so heavy swapping makes everything sluggish. Swappiness, from zero to one hundred, controls how eagerly the kernel swaps anonymous memory versus dropping page cache; the default is sixty, and lower values prefer keeping applications in RAM. For databases, I typically set swappiness low, around one to ten, because swapping out hot buffer pool pages hurts latency, but keep a small amount of swap as a safety net so the OOM killer isn't the first response. For Kubernetes, nodes have traditionally run with swap disabled, because the scheduler and kubelet assume memory limits are hard, and swap makes memory accounting and QoS unpredictable; newer versions support swap in a controlled way, with node-level configuration, limited to certain workloads. The trade-off is predictability versus resilience. Operationally, I monitor swap activity, not just swap used, since idle swapped pages are harmless, but constant swap in and out indicates memory shortage. Tools like zram provide compressed in-memory swap that helps on small systems. Size swap thoughtfully and fix the underlying memory demand.

Likely follow-ups

- Why is swap disabled on traditional Kubernetes nodes?
- What swappiness would you use for a database server?

WHAT SWAP IS

```
# Disk space used as overflow when RAM is full — lets the system survive
#   memory spikes instead of OOM-killing, but swapped pages are SLOW (disk).
swapon --show; free -h
vm.swappiness = 60 (default)   # 0-100: higher = swap more eagerly
# Lower (e.g. 10) = prefer keeping things in RAM, swap only when really needed.
```

certs.eknathalabs.com

DATABASES

```
# DBs hate swap: a hot page swapped to disk = sudden latency cliff.
# Common practice: low swappiness (1-10) so swap is a safety net, not routine.
# Some disable swap entirely and size RAM properly + rely on OOM tuning.
```

KUBERNETES

```
# Historically kubelet REQUIRED swap off (failed to start with swap on) so that
#   memory limits + OOM behavior were predictable. Newer K8s (1.22+ beta, GA later)
#   supports swap, but most clusters still run swap off for deterministic limits.
```

Interview tip: Explain swap as an overflow safety net that trades OOM-death for severe slowness. For latency-sensitive workloads (databases), keep `swappiness` very low so swap stays a last resort, not routine paging. For Kubernetes, the classic answer is swap off for predictable memory limits/OOM (kubelet historically required it), while noting newer versions can support swap. The principle: swapping hot memory = latency cliff.

📚 **Official docs**

- Linux kernel docs: /proc/sys/vm

Q98 How do you check disk health and predict failures? (SMART, dmesg, errors)

Storage & disks

🎙 **Spoken answer · ~84 s**

To check disk health and predict failures I use SMART data with smartctl. The overall health test gives pass or fail, but that's a late indicator, so I look at the attributes themselves: reallocated sector count, current pending sectors, offline uncorrectable sectors, and reported uncorrectable errors; rising values mean a failing disk. For SSDs, wear indicators such as percentage used, wear levelling count, and media wearout, plus available spare. I can run short or long self-tests. The kernel log is another source: I/O errors, resets, timeouts, and filesystem errors in dmesg often appear before SMART flags anything. Also watch latency spikes in iostat and RAID or ZFS error counters. For NVMe, nvme smart-log provides the equivalent. Automate it: run smartd or export metrics to Prometheus and alert on rising critical attributes, temperature, and error counts, so I replace disks proactively. When a disk is suspect, check redundancy and backups first, then replace it and rebuild. Limitations: SMART doesn't predict every failure, and sudden failures occur, so redundancy and tested backups are essential. In virtualised or cloud environments I can't see SMART, so I rely on provider health events, error counts in the guest, and replacing instances or volumes at the first signs.

Likely follow-ups

- Which SMART attributes are the strongest failure predictors?
- What do you do when you can't access SMART, as in the cloud?

SMART DATA

cors.eknathalabs.com

```
smartctl -a /dev/sda           # full SMART attributes
smartctl -H /dev/sda           # overall health PASSED/FAILED
# Watch these attributes (rising = trouble):
#   Reallocated_Sector_Ct, Current_Pending_Sector, Offline_Uncorrectable,
#   Reported_Uncorrect, and (SSD) Wear_Leveling / Percentage_Used / Media_Wearout
smartctl -t long /dev/sda      # run a self-test
```

KERNEL + FILESYSTEM ERRORS

```
dmesg -T | grep -iE 'I/O error|ata|medium error|reset'   # disk errors
journalctl -k | grep -i error
# EXT4-fs errors / "remounting filesystem read-only" = serious corruption signal.
```

PROACTIVE

```
smartd (smartmontools daemon)   # schedule self-tests + alert on SMART changes
# In cloud, you usually don't see SMART (managed storage) — watch volume
#   metrics, I/O errors in dmesg, and degraded-RAID alerts instead.
```

Reality check: SMART catches many but not all failures — disks can die with a clean SMART. Reallocated/pending sectors trending up is the strongest early warning. A filesystem going read-only or repeated I/O errors in `dmesg` means act now (replace + restore).

Interview tip: Lead with `smartctl` and the meaningful attributes (reallocated/pending sectors, SSD wear) plus `dmesg` I/O errors and the read-only-remount red flag. The honest senior caveat: SMART isn't a guarantee — disks fail clean — so pair it with proactive `smartd` alerts and good backups/RAID. The cloud nuance (no SMART on managed volumes; watch metrics/dmesg) shows breadth.

📚 **Start here (official docs)**

- Linux man-pages project

certs.eknathalabs.com

Q99 Explain NFS: how it works, common problems (stale handles, hangs), and tuning.

Storage & disks

🎙 Spoken answer · ~88 s

NFS lets a client mount a remote export as if it were local, which is convenient for shared data, but it adds network and server dependencies. I mount with the mount command or in fstab with the network-dependent option, and list exports with showmount. Classic problems. Stale file handle means the file or export on the server changed or was recreated under the client; the fix is to remount and, in the application, reopen files. Hangs happen with hard mounts when the server is unreachable: processes block in D state until it returns, which is by design to protect data integrity, and soft mounts avoid the hang at the risk of data corruption on writes. Slow performance can come from small rsize and wsize, high latency, attribute caching settings, a saturated server or network, or sync writes. Permission surprises come from UID and GID mapping mismatches or root squash. Tuning: use modern NFS versions, set larger rsize and wsize, choose appropriate attribute cache times, use TCP, and consider async on the server only when the risk is acceptable. For resilience use autofs, avoid putting NFS in critical boot paths, monitor latency and retransmits with nfsstat, and make the server highly available. For high-scale cases, consider object storage or a distributed filesystem instead.

Likely follow-ups

- What causes a stale file handle?
- Why does a hard mount hang when the server is gone?

WHAT NFS IS

```
# Network filesystem — mount a remote export as if local.
mount -t nfs server:/export /mnt/data
# /etc/fstab: server:/export /mnt/data nfs defaults,_netdev 0 0
showmount -e server          # list exports
```

THE CLASSIC PROBLEMS

```
"Stale file handle" : the server-side file/export changed/recreated under you
                      -> remount, or the mount points to a now-invalid inode.
Hard hang           : with the default 'hard' mount, if the server dies, I/O
                      BLOCKS FOREVER (processes stuck in D state, load climbs).
                      'soft' returns errors instead but risks data corruption.
'intr' (old) / use timeo, retrans, and consider 'soft,timeo=' carefully.
```

TUNING + GOTCHAS

```
- NFSv4 (stateful, single port 2049, better security/ID mapping) over v3
- rsize/wsize for throughput; noatime to cut metadata writes
- _netdev so it mounts after network; autofs to mount on demand (avoid boot hangs)
- A dead NFS server is the #1 cause of D-state pileups + unkillable processes.
```

Interview tip: The two facts that matter most: **hard vs soft mounts** (hard = block forever if the server dies, causing D-state hangs and rising load; soft = error out but risks corruption), and "stale file handle"

certs.eknathalabs.com

meaning the server-side object changed. Tie a hung NFS server to the unkillable-D-state question. Prefer NFSv4, use `_netdev`/autofs to avoid boot hangs. This is a very common production failure mode.

📚 **Start here (official docs)**

- Linux man-pages project

Q100 What's the difference between block, file, and object storage? When use each?

Storage & disks

🎙 **Spoken answer · ~82 s**

Block, file, and object storage differ in how data is organised and accessed. Block storage provides raw volumes, like EBS or iSCSI, on which you put a filesystem; it offers low latency and high IOPS, typically attached to one host at a time, like a virtual disk. It's the choice for databases, boot volumes, and workloads needing consistent performance. File storage provides a shared hierarchical filesystem, like NFS, SMB, or EFS, mounted by many hosts with POSIX-like semantics. It suits shared content, home directories, and legacy applications needing a shared filesystem, but with higher latency and scaling limits. Object storage, like S3 or GCS, stores objects as key, data, and metadata accessed over HTTP APIs, with practically unlimited scale, very high durability, cheap cost, and features like versioning and lifecycle policies, but higher latency, no in-place edits, and no POSIX semantics. It suits backups, logs, media, data lakes, and static assets. How I choose: databases and latency-sensitive workloads on block; shared read-write access across servers on file; massive, durable, cheap, internet-accessible data on object. Consider consistency, cost, throughput, and access patterns. In practice, systems combine them: databases on block volumes, with backups and archives in object storage.

Likely follow-ups

- Why can't you use object storage as a database disk?
- When would you choose file storage over block?

THE THREE

```
Block  : raw volumes (EBS, iSCSI, LVM). You put a filesystem on it. Low latency,
         attached to ONE host (usually). Like a virtual disk.
File   : a shared filesystem with a hierarchy (NFS, SMB, EFS). Multiple hosts
         mount it; POSIX-ish semantics. Like a shared network drive.
Object : key -> blob + metadata over HTTP (S3, GCS). Infinite scale, durable,
         NOT a filesystem (no in-place edit, no POSIX). Like a giant key-value store.
```

WHEN TO USE EACH

```
Block  -> databases, boot disks, anything needing a real filesystem + low latency
File   -> shared access across many servers (legacy apps, home dirs, shared assets)
Object -> backups, media, logs, static assets, data lakes, anything web-scale +
          accessed by URL/API. Cheapest + most durable + most scalable.
```

certs.eknathalabs.com

KEY DISTINCTIONS

Block = one writer, fast, you manage the fs. File = many writers, POSIX, network latency. Object = HTTP API, versioned, eventually-consistent historically (now strongly consistent on major clouds), but you can't `fopen+seek-write` it like a file.

Interview tip: One line each: block = a virtual disk (DBs, low latency, single attach), file = shared network filesystem (NFS/EFS, many hosts, POSIX), object = HTTP key-blob store (S3, web-scale, durable, no in-place edit). The clarifying point: object storage isn't a filesystem — no partial writes/POSIX semantics — which is why you don't run a database on S3. Map each to a workload to show practical judgment.

📚 **Official docs**

- POSIX Shell Command Language

Q101 **How do you design a backup strategy? Explain 3-2-1, RPO/RPO, and testing restores.** Storage & disks

🎙 **Spoken answer · ~83 s**

A backup strategy starts from requirements: RPO, the recovery point objective, is how much data loss is acceptable, which sets backup frequency; RTO, the recovery time objective, is how quickly I must be back, which sets the restore method and infrastructure. Then the three-two-one rule: three copies of the data, on two different types of storage, with one copy off-site; modern guidance extends it with one immutable or air-gapped copy so ransomware or a compromised admin account can't delete or encrypt it, and zero errors on verified restore. Design elements: decide what to back up, including databases consistently via snapshots or dumps with logs for point-in-time recovery, configuration, and secrets; use incremental backups with periodic fulls; encrypt backups and protect the keys; control access and separate credentials from production; set retention policies; and monitor job success, age, and size anomalies. The most important and most neglected part is testing restores: automatically restore into a scratch environment on a schedule, validate data integrity, and measure the actual recovery time against the RTO. A backup is not real until it has been restored. Also document runbooks, run disaster recovery drills, and assign ownership. Remember that RAID and replication are not backups.

Likely follow-ups

- What does an immutable backup protect against?
- How do you verify that a backup is actually restorable?

THE 3-2-1 RULE

```
3 copies of the data
2 different media/storage types
1 copy OFF-SITE (and ideally 1 OFFLINE/immutable — ransomware can't encrypt it)
# Modern extension 3-2-1-1-0: +1 immutable/air-gapped, 0 errors on verified restore.
```

certs.eknathlabs.com

RPO AND RTO DRIVE THE DESIGN

```
RPO (Recovery Point Objective): how much DATA LOSS is acceptable?
   -> dictates backup frequency (hourly snapshots vs daily vs continuous/WAL)
RTO (Recovery Time Objective): how long to be back UP?
   -> dictates restore method (hot standby = minutes; cold restore = hours)
# Tighter RPO/RTO = more cost. Set them per workload by business need.
```

BACKUP TYPES + THE CARDINAL RULE

```
Full / incremental / differential; app-consistent (quiesce/snapshot the DB,
  not just copy files mid-write). Encrypt backups; manage retention.
# THE RULE: a backup you haven't RESTORED is not a backup. Test restores regularly
#   (automated restore drills) and measure actual RTO. Untested backups fail silently.
```

Common failures: backups that were silently failing for months; file-level copies of a live database that aren't crash-consistent; backups stored only in the same account/region (ransomware/compromise takes them too). Immutable + off-site/offline copies defend against this.

Interview tip: Anchor on **3-2-1** (now 3-2-1-1-0 with immutable + verified), then frame frequency/method by **RPO/RTO** per workload. The line that lands: "an untested backup isn't a backup" — restore drills are mandatory. Call out app-consistent backups for databases (not raw file copies) and an immutable/air-gapped copy as the ransomware defense. RAID is not a backup, either.

📚 **Official docs**

- Google SRE Book: Data Integrity

certs.eknathalabs.com

Q102 Walk through the Linux boot process from power-on to login prompt.

Boot & systemd

🎙 Spoken answer · ~83 s

The Linux boot sequence goes through several stages. First, firmware, BIOS or UEFI, performs the power-on self-test, initialises hardware, finds a boot device, and loads the bootloader. Second, the bootloader, usually GRUB2, loads the kernel and the initramfs into memory and passes the kernel command line. Third, the kernel initialises hardware and drivers and mounts the initramfs as a temporary root. Fourth, the initramfs loads the modules and scripts needed to locate and mount the real root filesystem, such as storage drivers, LVM, RAID, or encryption unlocking. Fifth, the system switches root to the real filesystem. Sixth, the kernel starts PID one, systemd, which reads its units, mounts filesystems from fstab, starts services in parallel according to dependencies, and reaches the default target such as multi-user or graphical. Finally, a login prompt or display manager appears. Failure points map to stages: no bootloader or no bootable device means firmware, disk, or bootloader trouble; a kernel panic or being dropped into the initramfs shell means missing drivers or a wrong root device; emergency mode after the switch means a mount or critical service failure, commonly a bad fstab. systemd-analyze shows where boot time goes, and journalctl for the boot shows errors.

Likely follow-ups

- Where does a bad root= parameter fail?
- What does the initramfs do?

THE STAGES

```
1. Firmware (BIOS/UEFI) : POST, find a boot device, load the bootloader
2. Bootloader (GRUB2)   : loads the kernel + initramfs into memory, passes cmdline
3. Kernel               : initializes hardware/drivers, mounts initramfs (temp rootfs)
4. initramfs            : loads modules needed to find + mount the REAL root fs
5. switch_root          : pivot to the real root filesystem
6. init = PID 1 (systemd): brings up the system via units/targets
7. Reaches default.target (graphical/multi-user) -> getty -> login prompt
```

SYSTEMD'S JOB (STAGE 6)

```
systemctl get-default          # the target it boots to (multi-user/graphical)
# systemd activates units in dependency order (not a fixed sequence like SysV),
#   parallelizing where possible -> faster boot.
```

WHERE IT BREAKS

```
No bootloader/GRUB  -> "no bootable device"
Bad initramfs/missing driver -> "cannot mount root" / dropped to initramfs shell
Bad /etc/fstab      -> emergency mode (a mount failed)
journalctl -b       -> this boot's logs;  journalctl -b -1 -> previous boot
```

Interview tip: Hit the chain: firmware → GRUB → kernel → initramfs (loads drivers to find real root) → switch_root → systemd (PID 1) → target → login. The most useful debugging knowledge is mapping failures to stages: "can't mount root" = initramfs/driver/fstab, emergency mode = a failed mount, and

certs.eknathalabs.com

`journalctl -b` for the current boot. Knowing why initramfs exists (chicken-and-egg: need drivers to mount the disk holding the drivers) is a great detail.

📚 **Official docs**

- systemd: boot process

Q103 Explain systemd units, targets, and dependencies (Wants, Requires, After).

Boot & systemd

🎙 **Spoken answer · ~80 s**

Systemd organises everything as units of different types: services for daemons, sockets for socket activation, targets as groupings similar to runlevels, timers as cron replacements, mounts and automounts for filesystems, paths that activate on file changes, and slices for cgroup resource grouping. Dependency directives define relationships. Requires is a hard dependency: if the required unit fails or stops, this one does too. Wants is a soft dependency: start it, but don't fail if it doesn't work, and is the usual, safer choice. BindsTo is stricter, tying lifetimes together. Importantly, ordering is separate from dependency: After and Before define the order of starting, but don't pull units in. So a service that needs the network typically uses both Wants and After network-online.target. Without After, the two might start in parallel; without Wants or Requires, the other unit might not start at all. Targets group units; multi-user.target is the normal non-graphical state, and a unit's Install section with WantedBy decides what enabling it does. I inspect with systemctl list-dependencies, systemd-analyze critical-chain, and systemctl cat. Common mistakes: using Requires when Wants is enough, causing cascading failures, and forgetting ordering, creating race conditions at boot.

Likely follow-ups

- Why are Wants and After usually used together?
- What is the difference between Requires and BindsTo?

UNIT TYPES

```
.service  a daemon/process       .socket  socket activation
.target   a group of units (~runlevel: multi-user.target, graphical.target)
.timer    cron replacement        .mount/.automount  filesystem mounts
.path     activate on file change  .slice  cgroup resource grouping
```

DEPENDENCY DIRECTIVES

```
Requires=b.service  : hard dep — if b fails/stops, this unit is stopped too
Wants=b.service     : soft dep — start b, but DON'T fail if b fails (preferred)
After=b.service     : ORDERING only — start after b (does NOT pull b in)
Before=             : ordering the other way
BindsTo=            : even stronger than Requires (tied to b's state)
# KEY: Requires/Wants = "what to pull in"; After/Before = "in what ORDER".
#   They are independent — you often need BOTH Wants= and After=.
```

certs.eknathalabs.com

MANAGING

```
systemctl list-dependencies myapp.service
systemctl cat myapp.service          # effective unit file
systemctl edit myapp.service         # drop-in override (don't edit vendor files)
```

Interview tip: The classic gotcha interviewers probe: `After=` controls ORDER, not whether the dependency starts — you need `Wants=`/`Requires=` to actually pull it in. Prefer `Wants=` (soft) over `Requires=` (hard) to avoid cascading failures. Targets replace SysV runlevels. Use `systemctl edit` drop-ins rather than editing vendor unit files — a real-world best practice.

📚 **Official docs**

- systemd.service(5)

Q104 How do you create a robust systemd service? Cover Restart, dependencies, and hardening.

Boot & systemd

🎙 **Spoken answer · ~77 s**

A robust systemd service has sensible dependencies, restart behaviour, and hardening. In the Unit section: a description, and ordering and dependencies such as After and Wants network-online.target if it needs the network. In the Service section: ExecStart with the absolute path; Type set correctly, notify if the app signals readiness, otherwise simple or exec; Restart on-failure with RestartSec to retry after a pause; and start limits, like StartLimitIntervalSec and StartLimitBurst, so a crash loop gives up after a number of tries instead of flapping forever. Run as a dedicated User, set an EnvironmentFile for configuration, and set limits such as LimitNOFILE and MemoryMax. Add TimeoutStopSec and a proper stop signal for graceful shutdown. Hardening options: NoNewPrivileges, ProtectSystem strict, ProtectHome, PrivateTmp, PrivateDevices, restricted address families and capabilities, ReadWritePaths for the specific directories it needs, and system call filtering; systemd-analyze security scores a unit and suggests improvements. Logging goes to the journal. In the Install section, WantedBy multi-user.target. I use drop-in overrides for local changes, validate with systemd-analyze verify, run daemon-reload, and test failure by killing the process to confirm restarts, and add monitoring for restarts.

Likely follow-ups

- What do StartLimitBurst and StartLimitIntervalSec prevent?
- How does systemd-analyze security help?

A SOLID UNIT FILE

certs.eknathalabs.com

```
[Unit]
Description=My API
After=network-online.target
Wants=network-online.target

[Service]
ExecStart=/usr/bin/myapi --config /etc/myapi.yaml
Restart=on-failure
RestartSec=5
StartLimitIntervalSec=60
StartLimitBurst=5          # stop flapping: max 5 restarts/60s then give up
Type=notify                # app signals READY (or simple/exec/forking)
TimeoutStopSec=30          # grace before SIGKILL

[Install]
WantedBy=multi-user.target
```

HARDENING (SANDBOX THE SERVICE)

```
User=myapi
NoNewPrivileges=true
ProtectSystem=strict       # most of / read-only
ProtectHome=true
PrivateTmp=true
ReadWritePaths=/var/lib/myapi
MemoryMax=512M  CPUQuota=50%   # cgroup limits
```

Restart loops: `Restart=always` without `StartLimitBurst` can hammer a crashing service forever, hiding the real failure and burning resources. Bound it so systemd backs off and the failure surfaces.

Interview tip: Show you write production units, not just `ExecStart`: `Restart=on-failure` + `StartLimit*` (bounded restarts so flapping is visible), correct `Type=` (notify/simple/forking — getting this wrong makes systemd mis-track readiness), `TimeoutStopSec` for graceful stop, and sandboxing (`User=`, `ProtectSystem`, `NoNewPrivileges`, resource limits). That breadth — reliability + security + ordering — is the senior signal.

📚 **Official docs**

- systemd.service(5)

certs.eknathalabs.com

Q105 How do you read and manage logs with journalctl? How do you stop the journal filling disk?

Boot & systemd

🎙 Spoken answer · ~88 s

Querying: journalctl with minus u for a unit, minus f to follow, and since and until for time windows; minus b for the current boot and minus b minus one for the previous, which is key after a crash; minus p for priority, such as errors and above; minus k for kernel messages; and field filters like by PID or command. Output formats include json for scripts. To stop the journal filling the disk, configure limits in journald.conf: SystemMaxUse to cap total size, SystemKeepFree to always leave free space, SystemMaxFileSize, MaxRetentionSec, and rate limiting with RateLimitIntervalSec and RateLimitBurst to throttle floods from a chatty service. Check usage with journalctl disk-usage, and trim immediately with vacuum by size or time, which safely removes old entries. Decide on storage: persistent in var log journal survives reboots, volatile in run does not; for servers, persistent plus central shipping is best. Ship logs to a central system so local retention can stay small and logs survive host loss. Also fix the root cause of floods: a crash loop or debug logging. Keep logs on a separate filesystem where possible, so they can't fill root, and alert on disk usage and log volume spikes. Combined with logrotate for non-journal logs, this keeps the system healthy.

Likely follow-ups

- What does RateLimitBurst do?
- Why ship the journal off the host?

QUERYING

```
journalctl -u myapp -f                # follow one unit (like tail -f)
journalctl -u myapp --since "1 hour ago" --until "10 min ago"
journalctl -b                         # this boot;  -b -1 previous boot
journalctl -p err -b                  # priority err and above this boot
journalctl -k                         # kernel (dmesg) messages
journalctl _PID=1234 / _COMM=sshd     # by pid / command
journalctl -u myapp -o json-pretty    # structured fields
```

DISK USAGE CONTROL

```
journalctl --disk-usage               # how big is the journal?
journalctl --vacuum-size=500M         # trim to 500M now
journalctl --vacuum-time=7d           # keep only last 7 days
# Persist limits in /etc/systemd/journald.conf:
SystemMaxUse=1G
MaxRetentionSec=2week
```

PERSISTENT VS VOLATILE

```
# If /var/log/journal/ doesn't exist, the journal is in RAM and lost on reboot.
mkdir -p /var/log/journal && systemctl restart systemd-journald   # make persistent
```

Interview tip: Beyond basic querying, two things stand out: capping journal size (`SystemMaxUse` in `journald.conf`, or `--vacuum-size/--vacuum-time`) since an uncapped journal is a real "disk full" cause,

certs.eknathalabs.com

and knowing the journal is volatile (RAM-only) unless `/var/log/journal` exists — so logs can vanish on reboot. `journalctl -b -1` to read the previous boot after a crash is invaluable.

📚 **Official docs**

- journalctl(1)

Q106 A server won't boot. How do you recover? (GRUB, rescue/emergency mode, initramfs)

Boot & systemd

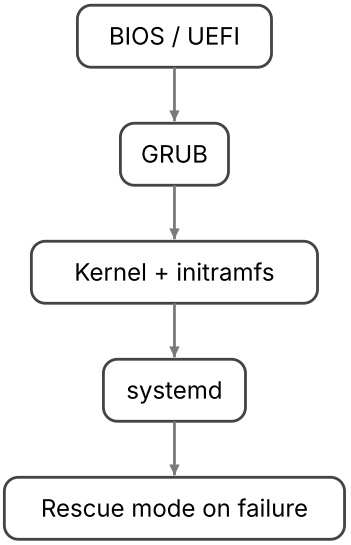
🎙 **Spoken answer · ~80 s**

For a server that won't boot, I first identify the stage of failure. No bootloader or no bootable device points to firmware, disk, or bootloader. GRUB appears but the kernel panics: kernel, initramfs, or root mount trouble. Boots but drops into emergency mode: a service or fstab mount failed. Recovery techniques: at the GRUB menu, edit the entry, remove quiet to see messages, add systemd.unit=rescue.target or emergency.target for a minimal shell, or init=/bin/bash as a last resort, then remount root read-write. Select an older kernel from the menu if the newest fails. If the system won't boot at all, boot from live or rescue media, mount the root filesystem, chroot into it, and repair: fix fstab, reinstall GRUB, regenerate the initramfs, and run filesystem checks on unmounted devices. Read the journal for the failed boot to see exactly what failed. In the cloud, use the serial console or detach the root volume, attach it to a healthy instance to repair, and reattach. Afterwards, prevent recurrence: nofail on non-critical mounts, keep earlier kernels, test changes in staging, take snapshots before risky changes, and keep out-of-band access working. Document the procedure for the team.

Likely follow-ups

- How do you repair a system from rescue media with chroot?
- What does init=/bin/bash do and when is it appropriate?

🗺 **Boot recovery**



certs.eknathalabs.com

IDENTIFY THE FAILURE STAGE

```
No GRUB / "no bootable device"  -> firmware/bootloader/disk
GRUB shows but kernel panics     -> kernel/initramfs/root mount
Boots then drops to emergency    -> a service or fstab mount failed
"Give root password for maintenance" -> emergency.target (fstab is the usual cause)
```

RECOVERY TECHNIQUES

```
# At GRUB: edit the entry (e) and append a kernel arg:
systemd.unit=rescue.target     # minimal single-user-ish (or emergency.target)
init=/bin/bash                 # bypass init entirely -> raw shell (then mount -o
remount,rw /)
# Boot a previous kernel from the GRUB menu if a kernel update broke it.
```

FROM A RESCUE/LIVE ENVIRONMENT

```
# Boot live USB / cloud rescue, then chroot into the broken system to fix it:
mount /dev/sda2 /mnt; mount --bind /dev /mnt/dev; mount --bind /proc /mnt/proc; mount --
bind /sys /mnt/sys
chroot /mnt
# now fix /etc/fstab, reinstall grub (grub-install), rebuild initramfs (dracut/update-
initramfs)
journalctl -b -1 -p err        # what failed on the last boot
```

Interview tip: Structure it by stage (bootloader vs kernel/initramfs vs failed-mount/service), then the toolkit: edit GRUB to add `systemd.unit=rescue.target` or `init=/bin/bash`, boot a prior kernel after a bad update, and the **chroot-from-rescue** flow to repair fstab / reinstall GRUB / rebuild initramfs. Mentioning `journalctl -b -1` to see why the last boot failed, and that a bad fstab drops you to emergency mode, shows you've done real recoveries.

📚 Official docs

- systemd: boot process
- Google SRE Book: Emergency Response
- GNU GRUB manual

certs.eknathalabs.com

Q107 What changed from SysV init to systemd? Why was systemd adopted despite controversy?

Boot & systemd

🎙 Spoken answer · ~83 s

SysV init started services through sequential shell scripts in init.d, ordered by runlevel symlinks with numbered prefixes. It was slow because startup was serial, had no real dependency awareness, no built-in supervision to restart crashed daemons, and each script reimplemented start, stop, and status inconsistently; daemons had to double-fork into the background. Systemd replaced this with declarative unit files; parallel startup with proper dependency resolution, which makes boot much faster; socket and path activation, starting services on demand; process supervision with restart policies; cgroup tracking so every process of a service is known and can be cleanly stopped; unified logging through the journal; resource controls; timers; and sandboxing features. Why it was adopted despite controversy: the practical benefits were large and consistent, and major distributions standardised on it, so administrators got one common way to manage services. The criticisms: it's a large project with a wide scope, which some feel violates the Unix principle of small tools, it has a binary journal format, and a steeper learning curve and tight coupling between components. In practice the operational advantages, like reliable dependency ordering, supervision, and observability, won out. For an interview, show you understand both the benefits and the trade-offs.

Likely follow-ups

- What problems did SysV init have with dependencies?
- What is socket activation and why is it useful?

SYSV INIT

```
# Sequential shell scripts in /etc/init.d, ordered by runlevel symlinks (S/K numbers).
# Slow (serial), no dependency awareness, no built-in process supervision,
#   each script reinvents start/stop/status. Daemons double-fork to background.
```

WHAT SYSTEMD ADDED

```
- Parallel startup with real dependency resolution -> much faster boot
- Process supervision (Restart=), proper readiness (Type=notify)
- cgroup-based tracking (knows ALL a service's processes — clean stop/limits)
- Socket + timer + path activation; unified logging (journald)
- Consistent declarative units instead of bespoke shell scripts
- Sandboxing directives (ProtectSystem, NoNewPrivileges, ...)
```

THE CONTROVERSY

Critics argued it violates Unix "do one thing well" (it absorbed logging, networking, login, etc.), increased complexity, and centralized too much in PID 1. It won anyway because the reliability, speed, supervision, and consistency solved real operational pain that SysV scripts couldn't.

Interview tip: Contrast concretely: SysV = serial bespoke shell scripts, no dependency/supervision; systemd = parallel, dependency-aware, cgroup-tracked, self-supervising, declarative. The cgroup tracking point is key — systemd reliably knows and controls every process a service spawns (clean

certs.eknathalabs.com

stop, resource limits), which SysV's PID-file guessing couldn't. Acknowledging the "too monolithic / un-Unix" criticism shows balanced judgment.

📚 **Official docs**

- systemd.service(5)

Q108 How do systemd timers work, and why prefer them over cron?

Boot & systemd

🎙 **Spoken answer · ~84 s**

A systemd timer consists of two units: a service unit that defines what to run, and a timer unit that defines when. The timer uses OnCalendar for calendar-style schedules, similar to cron but more readable, or monotonic triggers relative to boot or last activation. Useful options: Persistent equals true, so if the machine was off at the scheduled time, the job runs on next boot, which cron can't do without anacron; RandomizedDelaySec adds jitter so a fleet doesn't run a job simultaneously, avoiding thundering herds on shared backends; and AccuracySec. Why prefer timers over cron: output and errors go to the journal with the unit's name, so logging is built in; the job runs in a clean, defined environment instead of cron's minimal one; the service unit gives resource limits, sandboxing, and dependency ordering; systemd prevents overlapping runs of the same service by default; systemctl list-timers shows last and next run; and you can start the job manually with systemctl start to test it exactly as the timer would. Cron is still simple and universal for trivial tasks, but for production jobs, timers give better observability and control. I add failure alerting, using OnFailure, or a heartbeat check, and keep jobs idempotent.

Likely follow-ups

- What does Persistent=true do?
- How do timers avoid overlapping runs?

A TIMER = TWO UNITS

```
# backup.service (what to run) + backup.timer (when):
# /etc/systemd/system/backup.timer
[Timer]
OnCalendar=*-*-* 02:30:00      # daily 02:30 (cron-like calendar syntax)
Persistent=true               # if missed (machine off), run on next boot
RandomizedDelaySec=300        # jitter to avoid thundering-herd across hosts
[Install]
WantedBy=timers.target

systemctl enable --now backup.timer
systemctl list-timers          # next/last run for all timers
```

WHY OVER CRON

Certs.Eknathalabs.com

```
+ Persistent=true catches missed runs (cron just skips if the box was off — anacron needed)
+ Output goes to the journal (journalctl -u backup) — cron emails/loses output
+ Dependencies + ordering (After=) and resource limits via the .service
+ RandomizedDelaySec spreads load across a fleet
+ Same tooling as everything else (systemctl status, supervision, monotonic timers)
```

Interview tip: The advantages that matter: `Persistent=true` (runs a missed job after downtime — cron silently skips, needing anacron), output in the journal (cron's lost/emailed output is a real pain), dependency ordering, resource control, and `RandomizedDelaySec` to de-synchronize a fleet. Also `OnUnitActiveSec=` for monotonic intervals. Knowing a timer is a .timer + .service pair, and `systemctl list-timers`, shows hands-on use.

📚 **Official docs**

- systemd.service(5)
- systemd.timer(5)

Q109 Explain /etc/passwd, /etc/shadow, and /etc/group. What's in each field?

Users & permissions

🎙 **Spoken answer · ~82 s**

These three files hold local account data. etc passwd is world-readable and has seven colon-separated fields: username, a password placeholder x meaning the hash is in shadow, UID, primary GID, the GECOS comment or full name, the home directory, and the login shell. A shell like nologin or false means the account can't log in interactively, used for service accounts. etc shadow is readable only by root and holds the actual password data: username, the password hash with an identifier showing the algorithm and salt, last change date, minimum and maximum age, warning period, inactivity period, and expiry date. A hash starting with an exclamation mark or asterisk means the password is locked or disabled. etc group lists group name, a password placeholder, GID, and a comma-separated list of member users, and a user's primary group comes from passwd while supplementary groups come from here. Security relevance: keep shadow protected; audit for accounts with UID zero besides root, accounts with empty passwords, unexpected shells, and stale accounts; and use tools like useradd, usermod, and passwd instead of editing by hand, or vipw if editing is needed. On networked systems, LDAP or SSSD can supplement these files.

Likely follow-ups

- Why was the password hash moved out of /etc/passwd?
- What does an exclamation mark in a shadow hash field mean?

certs.eknathalabs.com

/ETC/PASSWD (WORLD-READABLE)

```
name:x:UID:GID:GECOS:home:shell
alice:x:1000:1000:Alice A:/home/alice:/bin/bash
# 'x' = password is in /etc/shadow (not here). Shell /usr/sbin/nologin or
#   /bin/false = account can't get an interactive login (service accounts).
```

/ETC/SHADOW (ROOT-ONLY, 0640)

```
name:$6$salt$hash:lastchange:min:max:warn:inactive:expire
# $6$ = SHA-512. A '!' or '*' in the hash field = locked / no password login.
# Aging fields drive password expiry policy.
chage -l alice          # view aging;  passwd -l alice  # lock
```

/ETC/GROUP

```
groupname:x:GID:member1,member2
# Primary group is in passwd (GID field); supplementary groups listed here.
id alice                # show uid, primary + supplementary groups
getent passwd alice     # works whether user is local OR from LDAP/SSSD
```

Interview tip: Key facts: passwords live in `/etc/shadow` (root-only), not `passwd` (world-readable); `$6$` = SHA-512; `!`/`*` = locked account; and a `nologin`/`false` shell is how service accounts are denied interactive login. Use `getent` (not `cat`) so it also resolves LDAP/SSSD users. Knowing primary (in passwd) vs supplementary (in group) groups, and `chage` for aging, rounds it out.

📚 **Start here (official docs)**

- Linux man-pages project

certs.eknathalabs.com

Q110 How does centralized authentication work (LDAP/AD, SSSD)? Why not local accounts everywhere?

Users & permissions

🎙 Spoken answer · ~85 s

Local accounts on every host don't scale: creating and removing users across hundreds of servers is error-prone, UIDs become inconsistent, there's no central audit trail, and offboarding misses a host, leaving a live account. Centralised authentication keeps identities in a directory such as LDAP, Active Directory, or FreeIPA, and hosts consult it. On Linux, SSSD is the common client: it connects to the directory, caches credentials and identity information so logins work during short outages, and plugs into the system through PAM for authentication and NSS for user and group lookups, configured in nsswitch.conf. Benefits: one place to create, change, and disable accounts; consistent UIDs and groups; central password and MFA policy; group-based access control and sudo rules distributed centrally; SSH keys or certificates stored in the directory; and a unified audit trail. For SSH access, combine with host-based access control rules. Considerations: the directory becomes critical infrastructure and needs high availability; use TLS to protect traffic; keep a local break-glass admin account for when the directory is unreachable; cache settings affect how quickly changes and revocations take effect; and in cloud environments, identity providers and short-lived credentials through SSO may replace long-lived accounts. Test offboarding to confirm access is actually removed everywhere.

Likely follow-ups

- What does SSSD cache and why?
- Why keep a local break-glass account?

THE PROBLEM WITH LOCAL ACCOUNTS

```
# Local /etc/passwd per host = unmanageable at scale: creating/removing users
#   on 500 boxes, inconsistent UIDs, no central audit, offboarding misses a host
#   and leaves a live account. Doesn't scale and is a security risk.
```

CENTRALIZED AUTH

```
# A directory (LDAP / Active Directory / FreeIPA) holds users/groups centrally.
# SSSD on each host queries it (with caching) and integrates via NSS + PAM:
#   NSS  -> resolves users/groups (getent sees directory users)
#   PAM  -> authenticates logins against the directory
# Add/disable a user once -> effective on every host. Offline cache = login works
#   even if the directory is briefly unreachable.
```

MODERN ALTERNATIVES

```
- SSH certificates (CA-signed, short-lived) — no per-host keys to manage/revoke
- Just-in-time access / bastions (SSM, Teleport, IAP) — no standing accounts at all
- Config mgmt (Ansible) can manage local users, but a directory scales better for humans
```

Interview tip: Frame it as scale + security + offboarding: local accounts don't scale and leave orphaned access. Centralized directory (AD/LDAP/FreeIPA) + SSSD wires in via **NSS** (identity resolution) and **PAM** (authentication), with caching for resilience. The forward-looking answer mentions short-lived SSH

certs.eknathalabs.com

certificates and JIT/bastion access (Teleport/SSM) replacing standing accounts entirely — strong for a platform/SRE role.

📚 **Official docs**

- AWS Systems Manager Session Manager

Q111 **Explain ACLs (setfacl/getfacl) beyond standard permissions. When do you need them?** Users & permissions

🎙 **Spoken answer · ~86 s**

Classic Unix permissions only provide three scopes: the owner, one group, and everyone else. That can't express a requirement like alice and bob, in different groups, both have read-write on a file, while carol has read-only, without creating awkward extra groups. POSIX ACLs add per-user and per-group entries. getfacl shows them, setfacl minus m adds or modifies entries, and removing works with minus x for an entry or minus b for all. Default ACLs on a directory are inherited by newly created files and subdirectories, which solves the shared-directory problem where new files get the wrong access. The ACL mask limits the maximum effective permission for named users and groups, which can silently reduce permissions, and a plus sign after the mode in ls indicates an ACL exists. When I need them: shared project directories with multiple teams, granting a service account access to another team's files without changing ownership, and fine-grained access on data directories. Considerations: the filesystem must have ACL support, which is the default on modern ones; copy and backup tools must preserve ACLs, for instance rsync with the A option; and too many ACL exceptions make access hard to audit, so prefer groups for simple cases and document the ACLs in configuration management.

Likely follow-ups

- What does the ACL mask do?
- How do default ACLs help shared directories?

THE LIMIT OF CLASSIC PERMS

```
# Standard mode bits give exactly THREE scopes: owner, ONE group, others.
# That can't express "alice and bob (in different groups) both read-write, but
#   carol read-only" without contortions. POSIX ACLs add per-user/per-group entries.
```

USING ACLS

```
getfacl file                          # view ACLs
setfacl -m u:alice:rw file            # grant alice rw (in addition to mode bits)
setfacl -m g:devs:rx dir              # grant a group
setfacl -x u:alice file               # remove an entry
# Default (inherited) ACLs on a directory -> new files inherit them:
setfacl -d -m g:devs:rwx /shared
ls -l file    # a '+' after the mode (rw-r--r--+) means ACLs are present
```

certs.eknathlabs.com

WHEN TO USE

Shared directories where multiple users/groups need different access than the single-owner/single-group model allows — collaborative project dirs, mixed-team file shares. For simple cases, prefer a well-designed group (SGID dir) over ACL sprawl, which gets hard to audit.

Interview tip: The core point: classic perms have only owner/group/other — ACLs add arbitrary per-user and per-group grants. Know the + in `ls -l` signals ACLs present, and default ACLs make a directory propagate permissions to new files. The balanced take: reach for ACLs when group design genuinely can't express the need, but don't over-ACL — it becomes an auditing headache. Prefer SGID-group folders for simple shared-write cases.

📚 **Official docs**

- Linux man-pages: path_resolution(7)

Q112 What is the difference between su, sudo, su -, and a login vs non-login shell?

Users & permissions

🎙 **Spoken answer · ~85 s**

su switches user, and sudo runs a command as another user, typically root. With su to a user, or su alone for root, you need the target's password and get a full shell. sudo needs your own password, is scoped by the sudoers policy, and logs every command, so it's preferred: there's no shared root password, authorisation is per command, and you get an audit trail. The dash matters. su alice starts a non-login shell, which keeps your current environment, working directory, and variables, and might use the wrong PATH. su dash alice starts a login shell, loading alice's profile and environment as if she had logged in, with her home directory as the working directory. Similarly sudo minus i gives a login shell as root, while sudo minus s gives a shell without a full login environment. The login versus non-login distinction determines which startup files are read: login shells read profile files, interactive non-login shells read bashrc, and scripts read neither unless told to. Practical implication: environment differences cause it works for me but not under sudo or cron problems, such as a different PATH. For security, avoid leaving root shells open, prefer sudo for single commands, and disable direct root login.

Likely follow-ups

- What is the difference between su and su -?
- Why do cron jobs not see your shell profile?

SU VS SUDO

```
su - alice        # become alice (needs ALICE's password) — full switch
su                # become root (needs ROOT's password)
sudo cmd          # run ONE command as root (needs YOUR password; audited; scoped)
# sudo is preferred: no shared root password, per-command authorization, logged.
```

THE CRUCIAL DASH: SU VS SU -

certs.eknathalabs.com

```
su  alice    # NON-login shell: keeps your env, your PWD, partial PATH
su - alice   # LOGIN shell: clean env as if alice logged in fresh —
             #   sources /etc/profile + ~/.bash_profile, correct PATH/HOME/umask
# The "-" (or -l) is why "su root" sometimes can't find sbin commands but "su -" can.
```

LOGIN VS NON-LOGIN SHELLS

```
Login shell    : reads /etc/profile + ~/.bash_profile|~/.profile (real login, ssh, su -)
Non-login shell: reads ~/.bashrc only (new terminal in a GUI, su without -, subshells)
# This is why "my PATH/alias works in one but not the other" — different rc files.
```

Interview tip: Two distinctions interviewers love: (1) prefer `sudo` over `su` (no shared root password, per-command scope, audit trail), and (2) the `-` in `su -` gives a full **login** shell with a clean environment, which is why commands/PATH sometimes only work with the dash. Tie it to login (`.bash_profile`) vs non-login (`.bashrc`) shells — the root cause of "my env var/alias isn't loaded" confusion.

📚 **Official docs**

- sudoers(5) manual

Q113 How do you audit and manage user access for security and compliance?

Users & permissions

🎙 **Spoken answer · ~82 s**

Auditing user access combines inventory, review, and monitoring. Inventory: list human accounts, those with UID of a thousand or more, accounts with empty passwords from the shadow file, anyone besides root with UID zero, members of sudo and wheel groups, and each user's authorized_keys files, to see who has key access. Find stale accounts with lastlog and last: users who haven't logged in for months should be disabled or removed. Review sudoers and sudoers.d rules for overly broad grants, such as NOPASSWD ALL. Check service accounts, shells, and password ageing policy. Compare against the source of truth, the HR or identity system, to catch accounts that should no longer exist. Monitoring: centralise authentication logs, sudo logs, and audit logs, and alert on unusual events such as new privileged users, logins from unexpected places, failed login spikes, and changes to sensitive files. For compliance: regular access reviews where managers confirm that access is still needed, evidence of joiner, mover, and leaver processes, MFA enforcement, and separation of duties. Automate the checks through configuration management and compliance scanners such as OpenSCAP or CIS benchmarks, and use centralised identity to make offboarding immediate. Document findings, remediate, and track completion.

Likely follow-ups

- How do you find stale accounts?
- Why compare against the HR system?

AUDIT WHO CAN ACCESS WHAT

certs.eknathalabs.com

```
getent passwd | awk -F: '$3>=1000'      # human accounts
awk -F: '($2==""){print $1}' /etc/shadow # accounts with NO password (danger)
grep -Po '^sudo:.*:\K.*' /etc/group      # who's in sudo/wheel
ls -l ~/.ssh/authorized_keys (per user)  # who has key access
lastlog                                  # last login per user (find stale accounts)
last / lastb                             # login history / failed logins
```

FIND RISKY CONFIGS

```
awk -F: '$3==0 {print $1}' /etc/passwd   # any UID 0 besides root = backdoor!
find / -perm -4000 -type f 2>/dev/null   # SUID-root binaries (attack surface)
# Review sudoers (visudo -c to validate), check for NOPASSWD: ALL grants.
```

ONGOING HYGIENE

```
- Disable (don't just delete) departed users: usermod -L; expire the account
- Centralized auth so offboarding is one action (not per-host)
- auditd to log access to sensitive files; ship logs to a SIEM
- Regular access reviews; least privilege; rotate/expire credentials
```

Interview tip: Show concrete checks, not platitudes: find **non-root UID-0 accounts** (instant backdoor), passwordless accounts, sudo/wheel membership, SUID binaries, and stale logins (`lastlog`). Then process: disable-not-delete on offboarding, centralized auth so removal is one action, auditd + SIEM for sensitive-file access, and periodic access reviews. The UID-0-backdoor check and "offboarding must be centralized" are standout points.

📚 **Official docs**

- Linux audit documentation (auditd)

certs.eknathalabs.com

Q114 Explain umask, default permissions, and how to set up a secure shared team directory.

Users & permissions

🎙 Spoken answer · ~89 s

New files get their permissions from a base, six six six for files and seven seven seven for directories, minus the umask. A umask of zero zero two gives files six six four and directories seven seven five, which is group-writable and good for shared directories. A umask of zero two two gives six four four and seven five five, the typical default, with group read-only. A umask of zero seven seven gives six zero zero and seven zero zero, fully private. A secure shared team directory: create a group for the team and add members; create the directory owned by that group; set the mode so owner and group have full access and others have none, such as two seven seven zero; the leading two is the SGID bit, so new files and subdirectories inherit the team group instead of the creator's primary group. Make sure users' umask allows group write, set via the shell profile, or use default ACLs on the directory so new files get the right group permissions regardless of umask. Optionally add the sticky bit if members shouldn't delete each other's files. Verify by creating files as different members, and test that outsiders are denied. Document the design, manage group membership centrally, and audit periodically for permissions drift.

Likely follow-ups

- What does the SGID bit do on a directory?
- How do default ACLs complement the umask?

HOW NEW FILES GET THEIR PERMISSIONS

```
# Base: files 666, dirs 777, minus the umask.
umask 002 -> files 664, dirs 775   (group-writable: good for shared dirs)
umask 022 -> files 644, dirs 755   (group read-only: typical default)
umask 077 -> files 600, dirs 700   (private)
```

SECURE SHARED TEAM DIRECTORY

```
groupadd projteam
chgrp projteam /srv/project
chmod 2775 /srv/project          # 2 = SGID: new files inherit 'projteam' group
# (optional) sticky bit so users can't delete each other's files:
chmod 3775 /srv/project          # 3 = SGID + sticky
# Ensure members create group-writable files:
#   set umask 002 for them (so files are 664, group can edit)
# (optional) default ACL to force inheritance regardless of umask:
setfacl -d -m g:projteam:rwx /srv/project
```

The common breakage: SGID makes new files inherit the group, but if users have `umask 022`, files are created group-*read-only* — teammates can read but not edit. You need SGID **and** a group-writable umask (002), or a default ACL, for true shared write.

certs.eknathalabs.com

Interview tip: The full recipe — SGID on the dir (group inheritance) + `umask 002` (group-writable files) + optional sticky bit (no deleting others' files) + optional default ACL for guaranteed inheritance. The insight that impresses: SGID alone isn't enough because `umask 022` still makes files group-read-only — the umask is the missing piece people forget. `/tmp` is the canonical sticky-bit example.

📚 **Official docs**

- Linux man-pages: path_resolution(7)

Q115 What makes a bash script robust? Explain set -euo pipefail and common pitfalls.

Scripting & automation

🎙 **Spoken answer · ~87 s**

A robust bash script starts with a safety preamble: set minus e to exit on any command failure, minus u to treat unset variables as errors, and minus o pipefail so a pipeline fails if any stage fails. The unset-variable check catches typos that could be catastrophic, like rm minus rf of a variable that is empty, which becomes rm minus rf of root. Setting IFS to newline and tab makes word splitting safer. Add a trap for cleanup on exit and for error reporting with the line number. Quote every variable expansion, use double brackets for tests, use local variables in functions, and use arrays for lists. Common pitfalls: set minus e has surprising exceptions, such as commands in conditions or after and-or lists, so don't rely on it blindly and check critical commands explicitly; parsing ls output; unquoted globs and spaces in filenames; forgetting that pipes run in subshells, so variables set inside a while loop in a pipeline are lost; race conditions with temp files, so use mktemp; and not handling signals. Also validate arguments, provide usage messages, log with timestamps, use meaningful exit codes, and avoid secrets on the command line. Lint with shellcheck, test with bats, and know when to move to Python.

Likely follow-ups

- What are the surprises of set -e?
- Why can variables set inside a piped while loop disappear?

certs.eknathalabs.com

THE SAFETY PREAMBLE

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
# -e : exit on any command failure (no silently continuing after an error)
# -u : error on UNSET variables (catches typos: rm -rf "$prefx/" -> empty -> rm -rf /)
# -o pipefail : a pipeline fails if ANY stage fails (not just the last)
# IFS: safer word splitting (spaces in filenames don't explode)
```

COMMON PITFALLS

```
- Unquoted variables: rm "$dir"/*  NOT rm $dir/*  (spaces/globs bite)
- ${var:?msg} to require a var; ${var:-default} for defaults
- Don't parse `ls`; use globs or find -print0 | xargs -0
- set -e has surprising exceptions (commands in if/&&/||, functions) — test!
- Trap for cleanup:  trap 'rm -f "$tmp"' EXIT
```

VALIDATE BEFORE RUNNING

```
shellcheck script.sh        # static analysis — catches most bugs before prod
bash -n script.sh           # syntax check without executing
```

The catastrophe pattern: `rm -rf "$DIR/"` where `$DIR` is unset/empty becomes `rm -rf /`. `set -u` (and quoting + `${DIR:?}`) prevents exactly this. Always guard destructive commands.

Interview tip: Lead with `set -euo pipefail` and explain each flag's failure it prevents — especially `-u` catching the unset-variable-in-`rm -rf` disaster. Mention quoting variables, `trap ... EXIT` for cleanup, avoiding `ls` parsing (use `find -print0`), and running `shellcheck`. Knowing `set -e`'s gotchas (it doesn't fire inside `if/||`) shows you've been burned by it for real.

📚 **Official docs**

- Google Style Guide: Shell
- GNU Bash reference manual

certs.eknathalabs.com

Q116 How do you make a script idempotent and safe to re-run? Why does it matter for automation?

Scripting & automation

🎙 Spoken answer · ~86 s

Idempotent means that running the script once or many times yields the same final state, without errors or duplicate effects. Non-idempotent examples: appending a line to a hosts file on every run, creating a user that already exists and failing, or making a directory that exists. The idempotent versions check before acting or use naturally idempotent operations: grep to see whether the line exists before appending, id before useradd, mkdir with the parents flag, ln with force, install for files, and writing config from a template and replacing it only if different. Declarative tools like Ansible, Terraform, and Puppet build this in by describing the desired state and acting only on differences. It matters for automation because reruns after partial failures are safe, so recovery is simply run it again; configuration can be applied continuously to correct drift; pipelines can retry steps; and the same code works for both bootstrap and update. Also consider check and dry-run modes to preview changes, locking so concurrent runs don't collide, and handlers so services restart only when configuration changed. Some things aren't naturally idempotent, such as database migrations or sending emails, so track state with markers or migration tables. Test by running twice and confirming the second run changes nothing.

Likely follow-ups

- How do you test that a script is idempotent?
- How do migrations achieve idempotency?

IDEMPOTENT = SAME RESULT NO MATTER HOW MANY TIMES YOU RUN IT

```
# NOT idempotent (breaks/duplicates on re-run):
echo "10.0.0.5 db" >> /etc/hosts          # appends a dup every run
useradd deploy                            # fails the 2nd time
mkdir /opt/app                            # fails if it exists

# Idempotent versions (check-then-act / declarative):
grep -q "db" /etc/hosts || echo "10.0.0.5 db" >> /etc/hosts
id deploy &>/dev/null || useradd deploy
mkdir -p /opt/app
ln -sfn /opt/app/v2 /opt/app/current      # -f overwrite, -n no-deref
```

WHY IT MATTERS

```
# Automation re-runs: retries after partial failure, config drift correction,
#   running the same playbook on 100 hosts in mixed states. A non-idempotent
#   script that half-ran leaves a mess and can't be safely retried.
# This is the whole model of Ansible/Terraform: declare desired state, converge.
```

Also make it safe: dry-run/`--check` mode, back up before modifying, and fail loudly (`set -e`) rather than continuing in a half-applied state. Re-running should never duplicate, error on "already done," or corrupt.

certs.eknathalabs.com

Interview tip: Define idempotency (re-runnable to the same end state) and show the check-then-act pattern (`grep -q || append`, `id || useradd`, `mkdir -p`, `ln -sfn`). The big-picture tie-in: this is exactly why config-management tools (Ansible/Terraform) are declarative — they converge to desired state idempotently. Critical for retries and running across a fleet of hosts in unknown states.

📚 **Official docs**

- RFC 9110: HTTP Semantics (status codes, methods)

Q117 Explain process substitution, here-docs, xargs, and parallelism in shell.

Scripting & automation

🎙 **Spoken answer · ~85 s**

Here-documents let me embed multi-line text in a script: a quoted delimiter prevents variable expansion so the text is literal, and an unquoted one expands variables, useful for generating config files. A here-string passes a single string as input. Process substitution treats a command's output as a file, for example diffing the output of two commands without temp files. xargs converts input into command arguments; with null-delimited input from find and the corresponding option in xargs, it handles odd filenames safely. For parallelism, xargs with the P option runs several processes concurrently, such as compressing logs eight at a time, with n controlling how many arguments each invocation takes. GNU parallel offers richer control, including job logs and retries. Caveats: parallel jobs interleave output, so write to separate files or use tools that group output; limit parallelism to avoid overwhelming a downstream service or disk; handle failures, as xargs returns an error if any invocation fails; and avoid shared state between parallel runs. Also use background jobs with wait for simple fan-out. Always quote variables and use null-separated pipelines for filenames. These features let shell scripts be concise and efficient, but if the logic grows complex, a language like Python becomes more maintainable.

Likely follow-ups

- Why use find -print0 with xargs -0?
- What does a quoted here-doc delimiter change?

certs.eknathalabs.com

HERE-DOCS / HERE-STRINGS

```
cat <<'EOF' > /etc/myapp.conf       # 'EOF' (quoted) = no variable expansion
verbatim $not_expanded
EOF
grep foo <<< "$mystring"            # here-string: feed a var as stdin
```

PROCESS SUBSTITUTION

```
# Treat a command's output as a FILE (no temp file needed):
diff <(sort a.txt) <(sort b.txt)    # compare two sorted streams
while read l; do ...; done < <(some_cmd)   # avoid subshell var-loss of a pipe
```

XARGS + PARALLELISM

```
find . -name '*.log' -print0 | xargs -0 -P 8 -n 1 gzip   # 8-way parallel, NUL-safe
seq 1 100 | xargs -P 16 -I{} ./process.sh {}             # 16 parallel workers
# GNU parallel for richer control (job logs, retries, --joblog).
# Always -print0 | xargs -0 to survive filenames with spaces/newlines.
```

Interview tip: The highest-value items: `diff <(cmd1) <(cmd2)` (process substitution — comparing command outputs as files), `xargs -P` / GNU `parallel` for easy parallelism, and the NUL-safe `find -print0 | xargs -0` idiom for filenames with spaces. Mentioning the pipe-creates-a-subshell gotcha (use `while ... done < <(cmd)` so variables persist) is an advanced detail that lands well.

📚 **Official docs**

- Terraform CLI: apply
- GNU coreutils manual

Q118 **How do you handle secrets in scripts and automation? What are the anti-patterns?** Scripting & automation

🎙 **Spoken answer · ~86 s**

Secrets in scripts and automation need careful handling, and the anti-patterns are common. Don't hardcode secrets in scripts, since they end up in git, backups, and logs. Don't pass them as command-line arguments, because they're visible to every user in the process list and in shell history. Avoid secrets in environment variables that get dumped by crash reporters, debug output, or child processes, and don't print them in logs or CI output. Don't store them in world-readable files or bake them into images. Better practices: fetch secrets at runtime from a secrets manager such as Vault, AWS Secrets Manager, or cloud-native equivalents, using the workload's identity so there's no long-lived credential to leak; or use short-lived credentials. If a file is needed, make it mode six hundred, owned by the service account, in a tmpfs, and delete it afterwards. Pass secrets via standard input or files read by the application, rather than arguments. In CI, use the platform's secret store with masking, and restrict which branches and jobs can access it. Use least privilege and scoped secrets, rotate them regularly, and have a rotation process ready for leaks. Add secret scanning in pre-commit and CI, and audit access. Turning off shell tracing around sensitive commands also matters.

Likely follow-ups

- Why is passing a secret as a CLI argument unsafe?
- What is better than a long-lived credential?

ANTI-PATTERNS (DON'T DO THESE)

certs.eknathalabs.com

```
- Hardcoding secrets in the script (ends up in git, shared, logged)
- Passing secrets as CLI args:  ./app --password hunter2
    -> visible in `ps aux`, shell history, process listing to ALL users!
- Secrets in environment that get dumped by error handlers / crash reports
- export VAR=secret in a sourced file that's world-readable
```

BETTER APPROACHES

```
- Fetch at runtime from a secrets manager (Vault, AWS Secrets Manager, SSM
  Parameter Store) using the host's IAM role/identity — never store the secret
- Read from a file with tight perms (chmod 600, owned by the service user)
- Pass via stdin, not argv:  printf '%s' "$secret" | mycmd --password-stdin
- Mounted secret (tmpfs / K8s secret) rather than baked into the image
```

HYGIENE

```
set +x near secret handling      # don't echo secrets when tracing (set -x)
# Avoid logging the secret; scrub it from error output; rotate regularly.
# git-secrets / gitleaks in CI to block accidental commits.
```

Interview tip: The strongest specific point: **never pass secrets as command-line arguments** — they're visible in `ps` and shell history to other users; use stdin (`--password-stdin`) or files instead. Then the modern pattern: fetch from a secrets manager at runtime via the host's IAM identity so the secret is never stored. Add `set +x` around secret handling and gitleaks in CI. Shows real security maturity.

📚 **Official docs**

- Microsoft: Anti-patterns (performance antipatterns)

certs.eknathalabs.com

Q119 Compare bash vs Python for automation. When do you reach for each?

Scripting & automation

🎙 Spoken answer · ~87 s

I choose based on what the task mostly is. Bash shines when I'm gluing command-line tools together with pipes, doing file and text wrangling, and quick one-offs; it's already available everywhere with no dependencies, and when the task is essentially running commands in order, bash is natural and concise. Python is better when there's real logic: data structures like lists and dictionaries, parsing JSON or YAML, calling APIs with proper error handling, retries and timeouts, working with dates, and anything needing tests, modules, and maintainability. Python also gives clearer error handling, cleaner string handling, a rich standard library, and portability across systems, while bash has quirky quoting, word splitting, and error semantics that make larger scripts fragile. Rule of thumb: if the script is mostly calling other programs and is under about fifty lines, use bash; if it needs data structures, complex conditions, concurrency, or will be maintained by a team, use Python. Hybrid approaches work too: Python calling subprocess for tools, or bash wrappers for deployment steps. Also consider dependencies: for minimal containers, bash or a single static binary may be the only option, while for long-lived tooling Go is another choice. Whatever I pick, I lint it, test it, handle errors, and keep it in version control.

Likely follow-ups

- At what point would you rewrite a bash script in Python?
- What are the main fragilities of larger bash scripts?

BASH SHINES WHEN

```
- Gluing CLI tools together (pipes), file/text wrangling, quick one-offs
- It's already there (no deps), and the task IS running commands
- Short, linear "do these commands in order" automation
# Rule of thumb: if it's mostly calling other programs, bash is natural.
```

PYTHON SHINES WHEN

```
- Real logic: data structures, JSON/YAML/API parsing, error handling, retries
- Anything > ~50-100 lines, or that needs tests/maintenance by a team
- HTTP/cloud SDK calls, structured data, complex conditionals
# Bash's arrays, error handling, and string handling get painful fast.
```

THE DECISION

```
# Reach for Python when you find yourself:
#   - parsing JSON with grep/sed/awk (use a real parser / jq at minimum)
#   - building nested data or doing real arithmetic/string manipulation
#   - needing robust error handling, retries, and unit tests
# A common smell: "this bash script now has functions and a config parser" -> Python.
```

Interview tip: Give a clear heuristic rather than dogma: bash for gluing commands and short linear tasks; Python once there's real logic, JSON/API handling, or it crosses ~100 lines / needs tests. The telltale sign to switch is "parsing structured data with sed/awk" or "bash script growing functions and

certs.eknathalabs.com

error handling." Mentioning `jq` as the middle ground for JSON-in-bash, and maintainability/testing as deciding factors, shows engineering judgment.

📚 **Official docs**

- GNU Bash reference manual
- Python Documentation

Q120 How do you debug a failing shell script? (set -x, trap, exit codes, common gotchas)

Scripting & automation

🎙 **Spoken answer · ~86 s**

To debug a failing shell script, I start with tracing: bash minus x prints each command with its expanded arguments as it runs; I can enable it for a section with set minus x and set plus x, and improve the output by setting PS4 to include the file and line number. bash minus n checks syntax without running anything. Exit codes are central: dollar question mark holds the last command's status, zero meaning success; with set minus e, the script exits at the first failure, so I use a trap on ERR to print the line number and failing command. A trap on EXIT handles cleanup. Common gotchas: unquoted variables splitting on spaces or expanding globs; unset variables, caught by set minus u; a pipeline hiding failures without pipefail; commands behaving differently in cron or CI because of a different PATH or environment; subshell variable scope in pipes; Windows line endings breaking the shebang; relying on bashisms when running under sh; and set minus e not triggering in conditionals. Run shellcheck, which catches many of these, and reproduce in a clean environment, for example env minus i. For tricky logic, add echo statements, test pieces interactively, and shrink the script to the smallest failing case.

Likely follow-ups

- How does PS4 improve set -x output?
- Why can a script work interactively but fail in cron?

TRACE EXECUTION

```
bash -x ./script.sh           # print each command + expansions as they run
set -x ... set +x             # trace just a section
PS4='+ ${BASH_SOURCE}:${LINENO}: '   # add file:line to the trace output
bash -n ./script.sh           # syntax check WITHOUT running
```

EXIT CODES + ERROR TRAPS

```
echo $?                       # exit status of last command (0 = success)
cmd || echo "failed with $?"  # check explicitly
# Trap errors with context:
trap 'echo "ERR line $LINENO: $BASH_COMMAND (exit $?)" >&2' ERR
set -e                        # then it stops + reports on first failure
```

COMMON GOTCHAS

certs.eknathalabs.com

```
- Unquoted vars / word-splitting; empty var in a path -> wrong target
- `cd foo && rm *` where cd silently failed (without set -e) -> rm in wrong dir!
- Pipeline hides failure without `pipefail` ($? is only the LAST command)
- Variable set inside a `cmd | while read` loop is lost (subshell) -> use < <(cmd)
- CRLF line endings (edited on Windows) -> "command not found" / weird errors
```

Interview tip: The first move is `bash -x` (with a rich `PS4` showing line numbers), plus `bash -n` for syntax. Then exit-code discipline (`$?`, `pipefail` so pipeline failures aren't hidden) and an `ERR` trap printing `$LINENO`/`$BASH_COMMAND`. Name a real gotcha or two — the `cd && rm *` wrong-directory disaster, subshell variable loss in `cmd | while`, or CRLF endings — to show scar tissue, not theory. `shellcheck` prevents most of these.

📚 **Official docs**

- Kubernetes: Determine the reason for Pod failure
- GNU Bash reference manual

Q121 A production server is unresponsive over SSH but seems 'up'. How do you diagnose?

Troubleshooting

🎙 **Spoken answer · ~87 s**

If a production server seems up but SSH is unresponsive, I first try an out-of-band path, since SSH may be the only broken thing: the cloud serial console, VM console, or IPMI, which bypass sshd and the network. I also test from another host in the same network with ping and ss to see whether it's the network or the host. Then I distinguish how it fails. If the connection times out, it's the network, a firewall or security group change, or the host is down or overloaded. If it connects but hangs at login, the host is struggling: a full disk that can't write logs or session files, memory exhaustion and swapping, an extreme load, a hung NFS mount blocking the home directory or profile scripts, PID exhaustion so sshd can't fork, or DNS and PAM lookups timing out. If authentication fails, check keys, permissions on home directories, expired accounts, or a changed sshd config. On the console I look at load, memory, disk, inodes, processes, and the kernel log. Mitigation might be killing a runaway process, freeing space, or rebooting as a last resort after capturing evidence. For prevention: monitor host health, keep out-of-band access working, set resource limits, reserve resources for sshd, and alert before saturation.

Likely follow-ups

- How do you tell a network problem from a host problem when SSH hangs?
- What would you capture before rebooting a hung server?

CAN YOU GET IN AT ALL?

```
# Try the out-of-band path FIRST (SSH may be the only thing broken):
- Cloud serial console / VM console / IPMI/iDRAC — bypasses sshd + network
- A second host in the same VPC -> ping/ss to see if it's network vs host
```

WHY SSH SPECIFICALLY HANGS

certs.eknathalabs.com

```
Connects but hangs at login   -> disk full (can't write logs/home), or PAM
                                  stalling on LDAP/DNS, or load so high sshd can't fork
Connection refused            -> sshd died (but host up) — restart via console
Connection timeout            -> network/firewall/route, or host wedged
Slow then in                  -> reverse-DNS lookup on the client IP (UseDNS) stalling
```

ONCE ON THE CONSOLE

```
uptime              # load — is it 500? (runaway / fork bomb / I/O storm)
df -h; df -i        # disk full / inodes? (a full / breaks sshd logins)
dmesg -T | tail     # OOM kills, I/O errors, kernel issues
free -h; top         # memory exhaustion / swap death spiral
ss -s; ip a          # network/socket state
```

Interview tip: First instinct should be the **out-of-band console** (serial/IPMI) — don't keep retrying SSH when SSH might be exactly what's broken. Then decode the failure mode: refused = sshd dead, timeout = network/wedged, hangs-at-login = disk full or PAM/DNS stall, slow-login = reverse-DNS (`UseDNS`). On the console, check the usual killers: load, disk/inodes full, OOM, swap death-spiral. The disk-full-breaks-login and serial-console points are what distinguish real ops experience.

📚 **Start here (official docs)**

- Linux man-pages project

certs.eknathalabs.com

Q122 Intermittent failures happen only under load. How do you approach this hard class of bug?

Troubleshooting

🎙 Spoken answer · ~87 s

Failures that happen only under load usually mean I'm hitting a limit or a resource that's fine when idle. The usual suspects: file descriptor limits, producing too many open files errors; ephemeral port exhaustion and TIME_WAIT buildup; the conntrack table filling; connection pool or thread pool exhaustion; database connection limits; listen backlog overflow; memory pressure and garbage collection pauses; CPU throttling or saturation; lock contention and race conditions that need concurrency to appear; and rate limits on dependencies. Approach: first capture the failure signature, exact error messages and when they start relative to load, since the threshold tells me which limit. Check the resource counters at that moment: ulimit values and open descriptors, socket states with ss, kernel log messages like conntrack full or SYN drops, pool metrics, and saturation graphs. Reproduce in a controlled way with a load test that ramps up gradually, watching every resource, rather than guessing. Use tracing and profiling under load to find where time goes, and thread dumps to spot lock contention. Once I find the limit, raise it or remove the cause, for example pooling connections, adding backpressure, or fixing the race. Add monitoring and alerts for approaching limits, and keep the load test in CI so the regression is caught.

Likely follow-ups

- Which kernel messages point to a hit limit?
- How would you reproduce a load-only bug safely?

WHY 'UNDER LOAD' IS A CLUE

```
# Load-only failures usually mean hitting a LIMIT or a RESOURCE that's fine when idle:
- file descriptors:  ulimit -n / LimitNOFILE  ("too many open files")
- ephemeral ports / TIME_WAIT pileup ("cannot assign requested address")
- conntrack table full ("nf_conntrack: table full")
- connection pool / thread pool exhaustion (requests queue, then time out)
- memory pressure -> swap/OOM only at peak
- accept queue overflow (somaxconn) -> SYN drops at peak
```

CATCH IT IN THE ACT

```
# Continuous sampling so you have data WHEN it happens (it's intermittent):
sar -A (sysstat)  / node_exporter + Prometheus    # historical metrics
watch -n1 'ss -s; cat /proc/sys/net/netfilter/nf_conntrack_count'
dmesg -T -w &     # capture kernel limit messages live
# Correlate failure timestamps with the metric that spikes/saturates.
```

METHOD

Reproduce with load testing if possible, watch the USE metrics (saturation is the giveaway), and check limits before app code. Most "only under load" bugs are a saturated resource or a hit ceiling, not a logic bug.

Interview tip: The mental model: "only under load" = a limit/resource that's fine at rest. Rattle off the usual ceilings — fd limits, ephemeral ports/TIME_WAIT, conntrack, pool exhaustion, `somaxconn`, OOM at

carlseknathalabs.com

peak. The method that impresses: continuous metric collection (you can't debug intermittent issues without historical data) + correlate failure times with saturation, and load-test to reproduce. Emphasize checking limits before suspecting app logic.

📚 **Official docs**

- Netfilter documentation

Q123 **How do you debug a service that works locally but fails in production/containers?** Troubleshooting

🎙 **Spoken answer · ~87 s**

When a service works locally but fails in production or in a container, I enumerate what differs. Environment: missing or different environment variables, config files, and secrets; compare the actual environment of the running process. Network: DNS resolving differently, being unable to reach a dependency because of a firewall or security group, or binding to 127.0.0.1 instead of 0.0.0.0, so it works locally but is unreachable from outside. Permissions: running as root locally but as a non-root user in production, so it can't write to a directory or bind a low port; a read-only root filesystem; SELinux or AppArmor denials. Resources: OOM kills or CPU throttling from container limits, or file descriptor limits. Filesystem and paths: mounts missing, files not copied into the image, or case-sensitivity and line-ending differences. Dependencies and versions: different library, runtime, or base image versions, or an architecture mismatch like arm64 versus amd64. Time and locale: timezone and encoding. Data: production data size and shape. Approach: read the actual error in logs, exec into the container or run the same image locally with the same environment and user, compare configurations, and use strace for failing file or network calls. Fix by making environments consistent, with the same image, configuration as code, and a staging environment.

Likely follow-ups

- Why does binding to 127.0.0.1 work locally but fail in a container?
- How would you reproduce a production container failure locally?

certs.eknathalabs.com

ENUMERATE WHAT DIFFERS

```
Environment : missing/different env vars, config, secrets (compare actual env)
Network     : DNS resolves differently, can't reach a dependency, firewall/SG,
              binding 127.0.0.1 vs 0.0.0.0 (works locally, unreachable in prod)
Permissions : runs as root locally, non-root in prod -> can't write/bind <1024
Filesystem  : read-only rootfs, missing volume, different paths, SELinux labels
Resources   : tighter CPU/memory limits -> OOM/throttle only in prod
Dependencies: different library/runtime versions, missing package in the image
```

INSPECT THE RUNNING CONTAINER/HOST

```
docker exec -it <c> sh / kubectl exec -it pod -- sh   # get inside
env; cat /proc/1/environ                              # what env does it ACTUALLY have
id; cat /proc/1/limits                                # user + ulimits in effect
getent hosts dependency; curl -v http://dep:port      # DNS + reachability from INSIDE
ls -l /data; mount | grep ro                          # volumes / read-only fs
dmesg | grep -i oom; kubectl describe pod (OOMKilled?) # resource limits
```

Top offenders: binding to `127.0.0.1` (works locally, refuses remotely — bind `0.0.0.0`), missing env/secret, non-root can't bind low ports or write a path, and tighter memory limits causing prod-only OOM. "Works on my machine" is almost always an environment delta.

Interview tip: Frame it as systematically diffing environments: env/config, network (DNS + reachability + 127.0.0.1-vs-0.0.0.0 binding), user/permissions (root local vs non-root prod), filesystem (read-only/volumes/SELinux), and resource limits (prod-only OOM). The move that lands: `exec` into the actual running container and inspect its real env, limits, and reachability from *inside* — not from your laptop. The localhost-binding gotcha is a frequent real culprit.

📚 **Start here (official docs)**

- Linux man-pages project

Q124 **Describe your end-to-end methodology for triaging a production incident.** Troubleshooting

🎙 **Spoken answer · ~84 s**

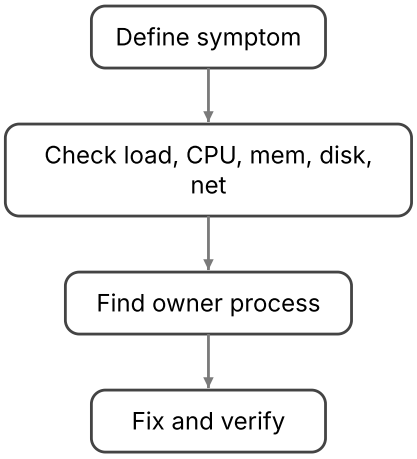
My incident triage methodology is: stabilise first, then diagnose. Step one, assess impact and set severity: who is affected, how badly, and since when. Mitigate before root-causing: stop the bleeding with the quickest safe action, such as rolling back the last deploy, scaling out, failing over, shedding load, or disabling a feature flag. Recovery beats diagnosis in the moment. Step two, correlate with change, because most incidents follow a change: check recent deploys, config changes, infrastructure changes, traffic shifts, and dependency incidents, using a change timeline on the dashboards. Step three, organise: declare an incident, name an incident commander, open one channel, assign roles for operations, communications, and a scribe, and set an update cadence. Step four, diagnose with data, narrowing from the symptom: use the golden signals and USE method to locate the failing layer, form specific hypotheses, test them one at a time with the least risky checks, and avoid changing several things at once. Step five, resolve and verify with user-facing metrics, not just that a command ran, and watch for recurrence. Step six, communicate throughout, to stakeholders and customers in plain language. Step seven, follow up with a blameless postmortem, timeline, contributing factors, and tracked action items.

Likely follow-ups

- Why mitigate before finding the root cause?
- What roles do you assign at the start of an incident?

certs.eknathalabs.com

🗺 **Triage method**



1. ASSESS IMPACT + STABILIZE FIRST

```
# Mitigate BEFORE root-causing — stop the bleeding:
- What's the user impact / blast radius? (set severity)
- Quick stabilizers: roll back the last deploy, scale out, failover, shed load,
  disable the bad feature flag. Recovery > diagnosis in the moment.
```

2. CORRELATE WITH CHANGE

```
# Most incidents follow a CHANGE. "What changed right before it started?"
- recent deploy / config push / migration / cert expiry / traffic spike /
  dependency incident / infra change. Check deploy + change logs first.
```

3. NARROW WITH SIGNALS (THE FOUR GOLDEN SIGNALS)

```
Latency, Traffic, Errors, Saturation -> which dashboard moved, and when?
# Follow the signal down: which service, which dependency, which resource.
# Logs + traces at the boundary that broke. Form a hypothesis, test it cheaply.
```

4. COMMUNICATE + CLOSE THE LOOP

```
- Keep stakeholders updated (status, ETA) on a clear channel
- After recovery: blameless postmortem -> root cause -> action items
  (monitoring gap? missing guardrail? add the alert/test/runbook that catches it next time)
```

Interview tip: The senior framing is **mitigate first, root-cause second** — roll back / failover / scale to restore service, THEN investigate. Then "what changed?" (most incidents are change-induced), narrow via the golden signals (latency/traffic/errors/saturation), and communicate throughout. Closing with a **blameless postmortem** that produces preventive action items (the alert/guardrail that would've caught it) is exactly the SRE maturity interviewers want.

📚 **Official docs**

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture

certs.eknathalabs.com

Q125 How do you avoid jumping to conclusions when debugging? Describe a disciplined approach.

Troubleshooting

🎙 Spoken answer · ~86 s

The trap in debugging is pattern-matching to a past incident, fixing the wrong thing, wasting an hour, or changing five things at once and never learning what worked. Confirmation bias makes it worse, because I look only for evidence that fits my first guess. A disciplined approach is a loop. First, state the problem precisely: what is failing, since when, for whom, and what changed. Second, gather evidence before theorising: logs, metrics, traces, and the exact error messages, and establish a timeline. Third, generate several hypotheses, not just one, and rank them by likelihood and by how cheap they are to test. Fourth, design a test that could disprove the favourite hypothesis, rather than looking for confirmation, and change one variable at a time. Fifth, record what I tried and what I observed, which keeps the team aligned and prevents repeating work. Sixth, when a hypothesis is confirmed, verify the fix actually resolves the symptom, and check that it explains all the observations, not just some. Also useful: bisecting, by time, by version, or by component, to cut the search space; asking a colleague to challenge assumptions; and timeboxing a theory so I move on if it isn't paying off. Afterwards, capture lessons in the postmortem.

Likely follow-ups

- How does bisection narrow down a problem?
- How do you avoid confirmation bias during an incident?

THE TRAP

```
# Common failure: pattern-match to a past incident, "fix" the wrong thing,
#   waste an hour, or change five things at once and never learn what worked.
# Confirmation bias: looking only for evidence that fits your first guess.
```

A DISCIPLINED LOOP

```
1. State the problem precisely (what, since when, who's affected, what's the delta vs
normal)
2. Gather evidence BEFORE theorizing (logs, metrics, recent changes)
3. Form a hypothesis that EXPLAINS the evidence — and predict what else must be true
4. Test the cheapest discriminating check first; change ONE variable at a time
5. Confirm the fix actually addresses the cause (not just correlated)
6. Write it down -> runbook / postmortem
```

USEFUL PRINCIPLES

```
- "What changed?" usually beats deep theory (correlation with a deploy/change)
- Bisect: cut the problem space in half (which layer? which host? which commit?)
- Don't trust assumptions — VERIFY (is it actually resolving DNS? actually reading
  that config? actually the version you think?). Reproduce if you can.
- Distinguish correlation from causation before declaring victory.
```

Interview tip: This is really about engineering temperament. Emphasize: evidence before hypotheses, change ONE thing at a time (so you know what fixed it), bisect the problem space, and verify

certs.eknathalabs.com

assumptions instead of trusting them ("is it *actually* doing X?"). "What changed?" as the first question, and confirming cause vs mere correlation before closing, signal the calm, methodical debugging that senior/SRE interviewers are really screening for.

📚 **Start here (official docs)**

- Linux man-pages project

Q126 **What are process groups, sessions and daemons, and how do nohup, setsid and disown keep a job alive after logout?** Processes

🎙 **Spoken answer · ~64 s**

Linux groups processes into process groups, and groups into sessions. A session usually has a controlling terminal, and when that terminal hangs up, the kernel sends SIGHUP to the foreground process group, which kills most jobs. To survive logout there are a few options. nohup makes the command ignore SIGHUP and redirect output to a file. disown removes a background job from the shell's job table so the shell does not forward the signal. setsid starts the process in a brand new session with no controlling terminal, which is closest to a classic daemon. The classic daemon recipe also forks twice, changes the working directory, closes file descriptors and redirects standard streams. In practice I avoid hand-rolled daemons and use systemd, either a unit file or systemd-run for one-off jobs, because it provides logging to the journal, restart policies, dependencies and resource limits, and cleans up the whole control group when the service stops.

Likely follow-ups

- What does the double fork do?
- Why is systemd-run better than nohup?

A session has a controlling terminal; when it closes the kernel sends SIGHUP to the foreground process group. nohup ignores SIGHUP, setsid starts a process in a new session, and disown removes a shell job from its table so the shell will not signal it. A proper daemon is better run under systemd.

```
nohup ./job.sh > job.log 2>&1 &
```

```
setsid -f ./job.sh > job.log 2>&1 < /dev/null
```

```
ps -o pid,pgid,sid,tty,cmd -p $$
```

```
systemd-run --user --unit=myjob ./job.sh
```

Interview tip: The modern answer is systemd-run or a unit file, which gives logging, restart and resource control that nohup does not.

certs.eknathalabs.com

📚 **Official docs**

- systemd.service(5)

Q127 **What is the difference between VSZ, RSS, PSS and the working set of a process, and which should you trust?**

Processes

🎙 **Spoken answer · ~59 s**

VSZ, virtual size, counts every address range a process has mapped, including libraries, memory-mapped files and reserved but untouched heap, so it is often huge and rarely meaningful. RSS, resident set size, is the amount of that memory currently in RAM, but shared pages, like a common library loaded by fifty processes, are counted fully in each process's RSS, so adding RSS values overstates real usage. PSS, proportional set size, divides shared pages by the number of processes sharing them, so the sum of PSS across processes is accurate, and tools like smem report it. Private dirty memory shows what would be lost if the process died. For containers, the kernel enforces limits at the cgroup level, so I look at memory.current and the working set, which excludes easily reclaimable cache, since that is what the OOM killer compares to the limit.

Likely follow-ups

- Why can VSZ be larger than RAM?
- What does the cgroup working set exclude?

VSZ is the total virtual address space mapped, much of it never touched. RSS is the resident physical memory including shared pages counted for every process. PSS divides shared pages proportionally and is the fairest per-process number. Containers care about cgroup working set.

```
ps -o pid,vsz,rss,cmd -p 1234
```

```
grep -E 'VmSize|VmRSS|VmSwap' /proc/1234/status
```

```
smem -P nginx -k
```

```
cat /sys/fs/cgroup/memory.current
```

Interview tip: Summing RSS across processes overcounts shared libraries, so use PSS or the cgroup's memory.current for capacity decisions.

📚 **Official docs**

- Linux kernel docs: networking scaling

certs.eknathalabs.com

Q128 What does 'too many open files' mean, and how do file descriptor limits work per process, per user and system-wide?

Processes

🎙 Spoken answer · ~65 s

A file descriptor is a small integer that refers to an open file, socket, pipe or device in a process. Each process has a soft limit and a hard limit, called RLIMIT_NOFILE, commonly one thousand twenty-four by default, and a busy server holding thousands of connections quickly hits it, producing too many open files errors on accept or open. The system also has a global cap, fs.file-max, and fs.nr_open caps the per-process maximum. To diagnose, I compare the count of entries in the process's fd directory with the limits file in proc, and use lsof to see what is open, which often reveals a descriptor leak such as unclosed sockets. To fix, I raise the limit for the service. For a systemd service that means LimitNOFILE in the unit or a drop-in, because etc security limits.conf applies only to PAM login sessions. Then I restart the service and verify the new limit in its proc limits file.

Likely follow-ups

- Why doesn't limits.conf affect a service?
- How do you find a descriptor leak?

Every file, socket and pipe uses a file descriptor. The soft RLIMIT_NOFILE limits one process, ulimit or limits.conf or the systemd LimitNOFILE sets it, and fs.file-max caps the system. Raise limits for the service, not just your shell.

```
ulimit -n
```

```
cat /proc/$(pidof nginx)/limits | grep 'open files'
```

```
ls /proc/$(pidof nginx)/fd | wc -l
```

```
systemctl show nginx -p LimitNOFILE
```

Interview tip: limits.conf does not apply to systemd services. Use LimitNOFILE in the unit file or a drop-in.

📚 Official docs

- Linux man-pages: ulimit / getrlimit(2)

certs.eknathalabs.com

Q129 What is overlayfs, and how does it relate to container image layers and the writable layer?

Filesystem

🎙 Spoken answer · ~63 s

Overlayfs is a union filesystem in the Linux kernel. It presents a single merged directory built from one or more read-only lower directories plus one writable upper directory and a work directory. Container runtimes use it to implement image layers: each image layer becomes a lower directory, all shared between containers that use the same image, and each running container gets its own upper directory for changes. Reading a file finds it in the topmost layer that has it. Writing to a file that exists only in a lower layer triggers a copy-up, where the whole file is copied into the upper layer and then modified, and deleting a lower file creates a special whiteout marker. That explains why the container layer disappears when the container is removed, why editing large files in a container is slow, and why databases and logs belong on mounted volumes, not on the overlay.

Likely follow-ups

- What is a whiteout file?
- Why is copy-up expensive for large files?

overlayfs merges a read-only lowerdir stack with a writable upperdir into one view. Container runtimes stack image layers as lowerdirs and give each container an upperdir. Writes use copy-up, so a modified file is copied into the upper layer first.

```
mount | grep overlay
```

```
docker inspect web --format '{{.GraphDriver.Data.UpperDir}}'
```

```
du -sh /var/lib/docker/overlay2/*/diff | sort -h | tail -3
```

Interview tip: Heavy writes to the container layer are slow and vanish with the container. Put them on a volume.

📚 Official docs

- Docker: Storage drivers (overlayfs)
- Kustomize reference

certs.eknathalabs.com

Q130 What do chattr and lsattr do, and when would an immutable flag explain a mysterious 'Operation not permitted' for root?

Filesystem

🎙 Spoken answer · ~60 s

Besides the normal permission bits, ext4 and XFS support file attributes managed with chattr and shown with lsattr. The immutable attribute prevents any change, including delete, rename, link creation and write, even by root, until the attribute is removed. The append-only attribute allows adding data to the end of a file but not rewriting it, which is useful for protecting logs. I may meet them when a configuration file keeps reverting, or a deletion fails with operation not permitted despite root privileges and correct ownership, in which case lsattr reveals the flag. Attackers sometimes use immutable flags to protect their persistence files, and administrators use them to guard critical files, like making resolv.conf unchangeable by network managers, though this is better handled through proper configuration. Also check mount options like read-only and noexec, and security modules, when permissions seem fine but access fails.

Likely follow-ups

- Why would an attacker set the immutable flag?
- What else can cause permission denied for root?

Extended file attributes such as immutable (+i) and append-only (+a) are enforced by the kernel regardless of ownership or permission bits. Even root cannot modify or delete a file with +i until the flag is removed.

```
lsattr /etc/resolv.conf
```

```
sudo chattr -i /etc/resolv.conf
```

```
sudo chattr +a /var/log/audit.log
```

Interview tip: If rm fails as root, run lsattr before anything else. It is a classic puzzle in interviews.

📚 Start here (official docs)

- Linux man-pages project

certs.eknathalabs.com

Q131 What are inotify limits, and what error appears when a tool such as a file watcher or Kubernetes runs out of watches?

Filesystem

🎙 Spoken answer · ~64 s

The inotify API lets a program subscribe to filesystem events instead of polling. The kernel limits how many instances and watches each user can create, because each watch uses unswappable kernel memory. Tools that monitor large directory trees, such as IDEs, webpack dev servers, log agents, and nodes running many containers each with config watchers, can exhaust the limits. The symptom is confusing: errors like no space left on device or too many open files, although the disk and descriptors are fine, or a failure to start a pod's sidecar. I check the sysctls for max user watches and max user instances, and use the proc filesystem to see which processes hold the most inotify instances. The fix is to raise the limits persistently in a sysctl configuration file, for example to half a million watches, after considering memory usage, and to exclude huge directories such as node_modules or build outputs from watchers.

Likely follow-ups

- Why is the error message misleading?
- How do you find the heaviest inotify user?

inotify lets programs watch files for changes. Limits on watches and instances per user, fs.inotify.max_user_watches and max_user_instances, can be hit by IDEs, log shippers and kubelet-heavy nodes, giving 'no space left on device' or 'too many open files' for inotify.

```
sysctl fs.inotify.max_user_watches fs.inotify.max_user_instances
```

```
sudo sysctl -w fs.inotify.max_user_watches=524288
```

```
find /proc/*/fd -lname 'anon_inode:inotify' 2>/dev/null | cut -d/ -f3 | sort | uniq -c | sort -nr | head
```

Interview tip: The misleading part is the message. ENOSPC from inotify has nothing to do with disk space.

📚 Official docs

- GNU coreutils manual

certs.eknathalabs.com

Q132 How do you use ss to troubleshoot sockets, listen queues and connection states?

Networking

🎙 Spoken answer · ~60 s

I use ss for almost all socket work. With tlnp, I list listening TCP sockets with numeric ports and the owning process, which answers whether the service is listening and on which address. Binding to localhost instead of all interfaces is a classic cause of unreachable services. For a LISTEN socket the Recv-Q column shows how many fully established connections are waiting to be accepted, and Send-Q shows the configured backlog, so if the queue is at its limit, the application is too slow to accept, or the backlog is too small. With ss minus s I get totals by state, which reveal thousands of TIME_WAIT or CLOSE_WAIT sockets. A growing CLOSE_WAIT count usually indicates an application not closing connections. With the info option, I see retransmissions, round-trip time and congestion window for each connection, which helps distinguish network problems from application problems.

Likely follow-ups

- What does a growing CLOSE_WAIT count mean?
- How do you check the accept backlog?

ss reads socket information directly from the kernel and is faster than netstat. Use -tlnp for listeners, -tan for connection states, -s for summary, and -ti for per-connection TCP internals such as retransmits and congestion window. Recv-Q and Send-Q on listeners show accept backlog.

```
ss -tlnp
```

```
ss -tan state established '( dport = :443 )' | wc -l
```

```
ss -s
```

```
ss -ti dst 10.0.0.5
```

Interview tip: For a LISTEN socket, Recv-Q is the current accept queue length and Send-Q is its maximum. A full queue means the app cannot accept fast enough.

📚 Start here (official docs)

- Linux man-pages project

certs.eknathalabs.com

Q133 How does conntrack work, and when does the connection tracking table cause problems?

Networking

🎙 Spoken answer · ~62 s

The kernel's connection tracking subsystem follows each flow through netfilter, storing source, destination, ports, protocol and state. Stateful rules such as accept established connections, and NAT, both depend on it. The table has a maximum size, and every tracked flow consumes an entry for the duration of its timeout, which is long for established TCP, around five days by default, shorter for others. Problems appear when there are very many short connections, such as DNS over UDP, or when a service handles huge numbers of clients through NAT, because entries pile up and the table fills. Then new packets are dropped and the kernel logs table full. Diagnostics are comparing the count to the maximum, and statistics showing insert failures or drops. Fixes include increasing nf_conntrack_max and the hash size, reducing timeouts, bypassing tracking for traffic that does not need it with the raw table's NOTRACK, and reusing connections.

Likely follow-ups

- How do you bypass conntrack?
- Why is the TCP established timeout so long?

conntrack records the state of every flow so NAT and stateful firewalling work. Entries live in a fixed-size table. When it fills, new connections are dropped; stale or short-lived flows and high UDP volume are common causes.

```
conntrack -L | head
```

```
conntrack -S
```

```
sysctl net.netfilter.nf_conntrack_count net.netfilter.nf_conntrack_max
```

```
dmesg | grep conntrack
```

Interview tip: The classic log line is 'nf_conntrack: table full, dropping packet'.

📚 Official docs

- Netfilter documentation

certs.eknathalabs.com

Q134 How do you troubleshoot DNS from a Linux host using dig, resolvectl and nsswitch, and why can dig work while an application fails?

Networking

🎙 Spoken answer · ~60 s

dig is a pure DNS client. It queries the server I specify, or those in resolv.conf, and does not consult etc hosts, mDNS, nscd or systemd-resolved caches. Applications call the C library, which follows the hosts line in nsswitch.conf, typically files first, then DNS, and applies search domains and the ndots setting. So dig can succeed while an application fails because of an entry in etc hosts, a wrong search domain producing a different name, a stub resolver on 127.0.0.53 with a broken upstream, or a negative cache. The right comparison is getent hosts, which uses the same resolution path as the application. With systemd-resolved, resolvectl status shows per-link DNS servers and domains, and resolvectl query shows which source answered. I also test with tcpdump on port 53 to see actual queries, and check for IPv6 AAAA lookups timing out, which produce multi-second delays.

Likely follow-ups

- What does ndots control?
- Why can AAAA lookups cause delays?

dig talks directly to a DNS server and ignores nsswitch, /etc/hosts and local caches. Applications use the libc resolver which follows nsswitch.conf order, search domains and ndots. Compare dig with getent hosts to see what the application sees.

```
dig +short api.example.com @10.0.0.2
```

```
getent hosts api.example.com
```

```
resolvectl status; resolvectl query api.example.com
```

```
cat /etc/nsswitch.conf | grep hosts
```

Interview tip: getent hosts uses the same path as most applications, so it is the right tool to reproduce their behaviour.

📚 Official docs

- Linux man-pages: nsswitch.conf(5)

certs.eknathalabs.com

Q135 What is path MTU discovery, and how do you diagnose MTU black holes with ping and tracepath?

Networking

🎙 Spoken answer · ~62 s

Every link has a maximum transmission unit, typically fifteen hundred bytes for Ethernet, less when tunnelling adds headers, as with VPNs, GRE, VXLAN or some cloud overlays. TCP avoids fragmentation using path MTU discovery: it sets the Don't Fragment bit, and a router that cannot forward the packet returns an ICMP fragmentation needed message with the next hop's MTU. If a firewall blocks that ICMP, the sender never learns and large packets disappear, creating a black hole. The signature is that connections establish and small requests work, but anything large, such as a TLS certificate exchange, hangs. I test with ping using the do not fragment flag and varying payload size, and use tracepath to find the MTU along the path. Fixes are to allow ICMP type three code four, lower the MTU on the interface, or clamp TCP MSS on the gateway so endpoints send smaller segments.

Likely follow-ups

- Why does a VPN reduce MTU?
- What does MSS clamping do?

If a packet is larger than a link's MTU and has the Don't Fragment bit set, the router should return ICMP fragmentation-needed. If ICMP is blocked, large packets silently vanish while small ones work. Test with ping -M do -s and tracepath, and fix with MSS clamping or allowing ICMP.

```
ping -M do -s 1472 10.0.0.5
```

```
tracepath 10.0.0.5
```

```
ip link show eth0 | grep mtu
```

```
sudo iptables -t mangle -A FORWARD -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --clamp-mss-to-pmtu
```

Interview tip: Symptoms: ping and small requests work, but TLS handshakes or large downloads hang. Think MTU first when tunnels or VPNs are involved.

📚 Official docs

- RFC 792: ICMP
- RFC 1191: Path MTU discovery
- Cloudflare Learning: What is MTU?

certs.eknathalabs.com

Q136 How do you read vmstat output to quickly classify a performance problem? Performance

🎙 Spoken answer · ~63 s

I run vmstat with a one second interval for ten or so samples and ignore the first line, which is an average since boot. The r column is the number of runnable processes: consistently higher than the number of CPU cores means CPU saturation. The b column counts processes blocked in uninterruptible sleep, usually waiting for IO. Swap in and swap out values that are non-zero for sustained periods mean real memory pressure. The bi and bo columns show block IO, and wa shows the percentage of time the CPU is idle waiting for IO, so high wa with blocked processes points to storage. High system time with many context switches can mean lock contention or too many threads. The st column shows CPU time stolen by the hypervisor on virtual machines. Once I classify the problem, I drill down with iostat, pidstat, perf or the pressure stall information files.

Likely follow-ups

- What is pressure stall information?
- How do you tell CPU saturation from high utilisation?

vmstat shows the run queue (r), blocked processes (b), swap in and out (si/so), block IO (bi/bo), interrupts, context switches and CPU split (us/sy/id/wa/st). Look at the pattern across lines, not a single snapshot.

```
vmstat 1 10
```

```
vmstat -w -S M 1
```

```
mpstat -P ALL 1 3
```

```
cat /proc/pressure/cpu /proc/pressure/memory /proc/pressure/io
```

Interview tip: r greater than the number of cores means CPU saturation. Non-zero si and so means memory pressure. High wa with b above zero points to storage.

📚 Official docs

- Brendan Gregg: Linux performance

certs.eknathalabs.com

Q137 What are eBPF and the bcc or bpftrace tools, and how do they help production troubleshooting?

Performance

🎙 **Spoken answer · ~63 s**

eBPF lets you load small sandboxed programs into the kernel and attach them to events such as system calls, kernel functions, network packets and tracepoints. A verifier checks that they terminate and cannot access invalid memory, so they are safe for production, and the overhead is low because aggregation happens in the kernel. The bcc and bpftrace projects provide ready-made tools. For example, execsnoop shows every new process launched, which finds runaway shell loops, opensnoop shows files being opened, biolatency gives a histogram of disk IO latency, runqlat shows scheduler queueing delay, tcpconnect and tcplife show connection attempts and lifetimes, and profile samples stack traces for CPU flame graphs. I use them when classic tools are not enough or too intrusive, such as finding which process makes a sudden burst of short-lived connections. They require a reasonably recent kernel and root privileges, and the same technology powers Cilium and Falco.

Likely follow-ups

- Why is strace risky in production?
- What does the verifier check?

eBPF runs small verified programs inside the kernel to observe events with low overhead. Tools such as execsnoop, opensnoop, biolatency, tcpconnect and runqlat answer questions that strace or tcpdump can't without heavy cost.

```
sudo execsnoop-bpfcc
```

```
sudo biolatency-bpfcc 1 5
```

```
sudo tcplife-bpfcc
```

```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat { @[comm] = count(); }'
```

Interview tip: Always say it is safe to run in production because the verifier guarantees programs terminate and cannot crash the kernel.

📚 **Official docs**

- Linux kernel docs: BPF
- ebpf.io: What is eBPF?

certs.eknathalabs.com

Q138 What are Transparent Huge Pages, and why do many database vendors recommend disabling them?

Performance

🎙 Spoken answer · ~60 s

Normally memory is managed in four kilobyte pages. Using two megabyte huge pages reduces page table size and TLB misses, which can speed up memory-intensive programs. Transparent Huge Pages tries to do this automatically, promoting regions when it can. The downside is that the kernel may have to compact memory to find contiguous blocks, causing stalls, and a khugepaged daemon merges pages in the background. For databases such as Redis, MongoDB and some Java workloads, this produces unpredictable latency spikes and can inflate memory use, for instance copy-on-write of a huge page after fork during a snapshot copies two megabytes for a one byte change. So vendors usually recommend setting it to madvise or never, via the sysfs file and persistently through the boot parameters or a tuned profile. Pre-allocated explicit huge pages remain a good option when I want the benefit with predictable behaviour.

Likely follow-ups

- THP versus explicit huge pages?
- Why does fork plus THP bloat memory?

THP lets the kernel back memory with 2 MB pages to cut TLB overhead. The background defragmentation and compaction can cause latency spikes and memory bloat for databases with sparse access patterns, so Redis, MongoDB and others advise madvise or never.

```
cat /sys/kernel/mm/transparent_hugepage/enabled
```

```
echo madvise | sudo tee /sys/kernel/mm/transparent_hugepage/enabled
```

```
grep -i huge /proc/meminfo
```

Interview tip: Explicit hugepages (hugetlbfs) are different. They are reserved up front, and databases like PostgreSQL can use them deliberately.

📚 Start here (official docs)

- Linux man-pages project

certs.eknathalabs.com

Q139 What is NUMA, and how can it cause uneven performance on large servers?

Performance

🎙 Spoken answer · ~65 s

On servers with more than one CPU socket, each socket has its own memory controller, so memory is split into NUMA nodes. A CPU reaches local memory fast and remote memory over an interconnect with higher latency and lower bandwidth. The kernel tries to allocate local memory, but a large process whose threads run on several nodes, or memory allocated at start by one thread, can end up mostly remote, giving lower and uneven performance. Another problem is imbalance: one node can run out of memory and begin reclaiming or swapping while the machine as a whole shows plenty of free memory. I inspect topology with numactl and lscpu, and per-node usage with numastat. Remedies include pinning processes and memory to a node with numactl or cpuset cgroups, interleaving memory for workloads that need the full capacity, tuning the database's NUMA settings, and, in Kubernetes, using the topology manager with static CPU policy for latency-sensitive pods.

Likely follow-ups

- What does interleaving do?
- How does Kubernetes topology manager help?

On multi-socket machines memory is attached to specific CPUs. Access to remote memory is slower. If a process runs on one node but its memory lives on another, latency rises; unbalanced allocation can exhaust one node while others are free, triggering swapping or OOM.

```
numactl --hardware
```

```
numastat -m
```

```
numactl --cpunodebind=0 --membind=0 ./db
```

```
lscpu | grep -i numa
```

Interview tip: Mention VMs and containers: pinning a database to a single NUMA node often gives steadier latency than letting threads roam.

📚 Start here (official docs)

- Linux man-pages project

certs.eknathalabs.com

Q140 How do SSH certificates differ from authorized_keys, and why do large fleets use an SSH certificate authority?

Security

🎙 Spoken answer · ~60 s

With plain authorized_keys, every server holds the public keys of everyone allowed, which becomes unmanageable and keys rarely expire, so former staff or lost laptops leave access behind. With an SSH certificate authority, the CA signs a user's public key together with metadata: a key identity for audit, a list of principals, meaning the account names it may log in as, and a validity period such as eight hours. Servers are configured once with the CA's public key using the TrustedUserCAKeys option and optionally principals files. Users get a short-lived certificate after authenticating to an identity system with multi-factor, using tools like Vault, Teleport or Smallstep. When the certificate expires, access ends without any cleanup on servers, and logs show the key identity. Host certificates similarly remove trust-on-first-use warnings. Revoking a lost key is handled by short lifetimes and a revoked keys list.

Likely follow-ups

- What are principals?
- How do host certificates help?

An SSH CA signs a user's public key with an identity, principals and a short validity. Servers trust the CA key once, so no per-user keys sit in authorized_keys, access expires automatically, and revocation and audit are simpler.

```
ssh-keygen -s ca_key -I alice@corp -n deploy,alice -V +8h id_ed25519.pub
```

```
grep -i TrustedUserCAKeys /etc/ssh/sshd_config
```

```
ssh-keygen -Lf id_ed25519-cert.pub
```

Interview tip: This is the same idea as short-lived cloud credentials: no standing access, a signed token that expires.

📚 Official docs

- OpenSSH manual pages
- Argo CD: ApplicationSet

certs.eknathalabs.com

Q141 firewalld versus ufw versus nftables: how do they relate and how do you choose?

Security

🎙 Spoken answer · ~63 s

The packet filtering engine in modern kernels is netfilter, now programmed with nftables, which replaced the older iptables interface, though iptables commands often still exist as a compatibility layer. On top of that sit management tools. firewalld, common on the RHEL family, organises interfaces into zones with trust levels and named services, supports runtime and permanent configuration, and can change rules without dropping existing connections. ufw, common on Ubuntu, offers a short, simple syntax for host firewalls. For complex or high-performance rules, such as sets and maps, or for automation in appliances, I may write nftables rules directly. The key is to pick one owner for the rule set. Mixing a manager with manual commands leads to rules being overwritten on reload. On container hosts, Docker and kube-proxy also manage rules, so I check how they interact. In the cloud I combine host firewalls with security groups as defence in depth.

Likely follow-ups

- What is a firewalld zone?
- Why can a firewall reload break Docker networking?

nftables is the kernel packet filter that replaced iptables. firewalld and ufw are front ends: firewalld uses zones and runtime versus permanent configuration (RHEL family); ufw is a simple rule syntax (Ubuntu). Use one manager, never mix with hand-edited rules.

```
sudo firewall-cmd --zone=public --add-service=https --permanent && sudo firewall-cmd --reload
```

```
sudo ufw allow from 10.0.0.0/8 to any port 22 proto tcp
```

```
sudo nft list ruleset
```

Interview tip: In Kubernetes or Docker hosts, remember the container runtime and kube-proxy also write rules, and a firewall manager reload can wipe them.

📚 Official docs

- nftables wiki
- RHEL 9 documentation

certs.eknathalabs.com

Q142 What do kernel hardening settings such as kptr_restrict, dmesg_restrict, module signing and lockdown mode protect against?

Security

🎙 **Spoken answer · ~59 s**

Many local privilege escalation exploits need information about the kernel layout, or the ability to inspect and manipulate other processes. kptr_restrict hides kernel pointers in files like proc kallsyms from unprivileged users, and dmesg_restrict stops them reading the kernel log, which can leak addresses. The Yama ptrace_scope setting prevents one process from attaching to another unless it is a child, which blocks credential harvesting from running processes. Disabling unprivileged BPF reduces attack surface in the kernel's BPF verifier. On the loading side, kernel module signing, secure boot and lockdown mode prevent root from loading unsigned modules or writing to kernel memory, which defends against rootkits. These are defence in depth, and I apply them through sysctl drop-ins, boot parameters and baseline profiles, validate with a compliance scanner, and test for application impact, as debugging tools like strace or perf may be restricted.

Likely follow-ups

- What does ptrace_scope protect?
- What is secure boot's role?

They reduce what a local attacker can learn or load. kptr_restrict hides kernel addresses, dmesg_restrict limits the kernel log, unprivileged_bpf_disabled and ptrace_scope limit tracing and BPF, and module signing or lockdown stops loading unsigned code into the kernel.

```
sysctl kernel.kptr_restrict kernel.dmesg_restrict kernel.yama.ptrace_scope
kernel.unprivileged_bpf_disabled
```

```
cat /sys/kernel/security/lockdown
```

```
mokutil --sb-state
```

Interview tip: Pair these settings with a baseline such as CIS benchmark profiles and verify them with an automated scanner.

📚 **Start here (official docs)**

- Linux man-pages project

certs.eknathalabs.com

Q143 How do LVM snapshots work, and what are the risks of using them for backups?

Storage & disks

🎙 Spoken answer · ~60 s

When I create an LVM snapshot, LVM reserves some space for it and records the original volume's state. When a block on the origin changes, the old content is first copied into the snapshot area, so the snapshot appears frozen while the origin keeps changing. This is useful for consistent file level backups, rollbacks before risky upgrades, or cloning data for tests. The risks are these. The snapshot space is limited, so if many blocks change and the space fills, the snapshot is invalidated and a backup in progress fails. Every write to the origin triggers an extra copy, so performance drops, sometimes drastically, while the snapshot exists. Snapshots are not backups by themselves, because they live on the same disks. I keep them short-lived, monitor the usage percentage, quiesce applications first, and copy the data off to another system. Thin provisioning improves flexibility.

Likely follow-ups

- Crash-consistent versus application-consistent?
- Why is a snapshot not a backup?

A snapshot is a copy-on-write view of a logical volume at a point in time. Changed blocks are copied into the snapshot space. If that space fills, the snapshot becomes invalid, and write performance on the origin degrades while snapshots exist.

```
sudo lvcreate -s -n data_snap -L 10G /dev/vg0/data
```

```
sudo lvs -o lv_name,lv_size,data_percent
```

```
sudo mount -o ro,nouuid /dev/vg0/data_snap /mnt/snap
```

```
sudo lvremove /dev/vg0/data_snap
```

Interview tip: For databases, flush and lock tables or use the database's backup API, because a snapshot is only crash-consistent.

📚 Official docs

- Linux kernel docs: LVM (device-mapper)

certs.eknathalabs.com

Q144 What do fstrim and the discard mount option do, and which should you use on SSDs and thin-provisioned storage?

Storage & disks

🎙 **Spoken answer · ~62 s**

When files are deleted, the filesystem knows blocks are free, but the underlying device does not. TRIM, or discard, informs the device, so an SSD can erase blocks in advance and keep write performance steady, and thin-provisioned storage, such as LVM thin pools, SANs and some virtual disks, can release the space back to the pool. There are two ways to send it. The discard mount option issues commands synchronously as files are deleted, which can add latency and slow deletes on some devices. The alternative is running fstrim periodically, and most distributions ship a systemd timer that does it weekly, which has minimal impact. I prefer the timer. It matters for thin-provisioned virtual machines too, because without it the backing store keeps growing even though the guest shows free space. Encrypted volumes need discard passed through explicitly, with the security trade-off of revealing which blocks are free.

Likely follow-ups

- Why avoid continuous discard?
- How does trim help thin provisioning?

TRIM tells the device which blocks are no longer used, helping SSD wear levelling and letting thin-provisioned or cloud storage reclaim space. Continuous discard can hurt performance; a weekly fstrim timer is the usual best practice.

```
lsblk --discard
```

```
sudo fstrim -av
```

```
systemctl status fstrim.timer
```

```
findmnt -o TARGET,OPTIONS | grep discard
```

Interview tip: On cloud volumes and VMs, trim support depends on the virtual disk type, so check lsblk --discard shows non-zero granularity.

📚 **Start here (official docs)**

- Linux man-pages project

certs.eknathalabs.com

Q145 On cloud VMs, why do device names like /dev/sdb or nvme1n1 change, and how do you mount disks reliably?

Storage & disks

🎙 Spoken answer · ~61 s

The kernel assigns names such as sdb or nvme1n1 in the order devices are discovered, which is not guaranteed to stay the same, especially when disks are attached or detached, or when instances use NVMe where the name order does not match the order shown in the cloud console. Mounting by those names in fstab is fragile and can mount the wrong disk. Instead I use the filesystem UUID or a label, which are stored on the disk, or the stable by-id symlinks that include the cloud volume's serial number. I add the nofail option for non-critical data disks, so an absent disk does not stop boot, and use x-systemd.device-timeout to avoid long waits. After editing fstab I always test with mount -a before rebooting. Automation can find the right device using the serial number from lsblk, which maps to the volume ID in the cloud API.

Likely follow-ups

- Why does nofail matter?
- How do you map a volume ID to a device?

Device names depend on probe order and can change across reboots or attachment. NVMe instances present EBS as nvme devices with a different order than the API mapping. Mount by UUID or label, or use stable /dev/disk/by-id paths, and use nofail in fstab for non-critical disks.

```
lsblk -o NAME,SIZE,UUID,MOUNTPOINT,SERIAL
```

```
blkid /dev/nvme1n1
```

```
ls -l /dev/disk/by-id/
```

```
echo 'UUID=abcd-1234 /data xfs defaults,nofail 0 2' | sudo tee -a /etc/fstab
```

Interview tip: A wrong fstab entry without nofail can drop the whole VM into emergency mode at boot, so always run mount -a to test.

📚 Start here (official docs)

- Linux man-pages project

certs.eknathalabs.com

Q146 What is initramfs, what does it do during boot, and how do you rebuild it?

Boot & systemd

🎙 Spoken answer · ~60 s

The kernel cannot mount the real root filesystem unless it has the drivers for the disk controller, filesystem, RAID, LVM or encryption layer, and those might be modules stored on that very filesystem. To solve this chicken and egg problem, the bootloader loads an initial RAM filesystem, a compressed archive that the kernel unpacks into memory and runs. It contains essential modules, udev, and scripts that assemble RAID, unlock LUKS, activate LVM and find root by UUID, then switch to the real root and start init. It's built with dracut on the RHEL family and update-initramfs on Debian-based systems, and must be regenerated after changing storage configuration, drivers, crypttab or when kernels are installed. If boot lands in an initramfs shell, I check for the root device, loaded modules and kernel messages, correct the cause, and rebuild the image from a rescue environment if needed.

Likely follow-ups

- How do you recover from an initramfs prompt?
- What does switch_root do?

The initramfs is a small temporary root filesystem loaded by the bootloader alongside the kernel. It contains drivers and scripts to find and mount the real root filesystem, for example LVM, RAID, LUKS or storage controllers, and then hands over with switch_root.

```
lsinitrd /boot/initramfs-$(uname -r).img | head -30
```

```
sudo dracut -f --kver $(uname -r)
```

```
sudo update-initramfs -u -k all
```

```
sudo grubby --info=ALL
```

Interview tip: If the system drops to an initramfs prompt, the root device was not found. Think missing driver, wrong UUID, or a degraded RAID.

📚 Official docs

- systemd: boot process

certs.eknathalabs.com

Q147 How do kernel command-line parameters and GRUB configuration work, and how do you change them safely?

Boot & systemd

🎙 Spoken answer · ~62 s

The bootloader, GRUB, loads the kernel and initramfs and passes a string of parameters that configure early boot. Examples are root to locate the root filesystem, ro to mount it read only until fsck, quiet to reduce messages, console to send output to a serial port, which is vital for cloud serial consoles, and tuning options such as isolcpus, hugepages, transparent_hugepage and net.ifnames. The permanent place to set extra options is the GRUB default file, after which I regenerate the configuration file with grub2-mkconfig or update-grub, and on the RHEL family, grubby can modify entries directly. The current values are visible in proc cmdline. When trying something risky, I press e at the boot menu and edit the line for that boot only, so a mistake disappears after reboot. On systems with BLS entries or UEFI, the paths differ, so I check where the real configuration lives before editing.

Likely follow-ups

- Why set console= on cloud VMs?
- What is a BLS entry?

GRUB reads its configuration and passes a kernel command line, for example root=, ro, quiet, console= and isolcpus. Edit /etc/default/grub then regenerate the config; for a one-off, press e at the GRUB menu to edit the line for a single boot.

```
cat /proc/cmdline
```

```
sudo vi /etc/default/grub && sudo grub2-mkconfig -o /boot/grub2/grub.cfg
```

```
sudo grubby --update-kernel=ALL --args='transparent_hugepage=madvise'
```

```
sudo update-grub
```

Interview tip: Test risky parameters through the temporary edit at the menu first. A mistake there is fixed by rebooting, but a bad permanent change needs a rescue boot.

📚 Official docs

- systemd: boot process
- GNU GRUB manual

certs.eknathalabs.com

Q148 What is kdump, and how do you capture and analyse a kernel crash?

Boot & systemd

🎙 Spoken answer · ~60 s

When the kernel panics, the system normally just reboots and the evidence is gone. kdump reserves a region of memory at boot via the crashkernel parameter and preloads a small capture kernel there. On panic, the machine boots into that capture kernel, which saves the crashed kernel's memory image as a vmcore to disk, network or another target, then reboots. To analyse it, I use the crash tool along with the debuginfo for the exact kernel version, and commands like bt for the backtrace, log for the kernel messages and ps to see processes at the time. This identifies a faulty driver, a hung task or memory corruption, and gives vendors the data to analyse. It must be configured and tested ahead of time, with enough reserved memory and disk space, and sometimes watchdog and panic-on-oops settings so hung machines actually trigger a dump.

Likely follow-ups

- What does the crashkernel parameter do?
- Why do you need matching debuginfo?

kdump reserves memory for a second capture kernel that boots after a panic and writes a vmcore of the crashed kernel's memory. You then analyse it with the crash utility and matching kernel debuginfo to see the panic stack and cause.

```
systemctl status kdump
```

```
cat /proc/cmdline | grep crashkernel
```

```
echo c | sudo tee /proc/sysrq-trigger   # test only, crashes the host
```

```
crash /usr/lib/debug/lib/modules/$(uname -r)/vmlinux /var/crash/*/vmcore
```

Interview tip: Never test the sysrq crash on a production host. Use a lab VM.

📚 Start here (official docs)

- Linux man-pages project

certs.eknathalabs.com

Q149 How do /etc/security/limits.conf and systemd resource settings interact for users and services?

Users & permissions

🎙 **Spoken answer · ~60 s**

Resource limits are set per process, inherited from the parent. For interactive logins, the PAM module pam_limits reads etc security limits.conf and the limits.d directory and applies soft and hard limits per user or group, such as open files, processes, core size and locked memory. A service started by systemd never goes through a PAM login session, so those files do not affect it. Instead, I set directives such as LimitNOFILE, LimitNPROC, LimitCORE or LimitMEMLOCK in the unit file, preferably through a drop-in created with systemctl edit, followed by a daemon reload and restart. There is also TasksMax controlling the number of tasks in the unit's cgroup, which limits threads and processes. Global defaults can be changed in system.conf. To check what applies, I read the limits file for the running process under proc, which is the source of truth, and not the config files.

Likely follow-ups

- What is TasksMax?
- How do soft and hard limits differ?

limits.conf is applied by pam_limits for login sessions. Services started by systemd ignore it and use LimitNOFILE, LimitNPROC, TasksMax and similar directives in unit files or drop-ins, with defaults in system.conf. Check the live values in /proc//limits.

```
sudo systemctl edit myapp   # add [Service] LimitNOFILE=65536
```

```
systemctl show myapp -p LimitNOFILE -p TasksMax
```

```
grep -r nofile /etc/security/limits.d/
```

```
cat /proc/$(pidof myapp)/limits
```

Interview tip: Always verify against the running process, because what the file says and what the process got can differ.

📚 **Official docs**

- systemd.service(5)

certs.eknathalabs.com

Q150 How should you set up service accounts on Linux: system users, nologin shells, and running services without root?

Users & permissions

🎙 Spoken answer · ~64 s

A service should never run as root unless there is a genuine need. I create a dedicated system account using the system flag, so it gets a low UID in the reserved range, no password, no home directory unless the app needs one, and a nologin shell so nobody can log in as it. In the unit file I set the user and group, and use systemd sandboxing options such as NoNewPrivileges, ProtectSystem, ProtectHome, PrivateTmp, and restricting capabilities and address families. If the program has to listen on a port below one thousand twenty-four, I grant just the net bind service capability, use socket activation, or put a reverse proxy in front. File ownership and permissions are restricted to the service account, with secrets readable only by it. Systemd-analyze security audits the unit and highlights what remains exposed. Each service gets its own account, so compromise of one does not give access to others.

Likely follow-ups

- What does NoNewPrivileges do?
- Why one account per service?

Create a system user with no login shell and no home unless needed, run the service under it with systemd User= and hardening options, bind low ports via capabilities or a proxy, and keep files owned by that user with minimal permissions.

```
sudo useradd --system --no-create-home --shell /usr/sbin/nologin appsvc
```

```
sudo systemctl edit app   # User=appsvc, NoNewPrivileges=yes, ProtectSystem=strict
```

```
sudo setcap 'cap_net_bind_service=+ep' /usr/local/bin/app
```

```
systemd-analyze security app.service
```

Interview tip: systemd-analyze security gives a score and shows which sandboxing options you are still missing.

📚 Official docs

- Kubernetes: Service accounts

certs.eknathalabs.com

Q151 How do you parse command-line options in a bash script, and when should you switch to a more capable language?

Scripting & automation

🎙 Spoken answer · ~64 s

For simple scripts I use the getopts builtin. I define the allowed short options in a string, loop over them in a case statement, handle unknown options and missing arguments explicitly, and shift away the processed options so the remaining positional parameters are available. For long options such as dash dash environment, I write a while loop with case matching on the argument and shift. I always validate required values, quote all variables, print a usage message to standard error on misuse and exit with a non-zero code, using two for usage errors and one for runtime failures. As soon as a script needs nested data structures, JSON handling, retries, concurrency, richer error handling or tests, I switch to Python or Go, where argparse or cobra gives better help text and validation. A good rule is that if the bash script passes about a hundred lines or needs arrays of arrays, rewrite it.

Likely follow-ups

- What does shift do?
- Why exit codes matter for automation?

Use getopts for short options in portable scripts, or a while-case loop for long options. Validate arguments, print usage on error and exit non-zero. When option handling, data structures or error handling become complex, move to Python or Go.

```
while getopts ':e:vh' opt; do case $opt in e) env=$OPTARG;; v) verbose=1;; h) usage; exit 0;; \?) usage; exit 2;; esac; done; shift $((OPTIND-1))
```

```
[[ -n ${env:-} ]] || { echo 'env required' >&2; exit 2; }
```

Interview tip: Send errors to stderr and use distinct exit codes, so automation can tell usage mistakes from runtime failures.

📚 Official docs

- GNU Bash reference manual

certs.eknathalabs.com

Q152 How do you use trap and flock in bash to clean up after failures and prevent concurrent runs?

Scripting & automation

🎙 Spoken answer · ~62 s

Robust scripts clean up after themselves. The trap builtin registers a command to run when a signal arrives or on pseudo-signals such as EXIT, which fires however the script ends, and ERR, which fires when a command fails. A common pattern is to create a temporary directory with mktemp and trap EXIT to remove it, so interrupted runs do not leak files. To stop two copies of a job, such as a backup scheduled by cron, overlapping, I use flock. I open a lock file on a file descriptor and call flock with the non-blocking option, exiting if it cannot take the lock. Because the kernel releases the lock when the process exits, even by crashing, there are no stale lock problems like with pid files. In cron I can also wrap the command with flock directly. Together with set minus euo pipefail, this makes scripts safe to re-run.

Likely follow-ups

- Why is a pid file unreliable?
- What does the ERR trap not catch?

trap runs a handler on EXIT, ERR or signals, so temp files are removed and locks released even on failure. flock takes an advisory lock on a file descriptor, so a second instance exits or waits instead of running concurrently, and the lock is released automatically if the process dies.

```
tmp=$(mktemp -d); trap 'rm -rf "$tmp"' EXIT
```

```
exec 9>/var/lock/backup.lock; flock -n 9 || { echo 'already running' >&2; exit 1; }
```

```
trap 'echo "failed at line $LINENO" >&2' ERR
```

Interview tip: Prefer flock over a hand-made pid file, since a crashed script leaves a stale pid file but cannot leave a stale flock.

📚 Official docs

- GNU Bash reference manual

certs.eknathalabs.com

Q153 How do you use jq (and yq) in shell pipelines to work safely with JSON and YAML?

Scripting & automation

🎙 Spoken answer · ~64 s

Parsing JSON with grep and sed is fragile, so I use jq. A filter starts from the input with dot, navigates fields, iterates arrays with brackets, and can pipe results to further filters. Select keeps items that match a condition, map transforms arrays, and output formats like at tsv and at csv produce shell-friendly rows. The raw output flag removes JSON quotes from strings, which matters when assigning to shell variables. When I need a shell value inside the filter, I pass it with the arg option rather than interpolating it into the program text, which avoids quoting bugs and injection. The exit-status flag makes jq return a non-zero code when the result is false or null, so scripts can fail fast on unexpected data. yq provides a similar syntax for YAML, handy for Kubernetes manifests and CI configuration, and I check which yq implementation is installed because there are two with different syntax.

Likely follow-ups

- Why pass values with --arg?
- What does jq -e do?

jq filters and transforms JSON with a small expression language. Use -r for raw strings, --arg to pass shell values safely, select to filter, map for arrays, and exit-status flags to fail the script on missing data. yq does the same for YAML.

```
aws ec2 describe-instances | jq -r '.Reservations[].Instances[] |
select(.State.Name=="running") | [.InstanceId,.PrivateIpAddress] | @tsv'
```

```
kubectl get pods -o json | jq -r --arg ns prod '.items[] | select(.metadata.namespace==$ns)
| .metadata.name'
```

```
jq -e '.status == "ok"' health.json > /dev/null
```

Interview tip: Never build jq filters by string-concatenating user input. Pass values with --arg so quoting cannot break or inject.

📚 Official docs

- jq manual
- yq documentation

certs.eknathalabs.com

Q154 Load average is very high but CPU utilisation is low. What is happening and how do you find the cause?

Troubleshooting

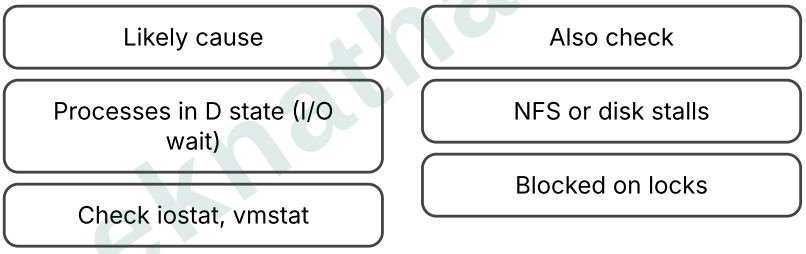
🎙 Spoken answer · ~65 s

On Linux, load average includes not just tasks that are running or ready to run, but also those in uninterruptible sleep, state D, which are normally waiting for IO to complete. So a machine whose processes are blocked on a slow or hung disk, a stuck NFS mount, a failing storage path or a lock can show a load of fifty while the CPU sits idle. I confirm by looking at vmstat for the blocked column and high wait percentage, then list processes in D state and the kernel function they wait in using wchan or proc stack. Iostat shows device latency and utilisation, and mount output reveals network filesystems. Typical causes are NFS server outages with hard mounts, a failing disk, an overloaded cloud volume that exhausted burst credits, or a backup job saturating IO. The fix is to restore the storage path, throttle the offender with ionice, or use soft mount options where appropriate.

Likely follow-ups

- Why can't you kill a D-state process?
- Why do hard NFS mounts hang clients?

🗺 High load, low CPU



Linux load average counts runnable tasks and tasks in uninterruptible sleep (D state), usually waiting on IO. High load with idle CPU points to blocked IO such as slow disks or hung NFS. Find D-state processes and what they wait on.

```
ps -eo state,pid,wchan:32,cmd | awk '$1=="D"'
```

```
vmstat 1 5
```

```
iostat -x 1 3
```

```
cat /proc/<pid>/stack
```

Interview tip: The wchan column tells you which kernel function the process is blocked in, a quick clue for NFS, disk or lock waits.

certs.eknathalabs.com

📚 **Official docs**

- Brendan Gregg: Linux performance

Q155 **Disk latency spikes on a cloud VM every few hours even though the workload is steady. What do you check?**

Troubleshooting

🎙 **Spoken answer · ~63 s**

I first confirm the problem in the guest with iostat: high await and queue size with utilisation near one hundred percent shows the device is saturated or throttled. A periodic pattern suggests a scheduled cause or a credit cycle. In clouds, volumes may have burst credits, like older gp2 volumes or small burstable disks, so the workload runs fast until credits are exhausted, then drops to the baseline until they refill. The instance also has its own storage bandwidth and IOPS limits, which can be hit even if the volume has capacity. I check the provider's metrics for burst balance, throttling and queue length. Locally I look for cron jobs, log rotation, backups, snapshots, database checkpoints or fstrim running at those times using iotop. Fixes include moving to a volume type with provisioned performance, increasing volume size, choosing a larger instance, spreading scheduled jobs, and using ionice for background tasks.

Likely follow-ups

- How do gp3 and gp2 differ for bursting?
- What does await measure?

Look for exhausted burst credits or throttling on the volume or instance, snapshots or backups running, filesystem maintenance, and noisy neighbours. Compare iostat await with the cloud provider's volume metrics.

```
iostat -x 1 10
```

```
aws cloudwatch get-metric-statistics --namespace AWS/EBS --metric-name BurstBalance --
dimensions Name=VolumeId,Value=vol-1 ...
```

```
sudo iotop -oPa
```

```
cat /sys/block/nvme1n1/queue/scheduler
```

Interview tip: gp2 burst balance and instance-level EBS bandwidth limits are two separate throttles. Check both.

📚 **Start here (official docs)**

- Linux man-pages project

certs.eknathalabs.com

Q156 Services fail with 'Too many open files' under load, but ulimit -n in your shell shows a high value. Why?

Troubleshooting

🎙 **Spoken answer · ~63 s**

Limits belong to each process, so what my interactive shell shows is irrelevant to a daemon that started earlier under systemd with its own settings. I read the running process's limits file to see the true soft and hard values, and compare with how many descriptors it has open, counted from its fd directory. If the count approaches the limit, I check whether this is legitimate growth or a leak. Using lsof and grouping by type shows whether the descriptors are sockets in CLOSE_WAIT, deleted files, or pipes, which points at the code not closing resources. As an immediate fix I raise LimitNOFILE with a systemd drop-in and restart, and for containers I check the runtime default and the node level maximums. If a leak is confirmed, raising limits only delays the failure, so I fix the code, add connection timeouts and pooling, and monitor the open descriptor count as a metric.

Likely follow-ups

- Why does raising the limit not fix a leak?
- How would you alert on descriptor usage?

The service process has a different limit from your shell: it inherited the systemd default or its own LimitNOFILE. Check /proc//limits for the real values, then set LimitNOFILE in the unit and look for a descriptor leak.

```
grep 'open files' /proc/$(pidof myapp)/limits
```

```
ls /proc/$(pidof myapp)/fd | wc -l
```

```
lsof -p $(pidof myapp) | awk '{print $5}' | sort | uniq -c | sort -nr
```

```
sudo systemctl edit myapp   # LimitNOFILE=131072
```

Interview tip: Also check fs.file-max and the container runtime's own limits when the service runs inside Docker or Kubernetes.

📚 **Official docs**

- Linux man-pages: ulimit / getrlimit(2)

certs.eknathalabs.com

Q157 SSH logins are slow, with a long pause before the password or key prompt. What are the usual causes?

Troubleshooting

🎙 Spoken answer · ~60 s

I run the client with triple verbose and watch where the output pauses. A delay before the key exchange or banner often comes from the server doing a reverse DNS lookup on the client's address, controlled by UseDNS, when the resolver is slow or unreachable. A pause during authentication method negotiation commonly comes from GSSAPI trying Kerberos with an unreachable KDC, so disabling GSSAPIAuthentication for clients that do not need it fixes it. If the delay occurs after authentication succeeds, the cause is on the server side in PAM modules, an SSSD or LDAP lookup that is slow, systemd-logind taking time to create the session, or a home directory on a slow or hung NFS mount. I check the sshd logs and the time getent takes for user lookups. The fix is correcting DNS settings, disabling unneeded features, and repairing the identity backend.

Likely follow-ups

- What does UseDNS do?
- How can SSSD cause slow logins?

The server does a reverse DNS lookup of the client (UseDNS), GSSAPI authentication tries Kerberos and times out, or the login path is slow due to PAM, SSSD or a hung home directory mount. Use ssh -vvv and server logs to find the stall.

```
ssh -vvv user@host 2>&1 | grep -E 'debug1: (Connecting|SSH2_MSG|Authentications)'
```

```
sudo journalctl -u sshd -f
```

```
grep -E 'UseDNS|GSSAPIAuthentication' /etc/ssh/sshd_config
```

```
time getent hosts 203.0.113.5
```

Interview tip: Timing the verbose client output shows exactly where the delay happens: before key exchange, before auth, or after login.

📚 Official docs

- OpenSSH manual pages

certs.eknathalabs.com

Q158 TLS errors, Kerberos failures or odd log timestamps appear on some servers. How could clock drift be the cause, and how do you fix it?

Troubleshooting

🎙 Spoken answer · ~62 s

Many security protocols depend on accurate time. TLS clients reject certificates that are not yet valid or already expired by the local clock, Kerberos allows only a few minutes of skew between client, server and KDC, and distributed systems such as databases and tokens rely on clocks for ordering and expiry. If a server drifts, I see certificate not yet valid errors, authentication failures, or logs that do not line up between hosts. I check timedatectl for synchronisation state, and chronyc tracking for the current offset and sources. Causes include NTP blocked by a firewall, a bad or unreachable time source, a paused VM or live migration, or conflicting sync mechanisms. I fix connectivity, point to reliable internal sources, and let chrony slew gently, using makestep for a large initial correction, being careful that stepping the clock can disrupt applications. Monitoring offset across the fleet catches drift early.

Likely follow-ups

- Slewing versus stepping the clock?
- Why use internal NTP servers?

Certificates have validity windows and Kerberos tolerates only minutes of skew. If NTP is down or the clock steps, validation fails. Check chronyc tracking, sync sources and virtualisation time sync, then step the clock safely and fix the time source.

```
timedatectl status
```

```
chronyc tracking; chronyc sources -v
```

```
sudo chronyc makestep
```

```
date -u; curl -sI https://example.com | grep -i '^date'
```

Interview tip: On VMs, avoid two competing time sync mechanisms such as the hypervisor tools and chrony both adjusting the clock.

📚 Start here (official docs)

- Linux man-pages project

certs.eknathalabs.com

Q159 A log file is not rotating and the disk is filling. How do you diagnose logrotate problems?

Troubleshooting

🎙 Spoken answer · ~65 s

I begin with a debug dry run of the specific config, which shows whether logrotate considers the file, why it skips it, and any syntax or permission errors. Then I check that logrotate is actually being triggered by its systemd timer or cron job, and the state file for the last rotation time. Common problems are a glob pattern that does not match, wrong ownership or permissions on the directory so logrotate refuses to rotate, SELinux denials, a size or time condition not yet met, or a failing postrotate script that aborts. Even when rotation works, space might not be freed if the application holds the old file open and continues writing to the renamed or deleted file. The fix is a postrotate command that signals the app to reopen logs, or copytruncate for apps that cannot, accepting a small risk of losing lines. I also add compression, a retention count and monitoring on disk usage.

Likely follow-ups

- copytruncate trade-offs?
- Why isn't space freed after rotation?

Test the config with logrotate -d, check the state file and timer or cron, permissions and SELinux, and whether the app keeps writing to the old inode. Use copytruncate or a postrotate signal so the app reopens its log.

```
sudo logrotate -d /etc/logrotate.d/myapp
```

```
sudo logrotate -vf /etc/logrotate.d/myapp
```

```
cat /var/lib/logrotate/status | grep myapp
```

```
systemctl list-timers | grep logrotate
```

Interview tip: After rotation the app must reopen its file, otherwise it keeps writing to the renamed file or a deleted inode and space is not freed.

📚 Start here (official docs)

- Linux man-pages project

certs.eknathalabs.com

Q160 The system is swapping heavily and everything is slow, but free shows a lot of memory used by cache. What is going on?

Troubleshooting

🎙 **Spoken answer · ~64 s**

I start by reading the available column from free, which estimates memory that can be given to applications without swapping, because cache and buffers can be reclaimed. If available is low, there is real pressure from anonymous memory such as heaps. Vmstat shows ongoing swap in and swap out activity, and sustained non-zero values mean the system is thrashing. I identify the biggest swap users by looking at VmSwap per process, and check whether one process is leaking, whether a memory cgroup limit forces its processes into swap, or whether many processes simply need more RAM than the host has. Swappiness influences how eagerly the kernel swaps versus dropping cache, but it is not the root cause. Remedies are fixing the leak, reducing workload, adding memory, tuning limits, and for databases lowering swappiness. If the system is unusable, I may need to restart the offending service, and use pressure stall information to monitor.

Likely follow-ups

- What does swappiness control?
- What is memory pressure stall information?

Page cache is reclaimable and counted in available memory. Heavy swapping suggests real anonymous memory pressure, a swappiness setting that favours swapping, memory cgroup limits, or a leak. Check available, si/so, and per-process swap usage.

```
free -m
```

```
vmstat 1 5
```

```
for f in /proc/*/status; do awk '/Name|VmSwap/{printf $2" "} END{print ""}' $f; done | sort -k2 -nr | head
```

```
sysctl vm.swappiness
```

Interview tip: Read the available column, not free. Cache will be dropped when needed, but anonymous memory has to be swapped.

📚 **Start here (official docs)**

- Linux man-pages project

certs.eknathalabs.com

Q161 A client sees 'connection refused' sometimes, 'timed out' other times, and 'reset by peer' on others. What does each tell you?

Networking

🎙 **Spoken answer · ~65 s**

Each message points at a different layer. Connection refused is quick: the destination host is reachable and its kernel replied with a reset because no process is listening on that port, or a firewall rule explicitly rejects. I check that the service is running, listening on the right address, and the rules. Timed out means no reply at all, so packets are being dropped, from a firewall with a drop rule, a security group, a routing problem, an unreachable host, or an overloaded listen queue. I trace the path and capture packets to see whether SYNs arrive. Connection reset by peer happens after the connection exists, when the other side, or a device in the middle like a load balancer, NAT or firewall, sends a reset, for instance after an idle timeout, an application crash or a protocol error. Packet captures show who sent the reset and when, which tells me where to look next.

Likely follow-ups

- Why would a firewall send RST instead of dropping?
- How do idle timeouts cause resets?

Refused means a host answered with RST because nothing is listening or a firewall rejects. Timed out means packets were dropped silently, or the path or host is down. Reset by peer means the connection was established and then forcibly closed by the peer, a proxy, or a firewall idle timeout.

```
nc -vz host 443
```

```
curl -v --connect-timeout 5 https://host/
```

```
sudo tcpdump -ni any host 10.0.0.5 and port 443
```

```
ss -tlnp | grep 443
```

Interview tip: Capture packets at both ends if you can. Seeing who sends the RST or where the SYN stops settles most arguments.

📚 **Official docs**

- RFC 9293: TCP

certs.eknathalabs.com

Q162 Latency is bad and ethtool shows RX drops or the listen queue overflows. How do you tell NIC, kernel and application limits apart?

Performance

🎙 Spoken answer · ~62 s

I look at three places. First, NIC level counters from ethtool: rx dropped or missed counts rising mean the card or its ring buffer overflowed because the kernel did not drain packets fast enough, which I can ease by increasing ring sizes or spreading interrupts across CPUs with receive side scaling and IRQ affinity. Second, the kernel network stack: softnet statistics showing drops or time squeeze, and one core saturated in softirq. Third, the application's accept queue: listen overflow and drop counters in nstat, together with the Recv-Q of a listening socket from ss, show that the app does not accept connections fast enough or its backlog is capped by somaxconn. Fixes depend on the layer: tune ring buffers and interrupt distribution, raise somaxconn and the application backlog, add worker threads or processes, or scale horizontally behind a load balancer. I validate under load with before and after counters.

Likely follow-ups

- What is RSS in networking?
- What does time squeeze mean in softnet_stat?

Check NIC counters (rx_dropped, rx_missed), softirq CPU usage, ring buffer sizes, the accept queue (ss, nstat ListenOverflows), and backlog sysctls. Tune ring buffers, RSS/IRQ affinity, somaxconn and the app's backlog, or scale out.

```
ethtool -S eth0 | grep -E 'drop|miss|err'
```

```
nstat -az | grep -E 'ListenOverflows|ListenDrops|TcpExtTCPBacklogDrop'
```

```
cat /proc/net/softnet_stat
```

```
sudo ethtool -G eth0 rx 4096
```

Interview tip: A single CPU at 100 percent softirq while others idle usually means interrupts are not spread across cores.

📚 Start here (official docs)

- Linux man-pages project

certs.eknathalabs.com

Q163 A filesystem suddenly became read-only on a running server. What happened and how do you recover?

Boot & systemd

🎙 Spoken answer · ~63 s

When the kernel detects metadata errors or IO failures, filesystems like ext4 by default remount themselves read-only to avoid making damage worse, which shows up as applications failing to write. I check dmesg for the first error, which usually points to the cause: hardware IO errors, a SCSI or NVMe timeout, a storage path failure, or corruption after an unclean shutdown. In the cloud I check whether the volume had an incident, was resized, or briefly lost connectivity. Recovery starts by stopping the applications, taking a backup or snapshot of the volume if possible, then unmounting the filesystem, or booting into a rescue environment for the root volume, and running fsck, or xfs_repair for XFS, to repair it, before remounting read-write. If errors recur, the underlying disk is unhealthy, so I check SMART data, replace it and restore from backup. Monitoring for ro mounts and IO errors gives early warning.

Likely follow-ups

- What does errors=remount-ro do?
- Why shouldn't you fsck a mounted filesystem?

ext4 and XFS remount read-only on errors by default (errors=remount-ro) to protect data. Causes include disk or controller errors, cloud volume detach, or corruption. Check dmesg, stop writers, run fsck from a rescue or after unmount, and replace failing hardware.

```
dmesg -T | grep -iE 'EXT4-fs error|I/O error|remount|xfs'
```

```
mount | grep ' ro,'
```

```
sudo fsck -fy /dev/nvme1n1p1   # only when unmounted
```

```
smartctl -a /dev/sda
```

Interview tip: On cloud VMs, a brief volume detach or storage incident often causes this, so check provider events before assuming corruption.

📚 Official docs

- Linux kernel docs: ext4

certs.eknathalabs.com

Q164 A file has mode 777 and you are root, yet you get 'Permission denied' executing or writing it. What else can block access?

Security

🎙 **Spoken answer · ~62 s**

Standard permission bits are only one layer. If execution fails, the filesystem may be mounted noexec, which is common on tmp directories, or read-only for writes, and findmnt shows the mount options. Mandatory access control systems like SELinux or AppArmor can deny the operation even for root, and the audit log or journal shows an AVC denial with the context involved. File attributes such as immutable block modification, ACL entries and the ACL mask can restrict effective permissions beyond the mode, and every directory in the path needs the execute bit for traversal. For scripts, a bad shebang line caused by Windows line endings or a missing interpreter produces a misleading no such file or permission error. On network filesystems, root squash maps root to an unprivileged user. I work through this checklist systematically, find the layer that denies access, and fix it correctly rather than loosening permissions broadly.

Likely follow-ups

- What is root squash?
- How do you read an AVC denial?

Check mount options (noexec, ro, nosuid), SELinux or AppArmor denials, immutable attributes, missing execute permission on a parent directory, a bad interpreter line (CRLF), ACL masks, and filesystem type limits such as FAT or NFS root squash.

```
findmnt -T /opt/app/run.sh
```

```
ls -lZ /opt/app/run.sh; getenforce; sudo ausearch -m avc -ts recent
```

```
lsattr /opt/app/run.sh; getfacl /opt/app/run.sh
```

```
head -1 /opt/app/run.sh | od -c | head -2
```

Interview tip: Work down a checklist rather than guessing: mount, MAC, attributes, ACL, path, interpreter.

📚 **Official docs**

- Linux man-pages: path_resolution(7)

certs.eknathalabs.com

Q165 A systemd service fails to start with status=203/EXEC or 'Failed at step USER'. How do you diagnose it?

Troubleshooting

🎙 **Spoken answer · ~64 s**

The status output includes the main process exit status, and the service journal gives more detail. Status 203 EXEC means systemd failed to execute the command in ExecStart, so I check that the path is absolute and exists, the file has the execute bit, the interpreter named in the shebang exists, the filesystem is not mounted noexec, and SELinux allows execution of that label. Messages like failed at step USER or GROUP mean the account named in the unit does not exist or cannot be resolved. Other step names help too: CHDIR for a missing working directory, NAMESPACE for sandboxing options that cannot be applied, and CAPABILITIES for capability issues. I run the command by hand as the same user and environment to reproduce, review the effective unit with systemctl cat to catch drop-in overrides, validate with systemd-analyze verify, then reload the daemon and restart. Setting Restart and rate limits avoids tight failure loops.

Likely follow-ups

- What does 'failed at step NAMESPACE' mean?
- How do drop-ins override units?

203/EXEC means systemd could not execute the binary: wrong path, no execute bit, missing interpreter, noexec mount or SELinux label. Failed at step USER means the configured user does not exist. Read systemctl status and journalctl -u, then test the command as that user.

```
systemctl status myapp.service
```

```
journalctl -u myapp.service -b --no-pager | tail -30
```

```
sudo -u appsvc /opt/app/bin/start
```

```
systemd-analyze verify /etc/systemd/system/myapp.service
```

Interview tip: systemctl cat shows the effective unit including drop-ins, which is where surprise overrides hide.

📚 **Official docs**

- systemd.service(5)

certs.eknathalabs.com

Q166 A cron job runs fine by hand but never runs or fails silently from cron. How do you debug it?

Scripting & automation

🎙 Spoken answer · ~66 s

Cron starts commands with a very small environment: a minimal PATH, no profile files, a home directory as the working directory, and no terminal, so scripts relying on aliases, environment variables or relative paths break. My first step is checking that the job is being triggered at all, by looking for CRON entries in syslog or the journal, which also reveals syntax errors and wrong user fields in files under etc cron.d, where a user column is required unlike in a user crontab. If it is triggered but fails, I redirect standard output and error to a log file, since by default cron tries to mail output. I use absolute paths for all commands, set PATH and needed variables at the top of the crontab or script, make the script executable with a proper shebang, and check percent signs, which cron treats specially. A flock wrapper prevents overlapping runs, and systemd timers are an alternative with better logging.

Likely follow-ups

- Why are percent signs a problem in crontab?
- Cron versus systemd timers?

Cron runs with a minimal environment (PATH, no profile), a different working directory and no TTY. Check syslog or journal for CRON entries, mail output, the crontab syntax and user, absolute paths, permissions, and redirect output to a log.

```
grep CRON /var/log/syslog | tail   # or journalctl -u cron
```

```
crontab -l; ls -l /etc/cron.d/
```

```
* * * * * /usr/bin/env > /tmp/cron-env.txt 2>&1
```

```
*/5 * * * * /opt/job.sh >> /var/log/job.log 2>&1
```

Interview tip: Dumping env from a test cron entry shows precisely how different the cron environment is from your shell.

📚 Start here (official docs)

- Linux man-pages project

certs.eknathalabs.com

Q167 Processes are stuck in D state after an NFS server outage, and the client hangs on ls or df. How do you recover?

Troubleshooting

🎙 Spoken answer · ~65 s

By default NFS uses hard mounts, so if the server disappears the client keeps retrying forever, and processes touching that mount sleep in uninterruptible state, which cannot be killed, and even df or ls can hang. The best recovery is to restore the server or the network path, after which the processes continue. If that cannot happen soon, I try a forced and lazy unmount, which detaches the mount point from the namespace so new accesses do not hang, though processes already blocked may stay stuck until the server returns or the host reboots. To find affected mounts safely I use commands wrapped in timeout. For prevention I design for failure: use soft mounts only for data where errors are acceptable, because they can corrupt writes, tune timeo and retrans, use autofs to mount on demand, avoid critical paths depending on NFS, monitor server health, and use NFS version four with a highly available server.

Likely follow-ups

- Soft versus hard NFS mounts?
- What does lazy unmount do?

Hard NFS mounts block forever waiting for the server by design. Restore the server or network, or use umount -f / -l and, if mounted with intr or soft options, interrupt. As a last resort, reboot. Prevent with soft,timeo options where safe, autofs and monitoring.

```
ps -eo pid,stat,wchan:30,cmd | awk '$2 ~ /D/'
```

```
mount -t nfs,nfs4
```

```
timeout 5 stat /mnt/data || echo 'NFS hung'
```

```
sudo umount -f -l /mnt/data
```

Interview tip: Never run df or ls on a suspected hung NFS mount without timeout, or your own shell joins the D-state pile.

📚 Start here (official docs)

- Linux man-pages project

certs.eknathalabs.com

Q168 A database server shows irregular latency spikes. How would you check transparent huge pages, swap, NUMA and IO scheduler settings together?

Performance

🎙 **Spoken answer · ~61 s**

I treat it as a checklist of known stall sources and measure each during a spike. For memory, I read the compaction and THP counters in vmstat, because direct compaction and huge page splitting can pause allocations, and check that transparent huge pages are set to madvise or never for the database. I check swap activity and the swappiness setting. For NUMA, I look at per-node memory with numastat, since one node running low can force reclaim while others are free. Writeback matters: large dirty page ratios allow big bursts of flushing that block writers, so I lower dirty background ratios so flushing is steadier. For storage I check the IO scheduler, such as none or mq-deadline for NVMe, the queue depth, and device latency in iostat. Finally, CPU frequency scaling and deep C-states can add wake-up latency. I change one thing at a time and measure.

Likely follow-ups

- Why lower dirty ratios for databases?
- Which IO scheduler suits NVMe?

Review each known source of stalls in order: THP and compaction, swap activity, NUMA imbalance, dirty page writeback bursts, IO scheduler and queue depth, and CPU frequency or C-states. Correlate spikes with kernel counters rather than guessing.

```
grep -E 'thp|compact' /proc/vmstat
```

```
numastat -m | head -20
```

```
cat /sys/block/nvme0n1/queue/scheduler; sysctl vm.dirty_ratio vm.dirty_background_ratio vm.swappiness
```

```
perf trace -s -p $(pidof postgres) -- sleep 10
```

Interview tip: Capture counters before and after a spike, so you compare numbers instead of opinions.

📚 **Start here (official docs)**

- Linux man-pages project

certs.eknathalabs.com

Q169 You find an unexpected listening port on a production server. How do you identify what owns it and decide whether it is malicious?

Security

🎙 Spoken answer · ~67 s

First I map the listener to a process with ss or lsof, which gives the process ID, user and command. Then I look at the evidence under proc: the executable path, command line, current working directory, environment and open files, and when it started. I check whether the binary belongs to an installed package with the package manager and verify its hash against the package or a known good source. Strong warning signs are execution from temp directories, a deleted binary that is still running, an unusual parent process, a name imitating a system process, or outbound connections to unfamiliar addresses. I compare against a baseline or configuration management inventory to see if the service is expected. If it looks malicious I isolate the host from the network rather than just killing the process, capture memory and process information, preserve logs, and follow the incident response process before cleaning up. If it is legitimate, I document and firewall it appropriately.

Likely follow-ups

- Why isolate before killing?
- How do you verify a binary against its package?

Map the port to a process with ss -tlnp or lsof, then examine the binary path, hash, parent, start time, package ownership, systemd unit and network connections. Compare to a baseline, isolate if suspicious, and preserve evidence before killing.

```
sudo ss -tlnp | grep :4444
```

```
sudo ls -l /proc/<pid>/exe; sudo cat /proc/<pid>/cmdline | tr '\0' ' '
```

```
rpm -qf /path/to/bin   # or: dpkg -S /path/to/bin
```

```
sudo lsof -p <pid> -i
```

Interview tip: A binary running from /tmp, a deleted executable (exe → (deleted)) or no package owner are strong red flags.

📚 Official docs

- sudoers(5) manual

certs.eknathalabs.com

Q170 The disk under /var/lib/docker is full on a container host. What do you clean up safely, and how do you stop it recurring?

Troubleshooting

🎙 Spoken answer · ~61 s

I start with docker system df to see how space is divided between images, containers, local volumes and build cache, and use du on the containers directory to spot giant JSON log files. Safe cleanups are removing stopped containers, dangling and unused images older than a week or so, and old build cache. I am careful with volumes, since they may hold real data, and never prune them blindly. A runaway log file can be truncated if the container must keep running. To prevent recurrence I configure the logging driver defaults in daemon.json with max-size and max-file, or use a driver that ships logs elsewhere, put Docker's data root on a dedicated volume so it cannot fill the root filesystem, schedule regular pruning in CI runners, use smaller base images and multi-stage builds, set ephemeral-storage requests and limits in Kubernetes, and alert at eighty percent usage.

Likely follow-ups

- Why not prune volumes blindly?
- How do you set log limits in daemon.json?

Find what consumes space with docker system df: unused images, stopped containers, build cache, volumes and oversized container logs. Prune carefully, set log rotation limits, put Docker data on its own volume, and alert on usage.

```
docker system df -v
```

```
docker image prune -a --filter 'until=168h'
```

```
docker builder prune --filter 'until=72h'
```

```
du -sh /var/lib/docker/containers/*/*-json.log | sort -h | tail
```

Interview tip: Container JSON logs without max-size are the most common silent disk eater. Set log-opts in daemon.json.

📚 Official docs

- Docker documentation

certs.eknathalabs.com

Q171 **A Java or Go service in a container gets OOMKilled, but the heap looks fine. Which non-heap memory and cgroup details do you check?**

Performance

🎙 **Spoken answer · ~65 s**

The kernel enforces the cgroup memory limit on everything the container uses, not just the language heap. For a JVM that includes metaspace, code cache, thread stacks, direct and mapped buffers, GC structures and native libraries, and the page cache for files the container reads and writes also counts, though it can be reclaimed. A Go program has stacks, runtime overhead and memory not yet returned to the operating system. I read the cgroup's current usage and its stat file to see the split between anonymous memory, file cache, kernel memory and sockets, and compare it with runtime metrics such as native memory tracking. Then I adjust: size the heap as a percentage of the container limit, limit thread counts and direct memory, set GOMEMLIMIT for Go, and raise the limit if the process legitimately needs it. I also check dmesg for the kill details and whether a sidecar, rather than the app, was the victim.

Likely follow-ups

- What is GOMEMLIMIT?
- Does page cache count toward the limit?

The cgroup limit covers heap plus metaspace, thread stacks, direct buffers, mmap'd files, native libraries and page cache attributed to the cgroup. Compare memory.current and memory.stat with runtime metrics and leave headroom.

```
cat /sys/fs/cgroup/memory.max /sys/fs/cgroup/memory.current
```

```
grep -E '^(anon|file|kernel|slab|sock) ' /sys/fs/cgroup/memory.stat
```

```
jcmd <pid> VM.native_memory summary
```

```
dmesg | grep -i 'killed process'
```

Interview tip: Rule of thumb: set JVM heap to roughly 60 to 75 percent of the container limit, not 100 percent.

📚 **Official docs**

- Kubernetes: Debug Pods
- Kubernetes: Determine the reason for Pod failure
- Kubernetes: Resource management for Pods & containers

certs.eknathalabs.com

Q172 What do umask and the setgid directory bit do together to create a secure shared project directory?

Users & permissions

🎙 Spoken answer · ~61 s

For a shared directory I create a dedicated group, make it the directory's group, and set mode two seven seven zero. The leading two is the setgid bit, so every file and subdirectory created inside inherits the project group instead of the creator's primary group, and the zero for others keeps outsiders out. The remaining problem is that new files are created according to the creator's umask, and with the common umask of zero two two, group members cannot write files made by someone else. I can set umask zero zero two for the team, but that depends on each user's shell setup. A more robust approach is a default ACL on the directory granting the group read, write and execute, which new files inherit regardless of umask. Removing others entirely and adding the sticky bit where users should not delete each other's files completes the design.

Likely follow-ups

- What does the sticky bit do?
- Why are default ACLs more reliable than umask?

umask removes permission bits from newly created files. The setgid bit on a directory makes new files inherit the directory's group. Together, with a project group and umask 002 or default ACLs, team members can collaborate without opening files to everyone.

```
sudo groupadd proj && sudo mkdir /srv/proj && sudo chgrp proj /srv/proj
```

```
sudo chmod 2770 /srv/proj
```

```
sudo setfacl -d -m g:proj:rwx /srv/proj
```

```
umask
```

Interview tip: Default ACLs are more reliable than relying on every user's umask, because they apply regardless of how the file was created.

📚 Official docs

- sudoers(5) manual

certs.eknathalabs.com

Q173 What are sparse files, and how can they make du, ls and df disagree?

Filesystem

🎙 **Spoken answer · ~67 s**

A sparse file is one where regions that were never written are not backed by disk blocks. When you read them you get zeros, but they consume no space. You can create one with truncate to extend the file size, or by seeking past the end before writing. This is why ls minus l, which reports the apparent length, can show ten gigabytes while du, which counts allocated blocks, shows almost nothing. Virtual machine images, some database files and core dumps are often sparse. The surprises come later. A naive copy or backup tool that reads the zeros and writes them out turns a small file into a full-sized one, so I use sparse-aware options in cp, rsync or tar. Also, a sparse file can fill the disk unexpectedly as it is written, because the filesystem promised more logical space than physical space it has, which is thin provisioning inside a file. Fallocate preallocates real blocks when I need guarantees.

Likely follow-ups

- How does fallocate differ from truncate?
- How do you copy a sparse file efficiently?

A sparse file has holes: ranges never written that take no disk blocks but read back as zeros. ls shows the apparent size, du shows blocks actually allocated, and a full-looking VM image can use far less space. Copying without sparse support can balloon it.

```
truncate -s 10G disk.img; ls -lh disk.img; du -h disk.img
```

```
cp --sparse=always big.img copy.img
```

```
filefrag -v disk.img
```

```
fallocate -l 1G prealloc.img
```

Interview tip: This explains why a database or VM image can fill a disk later than its apparent size suggests.

📚 **Start here (official docs)**

- Linux man-pages project

certs.eknathalabs.com

Q174 How do you use pidstat, top -H and /proc to find which thread of a multi-threaded process is hot?

Processes

🎙 Spoken answer · ~63 s

A process that appears to use one hundred fifty percent CPU may have one runaway thread or many moderately busy threads, and the fix is different. top with the H option shows individual threads, and pidstat with the t flag reports CPU use per thread over time. Once I have the busiest thread ID, I can look at proc for its name and state, and use perf top or perf record on that task ID to see where it spends time. For a Java application, I convert the thread ID to hexadecimal, take a thread dump with jstack, and search for the matching native ID, which shows the exact stack trace of the hot thread, perhaps a GC thread, a spin loop or a regex. For Go, pprof profiles goroutines, and for Python, py-spy attaches without restarting. This workflow narrows the problem from machine to process to thread to code line.

Likely follow-ups

- How do you profile a Go service?
- What does py-spy do?

Per-thread views show what a per-process view hides. top -H and pidstat -t list threads with their CPU use; the thread ID can be converted to hex to match a Java thread dump, or used with perf and gdb to see its stack.

```
top -H -p $(pidof java)
```

```
pidstat -t -p $(pidof java) 1 5
```

```
printf '%x\n' 12345
```

```
jstack $(pidof java) | grep -A15 nid=0x3039
```

Interview tip: The Java trick, converting the hot thread's TID to hex and finding nid in the thread dump, is a classic interview demonstration.

📚 Official docs

- Linux man-pages: proc(5)

certs.eknathalabs.com

Q175 Why should shell scripts quote variables and avoid parsing ls, and what are safe patterns for looping over files?

Scripting & automation

🎙 Spoken answer · ~64 s

In bash, an unquoted variable is split on whitespace and then subject to filename expansion, so a file named my report dot txt becomes two arguments, and a variable containing an asterisk expands to every file in the directory. That has deleted real data in production scripts, for example with rm of a path built from an empty variable. So I quote every expansion and use double dash to end option parsing for names that may start with a hyphen. Parsing ls output is wrong because filenames can contain spaces, newlines and other special characters. Safe patterns are looping over a glob directly, checking the file exists to handle no matches, or using find with print0 so names are separated by null bytes, consumed by xargs minus zero or a while read loop with the null delimiter. I also use set minus u to catch unset variables, and run shellcheck as part of CI.

Likely follow-ups

- Why does an empty variable make rm dangerous?
- What does set -u do?

Unquoted variables undergo word splitting and globbing, so filenames with spaces or asterisks break or expand unexpectedly. ls output is not reliably parseable. Use globs, find -print0 with xargs -0 or while read -r -d '', and quote everything.

```
for f in ./*.log; do [ -e "$f" ] || continue; gzip -- "$f"; done
```

```
find . -name '*.tmp' -print0 | xargs -0 rm --
```

```
while IFS= read -r -d '' f; do echo "$f"; done < <(find . -type f -print0)
```

```
shellcheck script.sh
```

Interview tip: Run shellcheck in CI. It catches most quoting bugs before they reach production.

📚 Official docs

- GNU Bash reference manual

certs.eknathalabs.com